STUDY GUIDE

# DATA SCIENCE IN PRACTICE: FROM BUSINESS QUESTION TO DECISION

*How working data scientists turn a vague business ask into a measured decision: framing, trustworthy data, honest statistics, features, experiments, causal thinking, forecasts, recommenders, domains, and the communication that gets work used.*

Taught the IIT Professor Way — Real Numbers. Real Problems. Real Understanding.

| | | | | |
|---|---|---|---|---|
| **CHAPTER 1** What a Data Scientist Actually Does | **CHAPTER 2** Getting Trustworthy Data: Collection, Point-in-Time Sets, Quality, Labels and Privacy | **CHAPTER 3** Exploratory Data Analysis for Modelling | **CHAPTER 4** Statistical Thinking: Ways Data Fools You | **CHAPTER 5** Feature Engineering I: Transformations, Encoding, Dates, Text and Geo |
| **CHAPTER 6** Feature Engineering II: Behavioural Features, Selection and Leakage-Safe Pipelines | **CHAPTER 7** Experimentation: A/B Testing Done Properly | **CHAPTER 8** Causal Inference When You Cannot Randomise | **CHAPTER 9** Forecasting and Time Series for Decisions | **CHAPTER 10** Recommendation, Ranking and Search |
| **CHAPTER 11** Applied Domains: Churn, Credit Risk, Fraud, Marketing, Demand, Healthcare and Operations | **CHAPTER 12** From Prediction to Decision: Cost Matrices, Thresholds, Optimization and Simulation | **CHAPTER 13** Communication, Stakeholders, Ethics, Reproducibility and Taking Work to Production | **CHAPTER 14** Capstone Projects and the Data Science Interview Bank | **CHAPTER 15** Project Lab 1: Churn, A/B Toolkit and Demand Forecasting |
| **CHAPTER 16** Project Lab 2: Recommender, Credit Scorecard and Causal Analysis | **APPENDIX** Cheat Sheet | | | |


## A Note From Your Professor

If you have ever planned a project, spotted what was going wrong and explained it to a manager, you already have more than half of what this document teaches, in a new setting. Data science is not "run an algorithm". It is turning a fuzzy question ("customers are leaving") into a decision someone can act on ("send this offer to these people, this week"), and proving with honest numbers that the decision worked.

Most books teach models. This document teaches the parts around the model, where careers are made and projects fail: choosing the right question, building a training table that matches real life, hunting leakage, knowing how data fools you, running experiments, reasoning about cause and effect, forecasting with intervals, and

writing the memo that makes a stakeholder say yes. The models themselves, such as logistic regression, trees and neural networks, live in their own document ( `study/ml/` ). Here we call them tools and focus on how to use them well.

Every chapter uses real code on synthetic datasets with fixed seeds, so you can reproduce each number. When the result is disappointing (for example a simple rule that nearly beats a model), we show it, because that is what real work looks like.

One rule, please: **never move on until the current topic feels obvious.** If a worked problem feels shaky, redo it by hand with fresh numbers. One hour a day with understanding beats five hours of skimming.

## What Every Topic Contains

1. **INTUITION** callout: the problem this concept solves, in everyday language, often with an analogy from everyday life (shops, queues, schools, hospitals).
2. **Formal definition / how it works**: precise but plain, with every new word defined at first use.
3. **Worked problems with real numbers**: every step shown, with a Check and a plain-language meaning (or runnable code with the exact output).
4. **The picture**: an ASCII diagram, table or figure, explained completely.
5. **Connection to the real job**: where it shows up in analytics, data science and ML work.
6. **Applications**: domains that use it.
7. **COMMON MISTAKE** callouts: what beginners get wrong and how to catch it.
8. **Check yourself**: 3 short questions with answers.

Each chapter ends with a recap (with a small formula and pattern table), a practice set of 8 to 12 graded problems with answers, and a prompt pack of 5 prompts you can paste into any AI assistant.

## Contents


1. **What a Data Scientist Actually Does** - the job and role types, the project lifecycle (CRISP-DM), turning a vague ask into a precise problem, baselines, and success metrics at three levels.
2. **Getting Trustworthy Data** - data sources, point-in-time correct training sets in SQL, quality checks, labels and ground truth (delay, noise, selection bias), privacy, DPDP and GDPR ideas.
3. **Exploratory Data Analysis for Modelling** - profiling, missingness, target analysis, segments, time and drift, and leakage hunting, with a full worked example.
4. **Statistical Thinking: Ways Data Fools You** - confounding, Simpson's paradox, selection bias, regression to the mean, base rates, Goodhart's law, multiple testing, and honest uncertainty.

5. **Feature Engineering I** - numeric transformations, encoding (target encoding done safely), dates and cycles, text and geo features.
6. **Feature Engineering II** - behavioural and aggregate features (RFM, rolling windows, lags, trends), feature selection and stability (PSI), leakage-safe pipelines.
7. **Experimentation** - A/B testing done properly: hypotheses, sample size and power, CUPED, sample ratio mismatch, peeking, sequential tests, bandits.
8. **Causal Inference** - potential outcomes, DAGs, propensity scores, difference-in-differences, synthetic control, regression discontinuity, instrumental variables, uplift modelling.
9. **Forecasting and Time Series for Decisions** - baselines, exponential smoothing, ARIMA ideas, ML with lags, rolling-origin backtests, prediction intervals, forecast to inventory decision.
10. **Recommendation, Ranking and Search** - two-stage systems, matrix factorisation, ranking metrics, cold start, offline versus online evaluation.
11. **Applied Domains** - churn and CLV, credit-risk scorecards (WoE, IV, KS, Gini, PSI), fraud, marketing mix and pricing, demand and supply chain, healthcare, performance-data analytics.
12. **From Prediction to Decision** - cost matrices, thresholds, optimisation, simulation, feedback loops.
13. **Communication, Ethics, Reproducibility and Production** - stakeholder management, writing, ethics checklist, reproducible work, prototype to production.
14. **Capstone Projects and the Interview Bank** - eight project briefs with acceptance criteria, mastery tests, and a data science interview question bank.
15. Project Lab 1: Churn, A/B Toolkit and Demand Forecasting
16. Project Lab 2: Recommender, Credit Scorecard and Causal Analysis

Appendix: **Cheat Sheet** (all key formulas and patterns by chapter) and **An Honest Closing Note**.

## How To Study This Document

Follow this loop for every chapter:

1. **Build first.** Read the motivating paragraph, then type the code of the first section yourself (do not paste) and run it. Before each run, write down your prediction of the output.
2. **Ask.** When something is unclear, use the chapter's Prompt pack. The prompts are self-contained: copy one, paste it into any AI assistant, and answer its questions.
3. **Just-in-time theory.** Read the formal part only when the code raises a question. The worked problems show every step; redo them with new numbers.
4. **Rebuild.** Close the document and rebuild the main example from a blank notebook, on your own data if you can (any spreadsheet or export you already work with is a good source).

5. **Teach.** Explain the chapter's main idea aloud in 5 minutes, as if to a colleague. If you get stuck, that is the topic to revisit.

**Pace.** About one chapter per week at one hour a day: three days for reading and running code, two days for the practice set, one day for the prompt pack and re-explaining, one day of rest or catching up. Chapters 1 to 6 are the foundation; do them in order. Chapters 7 to 12 can follow the role you are aiming for: product roles go to 7 and 8 first, supply chain and finance to 9 and 11, e-commerce to 10. Chapters 13 and 14 are for the last weeks before applying.

**Using the prompt packs.** Use "teach me" prompts before a chapter, "quiz me" prompts after, "review my work" prompts on your own mini-projects, and the "mock interview" prompt in the week before interviews. Always attempt before you ask for the answer.

**Reproducibility.** All code runs on Python 3 with `numpy`, `pandas`, `scikit-learn` and `scipy`, with fixed random seeds. The shop dataset used in Chapters 1 to 6 comes from the `ds_common.py` listing in Chapter 1. Output may differ slightly across library versions; where exact digits matter we say so.

## Prerequisites

- Python basics and `pandas` (the learner's Python study document).
- SQL basics: `SELECT`, `JOIN`, `GROUP BY`, CTEs (the learner's SQL study document).
- School-level statistics: mean, variance, normal distribution, probability (see the learner's Math PDF, statistics and probability pillars).
- No machine-learning background is needed to start; models appear as tools, and the algorithms are explained in `study/ml/`.

## Tools you need

All free.

- Python 3 with `numpy`, `pandas`, `scipy`, `scikit-learn`, `matplotlib` (the examples in this document were run with exactly these).
- Jupyter Notebook or JupyterLab, or VS Code with the Jupyter extension.
- SQLite (included with Python) for the SQL examples.
- Optional, named in chapters but not required to follow along: `statsmodels`, `lightgbm`, `lifelines`, `prophet`, `pulp` or `OR-Tools`, `PyMC`, `pandera`, `great_expectations`, `networkx`.
- Git for version control.

CHAPTER 1

# WHAT A DATA SCIENTIST ACTUALLY DOES

*The job, the project lifecycle, how to frame a vague question, and why the first model you build should be a boring one.*

Imagine your manager says on Monday: "Customers are leaving. Fix it with data." What do you do at 10 am? Most beginners open a notebook and train a model. Professionals first ask what decision will change, what is the simple thing the business does today, and how they will know they won. This chapter teaches you that first week of any project. By the end you will be able to turn a vague request into a one-page brief, choose the right framing (classification, survival, uplift, forecast), write the prediction time and label window, and compare any model to a baseline. We also meet the synthetic shop dataset that we use in many chapters, and we see a surprising result: a one-line rule almost beats a machine-learning model.

## 1.1 The Real Job and the Role Types

**INTUITION**

Think of a restaurant planning a new menu. Cooking the dish is maybe 20% of the work. The rest is asking "what do customers actually want?", "how many will come on a Friday?", "what will we do if it does not sell?", and then acting on the sales numbers. Data science is the same. The model is the visible part. The question, the data and the decision are the bigger part.

**How the time is really spent.** A typical project is about 60% understanding the problem and getting and cleaning data, 20% modelling, and 20% communicating and getting the result used. These numbers are rough, but the shape is true. Beginners invest in algorithms. Professionals invest in framing and in communication.

**Where data science creates value.** There are six main ways:

| Value type | Example | Typical output |
|---|---|---|
| Decisions (analytics) | Which city should we open a warehouse in? | A memo and a chart |
| Automation (ML) | Auto-route support tickets | A model in production |
| Personalisation | Show each user relevant products | A ranking system |
| Forecasting | How many items will we sell next month? | A forecast and an interval |

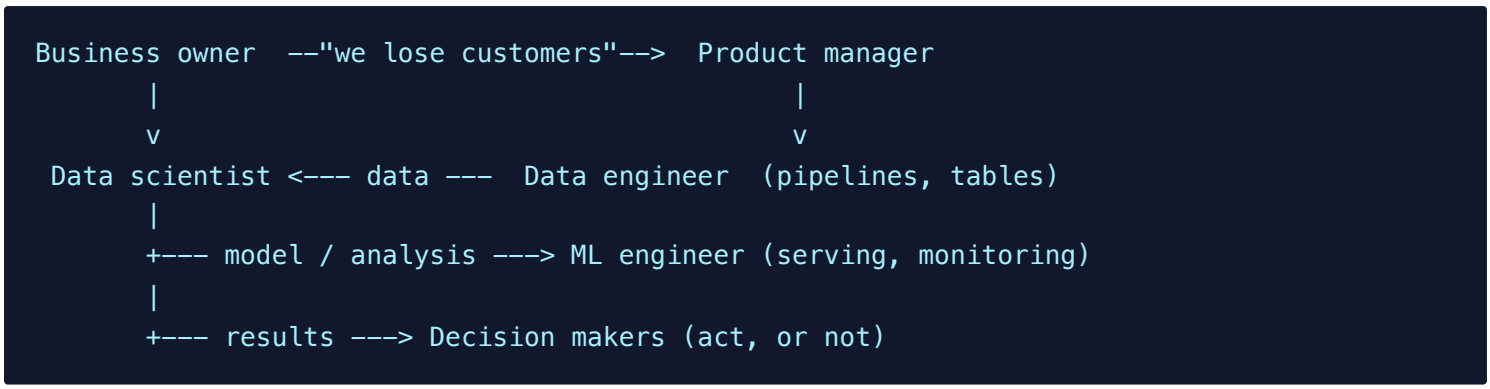
| Value type | Example | Typical output |
|---|---|---|
| Optimisation | Which customers get a discount, within budget? | A plan from a solver |
| Risk reduction | Block fraudulent payments | A score and a threshold |

**Role types.** Titles differ across companies, so read the job description, not only the title.

| Role | Main work | Where modelling sits |
|---|---|---|
| Data Analyst | Dashboards, SQL, ad-hoc questions | Light statistics |
| Data Scientist (product / decision) | Experiments, causal questions, metrics | Statistics heavy |
| Applied Scientist | Build and ship ML for a product area | Core |
| ML Engineer | Production ML systems, pipelines, serving | Engineering heavy |
| Research Scientist | New methods, papers | Core, often academic |
| Analytics Engineer | Clean, tested data models (dbt, SQL) | None to light |
| Decision Scientist | Predictions + optimisation + business rules | Mixed |
| Quant | Pricing and trading models in finance | Heavy maths |

You are a working professional moving into data roles. Two natural targets are *Data Scientist (product/decision)* and *ML/Data Platform or AIOps* roles, where knowledge of operations, delays and capacity is valuable. Pick one target and read job descriptions for it every month.

**How a DS team works with others.**




The data scientist is the translator in the middle. If any arrow is missing (no one owns the data, no one will act), the project fails regardless of model quality.

**Worked problems**

**Problem 1.1.1** A team spends 10 weeks on a project. Use the 60/20/20 rule. How many weeks go to each part, and how many days is "modelling" if a week has 5 working days?

| | |
|---|---|
| **Step 1** | 60% of 10 weeks = 6 weeks for understanding and data. |
| **Step 2** | 20% of 10 weeks = 2 weeks for modelling, and 2 weeks for communication and adoption. |
| **Step 3** | Modelling days = 2 × 5 = 10 working days. |
| **Answer** | 6 weeks problem and data, 2 weeks modelling (10 days), 2 weeks communication and adoption. |
| **Check** | 6 + 2 + 2 = 10 weeks, which matches the total. |
| **Why it matters** | This means that if you promise a stakeholder "a model in 2 weeks", you are really promising only the smallest slice; plan the other 8 weeks too. |

**Problem 1.1.2** A model scores 0.90 AUC. The business team never uses it because the output is a number between 0 and 1 and they have no action tied to it. The model took 8 weeks. Estimate the value created if each correct retention action is worth Rs 500.

| | |
|---|---|
| **Step 1** | Count the actions taken: zero, since nobody uses the output. |
| **Step 2** | Value = 0 actions × Rs 500 = Rs 0. |
| **Answer** | The value is Rs 0. Model quality multiplied by "usage" is what counts, and usage was zero. |
| **Check** | Any number multiplied by zero is zero, so a better AUC cannot rescue it. |
| **Why it matters** | In plain words: 8 weeks of work produced nothing because nobody agreed on the action first. |

**The picture**

```
Value created  =  quality of decision  x  how often it is used
                      (model)                (adoption)
A 0.90 AUC model used 0% of the time   ->  0
A 0.80 AUC model used 100% of the time ->  positive
```

**Connection to the real job.** In interviews you will be asked "tell me about a project". The best answers start with the decision and the impact, not with the algorithm.

**Applications**

- Retail: stock and pricing decisions.

- Banking: approve or reject a loan.
- Telecom: which customers to call before they leave.
- Operations: forecast how many orders a warehouse must pack before a sale event.

| COMMON MISTAKE | Building a clever model nobody asked for, or one that answers a question the business does not have. Before any code, write down "who will do what differently because of this?". If you cannot answer, stop. |
|---|---|

**Check yourself**

1. Name three of the six ways data science creates value.

Answer: for example decisions, forecasting, optimisation (also automation, personalisation, risk reduction).

1. Which role is most engineering-heavy: Data Analyst, ML Engineer or Quant?

Answer: ML Engineer.

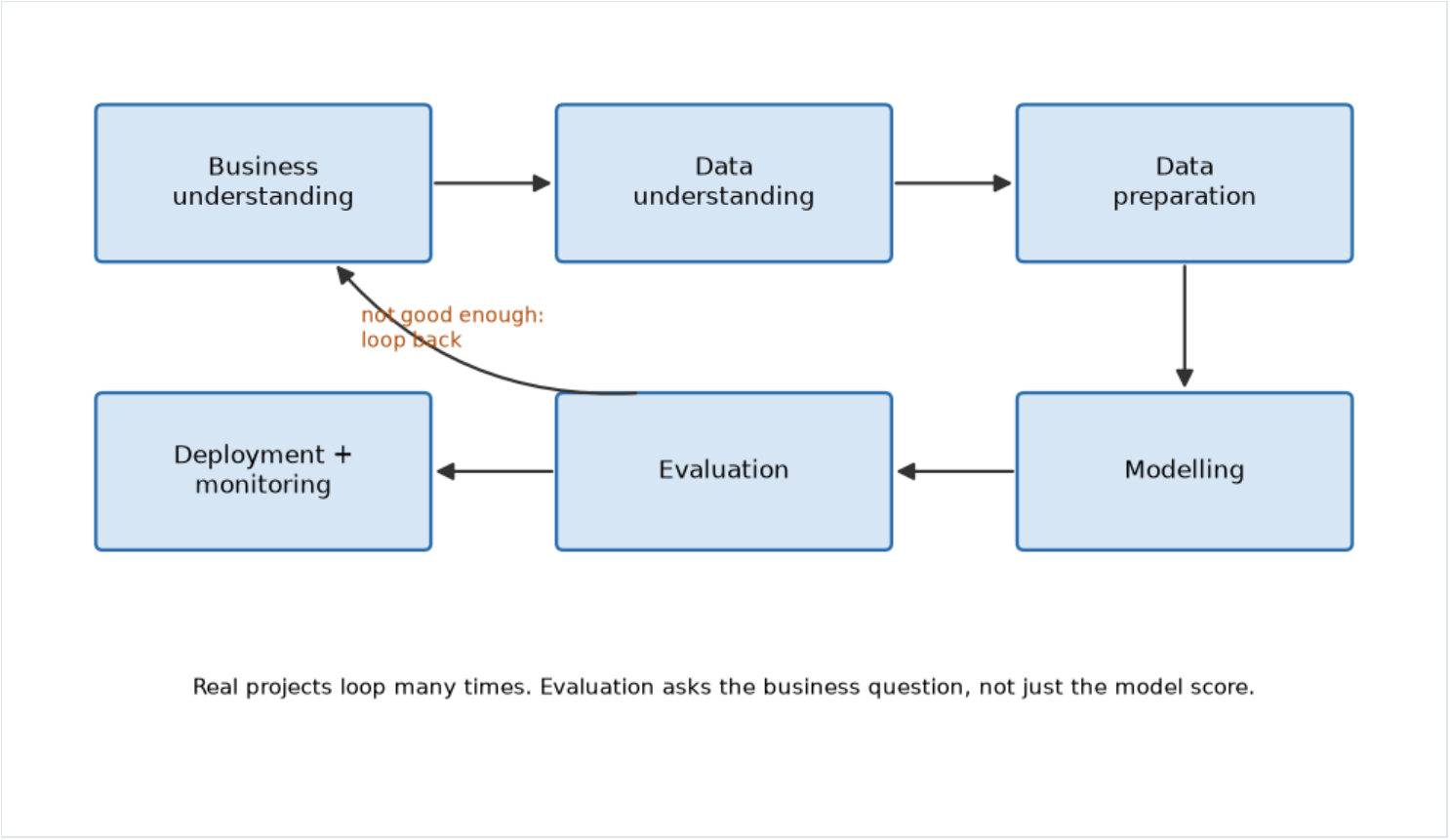
1. If usage is zero, what is the value of a very accurate model?

Answer: zero.

## 1.2 The Project Lifecycle (CRISP-DM and Modern Variants)

| INTUITION | Building a house has phases too: plan, get permits, lay the foundation, build, inspect, hand over. Skipping the planning phase gives you a beautiful house on the wrong plot. CRISP-DM is the same idea for data science projects: a checklist of phases so you do not skip the boring-but-vital ones. |
|---|---|

**CRISP-DM** (Cross-Industry Standard Process for Data Mining) has six phases:




*The CRISP-DM lifecycle: six phases with loops back*

How to read this picture: each box is a phase. The straight arrows show the usual order. The curved arrow from Evaluation back to Business understanding is the loop: if the result is not good enough for the business, you go back to the question, not only to the model.

The same flow as text:

```
Business understanding <-----------------------+
       |                                       |
       v                                       |
Data understanding  <--->  Data preparation    |
                                |              |
                                v              |
                           Modelling           |
                                |              |
                                v              |
                           Evaluation -------+   (not good enough: loop back)
                                |
                                v
                          Deployment + monitoring
```

Real projects loop many times. You find in Data understanding that the label is wrong, so you return to Business understanding. Modern variants add MLOps ideas (monitoring, retraining) and say "ship a thin version early".

| Phase | Questions to answer | Output |
|---|---|---|
| Business understanding | Who decides? What changes? What does success look like in money, time or risk? What is the current baseline? | One-page brief |
| Data understanding | What data exists? How is it generated? What is missing by design? | EDA notes, data dictionary |
| Data preparation | Grain, joins, cleaning, point-in-time features | Training table |
| Modelling | Baseline first, then improve | Candidate models, validation scores |
| Evaluation | Does it meet the business metric, not only the model metric? | Decision memo |
| Deployment | Who runs it, how is it monitored, who owns it? | Service or report, monitoring plan |

Here is how each phase looks in an everyday project, planning a new bus timetable, so the names feel familiar:

| CRISP-DM phase | Bus-timetable equivalent | Example value |
|---|---|---|
| Business understanding | Agree the goal and the peak-hour scenario | "90% of buses within 5 minutes of schedule at rush hour" |
| Data understanding | Read ticket logs to learn real passenger flow | Passengers peak at 8 am |
| Data preparation | Clean the ticket files, remove duplicates | One row per passenger trip |
| Modelling | Build the timetable and demand model | Buses every 10 minutes, 60/40 commuter/leisure |
| Evaluation | Compare results to the goal and last year's timetable | 92% on time vs 90% target |
| Deployment | Publish the timetable, set up daily monitoring | Daily on-time report with alerts |

**Feasibility check.** Before building, ask three questions:

1. Is there enough data and enough signal? (Do we have at least a few hundred positive examples? Do simple plots show a pattern?)

2. Is there an action that can be taken on the prediction? (A churn score without a retention offer is a nice number.)
3. Is the cost of a wrong prediction acceptable?

**The one-page project brief.** Write this before coding. Share it with the stakeholder and get a "yes, this is what we want".

```
Title:              Reduce 60-day churn among active customers
Problem:            Active customers who stop ordering cost ~Rs 4 cr/year
Users:              Retention team (acts), CRM manager (owns budget)
Decision:           Who receives a Rs 100 voucher this week?
Data:               orders, customers tables, snapshot on prediction date
Prediction time:    Every Monday. Label: no order in next 60 days
Metric (business): Retained revenue per Rs of voucher spend
Metric (model):     Precision@top-20%, AUC
Guardrails:         Discount cost, % of customers contacted, complaints
Baseline:           Rule: contact if no order in last 30 days
Risks:              Selection bias in labels, offers train people to wait
Timeline:           1 week baseline, 2 weeks model, 1 week A/B test
Use of result:      Weekly CSV to CRM; monitored monthly
```

**Worked problems**

**Problem 1.2.1** A manager says: "Reduce churn." Turn it into a precise problem using the brief fields.

| | |
|---|---|
| **Step 1** | Decide who: active customers (ordered in the last 90 days). |
| **Step 2** | Decide the prediction time and window: every Monday; label = no order in the next 60 days. |
| **Step 3** | Decide the action: a voucher, with a budget cap (say 20% of customers). |
| **Step 4** | Decide the metric: retained revenue per Rs of voucher. |
| **Answer** | "Every Monday, rank active customers by probability of no order in the next 60 days, send vouchers to the top 20%, and measure retained revenue per rupee of voucher against a no-voucher holdout." |
| **Check** | Read it aloud: can the retention team do it on Monday morning, and can a finance person measure it? Yes to both, so the ask is precise. |
| **Why it matters** | The vague ask "reduce churn" could be 10 different projects; the precise one is a single project with a measurable result. |

**Problem 1.2.2** In Data understanding you discover that 'order_ts' is stored in UTC but finance reports are in IST. A day's revenue is off by up to 5.5 hours of orders. Which CRISP-DM phase should you return to, and why?

| | |
|---|---|
| **Step 1** | The metric ("daily revenue") is defined in the business view in IST. |
| **Step 2** | If you do not fix it, your label windows and reports will not match finance numbers, and you lose trust. |
| **Answer** | Return to Data preparation to convert time zones, and note the rule in the data dictionary. If the label itself changes, go back to Business understanding. |
| **Check** | Convert one known order (UTC 2025-01-01 20:00) to IST: it becomes 2025-01-02 01:30. So the order moves to the next day; this is exactly the kind of mismatch that appears in daily totals. |
| **Why it matters** | Small definition mismatches cause big arguments ("your number is different from finance"). Fixing them early saves trust. |

**Connection to the real job.** Teams often use two-week sprints with the lifecycle compressed into "baseline, improve, ship, measure". You will write briefs, hold a kickoff with the stakeholder and present at the end. The brief protects you from scope creep.

**Applications**

- Every company that runs ML or analytics projects uses some form of this process.
- Consulting projects: the brief is often the contract.
- Regulated industries (banking, healthcare): documentation per phase is required.

| | |
|---|---|
| **COMMON MISTAKE** | Treating the phases as a straight line and being surprised when the "final" model fails. Plan for loops. Also: starting Modelling before you can state the baseline and the decision. |

**Check yourself**

1. List the six CRISP-DM phases.

Answer: business understanding, data understanding, data preparation, modelling, evaluation, deployment.

1. Name one feasibility question.

Answer: is there an actionable decision, enough signal, or acceptable cost of errors.

1. What goes in a brief under "Guardrails"?

Answer: metrics that must not get worse, such as discount cost or complaints.

# 1.3 Problem Framing Patterns

**INTUITION**

The same customer complaint can be a plumbing problem or an electrical problem. The word "leak" does not tell you which. In data science, "reduce churn" does not tell you if you need a classifier, a survival model, or an uplift model. Framing is choosing the right toolbox.

**Common framings.**

| Business wording | Framing | Typical target |
|---|---|---|
| Who will leave in 30 days? | Binary classification | 0 / 1 |
| When will they leave? | Survival / time-to-event | Time (with censoring) |
| Who will respond to the offer? | Uplift (causal) | Effect of treatment |
| How much will we sell? | Regression / forecasting | A number or series |
| What to show first? | Ranking | Ordered list |
| Group similar customers | Clustering | None |
| Something looks odd | Anomaly detection | Score |
| Do more X, will Y rise? | Causal inference | Effect of X on Y |
| Best plan under budget | Optimisation | Decision variables |

**Prediction, explanation, causation.** These are three different goals:

- **Prediction**: "Who will churn?" Needs accuracy. Any useful feature works, even one that is only a symptom.
- **Explanation**: "Why do customers churn?" Needs interpretable relationships.
- **Causation**: "What happens if we give a voucher?" Needs experiments or causal methods (Chapter 7 and 8). A high-risk customer is not necessarily a persuadable one.

**Prediction time and label window.** This is the single most useful discipline. Write two dates:

```
            past (features)          |        future (label)
-------------------------------------|-------------------------->
... Oct   Nov   Dec                  | T = 1 Jan    Feb    Mar
                                     |
you may use only data BEFORE T       |  label looks at T .. T+60 days
```

Every feature must be computable using data available before T. Every label must come from the window after T. This prevents leakage (using information from the future), the most common cause of "great offline score, bad real life".

**Unit and grain.** The unit of analysis is the thing you predict for (customer, session, order, device). The grain is what one row of the table means. "One row per customer per prediction date" is a grain. If you mix grains (some rows per order, some per customer), joins duplicate rows and averages mislead.

**Worked problems**

**Problem 1.3.1** Frame these three asks: (a) "Which loan applicants will default within 12 months?" (b) "How many riders will we need next Friday 6 to 9 pm?" (c) "Which users should we email to increase purchases?"

| | |
|---|---|
| **Step 1** | (a) default within a fixed window is binary classification; also note label delay of 12 months. |
| **Step 2** | (b) a number over time is forecasting, with an interval, to decide staffing. |
| **Step 3** | (c) "increase purchases by emailing" is about the effect of the email, so it is uplift modelling or an A/B test, not a plain purchase predictor. |
| **Answer** | (a) classification with delayed labels, (b) forecasting, (c) uplift / causal. |

**The shop dataset.** Our running example is a fake online shop with 6,000 customers and about 150,000 orders from January 2023 to April 2025. The full generator is the file `ds_common.py` below. Copy it and save it next to your notebooks; every later chapter starts with `from ds_common import *`. Customers have a plan (free or pro), a signup channel, a city and an age. Each customer orders at their own rate until they churn (stop for good). Higher churn risk is built in for the ads channel, the free plan and low order rate, so real patterns exist for us to discover.

```
import numpy as np
import pandas as pd

PRED_DATE = pd.Timestamp("2025-01-01")   # the day we "make the prediction"
END_DATE = pd.Timestamp("2025-04-30")    # last day of data we have

def make_data(n=6000, seed=42):
    """Synthetic subscription shop. Returns (customers, orders)."""
    rng = np.random.default_rng(seed)
    cid = np.arange(1, n + 1)
    signup = pd.Timestamp("2023-01-01") + pd.to_timedelta(rng.integers(0, 730, n),
unit="D")
    plan = rng.choice(["free", "pro"], n, p=[0.7, 0.3])
    channel = rng.choice(["organic", "ads", "referral"], n, p=[0.5, 0.35, 0.15])
    city = rng.choice(["Mumbai", "Delhi", "Bengaluru", "Pune", "Chennai", "Other"],
                      n, p=[0.2, 0.2, 0.2, 0.1, 0.1, 0.2])
    age = np.clip(rng.normal(32, 9, n), 18, 70).round()
    rate = rng.gamma(2.0, 0.04, n) * np.where(plan == "pro", 1.6, 1.0)  # orders per day
    # churn hazard per day (higher for ads channel, free plan, low order rate)
    hazard = 0.004 * np.exp(0.5 * (channel == "ads") - 0.7 * (plan == "pro")
                            - 0.3 * (channel == "referral") - 6.0 * (rate - 0.08))
    churn_day = signup + pd.to_timedelta(rng.exponential(1 / hazard).round(), unit="D")
    rows = []
    for i in range(n):
        stop = min(churn_day[i], END_DATE)
        days = (stop - signup[i]).days
        if days <= 0:
            continue
        k = rng.poisson(rate[i] * days)
        if k == 0:
            continue
        offs = np.sort(rng.integers(0, days + 1, k))
        amt = rng.lognormal(6.0 + 0.3 * (plan[i] == "pro"), 0.6, k).round(2)
        cat = rng.choice(["grocery", "fashion", "electronics", "home"], k, p=[0.5, 0.2,
0.1, 0.2])
        for o, a, c in zip(offs, amt, cat):
            rows.append((cid[i], signup[i] + pd.Timedelta(days=int(o)), a, c))
    orders = pd.DataFrame(rows, columns=["customer_id", "order_ts", "amount", "category"])
    orders.insert(0, "order_id", np.arange(1, len(orders) + 1))
    customers = pd.DataFrame({"customer_id": cid, "signup_date": signup, "plan": plan,
                              "channel": channel, "city": city, "age": age})
    return customers, orders

def make_label_table(customers, orders, pred_date=PRED_DATE, active_days=90, horizon=60):
    """One row per ACTIVE customer at pred_date; label = no order in next `horizon`
days."""
    past = orders[orders.order_ts < pred_date]
```

```
    last = past.groupby("customer_id").order_ts.max()
    active = last[last >= pred_date - pd.Timedelta(days=active_days)].index
    fut = orders[(orders.order_ts >= pred_date) &
                 (orders.order_ts < pred_date + pd.Timedelta(days=horizon))]
    buyers = set(fut.customer_id)
    df = customers[customers.customer_id.isin(active) & (customers.signup_date <
pred_date)].copy()
    df["churn"] = (~df.customer_id.isin(buyers)).astype(int)
    return df.reset_index(drop=True)
```

`make_label_table` is the whole prediction-time idea in 10 lines: it finds customers who ordered in the 90 days before T, then looks only at orders in the 60 days after T to build the label.

**Problem 1.3.2** Define prediction time, label and grain for 60-day churn in the shop dataset, then count the rows with code.

| | |
|---|---|
| **Step 1** | Prediction date T = 2025-01-01. Population: customers who signed up before T and ordered in the 90 days before T (the "active" ones). |
| **Step 2** | Label = 1 if no order in [T, T+60 days). Grain = one row per active customer. |
| **Step 3** | Before you run it: how many rows do you expect? (Hint: roughly 6,000 customers, but only some are active and signed up before T.) Then generate the data and build the table with `ds_common.py`. |

```
from ds_common import *
customers, orders = make_data()          # seed 42, 6000 customers
d = make_label_table(customers, orders)  # active customers at 2025-01-01
print(customers.shape, orders.shape)
print(d.shape, round(d.churn.mean(), 3))
```

```
(6000, 6) (151719, 5)
(2562, 7) 0.254
```

Line by line: `make_data()` returns two tables, 6,000 customers with 6 columns and 151,719 orders with 5 columns. `make_label_table` keeps only active customers and adds the `churn` column, so we get 2,562 rows and 7 columns. `d.churn.mean()` is the share of ones, which is the churn rate.

- **Answer** 2,562 active customers; 25.4% churned within 60 days. That 25.4% is the "base rate" every model must beat.
- **Check** 0.254 × 2,562 is about 651 churners, so about 1 in 4 active customers leaves.

- **Why it matters** It means a model that predicts nobody churns is right 74.6% of the time. That is the number any real model has to beat.

| KEY INSIGHT | Framing decides data, model and value. Spend the first hour of any project on framing. A change of label window from 60 to 30 days changes the whole dataset. |
|---|---|

**Connection to the real job.** Interviewers say "How would you predict churn?" A strong answer starts: "First, define churn, the prediction time, and what action we will take."

**Applications**

- Banking: default prediction (classification with delay), time-to-default (survival).
- Telecom: contract churn vs usage churn (different definitions).
- SaaS: activation, conversion, expansion (each its own label).
- Operations: "will this hospital ward run out of beds in the next hour?" is a classification with a prediction time.

| COMMON MISTAKE | Letting the label window overlap the feature window, for example computing "orders in last 30 days" including days after T. Always draw the timeline before writing SQL. |
|---|---|

**Check yourself**

1. Which framing handles "time until churn" with unfinished customers?

Answer: survival analysis (time-to-event with censoring).

1. What two dates define a supervised dataset?

Answer: the prediction time T, and the end of the label window.

1. What is "grain"?

Answer: what one row of a table represents.

## 1.4 Baselines, Success Metrics and the First Model

| INTUITION | Before a school judges a class, it keeps last year's results. An average mark of 62 is meaningless until you know last year's class averaged 71. In data science, a model score is meaningless until you know what a dumb method scores. |
|---|---|

**Three levels of metrics.**

| Level | Question | Example for churn |
|---|---|---|
| Business | Did we make money, save time, reduce risk? | Retained revenue per rupee spent |
| Model | Does the model rank or predict well? | AUC, precision at top 20%, calibration |
| Guardrail | What must not get worse? | Discount cost, complaint rate, customers contacted |

The business metric is the one that matters. The model metric is a proxy you can compute quickly offline. The guardrails stop you from "winning" by damaging something else.

**Baselines, from dumbest to smartest.**

1. *Constant*: predict the average (regression) or the majority class (classification).
2. *Last value / seasonal repeat*: for time series.
3. *Rule of thumb*: what the business does today, for example "no order in 30 days means churn risk".
4. *Simple model*: logistic regression or a small tree on three features.
5. *Complex model*: gradient boosting, neural net.

You only claim progress by comparing to the highest baseline you can build cheaply.

**Worked example (run with real output).** We build three features at T: *recency* (days since the last order), *orders_90d* (how many orders in the 90 days before T) and *is_pro* (1 if on the pro plan). Then we compare the baselines. We use `train_test_split`, which cuts the table into a training part (70%) and a test part (30%) so we score the model on rows it never saw. (Chapter 6 explains why a time-based split is better in real work; here we keep it simple.)

Before you run it, predict: will the logistic model beat the simple rule by a lot, a little, or not at all?

```
from ds_common import *
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, precision_score, recall_score

c, o = make_data()
d = make_label_table(c, o)
past = o[o.order_ts < PRED_DATE]                # only data before T
g = past.groupby("customer_id")
f = pd.DataFrame({
    "recency_days": (PRED_DATE - g.order_ts.max()).dt.days,
    "orders_90d": past[past.order_ts >= PRED_DATE - pd.Timedelta(days=90)]
                    .groupby("customer_id").size(),
})
d = d.merge(f, left_on="customer_id", right_index=True, how="left")
d["is_pro"] = (d.plan == "pro").astype(int)

tr, te = train_test_split(d, test_size=0.3, random_state=0, stratify=d.churn)
print("B0 accuracy (predict nobody churns):", round(1 - te.churn.mean(), 3))
rule = (te.recency_days > 30).astype(int)
print("B1 rule recency>30: precision", round(precision_score(te.churn, rule), 3),
      "recall", round(recall_score(te.churn, rule), 3),
      "AUC", round(roc_auc_score(te.churn, te.recency_days), 3))
X = ["recency_days", "orders_90d", "is_pro"]
m = LogisticRegression(max_iter=1000).fit(tr[X], tr.churn)
p = m.predict_proba(te[X])[:, 1]
print("M1 logistic AUC", round(roc_auc_score(te.churn, p), 3))
k = int(0.2 * len(te))
top = te.assign(p=p).sort_values("p", ascending=False).head(k)
print("top 20% size", k, "churn rate inside", round(top.churn.mean(), 3),
      "lift", round(top.churn.mean() / te.churn.mean(), 2))
top_r = te.sort_values("recency_days", ascending=False).head(k)
print("rule-based top 20% churn rate", round(top_r.churn.mean(), 3))
```

Real output:

```
B0 accuracy (predict nobody churns): 0.746
B1 rule recency>30: precision 0.759 recall 0.662 AUC 0.879
M1 logistic AUC 0.881
top 20% size 153 churn rate inside 0.778 lift 3.07
rule-based top 20% churn rate 0.778
```

Read this carefully. "Predict nobody churns" gets 74.6% accuracy while being useless: accuracy is a bad metric for imbalanced problems. The business rule already reaches AUC 0.879. The logistic model reaches 0.881, a

tiny gain. In the top 20% (153 customers) both find a churn rate of 77.8%, which is 3.07 times the base rate of 25.4%.

Line by line: `past` keeps only orders before T, which enforces the rule "no future data". `g.order_ts.max()` is each customer's last order date, so `PRED_DATE - ...` gives recency in days. `rule` is 1 when recency is above 30 days. `precision_score` asks: of those we flagged, how many really churned? `recall_score` asks: of those who churned, how many did we flag? `roc_auc_score` is the chance that a random churner gets a higher score than a random non-churner (0.5 is a coin flip, 1.0 is perfect). `lift` is the churn rate inside the chosen group divided by the overall churn rate.

Why is the rule so strong? In our data, customers who stopped ordering already churned, and recency shows that. The model adds only a little because recency carries most of the signal. This is common on real data and is the best argument for always building the rule first.

**Worked problems**

**Problem 1.4.1** Use the numbers above. The retention team can voucher 153 customers in the test set. Each churner saved is worth Rs 600 in revenue, and each voucher costs Rs 100. Assume (optimistically) that a voucher saves 30% of the churners who receive it. What is the expected net value of targeting the model's top 20%, versus 153 random customers?

| | |
|---|---|
| **Step 1** | Model top 20%: churners = 153 × 0.778 = 119.0 (rounded to 119). |
| **Step 2** | Saved = 119 × 0.30 = 35.7 customers. Revenue = 35.7 × 600 = Rs 21,420. Cost = 153 × 100 = Rs 15,300. Net = Rs 6,120. |
| **Step 3** | Random 153: churners = 153 × 0.254 = 38.9. Saved = 11.7. Revenue = 11.7 × 600 = Rs 7,020. Cost = Rs 15,300. Net = Rs -8,280. |
| **Answer** | Targeting the top 20% gives about +Rs 6,120, random gives about -Rs 8,280. Targeting is worth about Rs 14,400 per campaign of this size. Note the simple rule does as well as the model here, so the model's value over the rule is near zero on this metric. |
| **Check** | 6,120 - (-8,280) = 14,400, and 153 × (0.778 - 0.254) × 0.30 × 600 = 14,431, which agrees up to rounding. |
| **Why it matters** | In plain words: picking the right 153 people turns a money-losing campaign into a profitable one. That difference, not the AUC, is what you show the CRM manager. |

| | |
|---|---|
| **KEY INSIGHT** | The 30% save rate was an assumption. A churn model finds who is *at risk*, not who is *persuadable*. Chapter 8 (uplift) shows how to fix that. |

**Problem 1.4.2** A teammate reports "our model has 92% accuracy" on a fraud problem where 1% of transactions are fraud. Is it good?

| | |
|---|---|
| **Step 1** | A model that says "never fraud" gets 99% accuracy. |
| **Step 2** | So 92% accuracy is worse than doing nothing. |
| **Answer** | It is not good. Report precision, recall, and PR-AUC, and compare to a baseline. |
| **Check** | 0.99 > 0.92, so the do-nothing model wins on accuracy. |
| **Why it matters** | If you do not check the baseline, you can proudly present a model that is worse than doing nothing. |

**The picture: baseline ladder**

```
AUC
0.50  random / constant
0.879 rule: recency > 30 days            <- strong, cheap
0.881 logistic on 3 features             <- +0.002, is it worth it?
 ?    gradient boosting with 60 features <- must beat 0.881 meaningfully
```

**Connection to the real job.** In an interview or on the job you will often hear "the model improved AUC by 0.01". Ask: over what baseline, on what data, and what is that worth in rupees? A simple model with 90% of the value that ships next week often beats a complex one that ships in three months.

**Applications**

- Forecasting: seasonal-naive baseline ("same as last week").
- Fraud: existing rule engine is the baseline.
- Credit: the existing scorecard is the baseline.
- Support routing: current keyword rules.

| | |
|---|---|
| **COMMON MISTAKE** | Comparing your model to nothing, or to a weak baseline you built to look good. Always include the business's current rule. Another trap: reporting accuracy on imbalanced data. |

**Check yourself**

1. Name the three metric levels.

Answer: business, model, guardrail.

1. Why is "predict nobody churns" a useful baseline?

Answer: it shows how high accuracy is without any skill, so you know accuracy is meaningless.

1. A model beats a rule by 0.002 AUC. What do you do?

Answer: check if the gain has business value; maybe ship the rule or keep improving features.

## Chapter recap

- The job is mostly framing, data and communication; modelling is a smaller part.
- Value = quality of decision multiplied by adoption. A model nobody uses is worth nothing.
- Role titles vary; read job descriptions and aim at one target (decision science, applied science or platform).
- CRISP-DM has six phases and real projects loop through them many times.
- Always write a one-page brief before coding: problem, decision, data, metric, baseline, risks.
- Frame the same business question as classification, survival, uplift, forecasting or optimisation, depending on what decision is needed.
- Write down the prediction time, label window and grain. This prevents leakage.
- Use three metric levels: business, model, guardrail.
- Build baselines from dumb to smart, and report improvement over the best cheap one. In our example the rule almost equals the logistic model.
- Accuracy on imbalanced data is misleading; use rankings, precision, recall and money.

| Idea | Formula or pattern | Example from this chapter |
|---|---|---|
| Improvement % | `(old − new) / old × 100` | 480 ms to 360 ms is 25% |
| Base rate | `positives / rows` | 651 / 2,562 = 0.254 |
| Lift | `rate in group / base rate` | 0.778 / 0.254 = 3.07 |
| Precision | `true flagged / all flagged` | 0.759 for the 30-day rule |
| Recall | `true flagged / all true` | 0.662 for the 30-day rule |
| Expected value per target | `rate × save rate × value − cost` | 0.778 × 0.30 × 600 - 100 = 40 |
| Timeline rule | features before T, label after T | T = 2025-01-01, window 60 days |

# Practice set

**Problem 1.P.1** (easy) A project lasts 15 weeks. Using 60/20/20, how many weeks go to modelling? **Answer:** 20% of 15 = 3 weeks.

**Problem 1.P.2** (easy) Which is "prediction", "explanation" or "causation": "If we cut price by 5%, how will sales change?" **Answer:** Causation (effect of an action).

**Problem 1.P.3** (easy) A table has one row per order. You join to a customer table and compute "average per row" for customers. Why might the result mislead? **Answer:** Grain mismatch: customers with many orders get more weight in averages across rows. Aggregate to the customer grain first.

**Problem 1.P.4** (medium) The base churn rate is 25.4%. Find the lift if the model's top 100 customers (the top 10% of 1,000) contain 70 churners. **Answer:** Churn rate in the top group = 70/100 = 0.70. Lift = 0.70 / 0.254 = 2.76.

**Problem 1.P.5** (medium) What is the output? `1 - te.churn.mean()` where churn mean is 0.254. **Answer:** 0.746, the accuracy of "never churn".

**Problem 1.P.6** (medium) Find the bug: features include `orders_next_30d` for churn prediction at T. **Answer:** It looks into the future (after T), which is leakage. Remove it. Use only data before T.

**Problem 1.P.7** (medium) Fraud rate is 0.5%. A model flags 2% of transactions and catches 60% of all fraud. What is its precision? **Answer:** Per 10,000 transactions: fraud = 50, flagged = 200, caught = 30. Precision = 30/200 = 15%.

**Problem 1.P.8** (hard) You have budget for 200 vouchers, each costs Rs 100, a saved customer is worth Rs 600, and the save rate is 25% of churners targeted. At what churn rate inside the targeted group does net value become zero? **Answer:** Expected value per targeted customer = churn rate × 0.25 × 600 - 100 = 150 × churn rate - 100. Zero at churn rate 100/150 = 0.667. Below 66.7% the campaign loses money.

**Problem 1.P.9** (interview) "A stakeholder asks you to build a churn model. What do you do in the first week?" **Answer:** Meet the stakeholder; define churn, prediction time, label window, action and budget; check data

availability; compute base rate and a rule baseline; write the one-page brief; agree the metric and guardrails; plan an A/B test for the action.

**Problem 1.P.10** (interview) "Your new model has AUC 0.82; the old rule has 0.80. Do you ship?" **Answer:** Not automatically. Check the business metric at the operating threshold, cost of complexity, stability over time and calibration. If the rupee gain is small, ship the rule or iterate on features. Run an A/B test if the stakes justify it.

# Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, and English is my second language, so use simple words. Teach me "the data science project lifecycle (CRISP-DM)" step by step, using a retail churn example. After each phase, ask me one question and wait for my answer before continuing. Correct my mistakes kindly. Finish by asking me to write a one-page project brief for a problem I choose, and review it.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science and I study one hour a day. Topic: "problem framing in data science: classification vs survival vs uplift vs forecasting, and prediction time and label windows". Give me 8 graded exercises, from easy to hard, each starting from a vague business ask. Do not give solutions yet. After I answer, I will paste my answers and you will grade them and explain the gaps.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional new to data science. Topic: "baselines and success metrics (business, model, guardrail)". Quiz me one question at a time, mixing numerical questions (lift, precision, expected value of a campaign) and conceptual ones (why accuracy fails on imbalanced data). Wait for my answer, grade it, explain, then go to the next. Ask 10 questions.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Review my project brief for a data science project. Check the decision, prediction time, label window, grain, baseline, three levels of metrics, risks and how the
```

result will be used. Point out vague parts and leakage risks, and suggest concrete rewrites in simple English. Here is my brief: [PASTE HERE]

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional preparing for data science interviews in India. Run a mock interview on "what a data scientist does, lifecycle, framing and baselines". Ask me one scenario question at a time, such as "How would you predict churn?" or "Your AUC is great but nobody uses the model". After each answer, give a score out of 10, a better sample answer, and one follow-up question. Do 8 rounds.

CHAPTER 2

# GETTING TRUSTWORTHY DATA: COLLECTION, POINT-IN-TIME SETS, QUALITY, LABELS AND PRIVACY

*Most model failures start here, long before any algorithm runs. You will learn to build a training table that matches real life, to try to break it, and to treat people's data with care.*

Think about a hospital that studies waiting times from its appointment log, but the log misses every patient who gave up and left without being seen. The results look great and the real waiting room is overflowing. Data science has the same trap. If the training data is built wrongly, the best model still fails in real use. In this chapter you will learn how to find the right data sources, how to build a training table so that every feature uses only information that existed at the prediction time (called *point-in-time correctness*), how to run quality checks that catch broken data, how to think about labels that are late, noisy or biased, and how to respect privacy law and ethics. You will leave with SQL and pandas code you can reuse on any project.

We use the shop dataset from Chapter 1 ( `from ds_common import *` ).

## 2.1 Data Sources and Point-in-Time Training Sets

**INTUITION**

Imagine a teacher who prints an exam and, by mistake, includes the answer key on the back. Students who see the back score 100%, but they learned nothing. A training table that contains information from after the prediction time is the same: the model "sees the answer key". It will look brilliant in the notebook and fail in production, where the key does not exist.

**Words you need**

| Word | Plain meaning | Example |
|---|---|---|
| Source system | Where a record is first created | The orders database of the shop |
| Event log (clickstream) | An append-only list of "something happened" rows | One row per page view |
| Warehouse | A central database copy built for analysis | Tables like `orders` , `customers` |

| Word | Plain meaning | Example |
|---|---|---|
| Prediction time (T) | The moment the prediction is made | 2025-01-01 |
| Point-in-time correct | Each feature uses only data from before T | `orders_90d` counts orders before T only |
| Leakage | Information from after T (or the label itself) entering a feature | `days_to_last` uses all orders |

**Data sources and how they are generated.** Always ask "what event creates a row?" and "what is missing by design?".

| Source | What creates a row | Missing by design | Where it appears in the job |
|---|---|---|---|
| Operational DB (orders) | A completed purchase | Abandoned carts, failed payments | Revenue, churn labels |
| Event log / clickstream | A click or screen view | Users who blocked tracking | Behaviour features |
| Warehouse tables | A nightly copy of other systems | Edits made after the copy | Reporting and training sets |
| Third-party / APIs | A vendor call | Anyone the vendor does not cover | Credit bureau scores, weather |
| Surveys | A person who answered | Non-responders (often unhappy users) | NPS, satisfaction |
| Sensors / monitoring | A measurement at fixed intervals | Gaps when the device is offline | Latency, IoT |
| Manual labels | A human decision | Cases nobody reviewed | Fraud confirmed by analysts |

Notice the "missing by design" column. It is like a shop door counter that never records the customers who turned away at a locked door. The data cannot show what it was never able to record.

**Point-in-time training set.** Remember the timeline from Chapter 1. Features come from before T and the label from after T. Here is the SQL version (SQLite syntax; other databases differ in date functions). We load the shop tables into an in-memory SQLite database and build the table for T = 2025-01-01.

Before you run it, ask yourself: how many rows should this return? It should equal the number of active customers from Chapter 1.

```
import sqlite3
from ds_common import *

c, o = make_data()
con = sqlite3.connect(":memory:")
c.assign(signup_date=c.signup_date.dt.strftime("%Y-%m-%d")).to_sql("customers", con,
index=False)
o.assign(order_ts=o.order_ts.dt.strftime("%Y-%m-%d")).to_sql("orders", con, index=False)

sql = """
WITH base AS (
  SELECT c.customer_id, c.plan, c.channel,
         CAST(julianday(:T) - julianday(c.signup_date) AS INT) AS tenure_days
  FROM customers c WHERE c.signup_date < :T
),
past AS (
  SELECT customer_id,
         COUNT(*) AS orders_90d,
         SUM(amount) AS spend_90d,
         CAST(julianday(:T) - julianday(MAX(order_ts)) AS INT) AS recency_days
  FROM orders
  WHERE order_ts <  :T
    AND order_ts >= date(:T, '-90 day')
  GROUP BY customer_id
),
fut AS (
  SELECT DISTINCT customer_id FROM orders
  WHERE order_ts >= :T AND order_ts < date(:T, '+60 day')
)
SELECT b.*, p.orders_90d, p.spend_90d, p.recency_days,
       CASE WHEN f.customer_id IS NULL THEN 1 ELSE 0 END AS churn
FROM base b
JOIN past p USING (customer_id)          -- inner join = active customers only
LEFT JOIN fut f USING (customer_id)
"""
d = pd.read_sql(sql, con, params={"T": "2025-01-01"})
print(d.shape, round(d.churn.mean(), 3))
print(d.head(3).round(1).to_string())
```

```
(2562, 8) 0.254
   customer_id  plan   channel  tenure_days  orders_90d  spend_90d  recency_days  churn
0            1   pro  referral          666           3     2403.8             8      0
1            3  free   organic          254           8     3828.1            37      1
2            6   pro       ads          105           5     4034.3            34      0
```

Line by line. The `base` block lists customers who existed before T and computes tenure up to T. The `past` block has two guards, `order_ts < :T` and `order_ts >= date(:T, '-90 day')`: they say "only before T, and only the last 90 days". The `fut` block is the label window and is used only to create `churn`. `JOIN past` is an inner join, so customers with no order in the last 90 days drop out: this defines who is "active". `LEFT JOIN fut` keeps everyone and gives NULL when there was no future order, which we turn into `churn = 1`. We get 2,562 rows and a 25.4% churn rate, the same as the pandas helper (the code also checked that both methods agree: rows match and churn counts match).

**Worked problems**

**Problem 2.1.1** A feature is `orders_all` = the number of orders a customer ever made (all dates in the table), used to predict churn at T = 2025-01-01. A customer who churned on 2025-01-20 has 14 orders in total, 10 before T. A customer who kept ordering has 40 orders in total, 12 before T. Which numbers should the model see, and why?

| | |
|---|---|
| **Formula** | feature value must be computed from rows with `order_ts < T`. |
| **Plug in** | Customer A: 10 orders before T. Customer B: 12 orders before T. |
| **Compute** | The leaky feature gives 14 vs 40, a gap of 26. The honest feature gives 10 vs 12, a gap of 2. |
| **Answer** | The model must see 10 and 12. The leaky values 14 and 40 include orders after T, so the churner looks "low" only because they stopped later. |
| **Check** | On January 1st nobody knows how many orders a customer will make later; if a feature value cannot be known on that date, it is leakage. |
| **Why it matters** | It means the model learns "customers who will order more in the future will order more in the future", which is true and useless. |

**Problem 2.1.2** Measure the damage. Compare the AUC of a logistic regression on honest features ( `orders_90d` , `recency_days` , `spend_90d` ) with a version that adds `days_to_last` = (date of the customer's last order in the entire table) minus T.

| | |
|---|---|
| **Step 1** | Compute `days_to_last` from all orders, including those after T. |
| **Step 2** | Fit the same model on both feature sets, same 70/30 split, same seed. |
| **Step 3** | Compare. |

```
leaky = pd.read_sql("SELECT customer_id, MAX(order_ts) AS last_order FROM orders GROUP BY
customer_id", con)
leaky["days_to_last"] = (pd.to_datetime(leaky.last_order) - pd.Timestamp("2025-01-
01")).dt.days
dd = d.merge(leaky, on="customer_id")
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

def auc_for(cols, data):
    tr, te = train_test_split(data, test_size=0.3, random_state=0, stratify=data.churn)
    m = LogisticRegression(max_iter=2000).fit(tr[cols], tr.churn)
    return round(roc_auc_score(te.churn, m.predict_proba(te[cols])[:, 1]), 3)

print("honest AUC :", auc_for(["orders_90d", "recency_days", "spend_90d"], dd))
print("leaky  AUC :", auc_for(["orders_90d", "recency_days", "spend_90d", "days_to_last"],
dd))
```

```
honest AUC : 0.882
leaky  AUC : 0.972
```

- **Answer** Adding one future-looking column raises AUC from 0.882 to 0.972, a jump of 0.090.
- **Check** `days_to_last` is positive only for customers who ordered after T, which is exactly the opposite of the label `churn` . A jump this big from one column is a smell.
- **Why it matters** In plain words: a 0.97 AUC looks like a triumph in the notebook. In production, where the future is unknown, you would get about 0.88. If you had promised 0.97 to a manager, you just lost credibility.

**The picture**

```
time ------------------------------------------------------------------>
        |---- feature window (90 days) ----|T|---- label window (60 days) ----|
        orders_90d, recency, spend          ^   churn = no order here
                                            |
       LEAK: any column that looks to the right of T
```

**Sampling.** You rarely train on all data. Three common ways:

| Method | How | Use when | Danger |
|---|---|---|---|
| Random | Pick rows with equal chance | Rows are independent | Same customer in train and test |
| Stratified | Keep class shares equal in each part | Rare positives | Hides distribution shift |
| Time-based | Train on the past, test on the future | Anything with time (almost everything) | Fewer rows for training |

A sample misrepresents the population when selection depends on the outcome, for example "only customers who answered our survey".

**Join pitfalls.** Four traps:

- *Fan-out*: joining a customer table to an address table with two addresses per customer duplicates the customer row. In our toy test, 2 customers became 3 rows after the join (`2 customers -> 3 rows`), while distinct customers stayed 2. Then `SUM` or `COUNT` over customers double counts.
- *Many-to-many*: joining two tables on a key that is not unique in either side multiplies rows (3 × 4 = 12).
- *Inner vs left*: an inner join silently removes rows (it changed the population to "active customers" above, on purpose). Always print row counts before and after every join.
- *Time-misaligned joins*: joining today's customer profile (e.g., "current plan") to old rows leaks the future. Use the profile as it was at T (slowly changing dimension or snapshot table).

**Documenting.** Keep a data dictionary: column name, meaning, unit, time zone, how it is created, known issues, owner. Define metrics once ("active customer = at least 1 order in 90 days") in a source-of-truth table or view.

**Connection to the real job.** Building this table is usually the longest step of a project. In interviews, "write SQL to build a churn training set" is a standard question; the examiner looks for the `< T` guards.

**Applications**

- Data Science: churn, fraud and credit training tables.
- Data Engineering: feature tables with "as-of" joins and snapshots.

- MLOps: feature stores exist mainly to guarantee point-in-time correctness.
- Analytics: cohort reports where each user is measured from their signup date.

| COMMON MISTAKE | Using a "current state" column (current plan, current city, lifetime orders) in a historical training table. It looks harmless, but it carries future information. Ask for every column: "what would this value have been on date T?". |
|---|---|

**Check yourself**

1. What does `order_ts < :T` protect against?

Answer: using information from after the prediction time (leakage).

1. A join changes 1,000 rows into 1,400. What likely happened?

Answer: fan-out from a non-unique key on the right table.

1. Why prefer time-based splits?

Answer: they copy real use, where the model predicts the future from the past.

## 2.2 Data Quality Checks Before Any Modelling

| INTUITION | Before a bank audits its accounts it checks that the file is not empty, that account numbers are unique and that it is the right month. You do not trust it. A dataset deserves the same suspicion. The rule is: *never trust a dataset until you have tried to break it.* |
|---|---|

**Six dimensions of quality.**

| Dimension | Question | Example check | Example in our shop |
|---|---|---|---|
| Completeness | Are values present, and why not? | Null rate by column and by segment | `category` is null in 0.1% |
| Validity | Are values possible? | Ranges, formats, units, time zones | `amount <= 0` |
| Uniqueness | Are keys unique? | Duplicate `order_id` | 300 duplicates |
| Consistency | Do two computations agree? | Sum of orders = finance total | difference 0.17% |

| Dimension | Question | Example check | Example in our shop |
|---|---|---|---|
| Timeliness | Is the data final? | Complete counts for yesterday | 59% complete same day |
| Stability | Did the distribution change silently? | Monthly mean amount | around 570-582 |

**Null reasons.** A null can mean *not applicable* (a free plan has no discount code), *not recorded* (a sensor was off), or *error* (a bug dropped the field). They need different fixes, so always ask why.

We now damage a copy of the orders on purpose, so you can see what the checks catch. Predict first: which of the checks below will be non-zero for the damaged table?

```
bad = o.copy()
dup = bad.sample(300, random_state=1)
bad = pd.concat([bad, dup], ignore_index=True)                    # 300 duplicate rows
bad.loc[bad.sample(40, random_state=2).index, "amount"] = -99.0   # impossible values
bad.loc[bad.sample(25, random_state=3).index, "order_ts"] = pd.Timestamp("2030-01-01")
bad.loc[bad.sample(150, random_state=4).index, "category"] = None

def quality_report(df, name="orders"):
    r = {}
    r["rows"] = len(df)
    r["dup_order_id"] = int(df.order_id.duplicated().sum())
    r["null_category_pct"] = round(df.category.isna().mean() * 100, 2)
    r["amount<=0"] = int((df.amount <= 0).sum())
    r["order_ts_in_future"] = int((df.order_ts > END_DATE).sum())
    r["orphan_customer_ids"] = int((~df.customer_id.isin(c.customer_id)).sum())
    return pd.Series(r, name=name)
print(pd.concat([quality_report(o, "clean"), quality_report(bad, "damaged")],
axis=1).to_string())
```

```
                      clean   damaged
rows                 151719.0  152019.0
dup_order_id              0.0     300.0
null_category_pct         0.0       0.1
amount<=0                 0.0      40.0
order_ts_in_future        0.0      25.0
orphan_customer_ids       0.0       0.0
```

Line by line: `duplicated().sum()` counts rows whose key already appeared above. `isna().mean()` is the share of nulls (True counts as 1). `df.order_ts > END_DATE` finds dates later than the last day we could

possibly have data. `isin` checks that every order points to a real customer (an "orphan" has no parent). Four checks fire, one stays zero, which is also information: the foreign keys are intact.

**Worked problems**

**Problem 2.2.1** Reconcile with finance. The finance team says total order value is Rs 86,580,170. Your damaged table sums to Rs 86,723,834. How big is the gap in percent, and which defects could explain it?

| | |
|---|---|
| **Formula** | gap % = (yours - finance) / finance × 100 |
| **Plug in** | (86,723,834 - 86,580,170) / 86,580,170 × 100 |
| **Compute** | 143,664 / 86,580,170 = 0.001659, so 0.17% |
| **Answer** | The damaged table is 0.17% too high. The 300 duplicated orders add money (about 300 × Rs 570 = Rs 171,000) and the 40 negative amounts subtract 40 × (570 + 99), roughly Rs 27,000; the net is close to +Rs 144,000. |
| **Check** | 171,000 - 27,000 = 144,000, near the 143,664 observed. |
| **Why it matters** | A 0.17% gap looks small, yet it proves the pipeline is wrong. Reconciling to a trusted number is the cheapest test you have. |

**Problem 2.2.2** Timeliness. We simulate that each order arrives in the warehouse 0 to 3 days after it happened. On 2025-04-20, how complete is the data for each of the last four days?

```
2025-04-20 complete on 04-20: 125 / 211 59.2 %
2025-04-19 complete on 04-20: 191 / 223 85.7 %
2025-04-18 complete on 04-20: 220 / 237 92.8 %
2025-04-17 complete on 04-20: 229 / 229 100.0 %
```

- **Step 1** For each day divide the rows already arrived by the final count.
- **Step 2** Compute: 125/211 = 0.592, 191/223 = 0.857, 220/237 = 0.928, 229/229 = 1.
- **Answer** "Today" is only 59% complete, yesterday 86%. Data older than 3 days is final.
- **Check** The delays we simulated were at most 3 days, so day 4 back must be 100%, and it is.
- **Why it matters** If your churn label uses "no order in the last 7 days" and you build it on yesterday's partial data, you will label many active customers as churners. Use a *finalisation lag*: only use data older than the lag.

**Stability over time.** Compare simple summaries by month (mean amount, null rate, row count). Our mean order amounts for Jan to Apr 2025 are 580.6, 568.1, 576.0, 582.0: stable. A sudden jump to 1,160 in one month would suggest a currency or unit change. This is your "silent pipeline change" detector.

**Automating.** Write checks as code, run them every day: `pandera` (schema checks on DataFrames), `great_expectations` (declarative expectations), or plain SQL tests ( `SELECT COUNT(*) ... HAVING COUNT(*) > 1` ). These libraries were not run here; the idea is the same as our `quality_report` function.

**The picture**

```
raw data --> [schema check] --> [row-count check] --> [range check]
                  |                    |                   |
              fail: stop           fail: alert         fail: quarantine rows
--> [reconcile with finance] --> [distribution vs last month] --> training table
```

**Connection to the real job.** Data engineers own the pipelines; you own the sanity of what you use. A good habit is a 10-line `quality_report` at the top of every notebook.

**Applications**

- Data Engineering: data contracts and freshness SLAs.
- Data Science: checks before training, plus drift checks after deployment.
- Analytics: reconciling dashboards with finance.
- Operations: validating that sensor gaps are not mistaken for "zero faults".

| COMMON MISTAKE | Dropping duplicates blindly. Sometimes a repeated row is real (two identical purchases). Check the key: if `order_id` repeats, it is a duplicate; if only the other columns repeat, it may be real. |
|---|---|

**Check yourself**

1. A table's revenue is 0.5% above finance. What do you do?

Answer: investigate duplicates, test rows, currency and time zone issues until it reconciles.

1. What is a finalisation lag?

Answer: waiting some days before using data because late records are still arriving.

1. Null means what?

Answer: it depends: not applicable, not recorded, or error; find out which.

# 2.3 Labels and Ground Truth

| INTUITION | A label is a witness statement about what really happened. Some witnesses are late (the 12-month default arrives a year later), some disagree (two doctors read the same scan differently), and some only saw part of the street (we only see what happens to the customers we approved). Label quality caps model quality: a student cannot do better than a textbook with wrong answers. |
|---|---|

**Words you need**

| Word | Meaning | Example |
|---|---|---|
| Proxy label | A measurable stand-in for the real goal | "Clicked" for "liked" |
| Label noise | Some labels are simply wrong | A fraud case marked "genuine" by mistake |
| Label delay / censoring | The outcome is not yet known | Loan from last month: default within 12 months unknown |
| Selection bias in labels | Outcomes are seen only for a chosen subset | Defaults seen only for approved loans |
| Inter-annotator agreement | How much human labellers agree | Cohen's kappa |

**Cohen's kappa.** Two people label 100 items. They agree on 85, but part of that is luck. Kappa removes the luck: `kappa = (po − pe) / (1 − pe)`, where `po` is the observed agreement and `pe` is the agreement expected by chance from each person's label rates. 0 means no better than chance, 1 is perfect; values above about 0.6 are usually considered good, but the threshold depends on the problem.

**Worked problems**

**Problem 2.3.1** Annotator A and B label 100 tickets as "urgent" (1) or not (0). Both say urgent for 40. A says urgent and B not urgent for 10. A says not urgent and B urgent for 5. Both say not urgent for 45. Compute kappa.

| | |
|---|---|
| **Formula** | `kappa = (po − pe) / (1 − pe)` |
| **Step 1** | `po` = (40 + 45) / 100 = 0.85. |
| **Step 2** | A says urgent for 50 of 100, so 0.50. B says urgent for 45 of 100, so 0.45. |
| **Step 3** | `pe` = 0.50 × 0.45 + 0.50 × 0.55 = 0.225 + 0.275 = 0.50. |
| **Compute** | kappa = (0.85 - 0.50) / (1 - 0.50) = 0.35 / 0.50 = 0.70. |
| **Answer** | Kappa = 0.70. `sklearn.metrics.cohen_kappa_score` gives the same 0.7 (verified by code). |
| **Check** | If they agreed on everything, `po` = 1 and kappa = 1. If `po` = `pe`, kappa = 0. Ours is between them. |
| **Why it matters** | In plain words: even with 85% raw agreement, only 70% of the "above-chance" room is agreed. If kappa is low, your labelling guidelines are unclear and no model can learn a stable rule. |

**Problem 2.3.2** Label delay. A bank wants to predict default within 12 months and has loans from Jan 2024 to Jun 2025. Today is end of June 2025. Which loan months have mature labels?

```
  month  months_observed  label_mature
2024-11                7         False
2024-12                6         False
2025-01                5         False
...
2025-06                0         False
```

- **Step 1** A label is mature if the loan has been observed for at least 12 months.
- **Step 2** For Jan 2024 loans we have 17 months, so mature; for Jun 2024, 12 months, mature; for Jul 2024, 11 months, not.
- **Answer** Only loans from Jun 2024 and earlier have final labels. Training on the later ones treats "not yet defaulted" as "good", which makes the model think risk is lower than it is.
- **Check** June 2024 to June 2025 is exactly 12 months, so it is the last mature month.
- **Why it matters** It means your newest data is the least trustworthy. You either wait, use a shorter proxy (default within 3 months) or treat the recent rows as censored with survival methods (Chapter 11).

**Problem 2.3.3** Selective labels. A bank approves only applicants with income score above -0.2 (in standard-deviation units). We simulate 20,000 applicants. What default rate does the training data show, versus the truth?

```
true default rate, everyone      : 0.1885
default rate among approved only : 0.0652 (what the training data shows)
approval share: 0.58
```

- **Step 1** The model can only learn from approved applicants (58% of all).
- **Step 2** Their default rate is 6.52%, but the whole population's rate is 18.85%.
- **Answer** The training data understates risk by a factor of about 2.9 (18.85 / 6.52).
- **Check** 0.1885 / 0.0652 = 2.89.
- **Why it matters** A model trained only on approved applicants never sees the risky region, so it is overconfident there. **Reject inference** tries to guess outcomes for rejected applicants (for example, by reweighting or by small controlled approvals); it is covered again in Chapter 11.

**Human labelling and weak labels.** For human labels, write clear guidelines with examples, double-label a sample and track kappa, audit a random 5% each week, and use *active learning* (let the model choose the most uncertain items for humans to label first) to save effort. *Weak supervision* (the Snorkel idea) writes heuristic rules such as "contains the word refund means billing ticket", combines their noisy votes, and trains on that. It is fast but inherits the rules' errors, so always validate on a small hand-labelled set. **PU problems** (positive-unlabelled) arise when only positives are known (confirmed fraud) and the rest are simply "not yet flagged"; treating all unlabelled rows as negative biases the model, so use methods that estimate the share of hidden positives.

**The picture**

```
What we care about  ->  What we can measure  ->  What we label
 "customer is happy"      "clicked / reviewed"     "stars >= 4"
       ^ each arrow loses information; label quality caps model quality
```

**Connection to the real job.** In every project, spend an hour on "what does the label really mean, and when do we know it?". Ask the owner of the process, not only the table.

**Applications**

- Credit: delayed and selective labels, reject inference.
- Fraud: labels from analysts, only for flagged cases.
- NLP: human-labelled tickets, kappa audits.
- Medicine: label is a diagnosis, with disagreement between doctors.

| COMMON MISTAKE | Treating "no event yet" as "no event". For recent periods, the absence of a default or a purchase is not final. Always check the label maturity before using the newest rows. |
|---|---|

**Check yourself**

1. Kappa is 0 and raw agreement is 80%. How?

Answer: chance agreement is also 80%, for example when almost all items are one class.

1. Why are approved-only labels risky?

Answer: the model never sees outcomes for rejected people, so risk is underestimated and biased.

1. What is censoring?

Answer: the outcome has not happened yet within the observed time, so the true label is unknown.

## 2.4 Privacy and Ethics in Data Use

| INTUITION | Customer data is like the keys to someone's house that they lent you for one purpose. You may use the keys for that purpose, only as long as needed, and you must keep them safe. You do not copy them for a "maybe later" project. |
|---|---|

**Words you need**

| Word | Meaning | Example |
|---|---|---|
| PII (personally identifiable information) | Data that identifies a person alone or combined | Name, phone, Aadhaar, email, IP |
| Pseudonymisation | Replace identifiers by tokens; re-linking is possible with a key | `b1f22c3870` instead of customer 1 |
| Anonymisation | Remove identifiability so it cannot be reversed | Aggregates with large groups |
| Minimisation | Collect and keep only what is needed | Do not store date of birth if age band suffices |
| Purpose limitation | Use data only for the stated purpose | Support data not reused for ad targeting |

| Word | Meaning | Example |
|---|---|---|
| k-anonymity | Every combination of quasi-identifiers appears at least k times | k = 15 for city × age band |

**Laws.** India's **DPDP Act (Digital Personal Data Protection Act, 2023)** and the EU's **GDPR** share core ideas: consent (a clear, specific agreement), purpose limitation, minimisation, the right to access and erase one's data, breach notification and extra care for children. Rules and timelines change, so check the current text and your company's legal team; this chapter is awareness, not legal advice.

**Sensitive attributes and proxies.** Caste, religion, gender, health and similar attributes carry high risk of unfair treatment. Even if you drop them, *proxy variables* can leak them: pincode can stand in for caste or religion, first name for gender and region, shopping basket for health. Test it: try to predict the sensitive attribute from your features. If you can, the model can use it indirectly.

**Awareness: differential privacy and federated learning.** Differential privacy adds calibrated random noise so that no single person's presence changes the published result much. Federated learning trains a model on devices or sites without moving raw data to one place. Both are advanced; you only need to know the names and the idea now.

**Worked problems**

**Problem 2.4.1** Pseudonymise customer ids with a keyed hash and check how identifiable a table is. We create a token with HMAC-SHA256 and a secret key, and generalise age into bands.

```
import hashlib, hmac
SECRET = b"demo-secret-not-for-real-use"      # in real life: stored in a vault, never in
code
c["cust_token"] = [hmac.new(SECRET, str(i).encode(), hashlib.sha256).hexdigest()[:10]
                   for i in c.customer_id]
c["age_band"] = pd.cut(c.age, [17, 25, 35, 50, 70], labels=["18-25", "26-35", "36-50",
"51-70"])
print(c[["customer_id", "cust_token", "age_band"]].head(3).to_string(index=False))
kc = c.groupby(["city", "age_band"], observed=True).size()
print("smallest group (k) for city x age_band:", kc.min())
kc2 = c.groupby(["city", "age", "plan", "channel"], observed=True).size()
print("k for city x exact age x plan x channel:", kc2.min(), " groups of size 1:",
int((kc2 == 1).sum()))
```

```
 customer_id cust_token age_band
           1 b1f22c3870    18–25
           2 fc2a3076b3    36–50
           3 797f02a27e    26–35
smallest group (k) for city x age_band: 15
k for city x exact age x plan x channel: 1  groups of size 1: 226
```

- **Step 1** A plain hash without a secret can be reversed by hashing all ids; a keyed hash cannot unless the key leaks. So the token is *pseudonymous*, not anonymous: whoever holds the key can re-link.
- **Step 2** With city and an age band, the smallest group is 15 people: k = 15.
- **Step 3** With city, exact age, plan and channel, 226 groups have size 1. Those people are unique in the table and easy to re-identify with outside knowledge.
- **Answer** The coarse table has k = 15 (safe by a common rule of thumb such as k at least 5 to 10); the detailed table has 226 unique people, so it must not be shared.
- **Check** 15 is the minimum over all groups in the first table, so every group has at least 15 members.
- **Why it matters** It means "we removed the names" is not enough. Combinations of ordinary columns can single out a person.

**Problem 2.4.2** A lending model uses `pincode`. A test shows a simple classifier can predict a sensitive group from pincode with 85% accuracy while chance is 50%. What do you conclude, and what do you do?

| | |
|---|---|
| **Step 1** | 85% vs 50% means pincode is a strong proxy. |
| **Step 2** | The model can discriminate indirectly even if the sensitive column is deleted. |
| **Answer** | Treat pincode as sensitive. Options: remove or coarsen it, check approval and error rates by group, involve compliance, and document the business justification. |
| **Check** | If the proxy test had given 52%, there would be little concern. |
| **Why it matters** | Unfair credit decisions harm real people and can break the law; "we did not use the attribute" is not a defence. |

**Connection to the real job.** In real projects you will sign data access agreements, work in secured environments, mask columns and document the purpose of each dataset. Interviewers in BFSI and healthcare ask about this directly.

**Applications**

- Banking: KYC data, credit models with fairness reviews.
- Healthcare: patient data under strict consent and de-identification.
- Telecom: location data minimisation.

- Ad-tech: consent management for targeting.

| COMMON MISTAKE | Saying "the data is anonymised" when you only replaced names with ids. If a key exists, or if combinations of columns are unique, it is only pseudonymised. Responsible use also means not scraping or building face-recognition systems without a lawful basis. |
|---|---|

**Check yourself**

1. Pseudonymisation vs anonymisation?

Answer: pseudonymised data can be re-linked using a key; anonymised data cannot.

1. What is a proxy variable?

Answer: a feature that stands in for a sensitive attribute, such as pincode for caste or religion.

1. What does k = 1 mean for a group?

Answer: one person is unique in that combination and easy to re-identify.

## Chapter recap

- Know how every source creates its rows and what it misses by design.
- A point-in-time correct table uses only data before T for features and only data after T for the label. One leaky column moved our AUC from 0.882 to 0.972.
- Print row counts around every join; watch for fan-out and for current-state columns in historical tables.
- Run quality checks: completeness, validity, uniqueness, consistency, timeliness, stability; reconcile with a trusted total.
- Late-arriving data means "today" is incomplete; use a finalisation lag.
- Labels are proxies; check delay (censoring), noise (kappa), and selection (approved-only data hid a 18.9% default rate behind 6.5%).
- Pseudonymised is not anonymous; check group sizes (k) and proxy variables.
- Know the DPDP Act and GDPR ideas: consent, purpose limitation, minimisation, erasure.

| Idea | Pattern | Example |
|---|---|---|
| Point-in-time filter | `WHERE ts < :T` | `order_ts < '2025-01-01'` |
| Label window | `ts >= :T AND ts < :T + horizon` | next 60 days |
| Row-count guard | `len(before)` vs `len(after)` | 2 customers became 3 rows |

| Idea | Pattern | Example |
|---|---|---|
| Duplicate check | `df.key.duplicated().sum()` | 300 |
| Reconcile | `(yours - finance) / finance` | 0.17% |
| Kappa | `(po - pe) / (1 - pe)` | (0.85 - 0.5)/0.5 = 0.7 |
| k-anonymity | `groupby(quasi_ids).size().min()` | 15 |

## Practice set

**Problem 2.P.1** (easy) Which date filter belongs in a feature for prediction at T: `ts <= T + 30`, `ts < T`, or `ts >= T`? **Answer:** `ts < T`.

**Problem 2.P.2** (easy) Table A has 500 customers, table B has 1,200 address rows for those customers (non-unique key). How many rows does `A.merge(B, on="customer_id")` return, assuming all customers appear? **Answer:** 1,200 rows (each address row matches one customer); customers are duplicated.

**Problem 2.P.3** (easy) What is kappa if `po` = 0.9 and `pe` = 0.9? **Answer:** (0.9 - 0.9) / (1 - 0.9) = 0.

**Problem 2.P.4** (medium) Your table's total is Rs 1,000,000 and finance says Rs 990,000. What is the percent gap? **Answer:** (1,000,000 - 990,000) / 990,000 = 1.01%.

**Problem 2.P.5** (medium) What is the output? `kc.min()` where groups have sizes 15, 20 and 31. **Answer:** 15.

**Problem 2.P.6** (medium) Find the bug: `features = orders.groupby("customer_id").amount.mean()` is merged to a training table as of T = 2025-01-01, using all orders. **Answer:** No date filter, so orders after T are included. Add `orders[orders.order_ts < T]` first.

**Problem 2.P.7** (medium) Of 2,000 loans from the past 6 months, a team trains a 12-month default model and gets very low default rate. Why? **Answer:** The labels are not mature; most loans have not had 12 months to default, so "not defaulted" is mostly "not yet".

**Problem 2.P.8** (hard) In the detailed table, 226 of the 6,000 customers are unique in their combination of city, exact age, plan and channel. What share of the population is unique, and is that high? **Answer:** 226 / 6,000 =

3.8% unique. That is 226 people who could be singled out, which is too many for a public release; coarsen columns until k is at least 5 to 10.

**Problem 2.P.9** (interview) "What is data leakage? Give two examples." **Answer:** Leakage is when a feature carries information that would not be available at prediction time, or the label itself. Examples: using lifetime order count to predict churn at T; using a "refund issued" flag to predict fraud, which is created after the fraud is found. It inflates offline scores and fails in production. Prevent it with a fixed T, point-in-time features, time-based splits and a hunt for "too good" features.

**Problem 2.P.10** (interview) "Your fraud model's training labels come from analyst reviews, but analysts review only flagged transactions. What is the problem?" **Answer:** Selective labels: unflagged fraud is never labelled, so the model learns what the old rules flagged, not all fraud. Remedies: random audits of a small sample of unflagged transactions, importance weighting, feedback from chargebacks (a delayed but unbiased label), and exploration.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, and English is my second language. Teach me "point-in-time correct training sets and data leakage" step by step with a small SQL example on an orders table. Explain prediction time, feature window and label window with a timeline. After each step, ask me a question and wait. At the end, give me a customer-churn table to build and review my SQL.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: "data quality checks before modelling: completeness, validity, uniqueness, consistency, timeliness, stability". Give me 8 graded exercises with small pandas DataFrames, from easy to hard, including late-arriving records and reconciliation with a finance total. Do not give solutions until I answer; then grade me and explain the gaps.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional new to data science. Topic: "labels and ground truth: label delay, censoring, selective labels, reject inference, Cohen's kappa, weak supervision". Quiz me one question at a time with small
```

numerical examples. Wait for my answer, grade it, and explain with a real banking or fraud example. Ask 10 questions in total.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science in India. Review my data-handling plan for a project. Check for leakage, join fan-out, label maturity, selection bias in labels, personal data (PII), consent and purpose under the DPDP Act and GDPR ideas, and proxy variables for sensitive attributes. Give a prioritised list of risks in simple English with concrete fixes. My plan: [PASTE HERE]

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional preparing for data science interviews in India. Run a mock interview on "data collection, leakage, data quality, labels and privacy". Ask one question at a time, for example "write SQL for a point-in-time churn set" or "how do you handle delayed labels". After each answer give a score out of 10, a model answer, and a follow-up. Do 8 rounds.

CHAPTER 3

# EXPLORATORY DATA ANALYSIS FOR MODELLING

*A step-by-step method to learn what generated your data, find its bugs, and catch leakage, shown on one complete worked example.*

You have a clean-looking table and a deadline. Where do you start? A beginner draws twenty random charts. A professional follows a fixed order, writes down every surprise, and finishes with a short list of facts, hypotheses and decisions. In this chapter you learn that order. You will profile a table, study missing values, look at single columns, compare columns with the target, split by segments and time, and hunt for leakage. We do all of it on the churn table from Chapters 1 and 2, where we have secretly planted three problems for you to find: a default value, informative missingness and a leaky column. By the end you will be able to run an EDA you could hand to a colleague, who can rerun it and trust it.

**Method.** Broad first, then narrow: shape, types, missingness; then one column at a time; then each column against the target; then interactions and segments; then time. Write down every surprise.

```
Profile -> Missingness -> Univariate -> Target -> Bivariate -> Segments -> Time -> Leakage
-> Write-up
 (what)     (what's gone)   (shapes)     (the goal) (links)     (who)       (when)
(cheats)   (so what)
```

Setup used in this chapter (the full code builds a features table for any date `T`, using only data before `T`, then plants the three problems):

```
from ds_common import *
from sklearn.metrics import roc_auc_score, mutual_info_score

c, o = make_data()
rng = np.random.default_rng(11)

def features(T):
    d = make_label_table(c, o, pred_date=T)
    past = o[o.order_ts < T]
    w = past[past.order_ts >= T - pd.Timedelta(days=90)]
    g = w.groupby("customer_id")
    f = pd.DataFrame({"orders_90d": g.size(), "spend_90d": g.amount.sum(),
                      "avg_order": g.amount.mean(),
                      "recency_days": (T -
past.groupby("customer_id").order_ts.max()).dt.days})
    d = d.merge(f, left_on="customer_id", right_index=True, how="left")
    d["tenure_days"] = (T - d.signup_date).dt.days
    return d

d = features(pd.Timestamp("2025-01-01"))
d["age_raw"] = d.age.copy()
d.loc[rng.random(len(d)) < 0.07, "age_raw"] = -1                  # planted: default value
miss_p = np.where(d.churn == 1, 0.18, 0.05)
d.loc[rng.random(len(d)) < miss_p, "age_raw"] = np.nan             # planted: informative
missingness
d["cancel_ticket"] = ((d.churn == 1) & (rng.random(len(d)) < 0.55)).astype(int) \
                   | ((d.churn == 0) & (rng.random(len(d)) < 0.01)).astype(int)  #
planted: created AFTER T
```

## 3.1 Profiling and Missing Values

| INTUITION | Before a doctor treats you, she checks height, weight, pulse and temperature. She does not yet know what is wrong, but she learns what normal looks like for you. Profiling is the pulse check of a dataset: how many rows, what types, how many distinct values, how many blanks. |
|---|---|

**Words you need**

| Word | Meaning | Example |
|---|---|---|
| Profile | A quick summary of shape and columns | 2,562 rows, 15 columns |

| Word | Meaning | Example |
|---|---|---|
| Cardinality | Number of distinct values in a column | `city` has 6 |
| Mode spike | One value appears far too often | `age = −1` for 6.5% |
| MCAR | Missing completely at random: blanks unrelated to anything | A sensor drops 1% of readings by chance |
| MAR | Missing at random given other columns: blanks depend on other observed data | Younger users skip the income field |
| MNAR | Missing not at random: blanks depend on the missing value itself | High earners hide their income |

**Run the profile.** Before you run it, predict: how many rows, and will `age_raw` have a count of 2,562?

```
print("shape", d.shape, " memory MB", round(d.memory_usage(deep=True).sum() / 1e6, 2))
print(d[["plan", "channel", "city"]].nunique().to_dict(), " duplicate ids:",
d.customer_id.duplicated().sum())
print(d[["age_raw", "orders_90d", "spend_90d", "avg_order", "recency_days",
"tenure_days"]]
      .describe().round(1).T[["count", "mean", "50%", "max"]])
```

```
shape (2562, 15)  memory MB 0.66
{'plan': 2, 'channel': 3, 'city': 6}  duplicate ids: 0
              count    mean     50%      max
age_raw        2380.0    30.1    32.0     61.0
orders_90d     2562.0     9.2     7.0     67.0
spend_90d      2562.0  5260.5  3512.9  41356.7
avg_order      2562.0   548.4   528.7   2840.2
recency_days   2562.0    19.4     9.0     90.0
tenure_days    2562.0   258.4   210.0    730.0
```

Line by line. `d.shape` gives rows and columns. `memory_usage(deep=True)` tells you whether the table fits comfortably in memory (0.66 MB: tiny). `nunique()` is the cardinality: low for all three categoricals, so one-hot encoding is fine (Chapter 5). `duplicated().sum()` should be 0 for an id column. `describe()` gives count, mean, median (`50%`) and max. Notice `age_raw` has a count of 2,380, not 2,562: 182 values are missing. Also notice that spend has a mean (5,260) well above the median (3,513): a long right tail.

**Missingness analysis.** Two questions: how much is missing, and does the missingness *carry information*? Compare the missing rate across the target.

```
print(d[["age_raw"]].isna().groupby(d.churn).mean().round(3)
        .rename(columns={"age_raw": "age_missing_rate"}))
print("share with age == -1:", round((d.age_raw == -1).mean(), 3))
```

```
       age_missing_rate
churn
0                 0.044
1                 0.149
```

```
share with age == -1: 0.065
```

**Worked problems**

**Problem 3.1.1** Compute how much more often `age` is missing for churners than for non-churners, and decide whether the blank is informative.

| | |
|---|---|
| **Formula** | ratio = rate for churners / rate for non-churners |
| **Plug in** | 0.149 / 0.044 |
| **Compute** | 0.149 / 0.044 = 3.39 |
| **Answer** | Churners have a missing age 3.4 times as often (14.9% vs 4.4%). |
| **Check** | Overall missing share = (182 of 2,562) = 0.071. A weighted mix of 74.6% × 0.044 + 25.4% × 0.149 = 0.0328 + 0.0378 = 0.0707. It matches. |
| **Why it matters** | The blank itself predicts churn, so deleting those rows would remove signal, and filling with the mean would hide it. Keep an `age_is_missing` flag (Chapter 5). This is MAR or MNAR-style missingness, not MCAR. |

**Problem 3.1.2** In the output, 6.5% of ages equal -1. Explain what this is and decide what to do.

| | |
|---|---|
| **Step 1** | An age of -1 is impossible, so it is not a real age. Systems often store "unknown" as -1, 0 or 9999. This is a *mode spike*. |
| **Step 2** | Count it: 0.065 × 2,562 = 166 rows. |
| **Step 3** | Convert -1 to missing ( `NaN` ) and treat it as a second kind of missing, or keep the flag. |
| **Answer** | The -1 values are a default code meaning "not provided". Convert them to `NaN` , then handle with other missing values. |
| **Check** | With the -1 values left in, the mean age is 30.1; after `d.age_raw.replace(-1, np.nan).mean()` it is 32.5. So 166 fake values pulled the average down by 2.4 years (we ran this). |
| **Why it matters** | A model could treat -1 as "very young" and learn a false pattern about a data-entry habit. |

**Reasoning about MCAR, MAR, MNAR.** Ask: "what process creates blanks here?" If you can explain blanks using other columns (younger users, mobile app), they are MAR and imputation using other columns is reasonable. If blanks depend on the value itself (people hide high income), they are MNAR and no imputation fully repairs it, so the indicator flag becomes essential. A missingness heatmap ( `missingno` , not run here) shows patterns by row; our table-based rate by target is the most useful single check.

**The picture**

```
Column age_raw (2,562 rows)
|########################.| present: 2,380 (92.9%)
|                        ?| missing:   182 ( 7.1%)   <- 14.9% of churners, 4.4% of
stayers
|                       -1| default code in 6.5% of rows
```

**Connection to the real job.** Every notebook starts with these four lines. In interviews, "how do you handle missing data?" has one good first answer: "first I find out why it is missing".

**Applications**

- Data Science: informative missingness as a feature in credit and churn.
- Data Engineering: detecting columns that silently became null after a schema change.
- Analytics: avoiding averages distorted by default codes like 9999.

| COMMON MISTAKE | Filling missing values with the mean before looking at them. You lose the signal, and you also hide default codes. Look first, then decide. |
|---|---|

**Check yourself**

1. Cardinality means?

Answer: the number of distinct values in a column.

1. Missing rate is 15% for positives and 4% for negatives. What kind of feature could you add?

Answer: a missing-value indicator ( `is_missing` ).

1. Why is age = -1 dangerous?

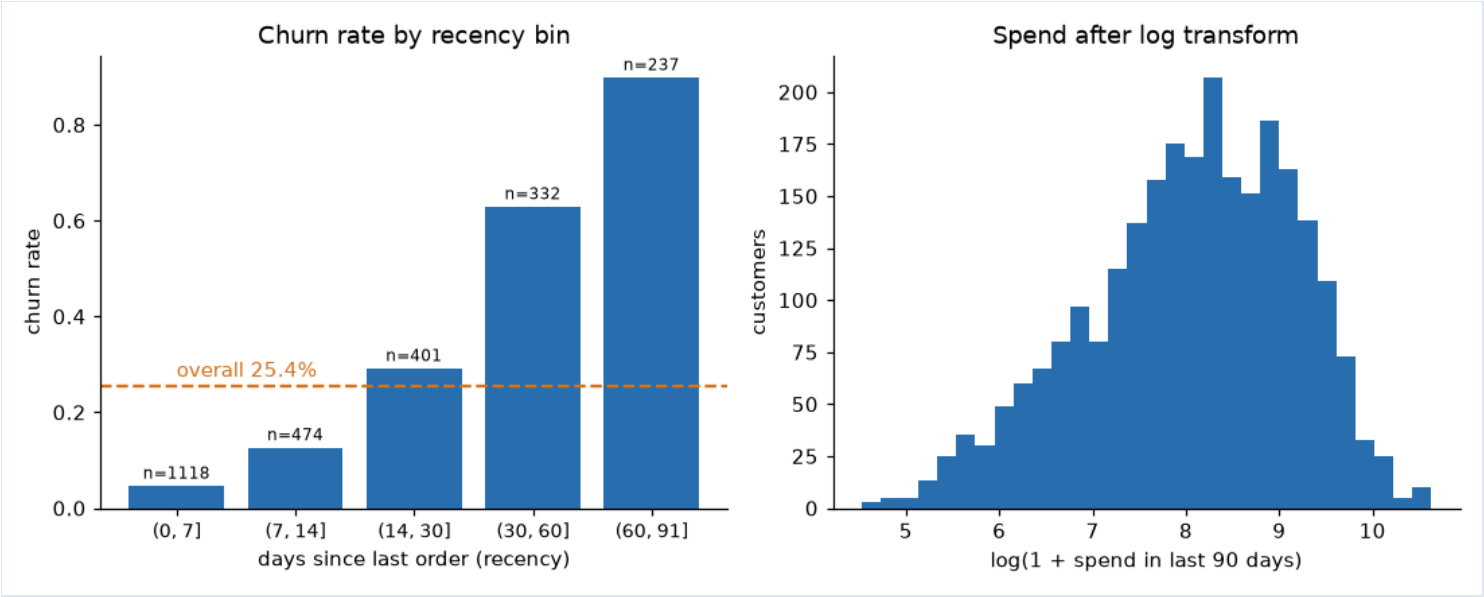
Answer: it is a default code, not a real age; a model may treat it as a number.

## 3.2 One Column at a Time, Then Against the Target

| INTUITION | Think of a delivery company's records. First you look at the histogram of delivery times (shape, long tail). Then you compare it against order volume (does it climb as orders grow?). EDA does the same: shape of each column, then its relationship to the thing you care about. |
|---|---|

**Univariate: shapes.** For a numeric column, look at a histogram (try a log scale), the median versus the mean, skewness (a number that is 0 for a symmetric shape and positive when the right tail is long), and outliers. An outlier is either an *error* (impossible) or a *real extreme* (a very big customer). Never delete before you know which.

Our spend column: skew is 2.17 (a long right tail). After `np.log1p(x) = log(1 + x)` the skew is -0.43 (nearly symmetric). That is the picture on the right below.




*Left: churn rate by recency bin rises from 4.7% to 89.9%. Right: spend after a log transform looks symmetric.*

How to read the figure. Left panel: each bar is a group of customers by days since last order; the bar height is the share who churned; the dashed line is the overall 25.4%; `n=` is the group size. Right panel: a histogram of log spend. The x-axis is log(1 + spend), the y-axis counts customers.

**Target analysis.** First the class balance: churn rate is 25.4%, so the classes are moderately imbalanced (not extreme). Then the target against each column:

```
d["rec_bin"] = pd.cut(d.recency_days, [0, 7, 14, 30, 60, 91])
for col in ["plan", "channel", "city"]:
    print(d.groupby(col).churn.agg(["mean", "size"]).round(3).T.to_string())
print(d.groupby("rec_bin").churn.agg(["mean", "size"]).round(3).T.to_string())
```

```
plan      free      pro
mean     0.313     0.17
size  1509.000  1053.00
channel      ads   organic  referral
mean       0.309     0.258     0.157
size     726.000  1372.000   464.000
city  Bengaluru  Chennai    Delhi  Mumbai    Other     Pune
mean       0.244    0.252    0.243    0.27    0.238    0.295
size     504.000  246.000  497.000  525.00  526.000  264.000
rec_bin    (0, 7]  (7, 14]  (14, 30]  (30, 60]  (60, 91]
mean        0.047    0.127     0.292      0.63     0.899
size  1118.000  474.000   401.000    332.00   237.000
```

`pd.cut` makes bins: `(0, 7]` means above 0 and up to and including 7 days.
`groupby(col).churn.agg(["mean","size"])` gives the churn rate and the group size for each level. Always

show the size: a rate on 20 customers is noise.

**Worked problems**

**Problem 3.2.1** Compute the lift of the recency group (60, 91] and of the group (0, 7], and say what they mean.

| | |
|---|---|
| **Formula** | lift = group churn rate / overall churn rate |
| **Plug in** | 0.899 / 0.254 and 0.047 / 0.254 |
| **Compute** | 0.899 / 0.254 = 3.54; 0.047 / 0.254 = 0.19 |
| **Answer** | Customers silent for more than 60 days churn 3.5 times as often as average; customers who ordered within a week churn at about one fifth of average. |
| **Check** | Weighted average of group rates: (0.047×1118 + 0.127×474 + 0.292×401 + 0.63×332 + 0.899×237) / 2562 = (52.5 + 60.2 + 117.1 + 209.2 + 213.1) / 2562 = 652.1 / 2562 = 0.2545, which matches the overall 0.254. |
| **Why it matters** | The relationship is strong and monotonic (it only goes up). This is the best single feature and also warns us that a recency rule is a strong baseline (Chapter 1). |

**Problem 3.2.2** Measure how strongly each categorical column is linked to churn using Cramér's V, and decide which matters.

Cramér's V is a number from 0 (no link) to 1 (perfect link) computed from a chi-square test on the cross-table of two categorical columns: `V = sqrt(chi² / (n × (min(rows, cols) − 1)))`. Mutual information (MI) is another measure: how much knowing the column reduces uncertainty about churn, in *nats* (units based on the natural log).

```
def cramers_v(a, b):
    ct = pd.crosstab(a, b).values; n = ct.sum()
    exp = ct.sum(1, keepdims=True) * ct.sum(0, keepdims=True) / n
    chi2 = ((ct - exp) ** 2 / exp).sum()
    return np.sqrt(chi2 / (n * (min(ct.shape) - 1)))
print("Cramer's V:", {k: round(float(cramers_v(d[k], d.churn)), 3) for k in ["plan",
"channel", "city"]})
print("MI (nats) rec_bin:", round(mutual_info_score(d.rec_bin.astype(str), d.churn), 4))
print(d[["churn", "orders_90d", "spend_90d", "recency_days", "tenure_days"]]
      .corr(method="spearman").round(2)["churn"].to_dict())
```

```
Cramer's V: {'plan': 0.161, 'channel': 0.116, 'city': 0.042}
MI (nats) rec_bin: 0.2042
{'churn': 1.0, 'orders_90d': -0.41, 'spend_90d': -0.39, 'recency_days': 0.57,
'tenure_days': 0.01}
```

- **Step 1** Plan has V = 0.161, channel 0.116, city 0.042.
- **Step 2** Spearman correlation (rank-based, so it catches any steadily rising or falling relation, not only straight lines) with churn: recency +0.57, orders -0.41, spend -0.39, tenure 0.01.
- **Answer** Recency, order count and spend are the strong numeric signals; plan and channel are moderate; city is almost nothing; tenure alone says nothing.
- **Check** The signs make sense: more recent orders and more frequent orders mean lower churn (negative correlation); longer silence means higher churn (positive). City's V of 0.042 matches the near-flat rates (23.8% to 29.5%) in the table.
- **Why it matters** It tells you where to spend feature-engineering time. Also, Pearson and Spearman only see monotone relations; a U-shape would give about zero, so always plot as well.

**The picture.** A feature-vs-target chart (binned rates, as in the left panel above) is also called a *partial view*. It is the fastest way to see non-linear effects, and it works for trees and linear models alike.

**Connection to the real job.** This table is what you show a business partner: "these are the strongest signals; here is how churn changes with each". It also guides the first model.

**Applications**

- Data Science: choosing features and spotting non-linear effects.
- Analytics: segmenting customers for campaigns.
- Credit risk: binning variables by default rate (Chapter 11, WoE).

| COMMON MISTAKE | Reading a rate without its group size. A 60% churn rate on 8 customers is noise; always print `n`, and for small groups add a confidence interval (Chapter 4). |
|---|---|

**Check yourself**

1. When do you use log scale?

Answer: for right-skewed positive values like spend or time, to see the shape.

1. What does lift 3.5 mean?

Answer: that group's rate is 3.5 times the overall rate.

1. Why can a correlation of 0 hide a strong relationship?

Answer: correlation measures linear or monotone links; a U-shape can give about 0.

## 3.3 Segments, Time and Simpson's Check

| INTUITION | The average temperature of India is useless for packing your suitcase. You need Shimla or Chennai. An average churn rate is the same: different groups can behave differently, and over time the whole picture can shift (like peak-hour versus night traffic). |
|---|---|

**Segment analysis.** Ask "who behaves differently?" by geography, device, cohort (customers who joined in the same period), and acquisition channel. In our data, plan and channel matter; city does not (Section 3.2). Combine two segments in a pivot table to see interactions (we do this for plan and channel in Problem 3.3.2 below). Also check that each segment has enough rows.

**Time analysis: drift.** *Data drift* means the data in the recent period differs from the training period; models trained on the past then fail on the present. Look at: trends, seasonality (weekday, month, festival), and recent change. We build the same churn table at five prediction dates and compare:

```
rows = []
for T in pd.to_datetime(["2024-04-01", "2024-07-01", "2024-10-01", "2025-01-01", "2025-02-
28"]):
    t = features(T)
    rows.append((T.date(), len(t), round(t.churn.mean(), 3), round(t.recency_days.mean(),
1), round(t.orders_90d.mean(), 1)))
print(pd.DataFrame(rows, columns=["T", "rows", "churn_rate", "mean_recency",
"mean_orders90"]).to_string(index=False))
```

```
         T  rows  churn_rate  mean_recency  mean_orders90
2024-04-01  2124       0.249          19.2            8.5
2024-07-01  2258       0.256          19.4            8.8
2024-10-01  2424       0.239          18.5            9.2
2025-01-01  2562       0.254          19.4            9.2
2025-02-28  2275       0.244          20.0           10.2
```

**Worked problems**

**Problem 3.3.1** Is the churn rate stable across the five dates? Use a simple range check.

| | |
|---|---|
| **Formula** | range = max - min of churn rate across dates |
| **Plug in** | max = 0.256 (2024-07-01), min = 0.239 (2024-10-01) |
| **Compute** | 0.256 - 0.239 = 0.017 |
| **Answer** | The rate moves by 1.7 percentage points over 11 months; there is no sign of a trend. |
| **Check** | With about 2,300 customers per date, the standard error of a rate near 0.25 is `sqrt(0.25 × 0.75 / 2300)` = 0.009, so a 1.7-point spread is within normal noise (about 2 standard errors). |
| **Why it matters** | A stable label rate and stable feature means (recency 18.5 to 20.0) give you comfort to train on older periods. If instead churn had risen from 25% to 35%, you would weigh recent data more or investigate a business change. |

Note: This data is synthetic and simple, so it has no weekday effect (each weekday has about 14.3% of orders) and no seasonal wave. Real order data almost always has both; you would plot daily counts and check holiday weeks. For India, mark Diwali, Holi, end-of-month salary days and IPL season, because they move traffic and orders sharply.

**Simpson's paradox check.** Simpson's paradox is when a trend in the whole data reverses inside every group. Checking costs one line: compute the effect overall and again within each segment, and compare the signs. We study the mechanism and a real numerical example in Chapter 4. In our table, the plan effect has the same direction in every channel, so there is no reversal here.

**Problem 3.3.2** Run the check for plan inside each channel: is "pro churns less" true in every channel?

| | |
|---|---|
| **Step 1** | Overall: pro 17.0%, free 31.3%. |
| **Step 2** | Within each channel (group by `channel`, `plan`) compute the churn rate. |
| **Step 3** | Look for any channel where pro is higher than free. |
| **Answer** | The check, in code, is `d.groupby(["channel","plan"]).churn.mean().unstack()`. The result is printed below; if every row shows pro below free, there is no reversal. |
| **Check** | The overall pro rate (17.0%) is a mix of the channel-level pro rates weighted by how many pro customers each channel has. The sign of the overall gap can flip only when the channel mix is very different between plans. |
| **Why it matters** | Without this check, an overall conclusion such as "channel X is better" might be reversed inside every plan. |

```
print(d.groupby(["channel", "plan"]).churn.mean().unstack().round(3))
```

```
plan         free    pro
channel
ads         0.402  0.192
organic     0.319  0.168
referral    0.171  0.136
```

Pro is below free in all three channels (0.192 < 0.402, 0.168 < 0.319, 0.136 < 0.171). No Simpson reversal. The gap is large for ads (21 points) and small for referral (3.5 points): the plan effect depends on the channel, which is an interaction worth a note in the EDA write-up.

**The picture**

```
churn
 rate   free  pro
 ads    .402 .192
 org    .319 .168      each row: free > pro   -> consistent
 ref    .171 .136
```

**Connection to the real job.** "DAU dropped 10%, what do you do?" The segment and time analysis here is the answer: split by platform, country, new vs old users, and check whether the drop started on a release date or a data-pipeline change.

**Applications**

- Product analytics: diagnosing metric drops by segment and date.
- Fraud: new attack patterns appear in one segment first.
- Forecasting: seasonal effects (festivals) drive plans for staffing and stock.
- Operations: peak-hour behaviour differs from average, so segment by hour.

| COMMON MISTAKE | Looking only at overall averages and at the whole time range. Always split by one or two key segments and plot recent vs older periods. |
|---|---|

**Check yourself**

1. What is data drift?

Answer: the data in recent time differs from the training period.

1. Why do you add `size` to a segment table?

Answer: small segments give noisy rates.

1. Simpson's check in one sentence?

Answer: compute the effect overall and within each group, and see if the sign flips.

## 3.4 Leakage Hunting and Writing It Up

| INTUITION | If a delivery time looks impossibly fast (2 minutes for a cross-city order), you do not celebrate; you suspect the order was never really delivered. A feature that predicts the target "too well" deserves the same suspicion. |
|---|---|

**Method 1: single-feature AUC.** AUC of one feature alone against the target. For a binary feature, AUC = 0.5 + (hit rate - false alarm rate) / 2. Any single feature above about 0.9 on a hard problem deserves a hard look. We take `max(AUC, 1 − AUC)` because a feature that perfectly *reverses* the target is just as informative.

```
rows = []
for col in ["orders_90d", "spend_90d", "avg_order", "recency_days", "tenure_days",
"cancel_ticket"]:
    x = d[col].fillna(d[col].median())
    a = roc_auc_score(d.churn, x)
    rows.append((col, round(max(a, 1 - a), 3)))
print(pd.DataFrame(rows, columns=["feature", "single_feature_AUC"])
        .sort_values("single_feature_AUC", ascending=False).to_string(index=False))
print(d.groupby("cancel_ticket").churn.agg(["mean", "size"]).round(3))
print("ticket share among churners:", round(d[d.churn==1].cancel_ticket.mean(), 3))
```

```
      feature  single_feature_AUC
 recency_days               0.880
   orders_90d               0.770
    spend_90d               0.761
cancel_ticket               0.753
    avg_order               0.576
  tenure_days               0.505
                mean  size
cancel_ticket
0              0.143  2206
1              0.944   356
ticket share among churners: 0.516
```

`cancel_ticket` ranks fourth by AUC, so AUC alone did not flag it, because it only covers half of the churners. But the second table is the alarm: customers with a cancellation ticket churn 94.4% of the time. A feature that

nearly determines the label, for a group of 356 customers, is suspicious.

**Method 2: the timestamp question.** For every strong feature ask: *when is this value written?* The ticket in our example is created when a customer asks to cancel, which happens *after* T (the planted bug). At prediction time on January 1st, the ticket does not exist yet. This is the decisive test; no statistic replaces it.

**Method 3: suspicious jumps.** If adding one column lifts AUC by 0.05 or more, look at it. (In Chapter 2, one future column moved AUC from 0.882 to 0.972.)

**Worked problems**

**Problem 3.4.1** Using the numbers above, how many churners does `cancel_ticket` flag, and how many false alarms does it produce?

| | |
|---|---|
| **Formula** | flagged churners = tickets × churn rate among tickets |
| **Plug in** | 356 × 0.944 |
| **Compute** | 356 × 0.944 = 336.1, so 336 churners flagged and 356 - 336 = 20 customers who got a ticket yet stayed |
| **Answer** | The ticket flags 336 churners with only 20 false alarms. |
| **Check** | Total churners are 0.254 × 2,562 = 651. Flagged share = 336 / 651 = 0.516, which matches "ticket share among churners 0.516". |
| **Why it matters** | Precision of 94% looks wonderful, which is the red flag. The ticket is an *effect* of the churn decision, not a *cause*; including it would make the model "predict" what we already know. |

**Problem 3.4.2** Decide what to do with each column: `recency_days`, `cancel_ticket`, `age_raw`, `city`.

| | |
|---|---|
| **Step 1** | `recency_days`: strong, computed from orders before T. Keep. |
| **Step 2** | `cancel_ticket`: created after T. Drop from the model. (If a version existed from before T, e.g. "complaint in last 30 days before T", that would be allowed.) |
| **Step 3** | `age_raw`: convert -1 to NaN, add `age_is_missing`, impute the median. Keep. |
| **Step 4** | `city`: almost no signal (V = 0.042), cheap to keep as one-hot but low priority; drop if you want a simpler model. |
| **Answer** | Keep recency and age (cleaned), drop the ticket, keep or drop city by simplicity. |
| **Check** | After dropping the ticket, the best single-feature AUC is 0.880 (recency), the same as our honest baseline in Chapter 1, so nothing is inflated. |
| **Why it matters** | The decision is documented with a reason. Next month, a colleague can see why `cancel_ticket` was excluded. |

**Writing findings.** Use three parts for each finding: *hypothesis, evidence, so what.* Example:

```
Finding 1
Hypothesis: Silence predicts churn.
Evidence:   Churn rate rises from 4.7% (ordered within 7 days) to 89.9% (silent >60 days);
lift 3.5x.
So what:    A recency rule is a strong baseline; the model must beat AUC 0.88 to be worth
shipping.

Finding 2
Hypothesis: Missing age is informative.
Evidence:   Age is missing for 14.9% of churners vs 4.4% of stayers (3.4x). 6.5% of ages
are -1.
So what:    Clean -1 to missing, add an is_missing flag.

Finding 3 (BUG)
Hypothesis: cancel_ticket leaks.
Evidence:   94.4% of ticket holders churn; the ticket is created after T.
So what:    Exclude; ask data engineering for a point-in-time version.
```

Keep an EDA notebook that someone else can rerun: fixed seed, relative paths, a top cell stating the data snapshot date. Tools that automate profiling include `ydata-profiling`, `sweetviz` and `missingno` (not installed or run here). Use them to save time, but still read every column yourself.

**The picture**

```
feature value -> "when is it written?"
   |-- before T  -> allowed
   |-- at T      -> allowed if available in the serving system at that moment
   |-- after T   -> LEAK, drop it
```

**Connection to the real job.** "Find the leakage in an unfamiliar notebook" is a classic mastery test and interview task. The method: list every feature, ask when it is written, look at the strongest features first, check the validation design.

**Applications**

- Credit: "account closed" flags or "collections status" used for default prediction.
- Healthcare: lab tests ordered because a doctor already suspected the disease.
- Fraud: "chargeback reason" fields filled after the investigation.
- Support: ticket categories assigned after resolution.

| COMMON MISTAKE | Celebrating a very strong feature. Before you celebrate, ask when the value is written and by which process. Strength is a reason for suspicion, not pride. |
|---|---|

**Check yourself**

1. Which question catches leakage best?

Answer: when is this value written, compared with the prediction time?

1. A binary feature with AUC 0.753 is leaky. How can that be?

Answer: it covers only a subset of cases but is almost perfect within them; check precision inside the flagged group.

1. What are the three parts of a finding?

Answer: hypothesis, evidence, so what.

## Chapter recap

- Follow a fixed order: profile, missingness, univariate, target, bivariate, segments, time, leakage, write-up.
- Missing values: find the reason (MCAR, MAR, MNAR), compare missing rate by target, and keep an indicator when it is informative (14.9% vs 4.4%).
- Mode spikes such as -1, 0, 9999 are default codes: convert to missing.
- Look at distributions on a log scale for skewed values; spend had skew 2.17, and after log1p -0.43.

- Always show group sizes next to rates; recency churn rises from 4.7% to 89.9% across bins.
- Use Cramér's V, Spearman correlation and mutual information to rank signals, but also plot.
- Check segments and time (drift) and run the Simpson check (compare overall and within-group effects).
- Hunt leakage with the question "when is this written?" and with suspiciously strong features (94.4% churn for ticket holders).
- Write findings as hypothesis, evidence, so what.

| Task | Code pattern | Meaning |
|---|---|---|
| Profile | `df.shape`, `df.describe()`, `df.nunique()` | size, summary, cardinality |
| Missing by target | `df[c].isna().groupby(df.y).mean()` | informative blanks |
| Mode spike | `(df[c] == -1).mean()` | default codes |
| Skew | `df[c].skew()` and `np.log1p` | tail shape |
| Rate by bin | `pd.cut` then `groupby().y.agg(["mean","size"])` | feature vs target |
| Lift | `group rate / overall rate` | strength of a segment |
| Single-feature AUC | `max(a, 1 - a)` | leak or signal |
| Simpson check | `groupby([seg, treat]).y.mean().unstack()` | sign flips? |

# Practice set

**Problem 3.P.1** (easy) 182 of 2,562 ages are missing. What percent? **Answer:** 182 / 2,562 = 7.1%.

**Problem 3.P.2** (easy) A group has churn rate 0.40; the overall rate is 0.25. What is the lift? **Answer:** 0.40 / 0.25 = 1.6.

**Problem 3.P.3** (easy) What does `pd.cut(x, [0, 7, 14])` do to the value 7? **Answer:** It goes to the bin (0, 7] because bins are right-closed.

**Problem 3.P.4** (medium) What is the output? `pd.Series([1, 2, 3, np.nan, -1]).replace(-1, np.nan).isna().sum()` **Answer:** 2 (the original NaN and the replaced -1).

**Problem 3.P.5** (medium) A feature has median 3,500 and mean 5,260. What shape do you suspect, and what transform do you try? **Answer:** Right-skewed with a long tail; try `log1p`.

**Problem 3.P.6** (medium) Find the bug: `d.groupby("city").churn.mean()` is used to claim "Pune is the worst city at 29.5%" without more. What is missing? **Answer:** The group size (Pune has only 264 customers) and the uncertainty. With n = 264 the standard error is about 0.027, so Pune (0.295) is not clearly different from the overall 0.254.

**Problem 3.P.7** (medium) Missingness: a feature is missing for 30% of positives and 10% of negatives, base rate 20%. What share of all rows are missing? **Answer:** 0.2 × 0.30 + 0.8 × 0.10 = 0.06 + 0.08 = 0.14, so 14%.

**Problem 3.P.8** (hard) The overall approval rate for loans is higher for men than women, but within each income band women are approved as often or more. What is this called, and what do you check? **Answer:** Simpson's paradox. Check how gender mix differs across income bands (women may apply more in lower-income bands where approval is lower), and report the within-band comparison.

**Problem 3.P.9** (interview) "Your feature has AUC 0.99 for a hard prediction problem. What do you do?" **Answer:** Suspect leakage. Ask when the feature is written; check whether it is derived from the label or from future data; drop it or rebuild it with a cutoff at T; retrain and see the AUC fall to a realistic value; document.

**Problem 3.P.10** (interview) "DAU dropped 10% last week. How do you investigate?" **Answer:** Check data first (pipeline, logging change, time zone), then segment by platform, country, new vs returning, channel and version;

line up the start date with releases, outages and campaigns; compare with the same week last year; quantify each segment's contribution to the drop; then propose a hypothesis and a test.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, and English is my second language. Teach me a step-by-step exploratory data analysis method for modelling: profile, missingness (MCAR, MAR, MNAR), univariate, target analysis, segments, time, leakage, write-up. Use a small customer churn table in pandas as the running example. After each step ask me one question and wait for my answer.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: "missing values and default codes in EDA". Give me 8 graded exercises with small pandas tables (include a -1 default code, informative missingness and a MAR example). Do not give solutions yet. After I answer I will paste my work and you will grade it, explain what I missed and show the pandas code.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional new to data science. Topic: "leakage hunting in notebooks". Quiz me one question at a time: show a short feature list and a validation design for a prediction problem, and ask me which feature leaks and why, using the question "when is this value written compared with the prediction time". Give feedback after each answer. Ask 10 questions with increasing difficulty.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Review my EDA notebook summary below. Check that I used a fixed order, looked at group sizes, checked drift over time, ran a Simpson check, tested for leakage, and wrote each finding as hypothesis, evidence, so what. Point out weak or missing steps in simple English and suggest concrete improvements. My summary: [PASTE HERE]
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional preparing for data science interviews in India. Run a mock interview on EDA: "DAU dropped 10%", "how do you
```

```
handle missing data", "your AUC is too good, why", "explain Simpson's paradox". Ask one question at a time, score me out of 10, give a model answer and one follow-up. Do 8 rounds.
```

CHAPTER 4

# STATISTICAL THINKING: WAYS DATA FOOLS YOU

*Nine classic traps, each reproduced with code so you can see it happen, plus how to put an honest "give or take" on every number you report.*

A stakeholder shows you a chart: "Campaign B converts 33.5%, campaign A only 20%. B wins." Another says: "We coached our worst-performing stores last month and they improved 13 points. The coaching works." A third: "Our new fraud model is 99% accurate." All three statements sound reasonable, and all three can be badly wrong. This chapter is the "know them cold" list of ways data fools people, and the habit of reporting uncertainty. You learn each trap by running a small simulation, so you do not just remember its name, you feel it. By the end you will be able to spot these traps in a dashboard, defend against them in an analysis, and report a result with an honest range. Statistics basics (mean, variance, normal distribution, p-value) are in the learner's Math PDF; here we use them as tools.

## 4.1 Correlation, Confounding, Simpson's Paradox and Selection Effects

**INTUITION**

In the summer, ice-cream sales and drownings both rise. Nobody thinks ice cream causes drowning; hot weather makes people eat ice cream and swim. The hidden third thing is called a *confounder*. Here is a similar trap: queue length at a bank and the number of people using umbrellas both rise on rainy lunch hours. Umbrellas did not cause the queue; the weather and the lunch rush did. Data science is full of hidden third things.

**Words you need**

| Word | Meaning | Example |
|---|---|---|
| Correlation | Two things move together | Ice cream and drowning |
| Confounder | A third factor that drives both | Temperature |
| Simpson's paradox | A trend in the combined data reverses inside every subgroup | Campaign B wins overall, A wins in each device group |
| Selection bias | The sample is not the population | Survey of app users only |

| Word | Meaning | Example |
|---|---|---|
| Survivorship bias | We see only the winners that remained | Funds that survived 5 years |
| Berkson's paradox | Selecting on a combined score creates a fake negative link | Admitted students: marks vs sports |

**Confounding, simulated.** We create 365 days. Temperature drives both ice-cream sales and drownings. Ice cream has no effect on drownings in the simulation.

```
import numpy as np
rng = np.random.default_rng(5)
n = 365
temp = rng.normal(30, 5, n)
ice = 50 + 4 * temp + rng.normal(0, 15, n)
drown = 2 + 0.3 * temp + rng.normal(0, 1.5, n)
print("corr ice,drown", np.corrcoef(ice, drown)[0, 1].round(2))
def res(y, x):
    b = np.polyfit(x, y, 1); return y - np.polyval(b, x)     # what is left after removing
temperature
print("partial corr given temp", np.corrcoef(res(ice, temp), res(drown, temp))[0,
1].round(2))
```

```
corr ice,drown 0.53
partial corr given temp -0.0
```

Line by line: `np.corrcoef` is the plain correlation: 0.53, fairly strong. `res(y, x)` fits a straight line of y on temperature and returns the leftover (residual). Correlating the leftovers is the *partial correlation*: the link between ice and drowning after removing what temperature explains. It is zero (printed as -0.0, a tiny negative rounding of 0). The apparent link was all temperature. Remember this pattern: *adjust for the confounder and see if the link survives.*

**Simpson's paradox, with real numbers.** A marketing team tests two campaigns. Each visitor is on mobile or desktop.

```
import pandas as pd
t = pd.DataFrame({"campaign": ["A","A","B","B"], "device":
["mobile","desktop","mobile","desktop"],
                  "visitors": [200,100,50,900], "buyers": [20,40,3,315]})
t["rate"] = (t.buyers / t.visitors).round(3)
print(t.to_string(index=False))
tot = t.groupby("campaign")[["visitors","buyers"]].sum(); tot["rate"] = (tot.buyers /
tot.visitors).round(3)
print(tot)
```

```
campaign  device  visitors  buyers  rate
       A  mobile       200      20  0.10
       A desktop       100      40  0.40
       B  mobile        50       3  0.06
       B desktop       900     315  0.35
          visitors  buyers   rate
campaign
A              300      60  0.200
B              950     318  0.335
```

**Problem 4.1.1** Which campaign is better? Compute by device and overall, and explain the reversal.

| | |
|---|---|
| **Formula** | conversion rate = buyers / visitors |
| **Plug in** | A mobile 20/200, A desktop 40/100, B mobile 3/50, B desktop 315/900 |
| **Compute** | A: 10% mobile, 40% desktop. B: 6% mobile, 35% desktop. Overall: A = 60/300 = 20.0%; B = 318/950 = 33.5%. |
| **Answer** | A beats B on both devices (10 > 6 and 40 > 35), yet B beats A overall (33.5 > 20). The reason: B's traffic is 900/950 = 95% desktop (where everybody converts well), while A's traffic is only 33% desktop. |
| **Check** | Weighted average for A: (0.10 × 200 + 0.40 × 100) / 300 = 60/300 = 0.20 ✓. For B: (0.06 × 50 + 0.35 × 900) / 950 = 318/950 = 0.335 ✓. The overall rates are the *weights times the group rates*, and the weights differ. |
| **Why it matters** | If traffic was assigned by the team and not by a coin flip, the overall comparison mixes "how good is the campaign" with "which visitors it got". If the device is the true driver and the campaigns were randomly assigned, the within-device comparison is right. If the campaign changed which device people use, the overall might be right. This is why a DAG (Chapter 8) is needed: statistics cannot decide which view is correct, causal reasoning can. |

**Problem 4.1.2** Survivorship bias. A fund list shows only funds that survived 5 years. In a simulation, 1,000 funds have yearly returns with *zero skill* (mean 0%, sd 20%). A fund dies if any yearly return is below -25%. What does the survivor list show?

| Step 1 | Simulate 1,000 funds x 5 years, mean 0, sd 0.2 (seed 2): |
|---|---|

```
r = np.random.default_rng(2)
ret = r.normal(0.0, 0.2, (1000, 5))
alive = ret.min(axis=1) > -0.25
print("all funds mean yearly return", ret.mean().round(3), " survivors only",
ret[alive].mean().round(3), " survivors share", alive.mean().round(3))
```

```
all funds mean yearly return 0.0  survivors only 0.041  survivors share 0.558
```

- **Step 2** Keep funds whose worst year was above -25%: 55.8% survive.
- **Step 3** Compare average yearly returns.
- **Answer** All funds average 0.0%; survivors average +4.1% per year, although nobody had skill.
- **Check** Removing the losers can only raise the mean. The survivor share (0.558) matches theory: each year's chance to stay above -25% is about 0.894, and $0.894^5 = 0.57$.
- **Why it matters** If you study only the companies that survived, the customers who stayed, or the loans that were approved, you learn the wrong lessons. It is the same mistake as Chapter 2's "approved-only" labels.

**Berkson's paradox.** Suppose marks (a) and sports skill (b) are independent in the population. A college admits students whose sum is high.

```
rng = np.random.default_rng(3)
a = rng.normal(0, 1, 100000); b = rng.normal(0, 1, 100000)     # independent marks and
sports
adm = (a + b) > 1.5                                             # admit if the sum is high
print("corr all", np.corrcoef(a, b)[0, 1].round(3), " corr admitted", np.corrcoef(a[adm],
b[adm])[0, 1].round(3), " admitted share", adm.mean().round(3))
```

```
corr all 0.0  corr admitted -0.67  admitted share 0.145
```

Among the admitted 14.5%, marks and sports show a strong negative correlation (-0.67), created only by the admission rule: if a student has low marks, they must have high sports skill to get in. Whenever your data is filtered by a combined score (approved loans, hired employees, shown products), expect fake negative links.

**The picture**

```
Confounder:        Temperature --> Ice cream
                        |
                        +--------> Drownings      (ice cream -> drownings? NO)

Selection (collider): Marks --> Admitted <-- Sports
                      Looking only at "Admitted" links marks and sports.
```

**Connection to the real job.** In product analytics, "users who use feature X retain better" is nearly always confounding: engaged users use more features. Interviewers love this question.

### Applications

- Product analytics: feature usage vs retention.
- Healthcare: sicker patients get more treatment, so treatment looks harmful.
- Finance: fund performance tables, credit approvals.
- HR analytics: performance of those hired only.

| | |
|---|---|
| **COMMON MISTAKE** | Saying "correlation is not causation" and stopping. Be specific: name the likely confounder, state how you would adjust or experiment, and say which view (overall or subgroup) matches the decision you need. |

### Check yourself

1. In Simpson's example, why does B win overall?

Answer: B's traffic is mostly on desktop, which converts better for everyone.

1. What is the first step to deal with a confounder?

Answer: name it and adjust for it, or randomise.

1. Why do admitted students show a negative link between marks and sports?

Answer: selection on the sum creates it (Berkson's paradox).

# 4.2 Regression to the Mean, Base Rates and Goodhart's Law

**INTUITION**

A cricketer scores a century in one match. In the next match he usually scores less. He did not get worse; the first score was partly skill and partly luck, and luck does not repeat. If you pick the extremes, you pick the lucky ones, and they drift back toward average.

**Regression to the mean (RTM).** When a measure has both skill and random noise, the top and bottom groups of the first measurement are partly lucky or unlucky. On the next measurement they move toward the average, *without any intervention*.

**Problem 4.2.1** A chain has 1,000 stores. Monthly sales index = true skill + random noise (both with sd 10, mean 100). We pick the worst 10% in month 1 and do nothing. What happens in month 2?

```
rng = np.random.default_rng(0)
n = 1000
skill = rng.normal(100, 10, n)
m1 = skill + rng.normal(0, 10, n)
m2 = skill + rng.normal(0, 10, n)
worst = m1 <= np.quantile(m1, 0.10)
print("worst 10% month1 mean", m1[worst].mean().round(1), "-> month2 mean (no
intervention)", m2[worst].mean().round(1))
print("corr month1 month2", np.corrcoef(m1, m2)[0, 1].round(2), " expected 0.5")
best = m1 >= np.quantile(m1, 0.90)
print("best 10% month1", m1[best].mean().round(1), "-> month2", m2[best].mean().round(1))
```

```
worst 10% month1 mean 73.7 -> month2 mean (no intervention) 86.7
corr month1 month2 0.49  expected 0.5
best 10% month1 124.7 -> month2 109.6
```

- **Formula** expected month-2 = mean + r × (month-1 - mean), where r is the correlation between months (here skill variance / total variance = 100/200 = 0.5).
- **Plug in** 100 + 0.5 × (73.7 - 100)
- **Compute** 73.7 - 100 = -26.3; × 0.5 = -13.15; 100 - 13.15 = 86.85
- **Answer** The worst stores "improved" from 73.7 to 86.7 (+13.0) without any coaching; the formula predicted 86.85. The best stores "fell" from 124.7 to 109.6.
- **Check** The measured correlation is 0.49, close to the theoretical 0.5, and the observed improvement matches the formula within 0.2.

- **Why it matters** If you had coached the worst stores, you would have claimed a 13-point success. The right comparison is with a *control group of similarly weak stores that were not coached* (an experiment, Chapter 7). RTM also explains the "Sports Illustrated cover jinx" and why medicine uses placebo controls.

**Base-rate neglect.** A test is 99% sensitive (finds 99% of true cases) and 99% specific (clears 99% of non-cases). A condition affects 0.1% of cases. You test positive. How likely are you truly positive?

**Problem 4.2.2** Use Bayes' rule with numbers per 100,000.

| | |
|---|---|
| **Formula** | PPV = true positives / (true positives + false positives) |
| **Step 1** | True cases = 100,000 × 0.001 = 100. Found: 100 × 0.99 = 99. |
| **Step 2** | Non-cases = 99,900. False alarms = 99,900 × 0.01 = 999. |
| **Compute** | PPV = 99 / (99 + 999) = 99 / 1,098 = 0.0902 |
| **Answer** | Only 9.0% of positives are real; about 9 in 10 alarms are false, although the test is "99% accurate". |
| **Check** | The code gave PPV 0.0902, with per 100,000 true positives 99 and false positives 999 ✓. |
| **Why it matters** | Fraud (0.1% to 1% of transactions), rare disease, intrusion detection and any "needle in haystack" task behave like this. Always ask for the base rate and report *precision at the operating threshold*, not accuracy. This is why a fraud team with capacity to review 500 alerts a day cares about precision@500. |

**Goodhart's law:** "When a measure becomes a target, it ceases to be a good measure." People (and models) optimise the number, not the goal. A toy model: a support team is rewarded for tickets per hour. Spending fewer minutes per ticket makes a ticket less likely to be truly resolved: P(resolved) = 1 - exp(-minutes / 8). A resolved ticket is worth Rs 100; an unresolved one costs Rs 400 (angry customer, repeat contact).

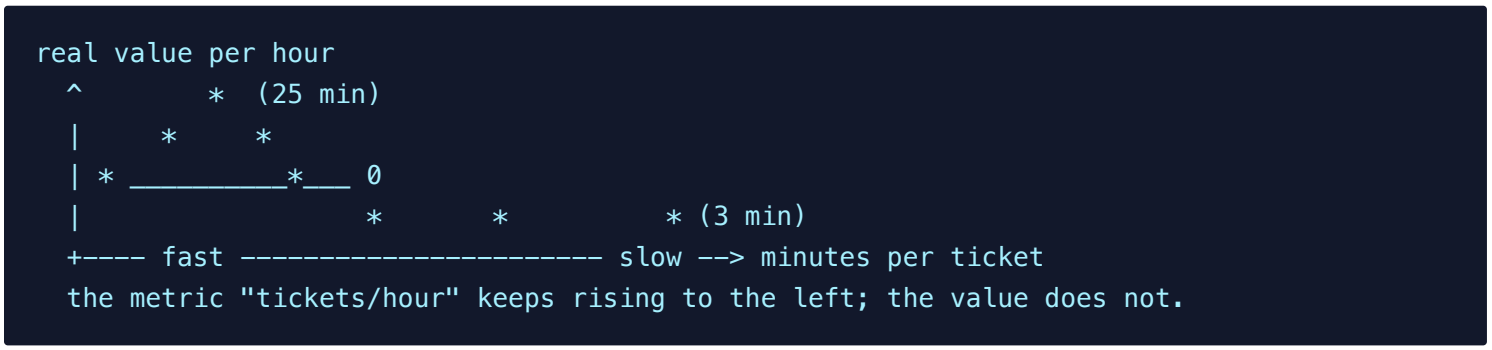
```
mins = np.array([30, 25, 20, 12, 6, 3])
p = 1 - np.exp(-mins / 8); tph = 60 / mins
print(pd.DataFrame({"minutes_per_ticket": mins, "tickets_per_hour": tph, "P_resolved":
p.round(3),
                    "value_Rs_per_hour": (tph * (100 * p - 400 * (1 -
p))).round(0)}).to_string(index=False))
```

```
 minutes_per_ticket  tickets_per_hour  P_resolved  value_Rs_per_hour
                 30               2.0       0.976              176.0
                 25               2.4       0.956              187.0
                 20               3.0       0.918              177.0
                 12               5.0       0.777              -58.0
                  6              10.0       0.528            -1362.0
                  3              20.0       0.313            -4873.0
```

**Problem 4.2.3** Read the table: which handling time maximises the *target metric* and which maximises the *real value*?

| | |
|---|---|
| **Step 1** | The target metric, tickets per hour, is largest at 3 minutes (20 per hour). |
| **Step 2** | The real value per hour peaks at 25 minutes (Rs 187) and is hugely negative at 3 minutes (Rs -4,873). |
| **Answer** | Optimising the metric (3 minutes) loses Rs 4,873 per hour; the best real choice is about 25 minutes. |
| **Check** | At 25 min: 2.4 × (100 × 0.956 - 400 × 0.044) = 2.4 × (95.6 - 17.6) = 2.4 × 78 = 187 ✓. |
| **Why it matters** | Any KPI used for bonuses will be gamed. Pair every target metric with a *counter-metric* (guardrail: resolution rate, reopen rate). The same happens to models: a recommender that optimises clicks learns clickbait. |

**The picture**




**Connection to the real job.** In any "we improved X after we targeted the worst" story, ask for the control group. In any "99% accurate" claim, ask for the base rate. In any KPI for staff or a model, ask what could be gamed.

**Applications**

- Operations: store or agent coaching programs (RTM).
- Fraud/medical screening: base rates and precision@capacity.
- Product: engagement metrics (time spent) that harm users.

- Operations: optimising the average delivery time while the slowest 1% of deliveries get worse.

| COMMON MISTAKE | Evaluating an intervention only on the "worst performers". A big improvement in an extreme group is the normal signature of RTM. Always keep an untreated comparison group. |
|---|---|

**Check yourself**

1. Stores in the bottom decile improved without help. Why?

Answer: regression to the mean; part of their low score was bad luck.

1. A 99% accurate test on a 0.1% condition; PPV?

Answer: about 9%.

1. How do you protect a KPI from Goodhart's law?

Answer: add counter-metrics and watch the real outcome.

## 4.3 Many Tests, Peeking at the Validation Set, and Look-Ahead

| INTUITION | Buy 20 lottery tickets and a win seems less surprising than with 1. If you test 20 metrics, one of them will "win" by luck. If you try 200 models and keep the one with the best validation score, it will look good simply because you picked the maximum of 200 random numbers. |
|---|---|

**Multiple comparisons.** Each test at the 5% level has a 5% chance of a false positive when nothing is going on. With 20 independent tests, the chance of at least one false positive is `1 - 0.95^20 = 0.642`, about 64%. Corrections:

- **Bonferroni**: use threshold `0.05 / m` for m tests (strict, safe).
- **Benjamini-Hochberg (BH)**: control the *false discovery rate* (FDR), the expected share of false findings among your discoveries. Sort p-values `p(1) <= ... <= p(m)`; find the largest k with `p(k) <= (k/m) × 0.05`; reject the first k.

**Problem 4.3.1** Run an A/A test (both groups from the same distribution) on 20 metrics with 2,000 users per group. How many look significant?

```
from scipy import stats
rng = np.random.default_rng(0)
ps = np.array([stats.ttest_ind(rng.normal(0, 1, 2000), rng.normal(0, 1, 2000)).pvalue for
_ in range(20)])
print("metrics with p<0.05 in an A/A test:", int((ps < 0.05).sum()), " min p",
ps.min().round(4))
print("Bonferroni threshold", 0.05 / 20, " significant:", int((ps < 0.05 / 20).sum()))
srt = np.sort(ps); m = len(ps)
ok = srt <= (np.arange(1, m + 1) / m) * 0.05     # BH rule: p(k) <= (k/m) x 0.05
print("BH rejections:", int(ok.sum()))
print("P(at least one false positive of 20 at 0.05):", round(1 - 0.95**20, 3))
```

```
metrics with p<0.05 in an A/A test: 1  min p 0.0297
Bonferroni threshold 0.0025   significant: 0
BH rejections: 0
P(at least one false positive of 20 at 0.05): 0.642
```

- **Step 1** One metric (the smallest p = 0.0297) falls under 0.05.
- **Step 2** Bonferroni threshold is 0.05 / 20 = 0.0025. 0.0297 > 0.0025, so no metric is declared significant.
- **Step 3** BH: the smallest p must be below (1/20) × 0.05 = 0.0025; it is not, so no rejections.
- **Answer** Without correction you would report one "winning" metric that does not exist; with Bonferroni or BH you correctly report nothing.
- **Check** All 20 differences are pure noise by construction, so the right number of discoveries is 0.
- **Why it matters** In experiment reports with 15 metrics and 6 segments, 90 comparisons produce about 4 to 5 fake wins. Pre-register one primary metric (Chapter 7) and treat the rest as exploration. The *garden of forking paths* (choosing the analysis after seeing the data) is the same problem in disguise: p-hacking.

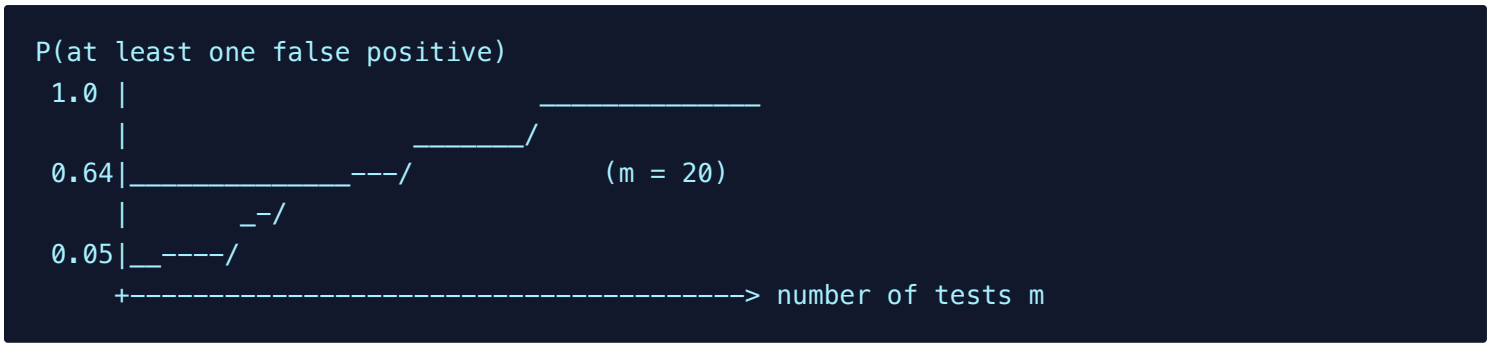
**Overfitting to the validation set.** Each time you look at the validation score and change something, the set leaks into your choices. We score 200 *random* models (no skill) on a validation set of 300 rows.

**Problem 4.3.2** What does the best validation AUC look like, and what happens on fresh data?

| | |
|---|---|
| **Step 1** | 200 random scorers, validation AUC for each. |
| **Step 2** | Pick the maximum; test it on 3,000 fresh rows. |
| **Answer** | Best validation AUC is 0.566 (the mean of all 200 is 0.498), but fresh data gives 0.499, i.e. pure chance. |
| **Check** | True skill is zero for every model, so the expected fresh AUC is 0.5 ✓. |
| **Why it matters** | A winner among many tries looks better than it is. Keep a final test set you touch once; use cross-validation or time-based folds for tuning; log how many things you tried. |

**Look-ahead bias and leakage in time data** (the Chapters 2 and 9 theme): any feature, label or split that uses information from after the prediction time. Time-based data must be split by time, never randomly.

### The picture




**Connection to the real job.** Experiment platforms apply these corrections for you; in your own analysis you must do it. In interviews: "You ran 20 metrics and one is significant. Ship?"

### Applications

- Experimentation: many metrics and segments per test.
- Genomics and finance: thousands of hypotheses, FDR control is standard.
- ML: model selection and hyperparameter search.

| | |
|---|---|
| **COMMON MISTAKE** | Slicing a failed experiment into 30 segments until one is significant and then reporting that segment as "the story". Treat such a finding as a new hypothesis for a new test. |

### Check yourself

1. What is the chance of at least one false positive in 10 independent tests at 0.05?

Answer: $1 - 0.95^{10} = 0.401$.

1. Bonferroni threshold for 50 tests?

Answer: 0.05 / 50 = 0.001.

1. Why keep an untouched test set?

Answer: repeated validation peeking makes validation scores optimistic.

## 4.4 Uncertainty: Intervals, Effect Size, Bayesian Thinking

**INTUITION**

A delivery firm reports average delivery time 480 minutes. Would you sign a contract at "480 minutes" if a re-check could give 520? You check three weeks and report "480 give or take 25". That "give or take" is uncertainty. A number without it is a guess dressed as a fact.

**Words you need**

| Word | Meaning | Example |
|---|---|---|
| Standard error (SE) | Typical size of the error of an estimate | SE of a rate: `sqrt(p(1–p)/n)` |
| Confidence interval (CI) | A range that, over many repeats, contains the true value 95% of the time | 11 to 17.5 points |
| Bootstrap | Re-draw rows with replacement many times to see how the estimate wobbles | 2,000 re-samples |
| Prediction interval (PI) | Range for a *new single* outcome | Next customer's value |
| Effect size | How big the difference is | 14 points, Cohen's d |
| Credible interval | Bayesian range: given data and prior, 95% probability inside | 0.18 to 0.34 |

**Problem 4.4.1** Pro customers churn at 17.0% (179 of 1,053); free at 31.3% (472 of 1,509). Give the difference with a 95% CI, analytically and with the bootstrap.

| | |
|---|---|
| **Formula** | `SE = sqrt(p₀(1−p₀)/n₀ + p₁(1−p₁)/n₁)`, CI = diff ± 1.96 × SE |
| **Plug in** | $p_0$ = 0.313, $n_0$ = 1,509, $p_1$ = 0.170, $n_1$ = 1,053 |
| **Compute** | diff = 0.313 - 0.170 = 0.143. SE = sqrt(0.313 × 0.687 / 1509 + 0.170 × 0.830 / 1053) = sqrt(0.0001425 + 0.0001340) = sqrt(0.0002765) = 0.0166. CI = 0.143 ± 1.96 × 0.0166 = 0.143 ± 0.0326 = (0.110, 0.175). |
| **Answer** | Free customers churn 14.3 points more than pro, 95% CI 11.0 to 17.5 points. The bootstrap (2,000 re-draws) gives (0.110, 0.176), nearly identical. |
| **Check** | Code printed `diff 0.143 SE 0.0166 95% CI 0.11 0.175` and `bootstrap 95% CI [0.11 0.176]` ✓. |
| **Why it matters** | In words for a manager: "Pro users churn about 14 points less, and we are fairly sure the true gap is between 11 and 18 points." Note: this is a *difference between groups*, not proof that giving someone Pro lowers churn (confounding, Chapter 8). |

The bootstrap is useful when no neat formula exists (a median, a ratio, AUC): resample rows with replacement, recompute the statistic 1,000+ times, take the 2.5th and 97.5th percentiles.

**Small data and Bayesian thinking.** A new segment has 3 churners among 10 customers.

**Problem 4.4.2** Compare the classical CI with a Bayesian credible interval.

| | |
|---|---|
| **Step 1** | Classical (normal approximation): 0.30 ± 1.96 × sqrt(0.3 × 0.7 / 10) = 0.30 ± 0.284 → (0.016, 0.584). Very wide and unreliable at n = 10. |
| **Step 2** | Bayesian with a flat prior Beta(1, 1): posterior is Beta(1 + 3, 1 + 7); mean 0.333; 95% credible interval (0.109, 0.610). |
| **Step 3** | Bayesian with an informative prior from history (average churn 25%, worth 100 customers: Beta(25, 75)): posterior Beta(28, 82); mean 0.255; interval (0.178, 0.340). |
| **Answer** | With almost no data, the flat prior says "somewhere 11% to 61%"; the history-informed prior says "about 25.5%, between 18% and 34%". |
| **Check** | Beta posterior mean = (a + k) / (a + b + n): flat: 4/12 = 0.333 ✓; informative: 28/110 = 0.255 ✓. |
| **Why it matters** | A *prior* is what you believed before seeing the data; Bayes' rule updates it with the data into a *posterior*. With small data, a sensible prior stabilises estimates and avoids silly conclusions ("segment X churns 30%!"). Bayesian methods also suit sequential decisions (Chapter 7). The prior must be defensible; document it. |

**Effect size vs significance.** With 2 million users per group, a difference of 0.18 points on a scale with sd 15 gives p = 8e-33, "highly significant", while the standardised effect (Cohen's d = 0.012) is tiny. Statistical significance says "not noise"; practical significance asks "does it matter in rupees?". Always report the effect size with its interval and translate it into business units.

**Confidence interval vs prediction interval.** From a straight-line fit on 200 points (true line y = 2 + 3x, noise sd 4), the prediction at x = 5 is 16.65. The CI for the *average* outcome at x = 5 has half-width 0.56; the PI for a *single new* outcome has half-width 7.99. The average is well known, an individual is not: forecasts for single days, orders or customers need the wider PI.

**Reporting uncertainty to non-technical audiences.** Use frequencies and ranges, not jargon:

| Instead of | Say |
|---|---|
| "p = 0.03" | "If there were really no difference, we would see a gap this big only 3 times in 100." |
| "95% CI 11 to 17.5" | "Our best estimate is 14 points; plausible range 11 to 18." |
| "AUC 0.88" | "If we pick one churner and one non-churner at random, the model ranks the churner higher 88 times in 100." |

| Instead of | Say |
|---|---|
| "Not significant" | "We could not tell; the data allow anything from a small loss to a small gain. We need more data." |

**The picture**

```
          |<------ 95% CI for the average ------>|
          |   |<------------ 95% prediction interval for one customer --------->|
  value:  ----[====(===*===)====]-----------------------------------------------
                    estimate
```

**Connection to the real job.** Every dashboard number, forecast and A/B test result should come with a range. Senior data scientists are trusted because they say "we are about 90% sure, and here is what would change my mind".

**Applications**

- Experimentation: CI on the lift decides ship or not.
- Forecasting: prediction intervals decide safety stock and staffing.
- Finance and credit: risk ranges, not point estimates.
- Operations: an interval on the 95th-percentile delivery time across weeks before saying "it got worse".

**COMMON MISTAKE**

Reading a 95% CI as "there is a 95% chance the true value is inside *this* interval". The frequentist meaning concerns the procedure over many repeats. The Bayesian credible interval does have the "95% probability" meaning, given the prior. And never confuse a CI for the mean with a PI for individuals.

**Check yourself**

1. Which is wider, a CI for the mean or a PI for one new value?

Answer: the prediction interval.

1. What does a 99.99% significant but tiny effect tell you?

Answer: it is real but may not matter; check the effect size in business terms.

1. With 3 events in 10 trials why use a prior?

Answer: the data alone is too weak; a prior stabilises the estimate.

# Chapter recap

- Correlation is not causation: name the confounder, adjust, or randomise (partial correlation of ice cream and drownings fell from 0.53 to about 0).
- Simpson's paradox: B beat A overall (33.5% vs 20%) while A beat B on each device (10% vs 6%, 40% vs 35%). Which view is right depends on the causal story.
- Selection and survivorship bias: survivors average +4.1% a year with zero skill; Berkson: selection makes a fake link of -0.67.
- Regression to the mean: the worst 10% rose from 73.7 to 86.7 with no action; always keep a control group.
- Base rates: a "99% accurate" test on a 0.1% condition has PPV 9%.
- Goodhart: a KPI that becomes a target gets gamed; add guardrail metrics.
- Multiple tests: 20 tests give a 64% chance of a false win; use Bonferroni or BH and one primary metric.
- Validation peeking: best of 200 random models scored 0.566 on validation and 0.499 on fresh data.
- Report uncertainty: CI, bootstrap, prediction interval, effect size, credible interval.

| Idea | Formula | Example |
|---|---|---|
| Rate SE | `sqrt(p(1−p)/n)` | 0.009 for p = 0.25, n = 2,300 |
| Two-rate CI | `diff ± 1.96 × sqrt(p₀q₀/n₀ + p₁q₁/n₁)` | 0.143 ± 0.033 |
| RTM | `mean + r × (x₁ − mean)` | 100 + 0.5 × (73.7 - 100) = 86.85 |
| PPV | `TP / (TP + FP)` | 99 / 1,098 = 0.090 |
| Family error | `1 − (1 − α)^m` | $1 - 0.95^{20} = 0.642$ |
| Bonferroni | `α / m` | 0.05 / 20 = 0.0025 |
| Beta update | `Beta(a + k, b + n − k)` | Beta(1,1) + 3/10 → Beta(4, 8) |

# Practice set

**Problem 4.P.1** (easy) 30 of 200 visitors buy. Find the rate and its SE. **Answer:** 30/200 = 0.15; SE = sqrt(0.15 × 0.85 / 200) = sqrt(0.0006375) = 0.0252.

**Problem 4.P.2** (easy) 12 tests at the 5% level, independent, no real effects. Probability of at least one false positive? **Answer:** $1 - 0.95^{12} = 1 - 0.5404 = 0.4596$, about 46%.

**Problem 4.P.3** (easy) A group of 1,000 stores: mean 100, r = 0.4. A store scored 70 in month 1. Expected month-2 score without action? **Answer:** 100 + 0.4 × (70 - 100) = 88.

**Problem 4.P.4** (medium) Prevalence 2%, sensitivity 90%, specificity 95%. PPV? **Answer:** Per 10,000: cases 200, found 180; non-cases 9,800, false alarms 490. PPV = 180 / 670 = 0.269, about 27%.

**Problem 4.P.5** (medium) Campaign X: 30/100 mobile, 10/300 desktop; Campaign Y: 4/20 mobile, 120/380 desktop. Rates by device and overall? **Answer:** X: mobile 30%, desktop 3.3%, overall 40/400 = 10%. Y: mobile 20%, desktop 31.6%, overall 124/400 = 31%. Y wins overall and on desktop; X wins on mobile. Always look at the mix.

**Problem 4.P.6** (medium) What is the output? `np.corrcoef(a[adm], b[adm])[0,1]` when a, b are independent and `adm = (a + b) > 1.5`. **Answer:** A clearly negative number (we measured -0.67), caused by selection.

**Problem 4.P.7** (medium) Find the bug: "We improved the 100 worst-performing agents with training. Their score rose 8 points, so training works." What is missing? **Answer:** A control group of equally poor agents without training; regression to the mean may explain the rise.

**Problem 4.P.8** (hard) Using BH at 5% for p-values 0.001, 0.008, 0.02, 0.04, 0.30 (m = 5): which are rejected? **Answer:** Thresholds (k/5) × 0.05 = 0.01, 0.02, 0.03, 0.04, 0.05. Compare sorted p: 0.001 <= 0.01 yes; 0.008 <= 0.02 yes; 0.02 <= 0.03 yes; 0.04 <= 0.04 yes; 0.30 <= 0.05 no. Largest k = 4, so reject the first four.

**Problem 4.P.9** (interview) "Explain a p-value to a manager." **Answer:** "Imagine nothing changed. The p-value is how often random luck alone would give us a difference at least this large. A small value, like 3 in 100, means luck is an unlikely explanation. It does not tell how big or how important the difference is."

**Problem 4.P.10** (interview) "Users who use our new feature retain 20 points better. Should we push everyone to use it?" **Answer:** Not yet. This is correlation; engaged users naturally use more features (confounding,

selection). Run a randomised test (promote the feature to a random half), or use causal methods with careful adjustment. Compare retention in the promoted vs not promoted groups, with guardrails.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, and English is my second language. Teach me Simpson's paradox, confounding, selection bias and survivorship bias with small numeric examples I can verify by hand. After each example, ask me to explain it in my own words and correct me. End with 3 real business cases where I must decide which view (overall or by group) is correct.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: "regression to the mean, base-rate neglect and Goodhart's law". Give me 8 graded exercises with real numbers (for example store performance, a rare-disease test, a support KPI). Do not give solutions yet. After I answer, grade me, show the calculation steps, and explain what business decision each result changes.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional new to data science. Topic: "multiple comparisons, p-hacking, Bonferroni and Benjamini-Hochberg, and validation-set overfitting". Quiz me one question at a time using small p-value lists and experiment scenarios. Wait for my answer, grade it, and explain. Ask 10 questions, increasing in difficulty.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Review my analysis below for statistical traps: confounding, selection bias, regression to the mean, base rates, multiple testing, effect size versus significance, and missing uncertainty (confidence or prediction intervals). List each problem, why it matters, and a concrete fix in simple English. My analysis: [PASTE HERE]
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional preparing for data science interviews in India. Run a mock interview on statistical thinking: "explain a p-value to a manager", "users of feature X retain better, so what do you do", "99% accurate
```

```
model on rare events", "how do you report uncertainty". Ask one question at a time, score me out of 10, give a model answer and a follow-up. Do 8 rounds.
```

CHAPTER 5

# FEATURE ENGINEERING I: TRANSFORMATIONS, ENCODING, DATES, TEXT AND GEO

*How to turn raw columns into numbers a model can learn from, and how to do it without cheating.*

On tabular data, good features usually beat a fancier algorithm. A model sees only numbers. If you give it a skewed amount column, a text field, a pincode with 400 levels and a timestamp, it cannot use them as they are. Feature engineering is the craft of turning each of them into numbers that carry the signal. In this chapter you will learn the core transformations for numbers, the encodings for categories (including the dangerous target encoding and how to do it safely), the tricks for dates, text and locations, and the habit of testing each idea instead of trusting it. By the end you will be able to take any raw table, list what a human expert would look at, and turn each answer into a feature computed only from data available at prediction time. (Chapter 6 continues with behavioural features, selection and pipelines.)

**Running example.** The churn table of Chapter 3 (2,562 active customers at T = 2025-01-01, churn rate 25.4%). The helper `features(T)` from Chapter 3 gives `orders_90d`, `spend_90d`, `avg_order`, `recency_days`, `tenure_days`. We fit a plain logistic regression (a basic classifier that adds up weighted features; see `study/ml/`) on a stratified 70/30 split with seed 0, and report AUC on the 30% test part. **Method.** Ask: "what would a human expert look at to make this decision?" Turn each answer into a number. Then test whether it helps.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score

d = features(pd.Timestamp("2025-01-01"))       # helper from Chapter 3
tr, te = train_test_split(d, test_size=0.3, random_state=0, stratify=d.churn)
def fit_auc(Xtr, Xte):
    m = LogisticRegression(max_iter=3000).fit(Xtr, tr.churn)
    return round(roc_auc_score(te.churn, m.predict_proba(Xte)[:, 1]), 3)
base = ["orders_90d", "recency_days"]
print("AUC base (orders_90d, recency):", fit_auc(tr[base], te[base]))
```

```
AUC base (orders_90d, recency): 0.883
```

Every idea below is compared against this 0.883.

# 5.1 Numeric Transformations

**INTUITION**

Imagine a ruler where every centimetre in the first metre is a real centimetre, but each metre after is squeezed so that 1 cm means 100 times more. Most of your data is in the first metre, and a few huge values are far away. A log transform is that squeezed ruler: it gives rich customers and normal customers a comparable scale.

**Why transform?** Many models (linear, distance-based, neural) work better when values are not heavily skewed. A few giant values can dominate a model. Tree models are less sensitive to monotone transforms (they only compare "greater than"), but still benefit from ratios and good bins.

**The tools**

| Tool | What it does | Formula | When |
|---|---|---|---|
| log1p | Compress a long right tail; works at 0 | `log(1 + x)` | money, counts, time |
| sqrt | Milder compression | `√x` | counts |
| Box-Cox / Yeo-Johnson | Learns the best power for you | `((x+1)^λ − 1)/λ` (Yeo-Johnson for x ≥ 0) | when log is too strong or too weak; Yeo-Johnson also allows zero and negatives |
| Clipping (winsorising) | Cap extremes at percentiles | `min(max(x, p1), p99)` | outliers that are real but extreme |
| Binning | Cut into groups | equal-width, quantile, or supervised | non-linear effects, interpretability |
| Scaling | Make units comparable | `(x − mean)/sd` | linear, distance, neural models (not needed for trees) |
| Ratios and differences | Compare two quantities | `a / b`, `a − b` | domain knowledge |

**Worked problem**

**Problem 5.1.1** Compute `log1p` and `sqrt` for the values 0, 9, 99, 999, and say which compresses the tail more.

| | |
|---|---|
| **Formula** | `log1p(x) = ln(1 + x)`, `sqrt(x) = √x` |
| **Plug in** | x = 0, 9, 99, 999 |
| **Compute** | log1p: ln(1) = 0, ln(10) = 2.303, ln(100) = 4.605, ln(1000) = 6.908. sqrt: 0, 3, 9.95, 31.61. |
| **Answer** | Log turns the range 0 to 999 into 0 to 6.9 (a 1000-fold spread squeezed to about 7 units); sqrt gives 0 to 31.6. Log compresses more. |
| **Check** | `ln(1 + 999) = ln(1000) = 6.908` because e^6.908 = 1000. |
| **Why it matters** | A model that adds numbers would treat a customer spending Rs 999 as 100 times a customer spending Rs 9 under the raw scale, but only 3 times under the log scale. Which is right depends on the business: log means "each doubling matters equally". |

**Run on the real data.** Before you run it, predict: which of log1p, sqrt and Yeo-Johnson removes the skew of `spend_90d` best?

```
from sklearn.preprocessing import PowerTransformer
print("skew spend:", round(d.spend_90d.skew(), 2), "log1p:",
round(np.log1p(d.spend_90d).skew(), 2),
      "sqrt:", round(np.sqrt(d.spend_90d).skew(), 2))
pt = PowerTransformer(method="yeo-johnson").fit(tr[["spend_90d"]])
print("yeo-johnson skew:",
round(float(pd.Series(pt.transform(d[["spend_90d"]]).ravel()).skew()), 2),
      " lambda", round(pt.lambdas_[0], 3))
lo, hi = d.spend_90d.quantile([0.01, 0.99]); print("1%/99% quantile:", round(lo),
round(hi), " max", round(d.spend_90d.max()))
clipped = d.spend_90d.clip(lo, hi)
print("clipped max", round(clipped.max()), " mean before/after",
round(d.spend_90d.mean()), round(clipped.mean()))
```

```
skew spend: 2.17 log1p: -0.43 sqrt: 0.78
yeo-johnson skew: -0.03  lambda 0.161
1%/99% quantile: 207 24601  max 41357
clipped max 24601  mean before/after 5261 5193
```

Line by line: `skew()` is 0 for a symmetric shape. The raw skew is 2.17 (long tail). `sqrt` brings it to 0.78 (still skewed), `log1p` overshoots to -0.43, and Yeo-Johnson, which learned a power `λ = 0.161` (close to 0, which is the log case) from the training rows only, gives -0.03. `quantile([0.01, 0.99])` finds the 1st and 99th percentile; `clip` caps values at them: the max falls from 41,357 to 24,601 and the mean from 5,261 to 5,193.

**Worked problem**

**Problem 5.1.2** Apply the Yeo-Johnson formula by hand for λ = 0.161 and x = 3,512.9 (the median spend), using `((x + 1)^λ − 1) / λ`.

| | |
|---|---|
| **Formula** | `((x + 1)^λ − 1) / λ` (for x ≥ 0 and λ ≠ 0) |
| **Plug in** | `((3513.9)^0.161 − 1) / 0.161` |
| **Compute** | `ln(3513.9) = 8.164`; `8.164 × 0.161 = 1.3144`; `e^1.3144 = 3.722`; `(3.722 − 1) / 0.161 = 16.9` |
| **Answer** | The median spend becomes about 16.9 on the transformed scale (a number the model sees instead of 3,513). (Scikit-learn then also standardises to mean 0, so you would see a different final number.) |
| **Check** | With λ near 0 the formula tends to `ln(x + 1)` = 8.16, which is the log case; 16.9 is the same idea on a stretched scale. |
| **Why it matters** | Using the same λ (learned on training data only) for test data keeps the pipeline honest. If you fit λ on all data you leak a tiny bit of test information. |

**Does it help the model?** We test binning, log, and a *ratio* feature. The ratio compares recency with the customer's own normal gap between orders: if a customer usually orders every 10 days (9 orders in 90 days) and has been silent for 30 days, the ratio is `30 / (90/9) = 3.0` ("three missed cycles").

```
from sklearn.preprocessing import KBinsDiscretizer
kb = KBinsDiscretizer(n_bins=5, encode="onehot-dense",
strategy="quantile").fit(tr[["recency_days"]])
print("quantile bin edges recency:", kb.bin_edges_[0].round(0))
Xb_tr = np.c_[tr[["orders_90d"]].values, kb.transform(tr[["recency_days"]])]
Xb_te = np.c_[te[["orders_90d"]].values, kb.transform(te[["recency_days"]])]
print("AUC with binned recency   :", fit_auc(Xb_tr, Xb_te))
for df in (tr, te):
    df["log_rec"] = np.log1p(df.recency_days)
    df["rec_ratio"] = df.recency_days / (90 / df.orders_90d)    # recency compared with
the usual gap
print("AUC with log recency      :", fit_auc(tr[["orders_90d", "log_rec"]],
te[["orders_90d", "log_rec"]]))
print("AUC + ratio feature (recency / usual gap):", fit_auc(tr[base + ["rec_ratio"]],
te[base + ["rec_ratio"]]))
```

```
quantile bin edges recency: [ 1.  3.  7. 14. 34. 90.]
AUC with binned recency   : 0.871
AUC with log recency      : 0.88
AUC + ratio feature (recency / usual gap): 0.889
```

Read it: the quantile bins put roughly 20% of customers in each of five groups (edges at 1, 3, 7, 14, 34, 90 days). Binning (0.871) and log (0.880) did *not* beat the raw recency (0.883), because logistic regression on a single raw column already catches the main pattern. The ratio feature, which encodes domain knowledge about each customer's own rhythm, gave the only gain: 0.883 to 0.889. This is the typical pattern: **domain ratios beat generic transforms**.

**The picture**

```
raw spend:  |#######.......  .   .      .           .     |  long right tail
log spend:  |    ..#######.....          |                    roughly bell-shaped
clipped:    |#######.......  .   .  [cap]                |  tail cut at the 99th
percentile
```

**Connection to the real job.** In credit scoring you bin income and age on purpose (so a scorecard is readable, Chapter 11). In forecasting you take the log of sales so that percentage changes become additive. In logistics, delivery times are right-skewed (a few very late parcels), so you analyse them on a log scale or by percentile.

**Applications**

- Data Science: log of amounts for linear models; ratios such as debt-to-income.
- Machine Learning: scaling for neural nets and k-nearest neighbours.
- Finance/BFSI: binning for scorecards, winsorising extreme transactions.
- Analytics: percent change and ratios in KPI design.

| COMMON MISTAKE | Fitting the transformer (mean, λ, quantile cut-offs) on the whole dataset before splitting. The test set must not influence any learned number. Fit on training rows, apply to test rows; a scikit-learn `Pipeline` (Chapter 6) does this automatically. |
|---|---|

**Check yourself**

1. Why `log1p` and not `log`?

Answer: `log(0)` is undefined; `log1p(0) = 0`.

1. Do tree models need scaling?

Answer: no; they only compare thresholds.

1. A 99th-percentile clip changes a few values. What does it protect?

Answer: the model from being dominated by a few extreme values.

## 5.2 Encoding Categories (and Target Encoding Done Safely)

**INTUITION**

A model cannot read "Pune". You must turn the city into numbers. Giving Pune the number 5 and Delhi the number 2 says "Pune is 2.5 times Delhi", which is nonsense. The art of encoding is to give the model the information in the category without inventing an order that is not there.

**Words you need**

| Word | Meaning | Example |
|---|---|---|
| Cardinality | Number of distinct categories | `city`: 6; `pincode`: about 400 |
| One-hot | One 0/1 column per category | `plan_pro` = 1 for pro |
| Ordinal | Integer codes for a real order | small = 1, medium = 2, large = 3 |
| Frequency encoding | Replace by how common the category is | Pune = 10.3% of rows |
| Target encoding | Replace by the average target in that category | pincode 560001 churns 31% |
| Hashing | Map categories into a fixed number of columns by a hash function | 400 pincodes into 16 columns |

**Choosing an encoding**

| Cardinality | Order exists? | Good choice |
|---|---|---|
| 2 to about 15 | no | one-hot |
| any | yes (sizes, grades) | ordinal |

| Cardinality | Order exists? | Good choice |
|---|---|---|
| 15 to thousands | no | frequency, target encoding (with care), hashing, or embeddings |
| huge (user_id) | no | usually do not use the id; use aggregates (Chapter 6); or learned embeddings |

For rare categories, group everything below a count (for example under 5 rows) into "other".

**Run the simple ones.**

```
print(d.city.value_counts().to_dict())
oh = pd.get_dummies(d[["plan", "channel"]], drop_first=True); print("one-hot columns:",
list(oh.columns))
freq = d.city.map(d.city.value_counts(normalize=True)); print("frequency encoding Pune:",
round(freq[d.city == "Pune"].iloc[0], 3))
```

```
{'Other': 526, 'Mumbai': 525, 'Bengaluru': 504, 'Delhi': 497, 'Pune': 264, 'Chennai': 246}
one-hot columns: ['plan_pro', 'channel_organic', 'channel_referral']
frequency encoding Pune: 0.103
```

`drop_first=True` removes one column per feature ( `plan_free` , `channel_ads` ): it is implied when the others are 0, and keeping all would make columns perfectly dependent. Pune frequency is 264 / 2,562 = 0.103.

**Target encoding, and why it can cheat.** Replace each category by the mean of the target in that category. It is powerful for high-cardinality columns, but dangerous: for a category with 3 rows, the mean *contains the row's own label*. The model will learn "the encoded value tells the label" on training data, which does not hold on new data. This is leakage through the encoder.

We create a `pincode` column with about 400 levels and *no real relationship with churn* (random). A safe method must give test AUC about 0.5.

**Problem 5.2.1** Compare three encoders: naive, out-of-fold with smoothing, and scikit-learn's `TargetEncoder` (which uses internal cross-fitting). We feed each encoded column alone into a logistic regression.

| | |
|---|---|
| **Step 1** | Naive: mean churn per pincode computed on the training rows, applied to the same training rows. |
| **Step 2** | Out-of-fold (OOF): split training rows into 5 folds; encode each fold using the means from the other 4 folds, so a row never sees its own label. Add *smoothing*: `encoded = (sum + m × global_mean) / (count + m)` with `m = 20`, which pulls small categories toward the global mean. |
| **Step 3** | Compare train and test AUC. |

```
rng = np.random.default_rng(5)
d["pincode"] = rng.integers(0, 400, len(d)).astype(str)          # random: no signal
tr, te = train_test_split(d, test_size=0.3, random_state=0, stratify=d.churn)
means = tr.groupby("pincode").churn.mean(); gm = tr.churn.mean()
tr_naive = tr.pincode.map(means); te_naive = te.pincode.map(means).fillna(gm)

from sklearn.model_selection import KFold
def oof_te(train, test, col="pincode", k=5, smooth=20, seed=0):
    gm = train.churn.mean(); out = pd.Series(index=train.index, dtype=float)
    for a, b in KFold(k, shuffle=True, random_state=seed).split(train):
        s = train.iloc[a].groupby(col).churn.agg(["sum", "count"])
        enc = (s["sum"] + smooth * gm) / (s["count"] + smooth)
        out.iloc[b] = train.iloc[b][col].map(enc).fillna(gm).values
    s = train.groupby(col).churn.agg(["sum", "count"]); enc = (s["sum"] + smooth * gm) /
(s["count"] + smooth)
    return out, test[col].map(enc).fillna(gm)

def single_auc(a_tr, a_te):
    m = LogisticRegression(max_iter=1000).fit(np.c_[a_tr], tr.churn)
    return (round(roc_auc_score(tr.churn, m.predict_proba(np.c_[a_tr])[:, 1]), 3),
            round(roc_auc_score(te.churn, m.predict_proba(np.c_[a_te])[:, 1]), 3))
print("levels:", d.pincode.nunique(), " mean rows per level (train):", round(len(tr) /
400, 1))
print("naive TE  train AUC, test AUC:", single_auc(tr_naive, te_naive))
a_tr, a_te = oof_te(tr, te)
print("OOF+smooth TE train AUC, test AUC:", single_auc(a_tr, a_te))
from sklearn.preprocessing import TargetEncoder
te_sk = TargetEncoder(target_type="binary", random_state=0)
print("sklearn TargetEncoder train, test AUC:",
      single_auc(te_sk.fit_transform(tr[["pincode"]], tr.churn).ravel(),
te_sk.transform(te[["pincode"]]).ravel()))
```

```
levels: 396  mean rows per level (train): 4.5
naive TE  train AUC, test AUC: (0.812, 0.476)
OOF+smooth TE train AUC, test AUC: (0.531, 0.469)
sklearn TargetEncoder train, test AUC: (0.529, 0.474)
```

- **Answer** The naive encoder reports train AUC 0.812 for a column that contains *no signal*, and 0.476 on test (no better than a coin flip). The out-of-fold and scikit-learn encoders give about 0.53 on train and 0.47 on test: honest, "no signal here".
- **Check** Since the column was random, true AUC must be 0.5; test AUC near 0.47 is noise (test set has only about 195 churners, so standard error of AUC is about 0.02 to 0.03). The naive train AUC of 0.81 is impossible for random data without cheating.
- **Why it matters** In a notebook with a random split, a naive target encoder on a real high-cardinality column (merchant id, pincode) can inflate offline AUC and hurt production. Rule: *always encode the training rows out-of-fold; encode validation/test/serving rows with means from all training rows.*

**Smoothing by hand.** A pincode has 3 training rows with churn [1, 0, 1]. Raw mean = 2/3 = 0.667. With global mean 0.254 and m = 20: `(2 + 20 × 0.254) / (3 + 20) = (2 + 5.08) / 23 = 0.308`. The small group is pulled almost to the global rate because 3 rows is weak evidence. A pincode with 300 rows would barely move.

**Hashing and learned embeddings.** `FeatureHasher(n_features=16)` maps each pincode to one of 16 columns by a hash, with no lookup table and a fixed size; different pincodes can collide, which is accepted. *Embeddings* are small learned vectors for each category (used in neural networks and recommenders, Chapter 10 and `study/ml/`).

**The picture**

```
Naive target encoding (category with 3 rows)   row's own label inside its own feature
   label y=1  -> encoded = mean(1,0,1) = 0.667   the model "sees" y

Out-of-fold:  rows of fold 3 are encoded with means from folds 1,2,4,5 only
   fold1 fold2 fold3 fold4 fold5
          |<- statistics come from the other folds ->|
```

**Connection to the real job.** Credit and fraud models have merchant, device and pincode columns with thousands of levels. Competitions are won and lost on safe target encoding. In interviews: "how do you encode a high-cardinality categorical?" has the OOF answer.

**Applications**

- Fraud: merchant and device categories (hashing, OOF target encoding).
- E-commerce: product category, brand (frequency, embeddings).
- Telecom: plan type, tower id.

- Healthcare: diagnosis codes (grouped, embedded).

| COMMON MISTAKE | Target encoding on the whole dataset before splitting, or encoding training rows with their own labels. Symptoms: huge train score, mediocre validation. A second mistake: ordinal-coding an unordered category (Pune = 5), which invents an order. |
|---|---|

**Check yourself**

1. Why `drop_first=True`?

Answer: one column is implied by the others; keeping all makes columns perfectly dependent.

1. What does smoothing do?

Answer: pulls small categories' encoded values toward the global mean.

1. How is a training row encoded in OOF target encoding?

Answer: using statistics from other folds, never its own label.

## 5.3 Dates, Cycles and Missing-Value Indicators

| INTUITION | A timestamp like "2025-03-04 20:15" carries many hidden facts: a Tuesday, evening, month end, near a festival. The model cannot see them in a raw date. You unpack the date into the facts a human would use. And hours are a clock, not a line: 11 pm and midnight are neighbours. |
|---|---|

**Typical date features**

| Feature | Code | Why |
|---|---|---|
| year, month, day of week, hour | `ts.dt.month`, `ts.dt.dayofweek`, `ts.dt.hour` | calendar effects |
| is_weekend, is_month_end, is_payday | boolean | salary-day shopping |
| is_holiday, days to next holiday | from a calendar list | Diwali, Holi, Eid, IPL season, sale events |
| days since an event | `(T - event_date).dt.days` | recency, tenure |

| Feature | Code | Why |
|---|---|---|
| time to next event | `(next_event − T).dt.days` | days until contract renewal |
| cyclical encoding | sin and cos of the angle | hour, day, month wrap around |

**Holiday features.** Keep a calendar table with date, name and region. Holidays differ across Indian states and religions, and festival dates move every year (check an official calendar; the dates below are an example, not a reference).

```
cal = pd.DataFrame({"date": pd.date_range("2024-10-28", "2024-11-03")})
hol = pd.to_datetime(["2024-10-31", "2024-11-01"])        # example festival dates; verify
with an official calendar
cal["dow"] = cal.date.dt.day_name().str[:3]
cal["is_weekend"] = (cal.date.dt.dayofweek >= 5).astype(int)
cal["is_holiday"] = cal.date.isin(hol).astype(int)
nxt = cal.date.map(lambda t: min([x for x in hol if x >= t], default=pd.NaT))
cal["days_to_next_holiday"] = (nxt − cal.date).dt.days
print(cal.to_string(index=False))
```

```
      date dow  is_weekend  is_holiday  days_to_next_holiday
2024-10-28 Mon           0           0                   3.0
2024-10-29 Tue           0           0                   2.0
2024-10-30 Wed           0           0                   1.0
2024-10-31 Thu           0           1                   0.0
2024-11-01 Fri           0           1                   0.0
2024-11-02 Sat           1           0                   NaN
2024-11-03 Sun           1           0                   NaN
```

(Read the columns by name: on 31 Oct, `is_weekend` is 0 and `is_holiday` is 1.) The countdown `3, 2, 1, 0` lets a model see demand building before a festival. After the last known holiday the value is `NaN`: you must supply the full calendar for the period you predict.

**Cyclical encoding.** Hours run 0 to 23, then wrap to 0. As a plain number, 23 and 0 are 23 apart, which is wrong. Map hour h to a point on a circle: `sin_h = sin(2π h / 24)`, `cos_h = cos(2π h / 24)`.

**Problem 5.3.1** Compute the distance between hour 23 and hour 0, and between hour 1 and hour 0, in raw form and on the circle.

| | |
|---|---|
| **Formula** | point(h) = (sin(2πh/24), cos(2πh/24)); distance = straight-line distance between points |
| **Plug in** | h = 23: (sin(−15°), cos(−15°)) = (−0.2588, 0.9659); h = 0: (0, 1); h = 1: (0.2588, 0.9659) |
| **Compute** | distance(23, 0) = √(0.2588² + (1 − 0.9659)²) = √(0.06699 + 0.00116) = √0.06815 = 0.261. The same for hour 1 vs hour 0 by symmetry. |
| **Answer** | On the circle, 23 and 0 are as close as 1 and 0 (0.261 each); as raw numbers they are 23 apart versus 1 apart. |
| **Check** | Chord for a 15° angle on a unit circle = `2 sin(7.5°) = 2 × 0.1305 = 0.261` ✓; the code printed 0.261 for both. |
| **Why it matters** | A model with raw hour cannot learn "midnight and 11 pm are alike". Cyclical features fix it for hours, weekdays and months. |

**Problem 5.3.2** Does it matter? We simulate two weeks of hourly order volume for an online shop that peaks at 8 pm, and fit a straight-line regression using (a) the raw hour and (b) sin/cos.

```
hrs = pd.date_range("2025-03-03", periods=24 * 14, freq="h")
h = hrs.hour.values
rng = np.random.default_rng(1)
load = 200 + 150 * np.sin(2 * np.pi * (h - 14) / 24) + rng.normal(0, 15, len(h))   # peak
at 20:00
ev = pd.DataFrame({"hour": h, "load": load})
ev["sin_h"] = np.sin(2 * np.pi * h / 24); ev["cos_h"] = np.cos(2 * np.pi * h / 24)
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
m1 = LinearRegression().fit(ev[["hour"]], ev.load); print("R2 linear on raw hour :",
round(r2_score(ev.load, m1.predict(ev[["hour"]])), 3))
m2 = LinearRegression().fit(ev[["sin_h", "cos_h"]], ev.load); print("R2 linear on sin/cos
:", round(r2_score(ev.load, m2.predict(ev[["sin_h", "cos_h"]])), 3))
```

```
R2 linear on raw hour : 0.388
R2 linear on sin/cos  : 0.984
```

- **Answer** A linear model explains 38.8% of the variance from the raw hour but 98.4% from sin and cos.
- **Check** $R^2$ (R-squared) is the share of variance explained, 1 is perfect. Because the true pattern is a sine wave plus noise (sd 15 against amplitude 150), 98% is the right ceiling.

- **Why it matters** The right encoding made a plain linear model nearly perfect. Tree models can approximate cycles by splitting, but they need more data to do so.

**Missing-value indicators.** Missingness is often informative (Chapter 3: age missing for 14.9% of churners vs 4.4% of stayers). Add `is_missing` next to the imputed value. We test with age only (median filled), using 5-fold cross-validation (the average of five train/test splits), because on a single split this weak signal is too noisy: on our seed-0 split the AUC went 0.563 to 0.466 by luck.

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
cv = StratifiedKFold(5, shuffle=True, random_state=0)
for cols in (["age_imp"], ["age_imp", "age_missing"], ["age_missing"]):
    s = cross_val_score(LogisticRegression(max_iter=2000), d[cols], d.churn, cv=cv,
scoring="roc_auc")
    print(cols, s.round(3), s.mean().round(3))
```

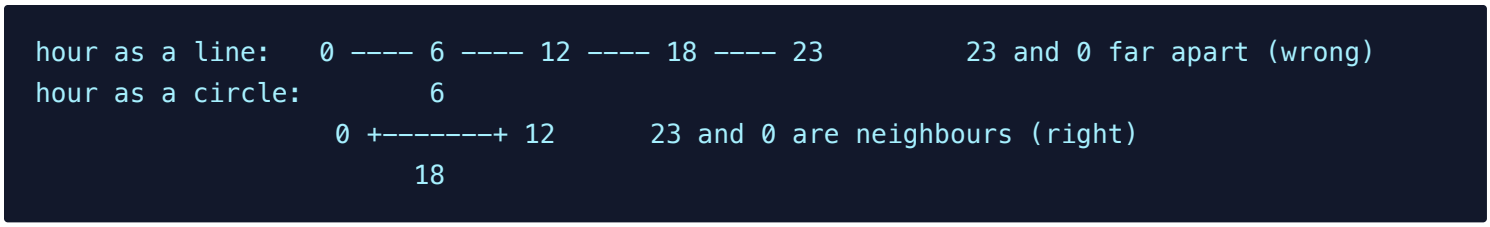
```
['age_imp'] [0.563 0.548 0.532 0.505 0.479] 0.525
['age_imp', 'age_missing'] [0.574 0.606 0.579 0.552 0.522] 0.567
['age_missing'] [0.52  0.566 0.556 0.553 0.54 ] 0.547
```

The two helper columns come from the Chapter 3 plant (seed 11) like this:

```
r = np.random.default_rng(11)
d["age_raw"] = d.age.copy()
d.loc[r.random(len(d)) < 0.07, "age_raw"] = -1
d.loc[r.random(len(d)) < np.where(d.churn == 1, 0.18, 0.05), "age_raw"] = np.nan
d["age_clean"] = d.age_raw.replace(-1, np.nan)          # -1 is a default code, so it
becomes missing
d["age_missing"] = d.age_clean.isna().astype(int)
d["age_imp"] = d.age_clean.fillna(d.age_clean.median())
```

- **Answer** Age alone is barely better than chance (0.525), the missing flag alone gives 0.547, and both give 0.567. The *blank* is more useful than the *value*.
- **Check** The five fold scores for each model vary by about 0.05, so differences of 0.01 are noise; 0.525 to 0.567 is a stable gain across all five folds.
- **Why it matters** If you only impute and forget the flag, you throw away the signal. Also: when a result is weak, use cross-validation, not one lucky split.

**The picture**




**Connection to the real job.** Demand, traffic and load all have daily and weekly rhythms, so calendar features are the first thing to add in forecasting (Chapter 9). For delivery or waiting-time work, hour-of-day, day-of-week and "is a sale day" are classic features to explain delays.

**Applications**

- Forecasting: day of week, month, festival flags, payday.
- E-commerce: "days to sale event", seasonality.
- AIOps: hour/weekday patterns in load and incidents.
- Telecom and banking: time since last transaction, time to contract end.

| COMMON MISTAKE | Using the *future* calendar in the wrong way, or computing "days since last holiday" with a calendar that already includes dates after T. Holidays are known in advance, so "days to next holiday" is allowed; "orders on the next holiday" is not. Another mistake: ignoring time zones, which shifts events by a day. |
|---|---|

**Check yourself**

1. Why use sin and cos for hours?

Answer: hours wrap around, so 23 and 0 should be close.

1. Is "days to next holiday" leakage?

Answer: no, holiday dates are known in advance.

1. Why keep a missing flag after imputing?

Answer: the fact that a value was missing may predict the target.

# 5.4 Text and Geo Features

| INTUITION | A support ticket is a paragraph of words; a customer is a point on a map. A model needs numbers. For text, you can count simple things (length, urgent words) or give each word a weight by how special it is (TF-IDF). For places, you can use distances ("how far is the nearest warehouse?") and areas, because "near" is what matters, not raw latitude numbers. |
|---|---|

**Text features, from simple to rich**

| Level | Idea | Example |
|---|---|---|
| Simple counts | length, number of words, `!` marks, contains "urgent" | `n_chars`, `has_urgent` |
| Bag of words | one column per word, value = count | "refund" appears 1 time |
| TF-IDF | count × how rare the word is across tickets | "refund" has more weight than "please" |
| Embeddings | a dense vector learned by a language model, close meanings = close vectors | semantic search ( `study/ml/` , Deep Learning documents) |

*TF-IDF* stands for term frequency, inverse document frequency: `tfidf(w, doc) = count(w in doc) × log(N / number of docs containing w)` (scikit-learn uses a smoothed version). Words that appear in every document ("please") get weight near 0; rare, specific words get larger weight.

**Problem 5.4.1** We generate 3,000 synthetic tickets; 30% are refund requests (words like "refund", "money back", "charged twice"), the rest are other topics, with filler words. Compare length features with TF-IDF for predicting `refund`.

```
rs = np.random.default_rng(3)
refund_w = ["refund", "money back", "cancel order", "charged twice", "reimburse"]
other_w = ["login", "password", "delivery late", "app crash", "address change", "invoice
copy", "update phone"]
filler = ["please", "help", "urgent", "hello", "thanks", "I am", "since morning", "again",
"very", "support"]
rows = []
for i in range(3000):
    y = rs.random() < 0.3
    words = list(rs.choice(refund_w if y else other_w, 1)) + list(rs.choice(other_w, 1)) +
list(rs.choice(filler, rs.integers(3, 9)))
    if y and rs.random() < 0.3: words = words[1:]
    rs.shuffle(words); rows.append((" ".join(words), int(y)))
tk = pd.DataFrame(rows, columns=["text", "refund"])
```

Now the features and the two models:

```
from sklearn.feature_extraction.text import TfidfVectorizer
tk["n_chars"] = tk.text.str.len(); tk["n_words"] = tk.text.str.split().str.len()
tk["has_urgent"] = tk.text.str.contains("urgent").astype(int)
ttr, tte = train_test_split(tk, test_size=0.3, random_state=0)
cols = ["n_chars", "n_words", "has_urgent"]
m = LogisticRegression(max_iter=1000).fit(ttr[cols], ttr.refund)
print("text: length/count features AUC:", round(roc_auc_score(tte.refund,
m.predict_proba(tte[cols])[:, 1]), 3))
tf = TfidfVectorizer(ngram_range=(1, 2), min_df=3).fit(ttr.text)
m = LogisticRegression(max_iter=1000).fit(tf.transform(ttr.text), ttr.refund)
print("text: TF-IDF AUC:", round(roc_auc_score(tte.refund,
m.predict_proba(tf.transform(tte.text))[:, 1]), 3), " vocab size", len(tf.vocabulary_))
coef = pd.Series(m.coef_[0],
index=tf.get_feature_names_out()).sort_values(ascending=False)
print("top words:", coef.head(5).round(2).to_dict())
```

```
text: length/count features AUC: 0.609
text: TF-IDF AUC: 0.99  vocab size 439
top words: {'reimburse': 4.96, 'refund': 4.76, 'money back': 2.68, 'back': 2.68, 'money':
2.68}
```

The tickets, for example `'very please password I am very refund'`, are made by this generator (each ticket is one topic phrase, one extra phrase and some filler words in random order; refund tickets sometimes

lose their refund phrase, which adds noise):

- **Answer** Length features give AUC 0.609; TF-IDF gives 0.990, and the model's top words are exactly the refund words.
- **Check** The top words are `reimburse`, `refund`, `money back`: the model learned the intended pattern. `ngram_range=(1, 2)` allows two-word phrases such as "money back" (vocabulary of 439 words and pairs). `min_df=3` ignores words seen in fewer than 3 tickets.
- **Why it matters** Cheap length features are a fine baseline, but the words carry the signal. Note that this dataset is easy by design; real tickets have noise, spelling mistakes, Hinglish (Hindi written in English letters) and sarcasm, so expect lower numbers and consider embeddings.

**Fit the vectoriser on training text only**, like any transformer: the vocabulary and the rarity weights are learned numbers.

**Geo features.** Latitude and longitude are not good raw features (a model would need to learn curved geography). Better:

| Feature | How | Example |
|---|---|---|
| Distance | haversine (great-circle distance on a sphere) | km to nearest warehouse, store, branch |
| Region labels | state, city, pincode (encode as in 5.2) | "Maharashtra" |
| Clusters | k-means on coordinates | customer hot-spots |
| Grid cells | round coordinates, or geohash / H3 hexagons | one id per 0.1-degree cell |
| Density | number of points per cell | orders per cell in the last 30 days |

Haversine formula: $a = \sin^2(\Delta\varphi/2) + \cos\varphi_1 \cos\varphi_2 \sin^2(\Delta\lambda/2)$, $\text{distance} = 2R \arcsin(\sqrt{a})$ with R = 6,371 km, φ latitude and λ longitude in radians.

```
def haversine(lat1, lon1, lat2, lon2, R=6371.0):
    p1, p2 = np.radians(lat1), np.radians(lat2); dphi = p2 - p1; dl = np.radians(lon2 -
lon1)
    a = np.sin(dphi / 2) ** 2 + np.cos(p1) * np.cos(p2) * np.sin(dl / 2) ** 2
    return 2 * R * np.arcsin(np.sqrt(a))
print("Mumbai–Pune km:", round(float(haversine(19.0760, 72.8777, 18.5204, 73.8567)), 1))
print("Delhi–Bengaluru km:", round(float(haversine(28.6139, 77.2090, 12.9716, 77.5946)),
1))
```

```
Mumbai–Pune km: 120.2
Delhi–Bengaluru km: 1739.8
```

**Problem 5.4.2** Build "distance to nearest warehouse" for 600 customers spread around Mumbai, Delhi and Bengaluru, with warehouses near Mumbai (19.2, 72.9) and Delhi (28.5, 77.3). Then cluster the customers and put them in 0.1-degree grid cells.

```
g = np.random.default_rng(4)
centers = np.array([[19.07, 72.88], [28.61, 77.21], [12.97, 77.59]])
pts = np.vstack([c0 + g.normal(0, 0.15, (200, 2)) for c0 in centers])
wh = np.array([[19.2, 72.9], [28.5, 77.3]])
dist = np.min([haversine(pts[:, 0], pts[:, 1], w[0], w[1]) for w in wh], axis=0)
print("median km", round(float(np.median(dist)), 1), " Bengaluru-cluster median",
round(float(np.median(dist[400:])), 0))
from sklearn.cluster import KMeans
km = KMeans(3, n_init=10, random_state=0).fit(pts)
print("KMeans cluster sizes:", np.bincount(km.labels_).tolist())
cell = (np.floor(pts[:, 0] * 10) / 10).astype(str) + "_" + (np.floor(pts[:, 1] * 10) /
10).astype(str)
print("grid cells of 0.1 degree:", len(set(cell)), "for", len(pts), "points")
```

```
median km 31.0  Bengaluru-cluster median 855.0
KMeans cluster sizes: [200, 200, 200]
grid cells of 0.1 degree: 151 for 600 points
```

- **Answer** The median customer is 31 km from the nearest warehouse; Bengaluru customers are about 855 km from the nearest one (Mumbai), which is a strong "too far to deliver quickly" feature. K-means recovers the three cities exactly (200 each). The 600 points fall in 151 grid cells, i.e. about 4 per cell.
- **Check** The distances make geographic sense: Bengaluru to Mumbai is about 840 km by air; our median of 855 km includes points up to 0.15 degrees away.
- **Why it matters** Delivery time, shipping cost and fraud ("billing address far from delivery address") are all distance features. H3 (a hexagon grid system) and geohash (a string code per rectangle) work like our grid cells but nest at several sizes; they were not run here ( `h3` library not installed). Responsible use: location is sensitive personal data (Chapter 2); use coarse cells where possible.

**Connection to the real job.** Ticket routing and review mining use text features; delivery, ride-hailing and quick-commerce use geo features all day. A typical interview exercise: "build features for ETA prediction": distance, time of day, weather, restaurant prep time.

### Applications

- NLP in business: ticket classification, review sentiment, invoice field extraction.
- Logistics: distance, route and zone features for ETA.
- Fraud: address mismatch distance, impossible travel between two card swipes.
- Real estate and retail: location density, footfall.

| COMMON MISTAKE | Fitting TF-IDF on all text (including test) and using raw latitude/longitude as numbers. The first leaks the vocabulary; the second hides distance relationships. Also never treat ZIP codes or pincodes as numbers: 560001 is not "bigger" than 400001 in any useful way. |
|---|---|

**Check yourself**

1. Which has more weight in TF-IDF: "please" or "refund"?

Answer: "refund", because it is rarer across documents.

1. Why use haversine and not plain subtraction of coordinates?

Answer: the Earth is a sphere; degrees of longitude shrink away from the equator.

1. Name two ways to turn coordinates into categories.

Answer: grid cells or geohash/H3; k-means clusters.

## Chapter recap

- Features are the highest-leverage skill on tabular data; ask what a human expert would look at.
- Skewed numbers: log1p, sqrt or Yeo-Johnson (spend skew 2.17 became -0.03); clip real outliers; fit all learned values on the training rows only.
- Generic transforms did not help our model (binning 0.871, log 0.880 versus 0.883); a domain ratio did (0.889).
- Encode categories by cardinality: one-hot (small), ordinal (real order), frequency, hashing, embeddings, or target encoding (large).
- Target encoding must be out-of-fold and smoothed: naive gave train AUC 0.812 on a random column; safe versions gave 0.53 and honest test 0.47.
- Unpack dates into calendar and event features; encode cycles with sin and cos (R-squared 0.388 to 0.984).
- Keep missing-value flags: age flag lifted cross-validated AUC from 0.525 to 0.567.
- Text: start with counts, then TF-IDF (0.609 to 0.990 on an easy task); fit on train only.
- Geo: distances (haversine), clusters and grid cells rather than raw coordinates.

| Task | Pattern | Example |
|---|---|---|
| Log of skewed value | `np.log1p(x)` | skew 2.17 to -0.43 |
| Clip outliers | `x.clip(q01, q99)` | max 41,357 to 24,601 |

| Task | Pattern | Example |
|---|---|---|
| Quantile bins | `KBinsDiscretizer(5, strategy="quantile")` | edges 1, 3, 7, 14, 34, 90 |
| One-hot | `pd.get_dummies(df, drop_first=True)` | `plan_pro` |
| Smoothed target mean | `(sum + m·gm) / (count + m)` | (2 + 20×0.254)/23 = 0.308 |
| OOF encoding | encode fold k with other folds | KFold(5) |
| Cyclical | `sin(2πh/24)`, `cos(2πh/24)` | 23 and 0 distance 0.261 |
| TF-IDF | `TfidfVectorizer(ngram_range=(1,2), min_df=3)` | AUC 0.99 |
| Haversine | `2R·arcsin(√a)`, R = 6371 | Mumbai-Pune 120.2 km |

# Practice set

**Problem 5.P.1** (easy) Compute `log1p(19)`. **Answer:** ln(20) = 2.996.

**Problem 5.P.2** (easy) A city appears in 150 of 1,000 rows. What is its frequency encoding? **Answer:** 0.15.

**Problem 5.P.3** (easy) How many columns does one-hot encoding of `channel` (3 levels) create with `drop_first=True`? **Answer:** 2.

**Problem 5.P.4** (medium) A category has 5 rows with churn [1, 1, 0, 1, 0]; global mean 0.25; smoothing m = 10. Smoothed encoding? **Answer:** (3 + 10 × 0.25) / (5 + 10) = 5.5 / 15 = 0.367.

**Problem 5.P.5** (medium) What is the output? `np.sin(2*np.pi*6/24).round(1)` and `np.cos(2*np.pi*6/24).round(1)`. **Answer:** 1.0 and 0.0 (hour 6 is at 90 degrees on the circle).

**Problem 5.P.6** (medium) Find the bug: `te = TargetEncoder().fit_transform(X_all, y_all)` then split into train and test. **Answer:** The encoder saw all labels, including those of test rows (leakage). Split first, fit the encoder on training rows, and use `fit_transform` (cross-fitted) for train and `transform` for test.

**Problem 5.P.7** (medium) A feature `days_to_next_holiday` is computed using a calendar that is published in advance. Is it leakage? What if the calendar is built from "days with unusually high sales"? **Answer:** The official calendar is fine (known before T). A calendar built from sales is leakage (it uses the outcome).

**Problem 5.P.8** (hard) Distance between two points on the equator, longitudes 72.0 and 73.0, same latitude 0. Use haversine. **Answer:** $\Delta\varphi = 0$, $\Delta\lambda = 1° = 0.017453$ rad; $a = \sin^2(0.0087266) = 7.6154\text{e-}5$; $\sqrt{a} = 0.0087266$; distance = 2 × 6371 × 0.0087266 = 111.19 km.

**Problem 5.P.9** (interview) "How do you handle a categorical column with 50,000 levels?" **Answer:** Do not one-hot it. Group rare levels, then use frequency encoding, out-of-fold smoothed target encoding, hashing, or embeddings. Check train versus validation gap to catch leakage, and handle unseen levels at serving by falling back to the global mean.

**Problem 5.P.10** (interview) "You have a timestamp column. What features do you create for a demand model?" **Answer:** Hour, day of week, month, weekend, month end, payday, holiday flag and days to next holiday, days

since last event, cyclical sin/cos for hour and month, and lag features (Chapter 6, Chapter 9). I would check time zone first and make sure every feature uses only information available at the prediction time.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, and English is my second language. Teach me numeric feature transformations (log1p, sqrt, Yeo-Johnson, clipping, binning, scaling, ratios) step by step with a small table of order amounts. For each transform tell me when it helps and when it does not, and ask me one check question before moving on.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: "categorical encoding and safe target encoding (out-of-fold, smoothing)". Explain with a tiny table of 12 rows with a pincode column, compute a naive and an out-of-fold smoothed encoding by hand, and show me why the naive one leaks. Then give me 6 exercises, without solutions, that I will answer and you will grade.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional new to data science. Topic: "date, holiday and cyclical features, and missing-value indicators". Quiz me one question at a time using Indian examples (Diwali, payday, IPL season, hourly web traffic). Include numeric questions on sin/cos encoding and on whether a feature leaks. Wait for my answer, grade it, and explain. Ask 10 questions.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Review my feature engineering plan below for a prediction problem. Check each feature for: when the value is known compared with the prediction time, leakage through encoders or scalers fitted on all data, treatment of missing values, handling of unseen categories and sensible use of text or location data. Suggest concrete fixes in simple English. My plan: [PASTE HERE]
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional preparing for data science interviews in India. Run a mock interview on feature engineering: "encode a
```

```
50,000-level categorical", "features from a timestamp", "features for delivery ETA", "why did your validation AUC drop in production". Ask one question at a time, score me out of 10, give a model answer and one follow-up. Do 8 rounds.
```

CHAPTER 6

# FEATURE ENGINEERING II: BEHAVIOURAL FEATURES, SELECTION AND LEAKAGE-SAFE PIPELINES

*The features that usually win: how a customer behaved recently, how that is changing, which ones to keep, and how to build them once so training and production agree.*

In Chapter 5 you prepared single columns. But for churn, fraud, credit and demand, the strongest signals are rarely in one column. They are in *behaviour over time*: how recently did the customer act, how often, how much, is the pace rising or falling? In this chapter you will build those features from an orders table (recency, frequency, monetary value, rolling windows, trends, gaps, category shares), test whether each helps, choose the ones to keep with several selection methods, check that they are stable over time, and package everything in a pipeline so the feature you test is exactly the feature you serve. You will end with a function `build_features(c, o, T)` that you can reuse on any event table. A warning you will see proven: more features is not always better, and in our data a handful of simple ones carry nearly all the signal.

**Running example.** The shop data (`make_data()`), churn at prediction date T. For training we build a *snapshot table* at several dates (T = 1 May, 1 June, ..., 1 Nov 2024), stack them (16,152 rows), and test on T = 2025-01-01 (2,562 rows). Using several snapshots gives the model more examples, and the time split (train earlier, test later) copies real use. The label window of the last training snapshot (1 Nov plus 60 days = 31 Dec) ends before the test date, so there is no overlap in time.

## 6.1 RFM and Rolling-Window Features

**INTUITION**

A shopkeeper who knows his regular customers does not need a model to guess who is drifting away. He thinks: "She came yesterday, she comes every week, she spends a lot: safe." "He last came two months ago, and used to come weekly: gone." Those are three numbers: how recently (R), how often (F), how much (M). Rolling windows are the same idea measured over the last 7, 30 and 90 days.

**Words you need**

| Word | Meaning | Example |
|---|---|---|
| RFM | Recency, Frequency, Monetary value | 8 days ago, 40 orders, Rs 23,746 |

| Word | Meaning | Example |
|---|---|---|
| Window | A time range ending at T | last 30 days: `age_d <= 30` |
| `age_d` | Days between an event and T (1 = yesterday) | an order 5 days before T has `age_d = 5` |
| Rolling aggregate | Count, sum, mean, std, min, max inside a window | `orders_30d`, `spend_90d` |
| Snapshot | The feature table as of one date T | train table at 2024-08-01 |

**The core function.** Everything is computed from rows with `order_ts < T`; this is the point-in-time rule from Chapter 2. Read it slowly; we use it for the rest of the chapter.

```
def build_features(c, o, T):
    """One row per ACTIVE customer at T. Uses only orders with order_ts < T. Label from
T..T+60d."""
    T = pd.Timestamp(T)
    base = make_label_table(c, o, pred_date=T)               # customers + churn label
    past = o[o.order_ts < T].copy()                           # <-- the leakage guard
    past["age_d"] = (T - past.order_ts).dt.days               # 1 = yesterday
    f = pd.DataFrame(index=base.customer_id)
    g = past.groupby("customer_id")
    f["recency_days"] = g.age_d.min()                         # R
    f["frequency_all"] = g.size()                             # F
    f["monetary_all"] = g.amount.sum()                        # M
    for w in (7, 30, 90):                                     # rolling windows
        s = past[past.age_d <= w].groupby("customer_id")
        f[f"orders_{w}d"] = s.size(); f[f"spend_{w}d"] = s.amount.sum()
    s = past[(past.age_d > 30) & (past.age_d <= 60)].groupby("customer_id")     # previous
windows
    f["orders_30_60d"] = s.size(); f["spend_30_60d"] = s.amount.sum()
    s = past[(past.age_d > 60) & (past.age_d <= 90)].groupby("customer_id")
    f["orders_60_90d"] = s.size(); f["spend_60_90d"] = s.amount.sum()
    f = f.reindex(base.customer_id).fillna({k: 0 for k in f.columns if k !=
"recency_days"})
    f["avg_order_90d"] = np.where(f.orders_90d > 0, f.spend_90d /
f.orders_90d.clip(lower=1), np.nan)
    f["ratio_7_30"] = f.orders_7d / (f.orders_30d + 1)
    f["trend_30_vs_prev60"] = (f.orders_30d + 1) / ((f.orders_30_60d + f.orders_60_90d) /
2 + 1)
    f["velocity_spend"] = f.spend_30d - f.spend_30_60d
    f["accel_spend"] = (f.spend_30d - f.spend_30_60d) - (f.spend_30_60d - f.spend_60_90d)
    # gaps between consecutive orders in the last 180 days
    p2 = past[past.age_d <= 180].sort_values(["customer_id", "order_ts"])
    p2["gap"] = p2.groupby("customer_id").order_ts.diff().dt.days
    gg = p2.groupby("customer_id").gap.agg(["mean", "max", "std"]); gg.columns =
["gap_mean", "gap_max", "gap_std"]
    f = f.join(gg, how="left")
    f["rec_over_gap"] = f.recency_days / (f.gap_mean.fillna(90) + 1)
    # customer x category shares in the last 90 days (cross-entity feature)
    q = past[past.age_d <= 90]
    cs = pd.crosstab(q.customer_id, q.category, normalize="index"); cs.columns =
[f"share_{x}" for x in cs.columns]
    f = f.join(cs, how="left").fillna({k: 0 for k in cs.columns})
    out = base.merge(f.reset_index(), on="customer_id", how="left")
    out["tenure_days"] = (T - out.signup_date).dt.days
    out["is_pro"] = (out.plan == "pro").astype(int)
    out["T"] = T
    return out
```

**Worked problem.** Before using it on 6,000 customers, we do one customer by hand.

**Problem 6.1.1** A customer's orders before T were, in days before T: 2, 5, 9, 20 and 45, for Rs 100, 200, 150, 300 and 250. Compute `orders_7d`, `orders_30d`, `spend_30d`, `orders_30_60d`, `spend_30_60d`, `trend_30_vs_prev60`, `gap_mean`, `gap_max`, and `rec_over_gap`.

| | |
|---|---|
| **Step 1 (windows)** | `orders_7d` counts `age_d <= 7`: ages 2 and 5, so 2. `orders_30d` counts `age_d <= 30`: ages 2, 5, 9, 20, so 4. `spend_30d` = 100 + 200 + 150 + 300 = 750. The window 30 to 60 days holds age 45 only, so `orders_30_60d` = 1 and `spend_30_60d` = 250. The window 60 to 90 days is empty: `orders_60_90d` = 0. |
| **Step 2 (trend)** | Formula `(orders_30d + 1) / ((orders_30_60d + orders_60_90d)/2 + 1)`. Plug in `(4 + 1) / ((1 + 0)/2 + 1) = 5 / 1.5 = 3.33`. The "+1" terms are *smoothing*: they avoid dividing by zero and calm down tiny counts. |
| **Step 3 (gaps)** | Sort the order times oldest to newest (ages 45, 20, 9, 5, 2). Gaps are 45-20 = 25, 20-9 = 11, 9-5 = 4, 5-2 = 3 days. Mean = (25 + 11 + 4 + 3) / 4 = 10.75; max = 25. |
| **Step 4 (recency over gap)** | `recency_days` = 2 (the most recent order was 2 days ago). `rec_over_gap = 2 / (10.75 + 1) = 0.17`. |
| **Answer** | 2, 4, Rs 750, 1, Rs 250, trend 3.33, mean gap 10.75, max gap 25, rec_over_gap 0.17. |
| **Check** | Total spend over all five orders = 1,000; windows 0-30 and 30-60 give 750 + 250 = 1,000 ✓. Verified in code too. |
| **Why it matters** | In plain words: this customer is buying about three times faster in the last month than before, and ordered 2 days ago against a usual gap of 11 days, so they look very active. A value of `rec_over_gap` above 3 would say "three missed cycles": a warning sign. |

**Run it.** Predict first: how many rows will the training snapshots give, and will churn rates differ much between train and test?

```
c, o = make_data()
Ts_train = pd.to_datetime(["2024-05-01", "2024-06-01", "2024-07-01", "2024-08-01",
                           "2024-09-01", "2024-10-01", "2024-11-01"])
tr = pd.concat([build_features(c, o, T) for T in Ts_train], ignore_index=True)
te = build_features(c, o, "2025-01-01")
print("train rows", len(tr), " test rows", len(te), " churn rates", round(tr.churn.mean(),
3), round(te.churn.mean(), 3))
print(te[["recency_days", "orders_7d", "orders_30d", "orders_90d", "gap_mean",
"trend_30_vs_prev60", "rec_over_gap"]]
      .describe().round(2).loc[["mean", "50%", "max"]].to_string())
```

```
train rows 16152  test rows 2562  churn rates 0.246 0.254
      recency_days  orders_7d  orders_30d  orders_90d  gap_mean  trend_30_vs_prev60
rec_over_gap
mean         19.35       0.75        3.12        9.17     11.65                1.28
1.83
50%           9.00       0.00        2.00        7.00      8.33                1.00
0.84
max          90.00       8.00       22.00       67.00    153.00               17.00
75.00
```

Reading it: the churn rate is steady (24.6% in train, 25.4% in test), as Chapter 3's drift check suggested. A median customer last ordered 9 days ago and has a gap of about 8 days. `trend_30_vs_prev60` has a median of 1.00 (steady pace) and a maximum of 17 (a customer who suddenly became very active).

**RFM scoring and segments.** Marketers rank each of R, F and M into 5 groups (5 = best) and combine them. `pd.qcut` cuts into quantile groups. For recency, a small number of days is good, so labels are reversed.

```
rfm = te[["customer_id", "recency_days", "frequency_all", "monetary_all", "churn"]].copy()
rfm["R"] = pd.qcut(rfm.recency_days, 5, labels=[5, 4, 3, 2, 1]).astype(int)
rfm["F"] = pd.qcut(rfm.frequency_all.rank(method="first"), 5, labels=[1, 2, 3, 4,
5]).astype(int)
rfm["M"] = pd.qcut(rfm.monetary_all.rank(method="first"), 5, labels=[1, 2, 3, 4,
5]).astype(int)
def seg(r):
    if r.R >= 4 and r.F >= 4: return "Champions"
    if r.R >= 4: return "Recent"
    if r.R <= 2 and r.F >= 4: return "At risk (were frequent)"
    if r.R <= 2: return "Lapsing"
    return "Middle"
rfm["segment"] = rfm.apply(seg, axis=1)
print(rfm.groupby("segment").churn.agg(["size", "mean"]).round(3).sort_values("mean"))
```

```
                         size   mean
segment
Champions                 608  0.028
Recent                    510  0.069
Middle                    423  0.121
Lapsing                   770  0.536
At risk (were frequent)   251  0.538
```

**Problem 6.1.2** Read the segment table: how much better is the best segment than the worst, and how many customers does the "At risk (were frequent)" group represent as churners?

| | |
|---|---|
| **Formula** | ratio = worst rate / best rate; churners = size × rate |
| **Plug in** | 0.538 / 0.028; 251 × 0.538 |
| **Compute** | 0.538 / 0.028 = 19.2; 251 × 0.538 = 135 |
| **Answer** | The worst segment churns 19 times as often as the best (53.8% vs 2.8%). The "At risk" group holds about 135 expected churners among its 251 customers. |
| **Check** | Sizes add up: 608 + 510 + 423 + 770 + 251 = 2,562 ✓. Overall rate: (608×0.028 + 510×0.069 + 423×0.121 + 770×0.536 + 251×0.538) / 2562 = (17.0 + 35.2 + 51.2 + 412.7 + 135.0) / 2562 = 651.1 / 2562 = 0.254 ✓. |
| **Why it matters** | A rule-based segmentation, with no model at all, separates customers 19 to 1. It is easy to explain to marketing ("call the At-risk segment first"), and a perfect baseline for any model (Chapter 1). |

**The picture**

```
time ------------------------------------------------------------> T
 |<----------- 90 days ----------->|
 |        |<------ 30 days ------->|
 |        |                |<-7d ->|
 prev30   last30            last7                orders in each window: counts, sums,
means
 (60–90)  (30–60) is between; trend = last30 / average of the two earlier windows
```

**Connection to the real job.** RFM-style features are the first thing built for churn, CLV (Chapter 11), fraud (transactions in the last hour, day, week) and credit (utilisation over the last 3, 6 and 12 months). Windows are almost always 7, 30, 90 days or similar, chosen to match the business rhythm.

**Applications**

- E-commerce and retail: churn and next-purchase propensity.
- Fraud: card transactions in the last 1 hour and 24 hours.
- Banking: average balance and credit utilisation over 3, 6, 12 months.
- Operations: failed-payment counts in the last 5, 15, 60 minutes (the same rolling windows).

| COMMON MISTAKE | Computing windows from the *whole table* and filtering by "last 30 days" relative to today, not to T. A training snapshot at 2024-08-01 must use 2024-08-01 as its "today". Always pass T into the function. |
|---|---|

**Check yourself**

1. What do R, F and M stand for?

Answer: recency, frequency, monetary value.

1. Why add +1 in a ratio of counts?

Answer: to avoid division by zero and to calm ratios with tiny counts.

1. What is a snapshot table?

Answer: the feature table as it looked at one prediction date T.

## 6.2 Trends, Lags, Cross-Entity Features, and Do They Help?

| INTUITION | Two customers both ordered 3 times last month. One used to order 10 times a month (falling), the other used to order once (rising). Same count, opposite story. Trends, gaps and changes carry that story. A car's speed tells you where you are; velocity and acceleration tell you where you are heading. |
|---|---|

**Types of behavioural features**

| Type | Example in our code | Meaning |
|---|---|---|
| Lag | `orders_30_60d` (the previous window) | what happened one window ago |
| Time since last event | `recency_days` | silence length |
| Gap statistics | `gap_mean` , `gap_max` , `gap_std` | usual rhythm and irregularity |

| Type | Example in our code | Meaning |
|---|---|---|
| Ratios between windows | `ratio_7_30` | is the last week quiet compared to the month? |
| Trend | `trend_30_vs_prev60` | pace now vs before |
| Velocity | `velocity_spend` = spend_30d - spend_30_60d | change in spend |
| Acceleration | `accel_spend` | change of the change |
| Streaks | consecutive weeks with an order | habit strength |
| Entity aggregates | per user, merchant, device | merchant's average ticket size |
| Cross-entity | `share_fashion` = customer x category | taste profile |
| Interactions / ratios | `rec_over_gap`, debt-to-income, price per unit | domain ratios |
| Graph (awareness) | number of accounts sharing a device or address | fraud rings |

We have not yet checked whether these help. A strong habit: **add features in groups and measure each group on the later test period.** We train a gradient boosting model (a tree-based model that handles missing values; see `study/ml/`) on the stacked snapshots and test on January 2025.

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score
def auc(cols, seed=0):
    m = HistGradientBoostingClassifier(max_iter=150, learning_rate=0.08,
random_state=seed).fit(tr[cols], tr.churn)
    return round(roc_auc_score(te.churn, m.predict_proba(te[cols])[:, 1]), 3)
static = ["is_pro", "tenure_days"]
A = ["recency_days", "orders_90d", "is_pro"]
B = A + ["orders_7d", "orders_30d", "spend_7d", "spend_30d", "spend_90d", "frequency_all",
"monetary_all"]
C = B + ["ratio_7_30", "trend_30_vs_prev60", "velocity_spend", "accel_spend", "gap_mean",
"gap_max", "gap_std", "rec_over_gap"]
D = C + [x for x in te.columns if x.startswith("share_")] + ["tenure_days",
"avg_order_90d"]
print("static only (plan, tenure):", auc(static))
for name, cols in [("A RFM-ish base", A), ("B + windows", B), ("C + trends/gaps", C), ("D
+ category shares, tenure", D)]:
    print(f"{name:32s} n_feat={len(cols):2d}  test AUC={auc(cols)}")
```

```
static only (plan, tenure): 0.612
A RFM-ish base                   n_feat= 3  test AUC=0.891
B + windows                      n_feat=10  test AUC=0.891
C + trends/gaps                  n_feat=18  test AUC=0.887
D + category shares, tenure      n_feat=24  test AUC=0.89
```

**Problem 6.2.1** Interpret the table. Which step is the "biggest win", and what does the rest tell you?

| | |
|---|---|
| **Step 1** | From static profile columns (plan, tenure) to behavioural features: 0.612 to 0.891, a gain of 0.279 AUC. |
| **Step 2** | Adding windows, trends, gaps and category shares: 0.891, 0.891, 0.887, 0.890: no real gain. The spread of 0.004 is within noise (with about 650 churners in the test set, the standard error of an AUC around 0.89 is roughly 0.01). |
| **Answer** | Behavioural features are the big win (+0.28 AUC over a profile-only model). The extra 21 features add nothing measurable in this dataset. |
| **Check** | Our synthetic customers churn according to their order rate and silence, so recency already captures nearly all of it (a single feature had AUC 0.880 in Chapter 3). Real data is richer, but the lesson holds. |
| **Why it matters** | In plain words: build behaviour first, then add features only when they earn their place on a *later* test period. More features mean more code, more monitoring and more ways to break in production. Do not add features to feel busy. |

**A fair test needs a time split.** We stacked several snapshots, so the same customer appears many times. A *random* split would put the same customer in train and test. We compare:

```
random split inside snapshots AUC: 0.887  (customers repeat across snapshots)
time split (train<=Nov, test Jan) AUC: 0.89
customers in train & test: 2085 of 2562
```

The code for these three lines:

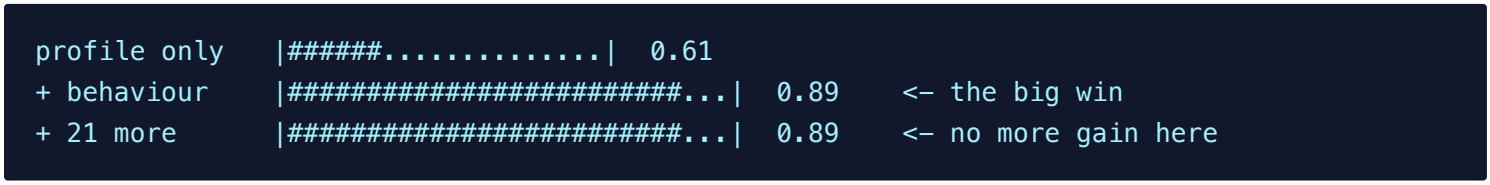
```
from sklearn.model_selection import train_test_split
rtr, rte = train_test_split(tr, test_size=0.3, random_state=0)
m = HistGradientBoostingClassifier(max_iter=150, learning_rate=0.08,
random_state=0).fit(rtr[D], rtr.churn)
print("random split inside snapshots AUC:", round(roc_auc_score(rte.churn,
m.predict_proba(rte[D])[:, 1]), 3))
print("time split AUC:", auc(D))
print("customers in train & test:", len(set(tr.customer_id) & set(te.customer_id)), "of",
te.customer_id.nunique())
```

Here both agree, because this synthetic world does not change over time. In real data with trends, promotions or a changing user base, the random split is *optimistic* and the time split is honest. 2,085 of the 2,562 test customers were also in training: this is normal in churn, but it means customer-level repeats must stay on one side of any random split (use group splits, `GroupKFold`, in that case).

**Problem 6.2.2** Design three more features for fraud on card transactions: velocity, entity aggregates and a graph feature. Write them as precise definitions.

| | |
|---|---|
| **Step 1 (velocity)** | `txn_count_1h` = number of transactions by this card in the 60 minutes before the current transaction (strictly before). |
| **Step 2 (entity aggregate)** | `merchant_avg_ticket_30d` = average amount at this merchant over the 30 days before the current transaction; then `amount / merchant_avg_ticket_30d` as a ratio feature. |
| **Step 3 (graph, awareness)** | `cards_on_device_7d` = number of distinct cards seen on this device in the last 7 days (a high value suggests a fraud ring). |
| **Answer** | Each feature has a window, an entity and a rule "strictly before the current event". |
| **Check** | For the first transaction of a card, `txn_count_1h` = 0, not missing; for a new merchant, `merchant_avg_ticket_30d` is missing and the ratio must handle that (fill, flag). |
| **Why it matters** | At serving time (a payment in 100 ms) these features must be computed from a fast store (a *feature store*); a feature you cannot compute in time is worthless in production (Chapter 13). |

**The picture**




**Connection to the real job.** In interviews: "what features would you build for X?" A good answer lists RFM-style windows, trends, ratios and entity aggregates, then says "and I will test groups of them on a later period".

**Applications**

- Churn and CLV: windows, trends, gaps.
- Fraud: velocity, device and merchant aggregates, graph features.
- Credit: utilisation trends, delinquency counts over windows.
- Demand forecasting: lags and rolling means of sales (Chapter 9).
- AIOps: rolling error rates and latency percentiles.

| | |
|---|---|
| **COMMON MISTAKE** | Adding every possible window and ratio (hundreds of features) with no test, then trusting feature importance. Also: dividing by zero or comparing a new customer's empty history with an old customer's long one without a tenure feature or a "has history" flag. |

**Check yourself**

1. What is velocity? And acceleration?

Answer: change in a quantity between windows; the change of that change.

1. Why use a time split when you stack several snapshots?

Answer: to avoid the same customer and the same period appearing in both train and test, and to copy real use.

1. 21 extra features gave AUC +0.000. What do you do?

Answer: drop most of them; keep the simple model.

## 6.3 Feature Selection and Stability

| INTUITION | Packing for a trip: you could take everything, but a heavy bag slows you down, and half of it you never use. Selection removes features that are useless (no information), redundant (same information twice), dangerous (leaky) or unstable (behave differently now than before). |
|---|---|

**Step 0: hygiene.** Remove (1) leakage features (Chapter 3), (2) constant and near-constant columns (`nunique() <= 1`), (3) duplicated information (correlation above 0.95).

**Selection methods**

| Family | Idea | Tools |
|---|---|---|
| Filter | Score each feature alone, keep the best | variance, correlation, mutual information (MI), `SelectKBest` |
| Wrapper | Train repeatedly, remove the weakest | `RFE` (recursive feature elimination) |
| Embedded | The model itself shrinks useless weights | L1 (lasso) logistic regression, tree importance |
| Permutation importance | Shuffle one column in the test set and see how much AUC drops | `permutation_importance` |
| SHAP / Boruta (awareness) | Game-theory contributions; compare with random shadow features | `shap`, `boruta` (not installed or run here) |

Run them all on our 25 candidate features. Predict: which single feature will matter most under permutation importance?

```
from sklearn.feature_selection import mutual_info_classif, RFE
from sklearn.linear_model import LogisticRegression
from sklearn.inspection import permutation_importance
from sklearn.preprocessing import StandardScaler
feats = ["recency_days", "frequency_all", "monetary_all", "orders_7d", "orders_30d",
"orders_90d",
         "spend_7d", "spend_30d", "spend_90d", "ratio_7_30", "trend_30_vs_prev60",
"velocity_spend",
         "accel_spend", "gap_mean", "gap_max", "gap_std", "rec_over_gap", "share_grocery",
"share_fashion",
         "share_electronics", "share_home", "tenure_days", "is_pro", "avg_order_90d",
"age"]   # 25 columns
Xtr, Xte = tr[feats], te[feats]
cm = Xtr.corr(method="spearman").abs()
print("pairs with |spearman|>0.95:", [(a, b, round(cm.loc[a, b], 3)) for i, a in
enumerate(feats) for b in feats[i+1:] if cm.loc[a, b] > 0.95])
mi = pd.Series(mutual_info_classif(Xtr.fillna(0), tr.churn, random_state=0),
index=feats).sort_values(ascending=False)
print("MI top 6:", mi.head(6).round(3).to_dict())
Xs = StandardScaler().fit_transform(Xtr.fillna(0))
l1 = LogisticRegression(penalty="l1", solver="liblinear", C=0.05).fit(Xs, tr.churn)
kept = [f for f, w in zip(feats, l1.coef_[0]) if abs(w) > 1e-6]
print("L1 (C=0.05) keeps", len(kept), "of", len(feats), ":", kept)
m = HistGradientBoostingClassifier(max_iter=150, learning_rate=0.08,
random_state=0).fit(Xtr, tr.churn)
pi = permutation_importance(m, Xte, te.churn, scoring="roc_auc", n_repeats=5,
random_state=0)
print(pd.Series(pi.importances_mean,
index=feats).sort_values(ascending=False).head(6).round(3).to_dict())
```

```
pairs with |spearman|>0.95: [('frequency_all', 'monetary_all', 0.985), ('orders_7d',
'spend_7d', 0.973), ('orders_30d', 'spend_30d', 0.953), ('orders_90d', 'spend_90d', 0.96)]
MI top 6: {'recency_days': 0.188, 'spend_30d': 0.177, 'orders_30d': 0.173, 'rec_over_gap':
0.145, 'trend_30_vs_prev60': 0.134, 'monetary_all': 0.125}
L1 (C=0.05) keeps 13 of 25 : ['recency_days', 'monetary_all', 'orders_7d', 'orders_30d',
'spend_7d', 'trend_30_vs_prev60', 'accel_spend', 'gap_mean', 'gap_std', 'rec_over_gap',
'share_fashion', 'share_home', 'is_pro']
permutation importance (AUC drop) top 6: {'recency_days': 0.198, 'tenure_days': 0.008,
'rec_over_gap': 0.006, 'is_pro': 0.004, 'gap_mean': 0.003, 'spend_90d': 0.003}
full model AUC: 0.889
top-5 by permutation: ['recency_days', 'tenure_days', 'rec_over_gap', 'is_pro',
'gap_mean']  AUC with only these: 0.894
RFE keeps: ['recency_days', 'orders_7d', 'orders_30d', 'rec_over_gap', 'is_pro']
```

The last two output lines come from these extra lines (RFE = recursive feature elimination: train, drop the weakest feature, repeat until 5 remain):

```
pis = pd.Series(pi.importances_mean, index=feats).sort_values(ascending=False)
top5 = list(pis.index[:5]); print("top-5 by permutation:", top5, " AUC with only these:",
auc(top5))
rfe = RFE(LogisticRegression(max_iter=2000), n_features_to_select=5).fit(Xs, tr.churn)
print("RFE keeps:", [f for f, k in zip(feats, rfe.support_) if k])
```

(The `age` column is the customer's age from the customers table.) Line by line: the correlation test finds four pairs that carry nearly the same information (spend and count of orders move together, since order values are similar). Mutual information ranks recency first. L1 shrinks 12 weights to exactly zero and keeps 13. Permutation importance for `recency_days` is 0.198: shuffling that one column drops the test AUC by 0.198, while every other feature drops it by under 0.01. RFE (keep 5) picks a different set, with recency, two window counts, `rec_over_gap` and `is_pro`.

**Problem 6.3.1** Compare the full model with a 5-feature model chosen by permutation importance.

| | |
|---|---|
| **Step 1** | Full model with 25 features: test AUC 0.889. |
| **Step 2** | Keep the top 5 by permutation importance (recency_days, tenure_days, rec_over_gap, is_pro, gap_mean): test AUC 0.894. |
| **Answer** | The small model is as good as (even slightly better than) the large one: 0.894 vs 0.889 with one fifth of the features. |
| **Check** | The difference of 0.005 is within noise, so the honest statement is "equal". Remember we looked at test importances to choose the features, which is a mild form of peeking (Chapter 4); in production you would pick the features on a *validation* period and keep the January test untouched. |
| **Why it matters** | A model with 5 features is cheaper to compute, easier to explain, easier to monitor and has fewer ways to break. Also, different methods choose different sets: that is a sign that many features are interchangeable, so prefer the cheapest to compute and the most stable. |

**Stability: PSI per feature.** A feature can be strong in training and then change meaning or shape in production. The *Population Stability Index* compares the distribution of a feature in an old period (expected) and a new period (actual) over bins:

`PSI = Σ (actual% − expected%) × ln(actual% / expected%)` over bins.

Common reading: below 0.1 stable, 0.1 to 0.25 moderate shift (watch), above 0.25 large shift (investigate or retrain). (These are rules of thumb from credit risk, not laws.)

**Problem 6.3.2** First by hand: a feature has three bins with expected shares 50%, 30%, 20% and actual shares 40%, 30%, 30%. Compute PSI.

| | |
|---|---|
| **Formula** | `PSI = Σ (a − e) × ln(a / e)` |
| **Plug in** | bin 1: (0.4 − 0.5) × ln(0.4/0.5); bin 2: (0.3 − 0.3) × ln(1); bin 3: (0.3 − 0.2) × ln(0.3/0.2) |
| **Compute** | bin 1: (−0.1) × (−0.2231) = 0.0223; bin 2: 0; bin 3: 0.1 × 0.4055 = 0.0405; total 0.0629 |
| **Answer** | PSI = 0.063: stable (below 0.1). |
| **Check** | Each term is positive (the product of a difference and the log of the same direction ratio), so PSI can never be negative ✓; the code reproduced 0.0629. |
| **Why it matters** | In plain words: 10 percentage points moved between bins, and PSI says "small shift". |

Now on real data: compare May 2024 customers (old) with January 2025 customers (new), then simulate a shift.

```
def psi(expected, actual, bins=10):
    qs = np.unique(np.nanquantile(expected, np.linspace(0, 1, bins + 1)))
    qs[0], qs[-1] = -np.inf, np.inf
    e = np.histogram(expected.dropna(), qs)[0] / expected.notna().sum()
    a = np.histogram(actual.dropna(), qs)[0] / actual.notna().sum()
    e = np.clip(e, 1e-4, None); a = np.clip(a, 1e-4, None)
    return float(np.sum((a - e) * np.log(a / e)))
old = tr[tr["T"] == pd.Timestamp("2024-05-01")]
print({f: round(psi(old[f], te[f]), 3) for f in ["recency_days", "orders_90d",
"spend_90d", "tenure_days", "gap_mean"]})
for k in (1.0, 1.2, 1.5, 2.0):
    print("spend x", k, "PSI", round(psi(old.spend_90d, te.spend_90d * k), 3))
```

```
{'recency_days': 0.009, 'orders_90d': 0.006, 'spend_90d': 0.02, 'tenure_days': 0.162,
'gap_mean': 0.015}
spend x 1.0 PSI 0.02
spend x 1.2 PSI 0.065
spend x 1.5 PSI 0.191
spend x 2.0 PSI 0.46
```

- **Answer** Behaviour features are stable (PSI under 0.03). `tenure_days` shows 0.162, a moderate shift, for an obvious reason: eight months later, the same customers are simply eight months older, and new signups arrive. If spend per order rose by 20%, PSI is 0.065 (stable), by 50% it is 0.191 (watch), and by 100% it is 0.46 (large).
- **Check** `spend x 1.0` is not zero (0.02) because January customers genuinely differ a bit from May ones; compare with the scale of 0.1.
- **Why it matters** Features that count "time since something" drift by design. Prefer ratios and rates, or retrain regularly. PSI is the standard monitor in credit risk (Chapter 11) and in MLOps (see `study/mlops/` if available).

**The picture**

```
PSI      0 ------ 0.1 ------- 0.25 -------------->
         stable    watch       act: investigate/retrain
tenure_days 0.162   (moderate: expected, ages grow)
spend x2    0.46    (large)
```

**Connection to the real job.** In regulated settings, a model with fewer, explainable and stable features is far easier to approve. Selection is also a cost question: each feature needs a data pipeline, monitoring and an owner.

### Applications

- Credit risk: Information Value and PSI for variable selection and monitoring.
- MLOps: per-feature drift dashboards.
- Healthcare: small interpretable feature sets for clinical use.
- Fraud: low-latency models need cheap features.

| COMMON MISTAKE | Choosing features using the test set (we did it above only as a demonstration, and noted it). Do selection on training or validation data only, and judge on a separate test. Another mistake: believing importance means causation (Chapter 8) or that a low-importance feature is "useless" when it is correlated with a stronger one. |
|---|---|

### Check yourself

1. Which selection method shuffles one column and measures the score drop?

Answer: permutation importance.

1. PSI is 0.3 for an important feature. What do you do?

Answer: investigate (pipeline change, real shift), and consider retraining.

1. Why did `tenure_days` drift?

Answer: it grows with time by definition.

## 6.4 Leakage-Safe Pipelines and Training-Serving Skew

| INTUITION | A recipe must be the same in the test kitchen and the restaurant. If the test kitchen uses a hand-crafted sauce and the restaurant a factory sauce, the dish tastes different, and nobody knows why customers complain. In ML this is *training-serving skew*: the features at training time are computed one way, in production another way. |
|---|---|

### Words you need

| Word | Meaning | Example |
|---|---|---|
| Pipeline | Ordered steps (impute, scale, encode, model) that are fitted together | `Pipeline([...])` |
| ColumnTransformer | Applies different steps to different columns | numbers vs categories |

| Word | Meaning | Example |
|---|---|---|
| Fit vs transform | `fit` learns numbers from training rows (median, mean); `transform` applies them | median from train used for test |
| Training-serving skew | Features differ between training and production | days computed in IST in training and UTC in serving |
| Feature store | A system that stores feature definitions and values for training and serving | Feast, Tecton (awareness) |

**Why pipelines.** Every learned number (medians, means, category lists, selected columns, encoders) must be learned from training rows only, and then applied unchanged to validation, test and live data. A scikit-learn `Pipeline` does this for you and also makes cross-validation honest: in each fold it refits all steps using only that fold's training part.

**Problem 6.4.1** Show the danger. We make 400 rows of *pure noise* (1,000 random features, random labels). The true AUC of any model is 0.5. Compare selecting the 20 "best" features before cross-validation versus inside a pipeline.

```
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline
rng = np.random.default_rng(0)
n, p = 400, 1000
X = rng.normal(size=(n, p)); y = rng.integers(0, 2, n)
cv = StratifiedKFold(5, shuffle=True, random_state=0)
Xk = SelectKBest(f_classif, k=20).fit_transform(X, y)                  # selection uses ALL
labels (leak)
leak = cross_val_score(LogisticRegression(max_iter=1000), Xk, y, cv=cv,
scoring="roc_auc").mean()
pipe = Pipeline([("sel", SelectKBest(f_classif, k=20)), ("clf",
LogisticRegression(max_iter=1000))])
safe = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc").mean()   # selection refit
inside each fold
print("pure-noise data: selection outside CV AUC", round(leak, 3), " selection inside
pipeline AUC", round(safe, 3))
```

```
pure-noise data: selection outside CV AUC 0.757  selection inside pipeline AUC 0.517
```

- **Answer** Selecting features on all the data first produced AUC 0.757 on random noise, a fake "skill". Inside the pipeline the AUC is 0.517, correctly near 0.5.

- **Check** The labels were random coin flips, so any honest AUC must be near 0.5 ✓.
- **Why it matters** With 1,000 features, a few correlate with the labels by chance; picking them with the labels in view and then "testing" on the same labels fools you. This is the same leak as Chapter 4's validation peeking. Rule: *every step that looks at the target (selection, target encoding, imputation by group mean, resampling) belongs inside the cross-validation loop.*

**A full production-style pipeline.** Numeric columns: median imputation (with missing flags), scaling. Categorical: one-hot with unknown categories ignored. Then logistic regression.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
num = ["recency_days", "orders_30d", "orders_90d", "spend_90d", "gap_mean",
"rec_over_gap", "tenure_days"]
cat = ["plan", "channel", "city"]
pre = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median", add_indicator=True)),
("sc", StandardScaler())]), num),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat)])
pipe = Pipeline([("pre", pre), ("clf", LogisticRegression(max_iter=2000))]).fit(tr[num +
cat], tr.churn)
print("pipeline AUC on Jan-25:", round(roc_auc_score(te.churn, pipe.predict_proba(te[num +
cat])[:, 1]), 3))
new = te[num + cat].head(2).copy()
new.loc[new.index[0], "city"] = "Jaipur"; new.loc[new.index[1], "gap_mean"] = np.nan
print("scores for odd rows (unseen city, missing gap):", pipe.predict_proba(new)[:,
1].round(3))
print("n transformed columns:", pipe.named_steps["pre"].transform(te[num +
cat].head(1)).shape[1])
```

```
pipeline AUC on Jan-25: 0.893
scores for odd rows (unseen city, missing gap): [0.102 0.631]
n transformed columns: 19
```

Line by line: `add_indicator=True` adds a 0/1 column for each numeric feature that had missing values in training ( `gap_mean` ), which is why we get 19 columns: 7 numeric + 1 flag + 2 + 3 + 6 one-hot levels = 19. `handle_unknown="ignore"` means a city never seen in training ("Jaipur") becomes all zeros instead of crashing. The odd rows still get scores. A plain logistic regression pipeline with 7 numeric features and 3 categoricals gives 0.893, equal to the boosted trees with many features. We can save the fitted pipeline as one object ( `joblib.dump(pipe, "model.joblib")` , not run here) and load it in the serving code, which guarantees the same median, same scaling and same categories.

**Problem 6.4.2** Why is our `build_features(c, o, T)` also a skew-prevention tool? Explain, and name two ways it could still go wrong in production.

| | |
|---|---|
| **Step 1** | The same function creates features for training (T = 1 May to 1 Nov) and for scoring (T = today). One definition, no rewriting in another language. |
| **Step 2** | Risk one: in production the orders table may be updated late (Chapter 2: today's data is 59% complete), so `orders_7d` is smaller than in training, where we used final data. Fix: train on data "as of T" or apply the finalisation lag. |
| **Step 3** | Risk two: time zones or a changed definition of an "order" (cancelled orders now counted). Fix: tests on feature distributions (PSI) and unit tests on `build_features`. |
| **Answer** | One shared function removes duplicated logic; the remaining risks are late data and definition changes, caught by monitoring. |
| **Check** | If you score yesterday's customers with partially loaded data and recency looks 1 day longer than it was, the model will think more of them have churned: monitor the mean of key features daily. |
| **Why it matters** | Most "the model got worse in production" stories are skew, not algorithms. Production topics continue in Chapter 13 and `study/mlops/`. |

**Tools (not installed or run here, so check their documentation):** `feature-engine` (transformers for scikit-learn), `category_encoders` (many encoders), `featuretools` (automatic features from related tables), `tsfresh` (automatic time-series features), `polars` and `duckdb` (fast aggregations on big tables), a feature store such as Feast.

**The picture**

```
TRAIN:  raw rows --build_features(T_i)--> features --[impute -> scale -> encode -> model]-
-> fitted pipeline
                                                                                        |
SERVE:  raw rows --build_features(today)-------------------------------------------------->
same pipeline.predict
       ^ same function                                  ^ same learned numbers
```

**Connection to the real job.** The ML engineer will ask you for exactly this: one function (or SQL view) that creates features at a given time, a fitted pipeline file, and a list of expected ranges. Giving them a messy notebook delays deployment by weeks.

**Applications**

- MLOps: shipping one fitted pipeline object; schema checks at the API.

- Fraud and credit: real-time feature computation with strict latency.
- Data Engineering: feature tables that are point-in-time correct.
- Any CV-based model comparison: pipelines inside cross-validation.

| COMMON MISTAKE | Fitting an imputer, scaler or selector on the full dataset "to save time". The effect is sometimes small (scalers) and sometimes huge (target-aware steps, as 0.757 vs 0.517 showed). Use a pipeline always; it costs nothing. |
|---|---|

**Check yourself**

1. What goes inside the cross-validation loop?

Answer: every step that learns from the data, especially those using the target (selection, target encoding, imputation, resampling).

1. What does `handle_unknown="ignore"` do?

Answer: unseen categories become all zeros instead of causing an error.

1. What is training-serving skew?

Answer: features are computed differently in training and in production.

## Chapter recap

- Behaviour over time (recency, frequency, monetary, windows, trends, gaps) is the strongest feature family: it lifted AUC from 0.612 (profile only) to 0.891.
- Always compute features from rows before T, with `age_d` relative to T, and pass T into the feature function.
- Do one example by hand before running it on millions of rows.
- RFM segments with no model separated customers 19 to 1 (53.8% vs 2.8% churn).
- Add features in groups and test on a later period: 21 more features added no AUC in our data.
- Time-split your validation; keep customer repeats on one side if you split randomly.
- Selection methods: hygiene, filter (correlation, MI), embedded (L1), wrapper (RFE), permutation importance; five features matched twenty-five.
- PSI per feature (below 0.1 stable, 0.1 to 0.25 watch, above 0.25 act); "time since" features drift by design.
- Put everything that learns from data, especially from the target, inside a pipeline; selection outside CV gave 0.757 on pure noise vs 0.517 inside.
- One feature function and one fitted pipeline for training and serving prevents skew.

| Task | Pattern | Example |
|---|---|---|
| Window mask | `past.age_d <= w` with `age_d = (T - ts).dt.days` | 7, 30, 90 days |
| Trend | `(a_now + 1) / (a_before + 1)` | 3.33 in Problem 6.1.1 |
| Gap stats | `groupby.order_ts.diff().dt.days` then `agg` | mean 10.75, max 25 |
| Recency over gap | `recency / (gap_mean + 1)` | 0.17 |
| RFM score | `pd.qcut(x, 5, labels=...)` | R, F, M in 1 to 5 |
| PSI | `Σ (a - e) ln(a/e)` | 0.0629 by hand |
| Permutation importance | `permutation_importance(m, X, y, scoring="roc_auc")` | recency drop 0.198 |
| Pipeline | `Pipeline([("pre", ColumnTransformer(...)), ("clf", model)])` | AUC 0.893 |

# Practice set

**Problem 6.P.1** (easy) An order happened 12 days before T. Which windows (7, 30, 90) contain it? **Answer:** The 30-day and 90-day windows, not the 7-day one.

**Problem 6.P.2** (easy) Compute the PSI for expected [0.6, 0.4] and actual [0.6, 0.4]. **Answer:** 0, since both differences and both logs are 0.

**Problem 6.P.3** (easy) `orders_30d` = 6, `orders_30_60d` = 2, `orders_60_90d` = 4. Trend using the chapter formula? **Answer:** (6 + 1) / ((2 + 4)/2 + 1) = 7 / 4 = 1.75.

**Problem 6.P.4** (medium) Orders at ages 1, 8 and 15 days before T. Gaps, mean gap, and `rec_over_gap`? **Answer:** Sorted oldest to newest ages 15, 8, 1 give gaps 7 and 7; mean 7. Recency = 1. `rec_over_gap` = 1 / (7 + 1) = 0.125.

**Problem 6.P.5** (medium) What is the output? `pd.qcut(pd.Series([1,2,3,4]), 2, labels=[0,1]).tolist()` **Answer:** `[0, 0, 1, 1]`.

**Problem 6.P.6** (medium) Find the bug: `df["orders_30d"] = (orders.order_ts >= pd.Timestamp.today() - pd.Timedelta(days=30)).groupby(orders.customer_id).sum()` used for a training table at T = 2024-08-01. **Answer:** The window is relative to today, not T, and has no upper bound `order_ts < T`: it includes future orders. Use `T - 30 days <= order_ts < T`.

**Problem 6.P.7** (medium) Expected shares [0.7, 0.2, 0.1], actual [0.5, 0.3, 0.2]. Compute PSI and say which action. **Answer:** (0.5-0.7)ln(0.5/0.7) = (-0.2)(-0.3365) = 0.0673; (0.3-0.2)ln(1.5) = 0.0405; (0.2-0.1)ln(2) = 0.0693; total 0.177. Moderate shift (0.1 to 0.25): watch and investigate.

**Problem 6.P.8** (hard) A random split on stacked snapshots gives AUC 0.93, a time split gives 0.86. Which do you report, and why do they differ? **Answer:** 0.86. The random split puts the same customers and similar periods in train and test, so it is optimistic; the time split copies real use (predict the future from the past).

**Problem 6.P.9** (interview) "Your model's AUC in production is 0.08 lower than in testing. How do you investigate?" **Answer:** Check training-serving skew first: compare feature distributions (PSI, means, missing rates) between training data and live data; check late-arriving data, time zones and definition changes; check label delay in monitoring; then check population drift and leakage in the original validation (random split, target encoding outside folds).

**Problem 6.P.10** (interview) "How would you select features for a model that must run in 50 ms?" **Answer:** Start with permutation importance and a cost column (compute time and data-source latency) per feature; drop

leakage, duplicates and unstable features; prefer features from fast stores (precomputed aggregates); keep the smallest set within 1 to 2 AUC points of the full model; test latency end to end.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, and English is my second language. Teach me behavioural features: RFM, rolling windows (7/30/90 days), lags, gaps, trends, velocity and acceleration, using a small orders table with 8 rows that we compute by hand together. Check my understanding with one question after each feature type, and show me the pandas code only after I try.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: "point-in-time correct feature building from an event table". Give me 8 graded exercises that start from tiny order lists and a prediction date T (compute windows, gaps, trends by hand), then 2 exercises where I must find a leakage bug in pandas code. Do not give solutions until I answer; then grade me and explain.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional new to data science. Topic: "feature selection (filter, wrapper, embedded, permutation importance) and feature stability with PSI". Quiz me one question at a time, mixing numerical PSI calculations with conceptual questions such as when permutation importance misleads. Wait for my answer, grade it, and explain. Ask 10 questions.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Review my scikit-learn pipeline and feature code below. Check whether any step that learns from data (imputer, scaler, selector, target encoder) is fitted outside the cross-validation loop, whether features use only data before the prediction time, whether unseen categories and missing values are handled, and whether training and serving use the same code. Give concrete fixes in simple English. My code: [PASTE HERE]
```

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional preparing for data science interviews in India. Run a mock interview on feature engineering and pipelines: "what features for churn, fraud or credit", "random vs time split", "AUC dropped in production", "training-serving skew". Ask one question at a time, score me out of 10, give a model answer and one follow-up. Do 8 rounds.

CHAPTER 7

# EXPERIMENTATION: A/B TESTING DONE PROPERLY

*How to design, size, analyse and stop an online experiment so that its answer can be trusted, shown on one retention-offer test.*

Your product manager says: "We built a new retention offer. Churn looks lower for people who got it. Ship it?" Looking at old data cannot answer that, because the people who got the offer were chosen in some way. An **A/B test** can answer it. You flip a fair coin for every customer: heads gets the new offer, tails gets the old one. Because the coin ignores everything about the customer, the two groups are alike in every way, including things you never measured. Any later difference in outcome is then caused by the offer. The maths of this is easy. What goes wrong in real companies is the design, the sample size, the stopping rule and the checks. By the end of this chapter you will be able to size a test with real numbers, analyse it, cut its noise with CUPED, detect a broken test with an SRM check, explain why peeking is dangerous, and decide when a bandit is the better tool.

**Running example.** The churn product from Chapters 1 to 3. The control group (A) gets the standard email. The treatment group (B) gets a new retention offer. The primary metric is "customer places an order in the next 30 days", with a baseline rate of 10%. All data below is synthetic with fixed seeds, and every output was produced by Python (`numpy`, `scipy`) in this session.

## 7.1 Designing the Test and Choosing the Sample Size

**INTUITION**

Think of a delivery firm testing a new route. If you time 20 deliveries and the average goes from 480 minutes to 450, you do not trust it. You need enough samples that real change stands out from random wobble. An A/B test is the same: before you start, you decide how many customers you need so that a real improvement of a size you care about will show up reliably.

### Words you need

| Word | Plain meaning | Example in our test |
|---|---|---|
| Hypothesis | A claim you can prove wrong | The new offer raises the 30-day order rate |

| Word | Plain meaning | Example in our test |
|---|---|---|
| Primary metric | The one number that decides ship or not | 30-day order rate |
| Guardrail metric | A number that must NOT get worse | Unsubscribe rate, refund rate, page latency |
| Randomisation unit | The thing the coin is flipped for | Customer (not email, not session) |
| Exposure criterion | Who counts as "in the test" | Customers who actually received the email |
| Baseline rate | The control group's expected rate | 10% |
| MDE (minimum detectable effect) | The smallest lift worth detecting | +1 percentage point (10% to 11%) |
| Alpha (α) | Chance of a false alarm when there is no effect | 0.05 |
| Power (1 − β) | Chance to catch the effect when it is real | 0.80 |

**Why randomise by customer?** If you randomise by email send, the same person could get both versions and remember both. The unit must match how the outcome is measured. Our outcome is per customer, so the coin is per customer.

**Why a guardrail?** A new offer can raise orders by giving a huge discount, while profit falls. Guardrails are your seatbelt.

## The sample size formula

For a yes/no metric, with control rate `p1`, treatment rate `p2`, `p̄ = (p1 + p2)/2`:

```
n per arm = ( z_α/2 · √(2·p̄·(1−p̄))  +  z_β · √(p1(1−p1) + p2(1−p2)) )²  /  (p2 − p1)²
```

`z_α/2` is the normal-curve cut-off for the false alarm rate. For α = 0.05 (two-sided) it is 1.96. `z_β` is the cut-off for power. For power 0.80 it is 0.8416. A handy rule: `n ≈ 16 · p(1−p) / δ²` where `δ` is the absolute lift you want to detect. The 16 assumes a two-sided test at α = 0.05 with 80% power (it is 2 × (1.96 + 0.84)² = 15.7, rounded up); Section 7.6 shows the constant for a one-sided test or 90% power.

**Problem 7.1.1** (tiny numbers) Using the rule of thumb, how many customers per arm to detect a lift from 10% to 11%?

| | |
|---|---|
| **Formula** | $n \approx 16 \cdot p(1-p) / \delta^2$ |
| **Plug in** | $16 \times 0.10 \times 0.90 / (0.01)^2$ |
| **Compute** | $16 \times 0.09 = 1.44$, and $0.01^2 = 0.0001$, so $1.44 / 0.0001 = 14{,}400$ |
| **Answer** | About 14,400 customers per arm. |
| **Check** | The exact formula in the next problem gives 14,751, close to the rule. |
| **Why it matters** | In a meeting you can do this on paper and tell the PM "we need about 15 thousand per arm" before anyone writes code. |

**Problem 7.1.2** (exact formula) Same test, full formula.

| | |
|---|---|
| **Formula** | the formula above, with $p1 = 0.10$, $p2 = 0.11$, $z_{\alpha/2} = 1.96$, $z_\beta = 0.8416$ |
| **Plug in** | $\bar{p} = 0.105$. First term: $1.96 \times \sqrt{(2 \times 0.105 \times 0.895)} = 1.96 \times \sqrt{0.18795} = 1.96 \times 0.43353 = 0.8497$. Second term: $0.8416 \times \sqrt{(0.10 \times 0.90 + 0.11 \times 0.89)} = 0.8416 \times \sqrt{0.1879} = 0.8416 \times 0.43347 = 0.3648$ |
| **Compute** | $(0.8497 + 0.3648)^2 = 1.2145^2 = 1.4750$, then $1.4750 / 0.0001 = 14{,}750$ |
| **Answer** | Python gives 14,750.8, so use 14,751 per arm, 29,502 in total. |
| **Check** | We simulated this: 2,000 fake experiments with exactly 14,751 per arm and a real lift 10% to 11%. The test found a significant result in 80.1% of them. That is the 80% power we asked for. |
| **Why it matters** | The formula is not magic. You can confirm it by simulation, and you should when the metric is unusual. |

```
import math
from scipy import stats

def n_per_arm(p1, p2, alpha=0.05, power=0.8):
    za = stats.norm.ppf(1 - alpha/2); zb = stats.norm.ppf(power)
    pbar = (p1 + p2) / 2
    num = (za*math.sqrt(2*pbar*(1-pbar)) + zb*math.sqrt(p1*(1-p1) + p2*(1-p2)))**2
    return num / (p2 - p1)**2

print(n_per_arm(.10, .11))    # 14750.79
print(n_per_arm(.10, .12))    # 3840.85
print(n_per_arm(.10, .105))   # 57762.65
```

Look at the three outputs. Halving the lift (1 point to 0.5 point) needs about **four times** the customers (14,751 to 57,763), because n depends on `1/δ²`. Doubling the lift (1 point to 2 points) needs about a quarter (3,841).

## The picture

```
Lift you want to detect (δ)   Customers per arm (10% baseline)
   2.0 points                   3,841    ####
   1.0 point                   14,751    ###############
   0.5 point                   57,763
############################################################
```

**Duration.** Say you get 2,000 eligible customers per day and need 29,502 in total. That is 14.75 days. Do not stop at day 15. Round up to **full weeks** (here, 3 weeks), because Saturday customers behave differently from Tuesday customers. A test that covers 2 Saturdays and 1 Tuesday is biased.

**Problem 7.1.3** (combine ideas) Your traffic is 1,200 eligible customers per day. The baseline is 10%. The PM says "we only care if the offer gives at least a 2-point lift". How long must the test run?

| | |
|---|---|
| **Step 1** | n per arm for 10% to 12% is 3,841 (Python output above). |
| **Step 2** | Total = 2 × 3,841 = 7,682 customers. |
| **Step 3** | Days = 7,682 / 1,200 = 6.4 days. |
| **Step 4** | Round up to full weeks: 7 days. |
| **Answer** | One full week is enough for a 2-point lift. If the PM later says "1 point matters", you need 29,502 / 1,200 = 24.6 days, so 4 weeks. |
| **Check** | 7 days × 1,200 = 8,400, which is above 7,682. Good. |
| **Why it matters** | The MDE conversation is really a business conversation: "how small a win is worth the weeks of waiting?" |

## Connection to the real job

Experiment platforms at product companies run this calculation for every test. As a data scientist you own the design doc: hypothesis, primary and guardrail metrics, unit, MDE, duration, and the decision rule written BEFORE the test starts.

## Applications

- **E-commerce:** checkout button, delivery fee display, search ranking changes.
- **Fintech/BFSI:** reminder message wording for loan repayment.
- **Streaming:** new home-page layout, measured on watch time.
- **Operations:** comparing two packing procedures in the same warehouse on the same days is an A/B test too.

| | |
|---|---|
| **COMMON MISTAKE** | Choosing the MDE from what you hope to see, not from what the business needs. If you size the test for a 2-point lift and the true lift is 1 point, you will most likely see "no significant difference" and wrongly conclude the offer does nothing. An underpowered test is not evidence of no effect. |

**Check yourself**

1. What happens to n if the lift to detect is cut to one third? Answer: n grows about 9 times, because n depends on $1/\delta^2$.

2. Why run in full weeks? Answer: weekday and weekend customers differ, so partial weeks bias the result.
3. What is a guardrail metric? Answer: a metric that must not get worse, such as refunds, even if the primary metric improves.

## 7.2 Analysing the Result and Cutting Noise with CUPED

| INTUITION | Two classrooms take the same exam. One class has a few brilliant students by luck, so its average is higher. If you also know each student's mark from last term, you can subtract "what we expected from them" and compare only the surprise. The noise from who happened to land in which class shrinks. That is CUPED. |
|---|---|

### The basic analysis

After the test ends, you compare two rates. The **two-proportion z-test** asks: if the real rates were equal, how surprising is a gap this big? The **p-value** is that surprise probability. A **confidence interval (CI)** gives a range of lifts that fit the data. Remember Chapter 4.4: a p-value does not say how big the effect is; the CI and the effect size do.

**Problem 7.2.1** Control: 1,000 of 10,000 customers ordered. Treatment: 1,120 of 10,000. Analyse.

| | |
|---|---|
| **Formula** | $z = (p_B - p_A) / SE$, with pooled rate $p = (c_A + c_B)/(n_A + n_B)$, $SE = \sqrt{p(1-p)(1/n_A + 1/n_B)}$ |
| **Plug in** | $p_A = 0.100$, $p_B = 0.112$, $p = 2{,}120 / 20{,}000 = 0.106$ |
| **Compute** | $SE = \sqrt{0.106 \times 0.894 \times 0.0002} = \sqrt{0.00001895} = 0.004353$. $z = 0.012 / 0.004353 = 2.756$. Two-sided p-value = 0.0058. |
| **Answer** | The absolute lift is 1.2 points, the relative lift is 1.2 / 10 = 12%. The 95% CI for the absolute lift (using the unpooled SE of 0.004353) is 0.35 to 2.05 points. We reject "no effect" at $\alpha = 0.05$. |
| **Check** | The CI does not include 0, which agrees with $p < 0.05$. |
| **Why it matters** | Report both: "+1.2 points (+12% relative), 95% CI 0.35 to 2.05 points". The interval tells the PM the win could be small. |

**Absolute vs relative lift.** Always state both and say which. "12% lift" sounds large, but it is 1.2 extra orders per 100 customers.

## CUPED: use the past to remove noise

**CUPED** means Controlled-experiment Using Pre-Experiment Data. Each customer has a **pre-period value** `X` (for example spend in the 30 days before the test) and an outcome `Y` (spend during the test). Because the coin was fair, `X` has the same average in both groups. So we can safely adjust:

```
θ = Cov(Y, X) / Var(X)
Y_adj = Y − θ · (X − mean(X))
```

The adjusted outcome keeps the same treatment effect but has smaller variance: `Var(Y_adj) = Var(Y) · (1 − ρ²)`, where `ρ` (rho) is the correlation between `X` and `Y`.

**Problem 7.2.2** We simulate 20,000 customers (seed 42). Pre-period spend X is skewed (gamma shaped). During-test spend Y = 0.8·X + noise + 100, and the offer truly adds ₹15 to spend. Compare the plain and CUPED estimates.

```
import numpy as np, math
rng = np.random.default_rng(42); N = 20000
pre = rng.gamma(2, 150, N)
noise = rng.normal(0, 200, N)
y0 = 0.8*pre + noise + 100
T = rng.integers(0, 2, N)
y = y0 + 15*T

def diff_se(y, T):
    a, b = y[T==1], y[T==0]
    return a.mean()-b.mean(), math.sqrt(a.var(ddof=1)/len(a) + b.var(ddof=1)/len(b))

print(diff_se(y, T))
theta = np.cov(y, pre)[0,1] / pre.var(ddof=1)
yc = y - theta*(pre - pre.mean())
print(theta, np.corrcoef(y, pre)[0,1])
print(diff_se(yc, T))
```

Real output:

```
plain  : diff 18.58, SE 3.71
theta  : 0.793, correlation rho 0.649
CUPED  : diff 17.82, SE 2.82
Var(Y) 68,959  ->  Var(Y_adj) 39,888   ratio 0.578 = 1 - rho^2 (0.5784)
SE ratio 0.760
```

- **Step 1** Plain estimate: 18.58 with SE 3.71. The 95% CI is 18.58 ± 1.96 × 3.71 = 11.3 to 25.9. True effect 15 is inside.
- **Step 2** θ = 0.793, nearly the 0.8 we built in. That is the slope of "spend now" on "spend before".
- **Step 3** Check the theory: ρ = 0.649, so 1 − ρ² = 1 − 0.4216 = 0.578. The variance really fell by exactly that factor (0.578).
- **Step 4** The SE fell from 3.71 to 2.82, which is √0.578 = 0.760 of the old one.
- **Answer** CUPED gave 17.82 ± 2.82 (CI 12.3 to 23.3), a tighter answer from the same customers.
- **Check** Sample size scales with variance, so CUPED here is like having 1 / 0.578 = 1.73 times as many customers. The point estimate moved a little (18.58 to 17.82) because both are noisy estimates of 15; CUPED's one is closer.
- **Why it matters** With ρ = 0.65 you finish the test in about 58% of the time. At ρ = 0.9 the variance drops by 81%.

| KEY INSIGHT | CUPED only works if X is measured BEFORE the test, so the treatment cannot have changed it. If you use a covariate the treatment can affect, you will bias the result. Pre-period values of the same metric are the safest choice. |
|---|---|

**Related methods. Stratification** splits customers into groups (such as new vs old users) and averages the group effects. **Regression adjustment** fits `Y ~ T + X` and reads the coefficient on T. They are close cousins of CUPED and give similar results with one or many covariates.

## Connection to the real job

CUPED is built into most experimentation platforms. Interviewers love it because it shows you understand variance, not just p-values.

## Applications

- **Revenue per user tests:** heavy-tailed, so variance reduction matters most.
- **Search and recommendation tests:** pre-period clicks predict test-period clicks.
- **Ads:** pre-period spend per advertiser.

| COMMON MISTAKE | Using "days since signup" measured at the END of the test as the covariate, or computing the pre-period window after some customers were already treated. Always define X from a window that closes before randomisation. |
|---|---|

**Check yourself**

1. If ρ = 0.5, by what factor does CUPED cut variance? Answer: 1 − 0.25 = 0.75, so variance falls by 25%.

2. Why is the point estimate unbiased? Answer: X has the same mean in both groups because of randomisation, so subtracting θ(X − mean) shifts both groups the same way.
3. Which is more informative, p-value or CI? Answer: the CI, because it shows the size and the uncertainty.

# 7.3 Pitfalls: SRM, Peeking and Awkward Metrics

| INTUITION | Before you read a result, check the experiment is not broken. If you flipped a fair coin 100,000 times and got 50,550 heads, would you be suspicious? You should be. The same thinking, applied to group sizes, is the SRM check. |
|---|---|

## Sample ratio mismatch (SRM)

You planned a 50/50 split. SRM means the real group sizes differ from the plan by more than chance allows. It nearly always means a bug: one arm crashes more often, bots are filtered in one arm, or a redirect drops users. When SRM shows up, the results are not valid, even if they look great.

Test it with a **chi-square test**: `χ² = Σ (observed − expected)² / expected`. With two groups (1 degree of freedom), the 0.001 p-value alarm level is a common choice.

**Problem 7.3.1** 100,000 customers, planned 50/50. Arm A has 50,550 and B has 49,450. Is there SRM?

| | |
|---|---|
| **Formula** | $\chi^2 = \Sigma\ (O - E)^2 / E$ |
| **Plug in** | E = 50,000 for each. $(50{,}550 - 50{,}000)^2 / 50{,}000 + (49{,}450 - 50{,}000)^2 / 50{,}000$ |
| **Compute** | $550^2 = 302{,}500$, so 302,500 / 50,000 = 6.05 for each arm. Sum = 12.1. p-value = 0.0005. |
| **Answer** | p = 0.0005 is below 0.001, so we declare SRM. A 1.1% imbalance looks small, but with 100,000 customers chance almost never makes it. |
| **Check** | A different split, 50,120 vs 49,880, gives $\chi^2 = 0.576$ and p = 0.45, which is normal. |
| **Why it matters** | Teams that skip this check ship wrong results with confidence. Investigate the assignment code and logging first. |

## Peeking and optional stopping

**Peeking** means checking the p-value every day and stopping the first time it is below 0.05. It sounds harmless, but each look is another chance for noise to cross the line.

**Problem 7.3.2** Simulate 20,000 A/A tests (both arms identical at 10%, so no real effect), 10,000 customers per arm, with 10 equally spaced looks. How often do we see a "significant" result?

```
import numpy as np
rng = np.random.default_rng(2); sims = 20000; n = 10000
c = np.arange(1, 11) * 1000
peek = end = poc = 0
for _ in range(sims):
    a = np.cumsum(rng.binomial(1, .1, n))[c-1]
    b = np.cumsum(rng.binomial(1, .1, n))[c-1]
    p = (a+b)/(2*c); se = np.sqrt(2*p*(1-p)/c)
    z = np.abs((b-a)/c/se)
    peek += (z > 1.96).any(); end += z[-1] > 1.96; poc += (z > 2.555).any()
print(peek/sims, end/sims, poc/sims)
```

Real output: `0.18885 0.0454 0.0483`.

- **Step 1** Look only at the end: 4.5% false alarms, as promised by α = 5%.
- **Step 2** Stop at the first of 10 looks with z above 1.96: 18.9% false alarms. Nearly 4 times the promise.
- **Step 3** Use a stricter bar at every look (z above 2.555, the Pocock constant for 10 looks): 4.8%, back near 5%.
- **Answer** Peeking with a fixed threshold turns a 5% error into about 19%. One in five useless offers would be declared winners.
- **Check** The 4.5% and 4.8% are within simulation error of 5% (with 20,000 runs, the standard error is about 0.15 points).
- **Why it matters** "The p-value is 0.03 on day 3, let's ship" is how companies fill their product with changes that do nothing.

You have three honest options: (a) fix the sample size and look once; (b) use a **sequential test** whose thresholds are built for repeated looks (Pocock, O'Brien-Fleming, or **always-valid p-values** from mSPRT); (c) use a **Bayesian** analysis with a pre-set decision rule, but note that Bayesian stopping also needs care and simulation to check its error rates.

## Other pitfalls, with fixes

| Pitfall | What it looks like | Fix |
|---|---|---|
| Novelty / primacy effect | Lift is big in week 1 then fades (or the opposite) | Run 2+ weeks, plot lift by day, look at new vs returning users |

| Pitfall | What it looks like | Fix |
|---|---|---|
| Interference / network effects | Treated users affect control users (marketplaces, social) | Randomise by cluster, region or time block |
| Many metrics and segments | 20 metrics, one is "significant" | One primary metric; correct for multiple tests (Chapter 4.3) |
| Simpson's paradox in rollouts | Traffic mix differs by day, so pooled result flips | Keep the ratio constant over the whole test; analyse by segment |
| Ratio metrics | Click-through = clicks / sessions, where the unit is the user | Delta method or bootstrap by user |
| Heavy-tailed revenue | A few huge buyers swing the mean | Winsorise (cap the top 1%), use bootstrap, or CUPED |

**Ratio metrics and the delta method.** If the metric is total clicks divided by total sessions, but users are the random unit, then sessions from one user are not independent. A naive SE treats each session as independent and is too small. The **delta method** uses `Var(ratio) ≈ Var(Y − r·S) / (n · mean(S)²)`.

**Problem 7.3.3** 5,000 users, sessions per user S = Poisson(4) + 1, clicks per user ~ Binomial(S, 0.2) (seed 5). Overall CTR = 0.1990. Compare the SE.

| | |
|---|---|
| **Naive** | √(r(1−r)/total sessions) = 0.002537 |
| **Delta method** | 0.002529 |
| **Answer** | Here they agree closely, because clicks within a user are independent in our simulation. In real data, a heavy user who clicks a lot makes the naive SE too small, and the delta method is the safe choice. |
| **Check** | Our data was generated with independent clicks, so agreement is expected. We did not simulate user-level correlation here. |
| **Why it matters** | When in doubt, bootstrap by user: resample users, not sessions. |

## Connection to the real job

The first thing a good experimentation team does with a new result is the SRM check and a "trust" checklist. Interview question: "A test shows +5% with p = 0.001 but the arms have 52% / 48% traffic. What do you do?" Answer: do not read the result; find the SRM cause.

### Applications

- **Platform teams:** automatic SRM alerts on every experiment.
- **Growth teams:** guardrails against novelty-driven wins.
- **Marketplaces:** cluster or switchback designs for interference.

| COMMON MISTAKE | Treating a significant result inside a segment you chose after seeing the data as a finding. If you slice by 12 segments, one will be significant by chance about half the time ($1 - 0.95^{12} = 46\%$). Pre-register segments, or label the finding "a hypothesis for the next test". |
|---|---|

**Check yourself**

1. What does SRM usually indicate? Answer: a bug in assignment or logging, so the result is invalid.
2. Why does peeking inflate false positives? Answer: each look is another chance for noise to cross the threshold.
3. Randomisation unit is user, metric is per session. What is the risk? Answer: sessions from the same user are correlated, so a naive SE is too small.

## 7.4 Sequential Tests, Bandits and the Decision

| INTUITION | A bandit is a gambler in a casino with three slot machines of unknown payout. A fixed A/B test says "play each machine 3,333 times, then decide". A bandit says "play, but shift more turns toward the machine that is winning so far". You lose less money while learning, but you learn the weaker machines less precisely. |
|---|---|

### Multi-armed bandits

**Thompson sampling** keeps a belief for each arm's success rate (a Beta distribution: starts at "1 success, 1 failure"). Each turn it draws one random value from each belief, plays the arm with the biggest draw, and updates that arm with the result. Arms that look good get played more, but every arm still gets some chances.

**Problem 7.4.1** Three offers with true order rates 10%, 12%, 11% (unknown to us). 10,000 customers. Compare an equal split with Thompson sampling, averaged over 100 repeats (seed 3).

```
import numpy as np
true = np.array([.10, .12, .11]); T_ = 10000
def run(policy, rng):
    s = np.ones(3); f = np.ones(3); rew = 0; pulls = np.zeros(3)
    for t in range(T_):
        arm = int(np.argmax(rng.beta(s, f))) if policy == "ts" else t % 3
        r = rng.random() < true[arm]; rew += r
        s[arm] += r; f[arm] += 1 - r; pulls[arm] += 1
    return rew, pulls
```

Real output:

```
equal split : orders 1096.4, customers per arm [3334, 3333, 3333]
Thompson    : orders 1161.4, customers per arm [1015, 6828, 2158]
best possible (always the 12% offer): 1200
```

- **Step 1** The equal split gets about (0.10 + 0.12 + 0.11)/3 × 10,000 = 1,100 orders (matches 1,096).
- **Step 2** Thompson sent 68% of customers to the best offer and got 1,161 orders.
- **Answer** Thompson won about 65 extra orders per 10,000 customers (1,161 vs 1,096). The cost of learning (regret) was 1,200 − 1,161 = 39 orders, against 104 for the equal split.
- **Check** With 3 arms and the best at 12%, 1,200 is the ceiling. Neither policy can pass it.
- **Why it matters** If every wasted customer costs money, bandits pay.

**When NOT to use a bandit.** A bandit gives you less data about the losing arms, so you cannot estimate the exact lift with a tight CI. It also struggles when the effect changes over time (a Monday winner may lose on Saturday) and when the outcome arrives late (30-day order rates). Use a standard A/B test when you must give a clear causal estimate, report it to leadership, or learn WHY. Use bandits for continuous optimisation of many cheap variants (headlines, banners, notification times).

## Bayesian A/B in one line

Instead of a p-value, you compute "the probability that B is better than A", from the two Beta beliefs. It answers the question the PM actually asks. It still needs a stopping rule fixed in advance.

## The decision and the experiment log

Write the decision rule before the test:

| Result | Action |
|---|---|
| Primary metric significantly up, guardrails flat | Ship |
| Primary flat, CI excludes your MDE, guardrails flat | Do not ship; the idea does not work at the size we care about |
| Primary flat, CI wide | Inconclusive; run longer or iterate on the idea |
| Primary up, a guardrail significantly worse | Do not ship; investigate the trade-off with the owner |

**Problem 7.4.2** In Problem 7.2.1 the lift is +1.2 points (CI 0.35 to 2.05) and the offer costs ₹30 per customer. A new order is worth ₹400 margin. Is it worth shipping?

| | |
|---|---|
| **Step 1** | Extra orders per customer = 0.012 (the point estimate). |
| **Step 2** | Extra margin per customer = 0.012 × 400 = ₹4.80. |
| **Step 3** | Cost per customer = ₹30. |
| **Answer** | Net = 4.80 − 30 = −₹25.20 per customer. Even at the top of the CI (0.0205 × 400 = ₹8.20), we lose money. The test is "statistically significant" and still a bad decision. |
| **Check** | The break-even lift is 30 / 400 = 7.5 points, much larger than anything in the CI. |
| **Why it matters** | Significance is not value. Always convert lift to money. |

Keep an **experiment log**: one row per test with hypothesis, dates, sample size, result with CI, decision and what you learned. Over time the log shows which ideas work, and prevents repeating failed ones. The standard book on all this is *Trustworthy Online Controlled Experiments* by Kohavi, Tang and Xu.

## Connection to the real job

Interviewers ask "design an experiment for feature X". Walk through: hypothesis, unit, metrics (primary, guardrails), MDE and n, duration, checks (SRM), analysis plan, decision rule. You now have every item.

## Applications

- **Notifications:** bandits to pick send time.
- **Pricing:** A/B tests with guardrails on margin.
- **Ad creatives:** bandits among many variants.
- **Medical-style trials:** sequential designs allow early stopping for benefit or harm.

| COMMON MISTAKE | Using a bandit and then analysing it with a plain A/B z-test. The data are not independent draws from fixed arms, so p-values are wrong. Use methods built for adaptive data. |
|---|---|

**Check yourself**

1. Which has lower regret, equal split or Thompson? Answer: Thompson (39 vs 104 orders in our run).
2. Why can a test be significant but not worth shipping? Answer: the effect can be too small to pay for the cost; convert lift to money.
3. Name one case where an A/B test beats a bandit. Answer: when you need a precise, defensible estimate of the effect, or the outcome is delayed.

# 7.5 Ratio Metrics and the Delta Method

Many headline metrics are ratios: click-through rate is clicks per session, revenue per session, orders per visit. The unit you randomise is the *user*, but the denominator counts *sessions*. Users with many sessions contribute many rows, and a user's sessions are not independent (a person who likes clicking clicks in most sessions). If you treat each session as an independent coin flip you get a standard error that is too small and too many false wins.

| INTUITION | Ten sessions from one enthusiastic user are not ten independent opinions. They are closer to one opinion counted ten times. The right count of independent pieces of evidence is the number of *users*, so the standard error must be built at the user level. |
|---|---|

## The delta method in one formula

Let each user have clicks `c` and sessions `s`. The metric is `R = Σc / Σs = c̄ / s̄`. A first-order Taylor expansion (the delta method) gives the variance of a ratio of two means across `n` users:

$$Var(R) \approx \left( Var(c) - 2 \cdot R \cdot Cov(c, s) + R^2 \cdot Var(s) \right) / \left( n \cdot \bar{s}^2 \right)$$

To test a difference between arms, compute `R` and `Var(R)` in each arm, add the two variances and take the square root for the standard error of the difference.

**Problem 7.5.1** Five users in one arm have sessions 1, 2, 4, 3, 10 and clicks 0, 1, 1, 1, 4. Find the CTR and its delta-method standard error.

| | |
|---|---|
| **Ratio** | R = 7 / 20 = 0.35. |
| **Moments** | $\bar{s} = 4$, Var(s) = 12.5, Var(c) = 2.3, Cov(c, s) = 5.25 (sample versions with n − 1). |
| **Numerator** | $2.3 - 2 \times 0.35 \times 5.25 + 0.35^2 \times 12.5 = 2.3 - 3.675 + 1.53125 = 0.15625$. |
| **Divide** | $n \cdot \bar{s}^2 = 5 \times 16 = 80$, so Var(R) = 0.15625 / 80 = 0.001953 and SE = 0.0442. |
| **Check** | A bootstrap over users (code below) gives 0.0485. With only 5 users the first-order approximation is rough; with thousands of users the two agree closely. |

```
import numpy as np
# Tiny hand example: 5 users in one arm
s = np.array([1, 2, 4, 3, 10.0])      # sessions
c = np.array([0, 1, 1, 1, 4.0])       # clicks
n = len(s)
R = c.sum() / s.sum()
ms, mc = s.mean(), c.mean()
vs, vc = s.var(ddof=1), c.var(ddof=1)
cov = np.cov(c, s, ddof=1)[0, 1]
var_R = (vc - 2 * R * cov + R**2 * vs) / (n * ms**2)
print("R", R, "var_c", vc, "var_s", vs, "cov", cov)
print("delta var", round(var_R, 5), "se", round(var_R**.5, 4))
rng = np.random.default_rng(0)
idx = rng.integers(0, n, (20000, n))
boot = c[idx].sum(1) / s[idx].sum(1)
print("bootstrap se", boot.std().round(4))
```

```
R 0.35 var_c 2.3 var_s 12.5 cov 5.25
delta var 0.00195 se 0.0442
bootstrap se 0.0485
```

**Problem 7.5.2** Now a realistic arm of 20,000 users. Compare the naive session-level standard error with the delta method and a user-level bootstrap.

```
import numpy as np
rng = np.random.default_rng(1)
n = 20000
# heavy users have many sessions AND a user-specific click rate (users are clustered)
rate = rng.beta(2, 18, n)                        # user click propensity, mean 0.10
sess = rng.poisson(1, n) + 1 + (rng.random(n) < 0.05) * rng.poisson(30, n)
clicks = rng.binomial(sess, rate)
R = clicks.sum() / sess.sum()
# naive: treat every session as independent Bernoulli
se_naive = np.sqrt(R * (1 - R) / sess.sum())
# delta method at user level
ms = sess.mean()
cov = np.cov(clicks, sess)[0, 1]
var_R = (clicks.var(ddof=1) - 2 * R * cov + R**2 * sess.var(ddof=1)) / (n * ms**2)
# bootstrap users
idx = rng.integers(0, n, (400, n))
boot = clicks[idx].sum(1) / sess[idx].sum(1)
print("CTR", round(R, 4), "sessions per user", round(ms, 2))
print("naive SE", round(se_naive, 5), " delta SE", round(var_R**.5, 5), " bootstrap SE",
round(boot.std(), 5))
print("naive understates SE by factor", round(var_R**.5 / se_naive, 2))
```

```
CTR 0.0963 sessions per user 3.48
naive SE 0.00112  delta SE 0.00146  bootstrap SE 0.00152
naive understates SE by factor 1.3
```

**How to read it.** The naive standard error is about 23% too small (the delta SE is 1.30 times larger). Using the naive number would turn a borderline result into a false win. The delta method and the bootstrap agree with each other (0.00146 and 0.00152). The gap grows with the clustering of sessions inside users and with how unequal the session counts are.

| COMMON MISTAKE | Running a two-proportion z-test on *sessions* when randomisation was by *user*. The test assumes independent rows. Either analyse at user level (clicks per user, then the delta method for the ratio) or bootstrap by user. Also check that the denominator itself is not affected by the treatment: if the new design changes the number of sessions, the ratio mixes two effects. |
|---|---|

### Going deeper

A neat equivalent view is **linearisation**: replace each user's pair (c, s) with $c - R \cdot s$ where $R$ is the control ratio. The treatment effect on that single per-user number is, to first order, the effect on the ratio, and you can use ordinary t-tests, CUPED and sample-size formulas on it. This is how most experimentation platforms implement ratio metrics.

## 7.6 Sample Size for Non-Binary Metrics and Smarter Variance Reduction

Section 7.1 sized a yes/no metric. The rule of thumb $n \approx 16\, p(1-p)/\delta^2$ assumes a two-sided 5% test and 80% power ($2 \cdot (1.96 + 0.84)^2 = 15.7$, rounded to 16). For other settings change the 15.7: a one-sided 5% test needs about 12.4 ($2 \cdot (1.645+0.842)^2$), and 90% power at two-sided 5% needs about 21.0 ($2 \cdot (1.96+1.282)^2$). In general, for a metric with standard deviation $\sigma$ per user, $n \text{ per arm} = 2 \cdot (z_{\alpha/2} + z_{\beta})^2 \cdot \sigma^2 / \delta^2$.

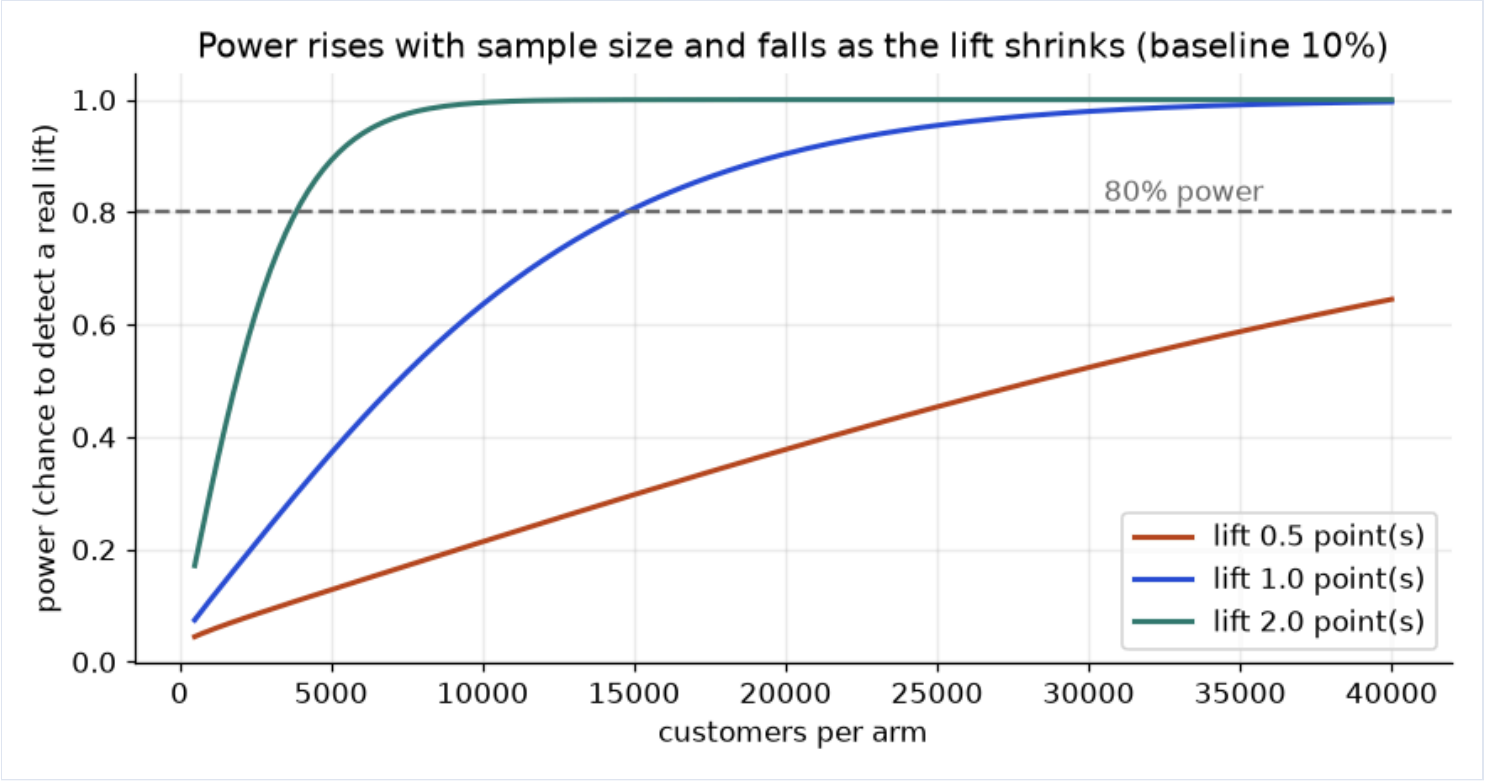
**Problem 7.6.1** Three sample-size calculations. (a) Revenue per user has σ = 120 rupees; detect +2 rupees. (b) The same with CUPED where ρ = 0.5. (c) A ratio metric (CTR 0.10) whose linearised per-user value has σ = 0.35 (assumed); detect +0.005. (d) The 10% to 11% binary case under three settings.

```
from scipy import stats
za, zb = stats.norm.ppf(0.975), stats.norm.ppf(0.8)
k = 2 * (za + zb) ** 2
print("2*(za+zb)^2 =", round(k, 3))
# continuous: revenue per user mean 50, sd 120, detect +2
sd, delta = 120, 2
print("continuous n per arm", round(k * sd**2 / delta**2))
# CUPED with rho = 0.5 cuts variance by 25%
print("with rho 0.5:", round(k * sd**2 * (1 - 0.25) / delta**2))
# ratio metric: delta-method variance per user of the linearised metric
# CTR 0.10, sd of linearised value per user = 0.35, detect +0.005
print("ratio n per arm", round(k * 0.35**2 / 0.005**2))
# one-sided 5% and power 90% for the binary 10% vs 11% case
def n_bin(p1, p2, alpha, power, two=True):
    za = stats.norm.ppf(1 - alpha / (2 if two else 1)); zb = stats.norm.ppf(power)
    pb = (p1 + p2) / 2
    return (za*(2*pb*(1-pb))**.5 + zb*(p1*(1-p1)+p2*(1-p2))**.5)**2 / (p2-p1)**2
print("binary 10 to 11:", round(n_bin(.10,.11,.05,.8)),
round(n_bin(.10,.11,.05,.8,False)), round(n_bin(.10,.11,.05,.9)))
```

```
2*(za+zb)^2 = 15.698
continuous n per arm 56512
with rho 0.5: 42384
ratio n per arm 76919
binary 10 to 11: 14751 11619 19747
```

| | |
|---|---|
| **(a) by hand** | 15.698 × 120² / 2² = 15.698 × 14,400 / 4 = 56,513 per arm (the code rounds the unrounded constant and prints 56,512). |
| **(b) by hand** | CUPED multiplies the variance by (1 − ρ²) = 0.75, so n falls by a quarter: 42,384. |
| **(c) by hand** | 15.698 × 0.35² / 0.005² = 15.698 × 0.1225 / 0.000025 = 76,920 (code: 76,919). |
| **(d)** | 14,751 for two-sided 5% with 80% power. A one-sided test needs 11,619 (21% fewer), and 90% power needs 19,747 (34% more). |

**Reading the answers.** Continuous revenue metrics are heavy-tailed, so σ is large relative to a small lift, and they need far more users than a conversion rate. This is why teams cap outliers (winsorise at the 99th percentile), use CUPED, or test a conversion proxy.




*Power against customers per arm for three lifts. The dashed line is 80% power.*

**How to read it.** Find where each curve crosses the dashed line: that x value is the sample size for that lift (about 3,800, 14,750 and 57,800 per arm). A curve only reaches the top of the chart when n is large for that lift. A test that stops at 10,000 per arm has only about 64% power for the 1-point lift and about 21% for 0.5 points.

## Variance reduction beyond CUPED

CUPED is one case of **regression adjustment**: add pre-experiment covariates to a regression of the outcome on treatment. The treatment coefficient is still unbiased because assignment is random, and the standard error shrinks by $\sqrt{(1 - R^2)}$ of the covariates. **Post-stratification** instead computes the effect inside each segment (for example free and premium customers) and averages with population weights; it removes the variance that comes from random imbalance in segment mix.

**Problem 7.6.2** Simulate 8,000 customers with a pre-period spend and a premium flag, true effect +3. Repeat the random assignment 600 times and compare the spread of three estimators.

```
import numpy as np
rng = np.random.default_rng(3)
n = 8000
pre = rng.gamma(2, 25, n)                       # last quarter spend
seg = (rng.random(n) < 0.3).astype(int)         # 1 = premium segment
y0 = 0.6 * pre + 40 * seg + rng.normal(0, 30, n)
def one_trial(rng):
    t = rng.integers(0, 2, n)
    y = y0 + 3 * t + rng.normal(0, 5, n)        # true effect +3
    plain = y[t == 1].mean() - y[t == 0].mean()
    # post-stratified by segment
    est = 0
    for g in (0, 1):
        m = seg == g
        est += m.mean() * (y[m & (t == 1)].mean() - y[m & (t == 0)].mean())
    # regression adjustment: y ~ t + pre + seg
    X = np.c_[np.ones(n), t, pre - pre.mean(), seg - seg.mean()]
    beta = np.linalg.lstsq(X, y, rcond=None)[0]
    return plain, est, beta[1]
res = np.array([one_trial(rng) for _ in range(600)])
print("mean estimate (truth 3):", res.mean(0).round(2))
print("empirical SE: plain, stratified, regression:", res.std(0).round(2))
print("variance vs plain:", (res.var(0) / res.var(0)[0]).round(2))
```

```
mean estimate (truth 3): [3.1  3.06 3.04]
empirical SE: plain, stratified, regression: [0.9  0.79 0.65]
variance vs plain: [1.   0.77 0.52]
```

**How to read it.** All three are centred near the truth (3), so none is biased. The spread differs: stratifying by segment cuts the variance to 77% of the plain estimator, and regression on pre-spend and segment cuts it to 52%, the same as using a sample about twice as large. Covariates must be measured *before* assignment; one measured after can be changed by the treatment and will bias the estimate.

# 7.7 Many Metrics: Guardrails and False Discovery Control

A real test tracks 20 or 30 metrics. At a 5% level each, a null test shows about one "significant" metric by luck. Two standard fixes: **Bonferroni** (use $\alpha/m$ for each of $m$ metrics; strict, controls the chance of *any* false alarm) and **Benjamini-Hochberg (BH)**, which controls the expected share of false discoveries among the metrics you call significant (the false discovery rate, FDR).

**Problem 7.7.1** Ten metrics have p-values 0.001, 0.008, 0.012, 0.041, 0.049, 0.20, 0.30, 0.50, 0.70, 0.90. Apply Bonferroni and BH at 5%.

| | |
|---|---|
| **Bonferroni** | Threshold 0.05 / 10 = 0.005. Only 0.001 passes: 1 metric. |
| **BH step 1** | Sort the p-values (already sorted). The k-th smallest is compared with k × 0.05 / 10: 0.005, 0.010, 0.015, 0.020, 0.025, ... |
| **BH step 2** | k = 1: 0.001 ≤ 0.005 yes. k = 2: 0.008 ≤ 0.010 yes. k = 3: 0.012 ≤ 0.015 yes. k = 4: 0.041 ≤ 0.020 no. k = 5: 0.049 ≤ 0.025 no. |
| **BH step 3** | Take the largest k that passes: k = 3. Reject the three smallest p-values. |
| **Answer** | Bonferroni finds 1, BH finds 3. BH is less strict because it accepts a few false findings in exchange for power. |

```
import numpy as np
p = np.array([0.001, 0.008, 0.012, 0.041, 0.049, 0.2, 0.3, 0.5, 0.7, 0.9])
m, q = len(p), 0.05
order = np.argsort(p); ps = p[order]
thr = q * np.arange(1, m + 1) / m
ok = ps <= thr
k = np.max(np.where(ok)[0]) + 1 if ok.any() else 0
print("BH thresholds", thr.round(3)[:5], "largest k", k, "rejected", ps[:k])
print("Bonferroni rejects", (p <= q / m).sum())
adj = np.minimum.accumulate((ps * m / np.arange(1, m + 1))[::-1])[::-1]
print("BH adjusted p-values", adj.round(3))
```

```
BH thresholds [0.005 0.01  0.015 0.02  0.025] largest k 3 rejected [0.001 0.008 0.012]
Bonferroni rejects 1
BH adjusted p-values [0.01  0.04  0.04  0.098 0.098 0.333 0.429 0.625 0.778 0.9  ]
```

| | |
|---|---|
| **KEY IDEA** | Use a hierarchy. Pick *one* primary metric before the test (no correction needed). Treat a small set of secondary metrics with BH. Treat guardrails (latency, refunds, complaints) differently: you are not trying to prove they moved, you are trying to rule out harm, so use a one-sided non-inferiority bound such as "refund rate rose by less than 0.3 points with 95% confidence". |

| | |
|---|---|
| **COMMON MISTAKE** | Slicing a flat result by 12 segments and announcing the one that is significant. That is 12 tests; the chance of at least one p below 0.05 is $1 - 0.95^{12} = 46\%$. Pre-register the slices, correct with BH, and treat a surprise segment as a hypothesis for the next test. |

## 7.8 When Users Affect Each Other: Interference, Switchbacks and Geo Tests

A/B testing assumes each user's outcome depends only on their own assignment (no interference, the "stable unit" assumption). That fails in marketplaces (drivers, riders, inventory), social products (friends see each other's changes) and auctions (ad budgets). Then user-level randomisation can show a lift that does not exist after launch.

**Problem 7.8.1** A market has 100,000 users and only 45,000 delivery slots. Each user requests a slot with probability 0.5 (control) or 0.6 (new design). Requests are served at random up to capacity. Compare the user-level A/B lift with the true effect of launching to everyone, and with a daily switchback.

| | |
|---|---|
| **Truth, hand** | All control: 50,000 requests, 45,000 slots, so 90% served and 0.5 × 0.9 = 0.45 orders per user. All treatment: 60,000 requests, 75% served, 0.6 × 0.75 = 0.45. Launching changes nothing: the extra requests just fight for the same slots. |
| **A/B, hand** | Half and half: requests = 100,000 × (0.25 + 0.30) = 55,000, so 45,000 / 55,000 = 81.8% served. Control: 0.5 × 0.818 = 0.409. Treatment: 0.6 × 0.818 = 0.491. The test says +0.082 (+20%). |
| **Why** | Treated users steal slots from control users. The comparison between arms is not a comparison between worlds. |

```
import numpy as np
rng = np.random.default_rng(0)
N, C, p0, p1 = 100000, 45000, 0.5, 0.6
def served(requests): return min(1.0, C / requests)       # share of requests that get a
slot
# truth by rolling out to everyone
ctrl = p0 * served(N * p0) ; trt = p1 * served(N * p1)
print("global control orders per user", round(ctrl, 3), " global treatment", round(trt,
3), " true effect", round(trt - ctrl, 3) + 0.0)
# user-level A/B inside the same market
treated = rng.random(N) < 0.5
req = rng.random(N) < np.where(treated, p1, p0)
sv = served(req.sum())
order = req & (rng.random(N) < sv)
print("A/B measured: control", round(order[~treated].mean(), 3), "treatment",
round(order[treated].mean(), 3),
      "lift", round(order[treated].mean() - order[~treated].mean(), 3))
# switchback: whole market alternates by day, 28 days
days = np.arange(28); tday = days % 2 == 1
o = []
for t in tday:
    r = rng.random(N) < (p1 if t else p0)
    o.append((r & (rng.random(N) < served(r.sum()))).mean())
o = np.array(o)
print("switchback estimate", round(o[tday].mean() - o[~tday].mean(), 3) + 0.0)
```

```
global control orders per user 0.45  global treatment 0.45  true effect 0.0
A/B measured: control 0.409 treatment 0.492 lift 0.083
switchback estimate 0.0
```

**How to read it.** The A/B test reports +0.083 orders per user (a 20% lift) when the true launch effect is zero. The switchback, which flips the *whole market* between control and treatment on alternating days, recovers the zero. Its cost is fewer independent units (28 days instead of 100,000 users), so it needs a longer run and care about carry-over from one day to the next (drop a washout period) and weekday effects (alternate so each weekday appears in both arms).

| Design | Randomise | Use when | Main cost |
|---|---|---|---|
| User-level A/B | Person | No sharing of supply, no network | Biased under interference |
| Cluster randomisation | Friend groups or cities | Social or local effects stay inside the cluster | Few clusters, less power |

| Design | Randomise | Use when | Main cost |
|---|---|---|---|
| Switchback | Time blocks for the whole market | Marketplaces, pricing, dispatch | Carry-over, few units |
| Geo experiment | Matched regions | Ads and offline effects, no user-level tracking | Needs a synthetic control analysis (Chapter 8) |

### Novelty, primacy and long-term holdouts

Plot the daily lift. A **novelty effect** starts high and decays as users get used to a change; a **primacy effect** starts low as users learn the new layout and then rises. A two-week test can mislead in either direction. For decisions with long-term consequences, keep a small **holdout** (for example 1% of users who never receive the feature for months) and compare it with everyone else. The cost is that 1% of users miss the feature; the gain is a clean estimate of long-run impact.

## 7.9 Looking Early Without Cheating: Group-Sequential and Always-Valid Tests

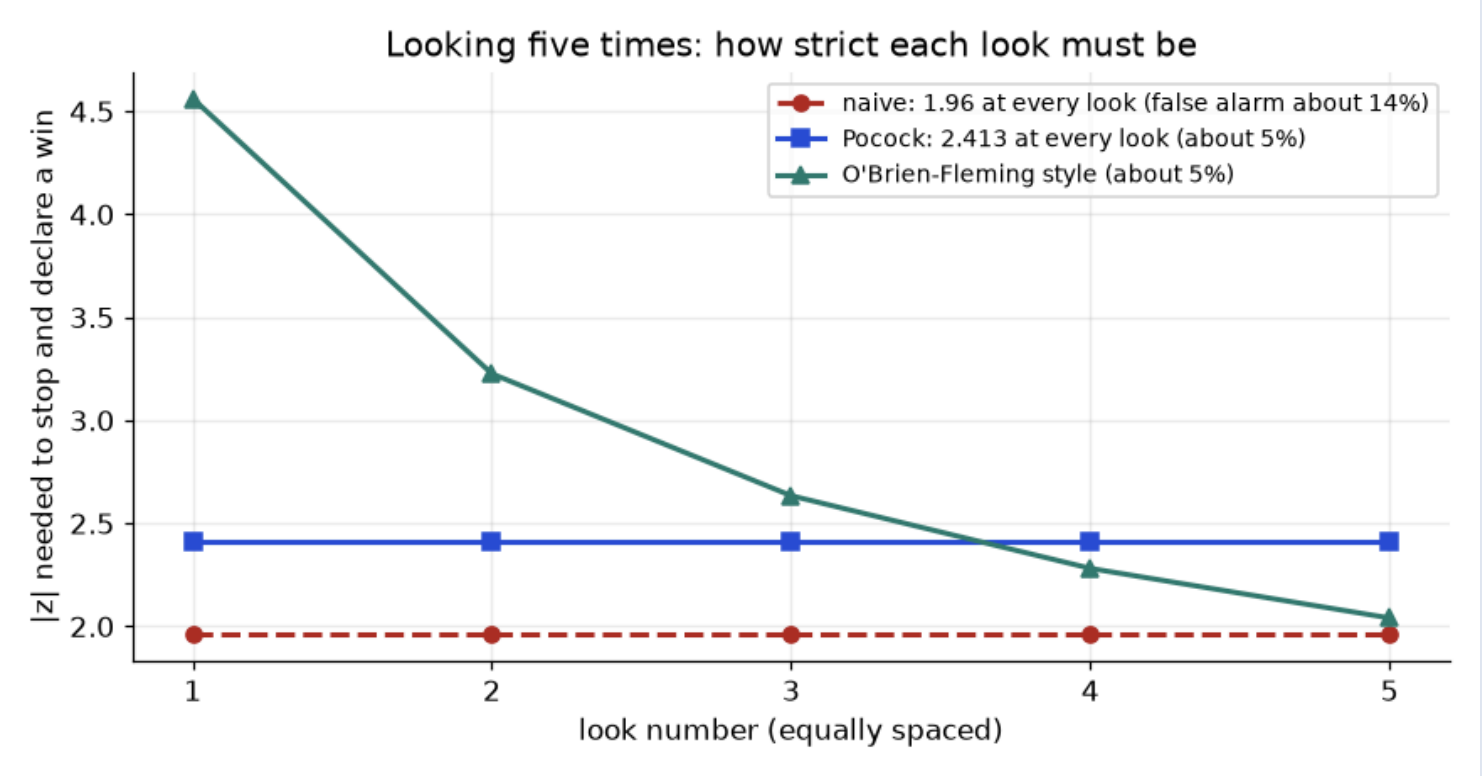
Section 7.3 showed that peeking at a fixed-sample test inflates false positives. The answer is not "never look". It is to use a method whose error rate is correct *because it expects you to look*. Two families: group-sequential designs (a few planned looks with stricter cut-offs) and always-valid tests such as the mixture sequential probability ratio test (mSPRT), which allows continuous monitoring.

**Problem 7.9.1** Plan 5 equally spaced looks at α = 0.05. How bad is naive testing at 1.96, and what cut-offs fix it?

```
import numpy as np
rng = np.random.default_rng(5)
paths = 400000
z = np.cumsum(rng.normal(size=(paths, 5)), axis=1) / np.sqrt(np.arange(1, 6))   # z-stat
at each of 5 equal looks
naive = (np.abs(z) > 1.96).any(1).mean()
pocock = (np.abs(z) > 2.413).any(1).mean()
ob = 2.04 * np.sqrt(5 / np.arange(1, 6))
obf = (np.abs(z) > ob).any(1).mean()
print("naive 1.96 at every look:", round(naive, 4))
print("Pocock 2.413 at every look:", round(pocock, 4))
print("O'Brien-Fleming style bounds", ob.round(3), "false positive rate", round(obf, 4))
print("independent-looks formula 1-0.95^5 =", round(1 - 0.95**5, 4))
```

```
naive 1.96 at every look: 0.1413
Pocock 2.413 at every look: 0.0505
O'Brien-Fleming style bounds [4.562 3.226 2.634 2.281 2.04 ] false positive rate 0.0493
independent-looks formula 1-0.95^5 = 0.2262
```

**How to read it.** Naive testing at five looks gives 14.1%, not the 22.6% from $1 - 0.95^5$. That formula assumes the five looks are independent, but the z-statistic at look 2 includes the data from look 1, so the looks are positively correlated and the true inflation is smaller. Treat 22.6% as an upper bound. A constant Pocock cut-off of 2.413 restores 5%. An O'Brien-Fleming style plan (4.56, 3.23, 2.63, 2.28, 2.04) also gives 5%, and keeps the final look nearly as sensitive as a fixed test, so it is the usual choice: it makes it hard to stop early on a lucky start, yet costs almost no power at the end.




*The z-statistic needed to stop at each of five looks.*

**How to read it.** The red dashed line is the cut-off many people use without thinking: 1.96 at every look. The green line starts very strict and relaxes towards the final look. Early on, only an enormous effect can stop the test.

## Always-valid inference (mSPRT)

With a normal metric of known variance 1 and a prior of N(0, $\tau^2$) on the effect, the mixture likelihood ratio after $n$ paired observations with mean $\bar{x}$ is $\Lambda = \sqrt{1/(1+n\tau^2)} \cdot \exp\left( n^2\tau^2\bar{x}^2 \,/\, (2(1+n\tau^2)) \right)$. Reject the null the first time $\Lambda \geq 1/\alpha$. Because $\Lambda$ is a martingale under the null, the chance it *ever* crosses $1/\alpha$ is at most $\alpha$, however often you look.

```
import numpy as np
rng = np.random.default_rng(2)
paths, n, tau2, alpha = 4000, 3000, 0.01, 0.05
x = rng.normal(size=(paths, n))                  # paired differences under the null,
variance 1
k = np.arange(1, n + 1); xbar = np.cumsum(x, 1) / k
lam = np.sqrt(1 / (1 + k * tau2)) * np.exp(k**2 * tau2 * xbar**2 / (2 * (1 + k * tau2)))
print("share of null paths that EVER cross 1/alpha:", (lam.max(1) >= 1 /
alpha).mean().round(4))
d = x + 0.1                                      # true effect 0.1 sd
xbar = np.cumsum(d, 1) / k
lam = np.sqrt(1 / (1 + k * tau2)) * np.exp(k**2 * tau2 * xbar**2 / (2 * (1 + k * tau2)))
hit = (lam >= 1 / alpha).argmax(1)[(lam >= 1 / alpha).any(1)] + 1
print("with a real effect of 0.1 sd: stop rate", (lam >= 1/alpha).any(1).mean().round(3),
"median stop at n =", int(np.median(hit)))
```

```
share of null paths that EVER cross 1/alpha: 0.0355
with a real effect of 0.1 sd: stop rate 0.996 median stop at n = 727
```

**How to read it.** Under the null only 3.6% of paths ever cross the line, below 5% even with 3,000 looks (the guarantee is "at most 5%", and it is conservative). With a true effect of 0.1 standard deviations, 99.6% of paths stop, with a median stopping time of 727 observations. The price of always-valid monitoring is some power compared with a fixed-sample test of the same length, and the choice of $\tau$ matters: pick it near the effect sizes you care about.

**REAL JOB**

Experiment platforms usually offer one of these methods. Your job is to choose the plan before launch: how many looks, which boundary, which metric, and what you will do for each outcome. Write it in the design doc next to the sample-size calculation.

## Chapter recap

- Randomising by a fair coin makes groups alike, so a difference in outcome is caused by the change.
- Decide hypothesis, primary metric, guardrails, unit and exposure BEFORE the test.
- Sample size depends on baseline, MDE, $\alpha$ and power. Halving the lift needs 4 times the customers.
- Run whole weeks to cover weekday and weekend behaviour.
- Report absolute lift, relative lift and a CI; a p-value alone is not enough.
- CUPED subtracts the part of the outcome predicted by pre-test behaviour; variance falls by $1 - \rho^2$.
- Always check SRM first. A failed SRM check means do not trust the result.

- Peeking with a fixed threshold made a 5% error rate into about 19%. Use one look or a sequential method.
- Bandits lose less while learning, but give weaker estimates. Use them for many cheap variants.
- Convert lift to money, then decide. Log every test.

| Item | Formula or pattern |
|---|---|
| Sample size (rule of thumb) | $n \approx 16 \cdot p(1-p)/\delta^2$ per arm |
| Sample size (exact) | $(z_{\alpha/2}\cdot\sqrt{(2\bar{p}(1-\bar{p}))} + z_{\beta}\cdot\sqrt{(p1q1+p2q2)})^2 / \delta^2$ |
| z-test | $z = (pB-pA)/\sqrt{(p(1-p)(1/nA+1/nB))}$ |
| CUPED | $Y_{adj} = Y - \theta(X - \text{mean } X)$, $\theta = \text{Cov}(Y,X)/\text{Var}(X)$, variance × $(1-\rho^2)$ |
| SRM | $\chi^2 = \Sigma(O-E)^2/E$, alarm if $p < 0.001$ |
| Peeking fix | Pocock z about 2.555 for 10 looks; or mSPRT |
| Ratio metric | Delta method or user-level bootstrap |
| Bandit | Thompson: draw from Beta(s,f), play the max, update |

## Practice set

**1. (easy)** Control rate 20%, you want to detect a 2-point lift. Use the rule of thumb for n per arm. **Answer:** 16 × 0.2 × 0.8 / 0.02² = 2.56 / 0.0004 = 6,400 per arm.

**2. (easy)** True or false: a test that finds "no significant difference" proves the change does nothing. **Answer:** False. It may be underpowered; check the CI and compare to the MDE.

**3. (easy)** Traffic is 500 eligible customers per day and you need 8,000 in total. How many days, rounded to full weeks? **Answer:** 8,000 / 500 = 16 days, round up to 3 weeks (21 days).

**4. (medium)** A: 200 of 2,000 ordered. B: 240 of 2,000. Compute the absolute lift and the z-statistic. **Answer:** pA = 0.10, pB = 0.12, lift 2 points. Pooled p = 0.11, SE = √(0.11 × 0.89 × (2/2000)) = √0.0000979 = 0.009894. z = 0.02 / 0.009894 = 2.02, p about 0.043. Barely significant; the unpooled 95% CI is about 0.06 to 3.94 points, so it nearly touches 0.

**5. (medium)** Planned 50/50. Observed 10,300 vs 9,700 (total 20,000). SRM check? **Answer:** E = 10,000. $\chi^2$ = 300²/10,000 × 2 = 9 × 2 = 18, p about 0.00002. SRM: do not trust the result.

**6. (medium, find the bug)** An analyst computes CUPED using "number of orders during the test" as X. What is wrong? **Answer:** X is measured after treatment and may be affected by it, so adjusting removes part of the real

effect and biases the estimate. X must be pre-test.

**7. (medium, what is the output)** Correlation between X and Y is 0.8. By what fraction does CUPED cut the SE? **Answer:** Variance factor = 1 − 0.64 = 0.36. SE factor = √0.36 = 0.6, so SE falls by 40%.

**8. (hard)** You look at a test at 10 equally spaced points and stop at the first $p < 0.05$. Roughly how often do you declare a false win on an A/A test? **Answer:** About 19% in our simulation (18.9%), not 5%.

**9. (hard, scenario)** Your offer lifts orders by 0.8 points (CI 0.1 to 1.5), costs ₹10 per customer, and an order is worth ₹300 margin. Ship? **Answer:** 0.008 × 300 = ₹2.40 versus a cost of ₹10, net −₹7.60. Even the top of the CI (0.015 × 300 = ₹4.50) loses money. Do not ship; break-even needs 10/300 = 3.3 points.

**10. (interview)** "We ran a test for 3 days and the p-value is 0.01. Can we ship?" **Answer:** Not yet. 3 days misses the weekly cycle, and if the sample size was fixed at 3 weeks, stopping early because p is low is peeking. Check SRM, the planned n, and guardrails. Also check novelty by looking at lift by day.

**11. (interview)** "When would you use a bandit instead of an A/B test?" **Answer:** Many cheap variants, a quick feedback metric, and a high cost of showing weak options (headlines, notification times). Use A/B when you need a precise, defensible effect estimate or outcomes are delayed.

**12. (easy)** State the assumptions behind $n \approx 16\,p(1-p)/\delta^2$. **Answer:** Two-sided test at $\alpha = 0.05$, 80% power, and a binary metric with variance about $p(1-p)$ in both arms. For one-sided 5% replace 16 by about 12.4, and for 90% power by about 21.

**13. (medium)** Revenue per user has $\sigma = 80$ rupees. Detect +1.5 rupees at two-sided 5%, 80% power. Per arm? **Answer:** $n = 15.698 \times 80^2 / 1.5^2 = 15.698 \times 6{,}400 / 2.25 = 44{,}653$ per arm.

**14. (medium)** With the same test, a pre-period covariate has correlation 0.6 with revenue. New sample size? **Answer:** Variance factor 1 − 0.36 = 0.64, so $n = 44{,}653 \times 0.64 = 28{,}578$ per arm.

**15. (medium)** Twenty metrics are all truly unaffected. What is the chance that at least one has $p < 0.05$, if they were independent? Under Bonferroni? **Answer:** $1 - 0.95^{20} = 0.642$. With Bonferroni each metric uses 0.05/20 = 0.0025, so the chance is about $1 - 0.9975^{20} = 0.049$.

**16. (hard)** p-values 0.003, 0.010, 0.020, 0.030, 0.40 for five metrics. Which are significant under BH at 5%? **Answer:** Thresholds $k \times 0.05/5 = 0.01, 0.02, 0.03, 0.04, 0.05$. $0.003 \le 0.01$ yes; $0.010 \le 0.02$ yes; $0.020 \le 0.03$ yes; $0.030 \le 0.04$ yes; $0.40 \le 0.05$ no. Largest passing $k = 4$, so the first four are significant. Bonferroni (0.01) would keep only two (0.003 and 0.010).

**17. (hard)** (find the flaw) A ride-hailing team randomises riders 50/50 to a coupon and sees a 15% rise in trips. After a full launch trips are flat. Give the likely reason and a better design. **Answer:** Interference through shared supply: coupon riders take drivers that control riders would have used, so the arms are not independent worlds. Use a switchback by city-hour or randomise whole cities, and compare to the user-level result to size the bias.

**18. (hard)** (interview) "Your CTR test has 200,000 sessions per arm but only 20,000 users per arm. The z-test on sessions gives p = 0.03. Do you ship?" **Answer:** Not on that number. Randomisation was by user and sessions are clustered inside users, so the session-level SE is too small (in our simulation by about 23%). Recompute at user level with the delta method or a user bootstrap, check SRM at user level, and decide on the corrected interval.

# Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, studying one hour a day. Topic: sample size, power and MDE for an online A/B test with a yes/no metric. Explain step by step with small numbers, using the idea of how many timed deliveries you need before an average is trustworthy. Then give me a baseline of 8% and an MDE of 0.5 points and walk me through the full calculation. Finish by asking me three questions, one at a time, and check my answers before moving on.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: CUPED variance reduction in A/B tests. Explain the idea with a classroom analogy, derive why variance falls by (1 − rho squared), and show a tiny numeric example I can verify by hand. Then give me 8 graded exercises, from easy to hard, without solutions, including one where the covariate is wrongly chosen. After I answer, grade me honestly.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: sample ratio mismatch (SRM) and peeking in A/B tests. Quiz me one question at a time. Use realistic numbers (group sizes, chi-square, p-values, number of looks). After each answer, tell me what I got right, what I missed, and explain the correct reasoning in simple English. Start easy and get harder over 10 questions.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: reviewing an A/B test design document. Here is my design doc: [PASTE HERE]. Review it against this checklist: hypothesis, primary metric, guardrails, randomisation unit, exposure criteria, MDE, sample size, duration, SRM check, analysis plan, decision rule. Point out each gap in plain English, explain why it matters, and suggest a fix with example numbers.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science and preparing for interviews. Topic: experimentation (A/B testing, bandits, sequential testing). Run a mock interview with me. Ask one question at a time, such as "design an experiment for a new checkout flow" or "when would you use a bandit". After each answer, score me from 1 to 5, show a model answer, and ask a follow-up. Do 8 rounds, then summarise my weak spots and give me a study plan.
```

CHAPTER 8

# CAUSAL INFERENCE WHEN YOU CANNOT RANDOMISE

*How to recreate a fair comparison from messy observational data: DAGs, propensity scores, difference-in-differences, synthetic control, regression discontinuity, instrumental variables and uplift, each with a worked example.*

Chapter 7 gave you the gold standard: flip a coin. But often you cannot. You cannot randomly decide which stores get a new delivery fee because the regional head already picked them. You cannot randomly deny loans. The offer was sent last month to the customers someone thought were at risk, and now the CEO asks "did it work?". The data is **observational**: you watch what happened, you did not control it. The danger is **confounding**: the people who got the treatment differ from those who did not, in ways that also change the outcome. This chapter teaches you to name the problem with a picture (a DAG), to fix it with a fair-comparison method that fits your situation, and to state the assumptions each method needs. By the end you will pick the right tool for a given business question and know how to test it for obvious failure. Everything below is synthetic with fixed seeds, so we KNOW the true answer and can see whether each method finds it.

**Running example.** Churn offers, again. A retention offer is sent mainly to customers with low engagement. We want its effect on "orders in next 30 days" (1 = ordered). Later sections reuse the same business world with stores, cities and credit scores.

Most code here uses `numpy`, `pandas`, `scikit-learn` and `scipy`. Specialised libraries (`DoWhy`, `EconML`, `CausalML`, `linearmodels`, `statsmodels`) wrap the same ideas; they were not installed here, so we wrote the steps by hand. That is better for learning anyway.

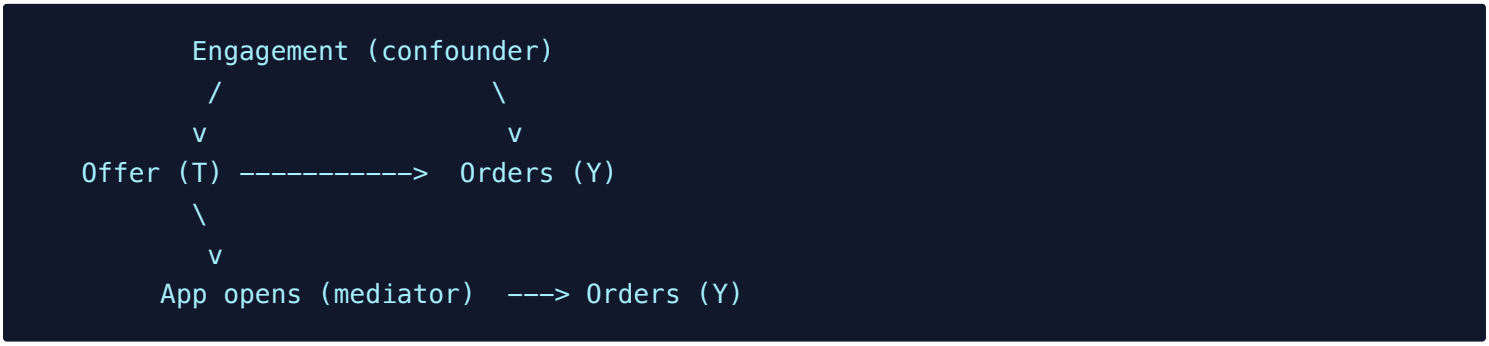
## 8.1 Potential Outcomes, DAGs and Confounding

| | |
|---|---|
| **INTUITION** | You want to know if an umbrella keeps people dry. You see that people carrying umbrellas are wetter, because they carry umbrellas when it rains. A naive comparison says "umbrellas make you wet". Rain is the confounder: it causes both carrying and wetness. A causal analyst draws that picture first, then compares umbrella and no-umbrella people within the same weather. |

## Words you need

| Word | Plain meaning | In our churn example |
|---|---|---|
| Treatment `T` | The action you did or did not take | Offer sent (1) or not (0) |
| Outcome `Y` | What you measure afterwards | Ordered within 30 days |
| Potential outcomes `Y(1)`, `Y(0)` | What would happen to the same person with and without the treatment | Would Asha order with the offer? Without? |
| Fundamental problem | For one person we see only one of `Y(1)`, `Y(0)` | Asha got the offer; we never see her without it |
| ATE | Average treatment effect over everyone: `E[Y(1) − Y(0)]` | Average lift if we gave the offer to all |
| ATT | Effect on those actually treated | Lift among customers who got the offer |
| CATE | Effect for a subgroup or type of customer | Lift for low-engagement customers |
| Confounder | Causes both `T` and `Y` | Engagement |
| Mediator | Lies on the path `T → M → Y` | Offer causes app opens, app opens cause orders |
| Collider | Caused by two things: `A → C ← B` | "Customer called support" is caused by both bad delivery and low patience |

A **DAG** (directed acyclic graph) is a drawing with arrows for "causes". Directed means arrows have a direction. Acyclic means no loops.

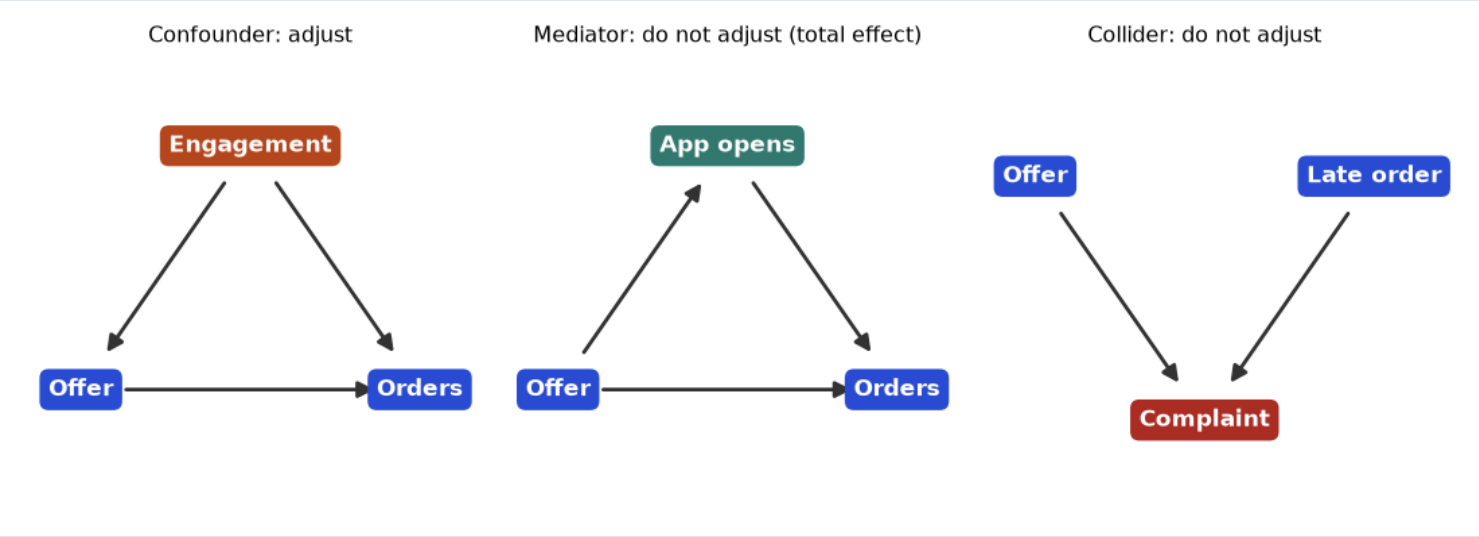



**The backdoor rule.** A "backdoor path" is any path from `T` to `Y` that starts with an arrow INTO `T`. Here: `T ← Engagement → Y`. This path carries a false (non-causal) association. To get the true effect, block every backdoor

path by adjusting for (holding fixed) a variable on it, without opening a new one.

What to adjust for:

- **Confounders:** YES, adjust.
- **Mediators:** NO if you want the total effect, since you would remove part of the effect you are after.
- **Colliders:** NO. Adjusting for a collider creates a false link between its two causes.




*The three patterns that decide what you adjust for.*

**How to read it:** an arrow means "causes". Left: the orange node sends arrows into both offer and orders, so it creates a false link and you must adjust for it. Middle: the teal node sits on the path from offer to orders, so adjusting for it removes part of the real effect. Right: the red node receives arrows from two causes, so adjusting for it (or filtering on it) invents a link between them.

**Problem 8.1.1** (tiny numbers, by hand) 1,500 customers in two engagement groups.

| Group | Offer | Customers | Order rate |
|---|---|---|---|
| Low engagement | yes | 800 | 0.30 |
| Low engagement | no | 200 | 0.20 |
| High engagement | yes | 100 | 0.70 |
| High engagement | no | 400 | 0.60 |

- **Step 1 (naive)** Treated rate = (800×0.30 + 100×0.70) / 900 = (240 + 70) / 900 = 0.3444. Control rate = (200×0.20 + 400×0.60) / 600 = (40 + 240) / 600 = 0.4667.
- **Step 2** Naive difference = 0.3444 − 0.4667 = −0.122. It says the offer HURTS by 12.2 points.

- **Step 3 (within groups)** Low group: 0.30 − 0.20 = +0.10. High group: 0.70 − 0.60 = +0.10.
- **Step 4** Average with the population weights (1,000 low and 500 high, so two thirds and one third): (2/3)(0.10) + (1/3)(0.10) = 0.10.
- **Answer** The true effect is +10 points. The naive comparison is wrong because most treated customers were low-engagement, who order less anyway.
- **Check** The sign flipped from −12 to +10. This is exactly Simpson's paradox from Chapter 4.1, now seen as confounding.
- **Why it matters** The adjustment did not use any fancy model. It compared like with like, then averaged.

## Same story at scale, with real code

We simulate 20,000 customers. Engagement `eng` is a number (a standard normal). Low engagement raises the chance of getting the offer. High engagement raises the chance of ordering. The offer truly adds 0.4 to the log-odds of ordering. Because we built the data, we can compute the true ATE: **+8.15 points**.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
sig = lambda x: 1/(1+np.exp(-x))
rng = np.random.default_rng(0); N = 20000
eng = rng.normal(0, 1, N)
t = (rng.random(N) < sig(-0.3 - 1.2*eng)).astype(int)       # low eng -> more offers
y = (rng.random(N) < sig(-0.5 + 1.0*eng + 0.4*t)).astype(int)
print(y[t==1].mean() - y[t==0].mean())                        # naive
```

Real output (full script outputs are collected below):

```
true ATE            +0.0815
naive difference    -0.1175   (treated mean engagement -0.51, control +0.41)
```

The naive method says the offer lowers orders by 11.7 points when it raises them by 8.2. The treated group has engagement 0.92 standard deviations lower than the control group.

**Colliders, a quick demonstration.** `a` and `b` are independent. `c = a + b + noise`. Corr(a,b) = 0.012. If you restrict to cases with `c > 1` (that is, "adjust for the collider"), Corr(a,b) becomes −0.50. In business: among customers who contacted support (the collider), bad delivery and low patience look negatively linked, even though they are unrelated overall.

## Connection to the real job

Before any model, draw the DAG on paper with the PM and the domain expert. It forces the question "what else drives both the decision and the outcome?" and shows which columns you must NOT put in the regression.

### Applications

- **Marketing:** who was targeted is rarely random.
- **Credit:** approved customers differ from rejected ones.
- **Healthcare:** sicker patients get stronger treatments.
- **Operations:** "branches that adopted the new till system are faster", but they were also the newer, bigger branches.

| COMMON MISTAKE | "Controlling for everything." Adding every column to the regression can include mediators (hiding part of the effect) and colliders (inventing a bias). Choose controls from a DAG, not from the list of available columns. |
|---|---|

**Check yourself**

1. What is the fundamental problem of causal inference? Answer: for each person we see only one of the two potential outcomes.
2. Should you adjust for a mediator when you want the total effect? Answer: No, it would remove part of the effect.
3. What is a backdoor path? Answer: a path from T to Y that begins with an arrow into T; it carries false association unless blocked.

## 8.2 Adjustment and Propensity Scores

| INTUITION | To compare fairly, find for every treated customer a "twin" who looks the same but was not treated. Matching on ten characteristics at once is almost impossible. The **propensity score** squeezes all characteristics into one number: "how likely was this customer to be treated?" Two people with the same score had the same chance of getting the offer, so among them, who got it behaves like a coin flip. |
|---|---|

**Definition.** `e(x) = P(T = 1 | X = x)`. Estimate it with logistic regression (or a tree model). Then you can:

- **Stratify:** group by score into bands, compare within bands, average.
- **Match:** pair each treated customer with a control with a nearby score.
- **Weight (IPW, inverse probability weighting):** weight treated customers by `1/e` and controls by `1/(1−e)`. This builds a pseudo-population where treatment is independent of `X`.

Two checks are mandatory.

- **Overlap (positivity):** for every kind of customer there must be both treated and controls. If the offer was ALWAYS sent to low-engagement customers, there is nothing to compare.
- **Balance:** after weighting, the groups should look alike. The standard measure is the **standardised mean difference** $\text{SMD} = (\text{mean}_1 - \text{mean}_0) / \sqrt{((\text{var}_1 + \text{var}_0)/2)}$. A common rule: absolute SMD below 0.1 is good.

**Problem 8.2.1** (hand check of IPW, same table as 8.1.1) Propensity: Low group 800/1,000 = 0.8, High group 100/500 = 0.2.

| | |
|---|---|
| **Step 1** | Weights: treated-Low = 1/0.8 = 1.25. Treated-High = 1/0.2 = 5. Control-Low = 1/(1−0.8) = 5. Control-High = 1/(1−0.2) = 1.25. |
| **Step 2** | Weighted treated rate = (800×1.25×0.30 + 100×5×0.70) / (800×1.25 + 100×5) = (300 + 350) / (1,000 + 500) = 0.4333. |
| **Step 3** | Weighted control rate = (200×5×0.20 + 400×1.25×0.60) / (200×5 + 400×1.25) = (200 + 300) / 1,500 = 0.3333. |
| **Answer** | IPW effect = 0.4333 − 0.3333 = **+0.10**, matching the stratified answer. |
| **Check** | After weighting, Low is 1,000 of 1,500 customers in BOTH groups (two thirds), and High is 500 of 1,500. The groups now have the same mix. |
| **Why it matters** | IPW is just "reweight so the groups look alike". |

**Problem 8.2.2** (the 20,000-customer data) Estimate the ATE with four methods.

```
ps = LogisticRegression(C=1e6).fit(eng.reshape(-1,1), t).predict_proba(eng.reshape(-1,1))
[:,1]
w = t/ps + (1-t)/(1-ps)
ipw = np.sum(w*t*y)/np.sum(w*t) - np.sum(w*(1-t)*y)/np.sum(w*(1-t))
```

Real output:

```
True ATE                          +0.0815
Naive                             -0.1175
Regression adjustment (logit)     +0.0773
IPW                               +0.0836
Doubly robust (AIPW)              +0.0785
Stratified, 5 propensity bands    +0.0646
Propensity range                  0.007 to 0.989  (1st/99th pct 0.046 / 0.924)
SMD of engagement                 -1.045 before, +0.036 after weighting
```

- **Step 1** Overlap: scores span 0.007 to 0.989 but 98% are between 0.05 and 0.92. Some extreme weights exist; trimming or capping weights is common practice.
- **Step 2** Balance: SMD goes from −1.045 (huge) to 0.036 (fine, below 0.1).
- **Step 3** All adjusted estimates land between 6.5 and 8.4 points, near the true 8.15. The naive one has the wrong sign.
- **Answer** The offer raises the 30-day order rate by about 8 points (true value 8.15), not −11.7.
- **Check** Estimates from different methods agree within sampling noise. The 5-band stratified estimate (6.5) is lowest because five wide bands leave some leftover confounding inside each band. More bands, or a finer method, reduces this.
- **Why it matters** If three methods disagree a lot, do not report any until you know why.

**Doubly robust (AIPW).** It combines an outcome model and a propensity model. If EITHER one is right, the estimate is still correct. That is why it is popular. **Double machine learning (DML)** extends the idea: use flexible ML models for both parts, with cross-fitting (fit on one fold, predict on the other) so overfitting does not bias the effect. `EconML` and `DoWhy` implement it.

**Limits.** Propensity methods only fix **measured** confounders. If the real driver is something you did not record (motivation, say), no propensity score can fix it. **Sensitivity analysis** asks "how strong would an unmeasured confounder have to be to erase my result?" A well-known summary is the E-value. Always run one before you present a causal claim from observational data.

## Connection to the real job

Propensity scores and IPW are the standard first approach at companies for "did the campaign work" when no holdout was kept. Also, the same weighting idea is used to correct selection bias in recommender evaluation (Chapter 10) and reject inference in credit scoring.

## Applications

- **CRM/marketing:** incremental effect of past campaigns.
- **Healthcare:** treatment effect from hospital records.
- **Pricing:** effect of discounts given to selected customers.
- **Product:** effect of using a feature (users who adopt it are different).

| COMMON MISTAKE | Using the propensity model's accuracy as the goal. A propensity model that predicts treatment perfectly is BAD: it means no overlap. You want a model that is well calibrated and a result that balances covariates, not a high AUC. |
|---|---|

**Check yourself**

1. What does IPW weight a treated customer with propensity 0.2 by? Answer: 1/0.2 = 5.
2. SMD of 0.04 after weighting. Good or bad? Answer: Good (below 0.1).
3. Which confounders can propensity scores handle? Answer: only the measured ones.

## 8.3 Difference-in-Differences

| INTUITION | Your shop's checkout got faster after a process change, and waiting times fell 10 seconds. But customer patterns also changed that week. If you have a similar group of branches that did not get the change, you can see how much the "similar" group improved on its own, and subtract that. What is left is the change's effect. |
|---|---|

**Definition. Difference-in-differences (DiD)** compares the before-to-after change in a treated group with the before-to-after change in an untreated group:

```
DiD = (Treated_after − Treated_before) − (Control_after − Control_before)
```

The key assumption is **parallel trends**: without the treatment, both groups would have moved in the same way. The groups may start at different levels. They must not drift at different speeds.

**Running example.** 20 stores, 12 weeks. Ten stores (treated) get a new delivery-fee policy in week 7. We simulate weekly sales per store. The treated stores are bigger (+15), all stores grow +0.8 per week, and the true effect is +6.

```
m = df.groupby(['treat','post']).y.mean().unstack()
did = (m.loc[1,1]−m.loc[1,0]) − (m.loc[0,1]−m.loc[0,0])
```

Real output:

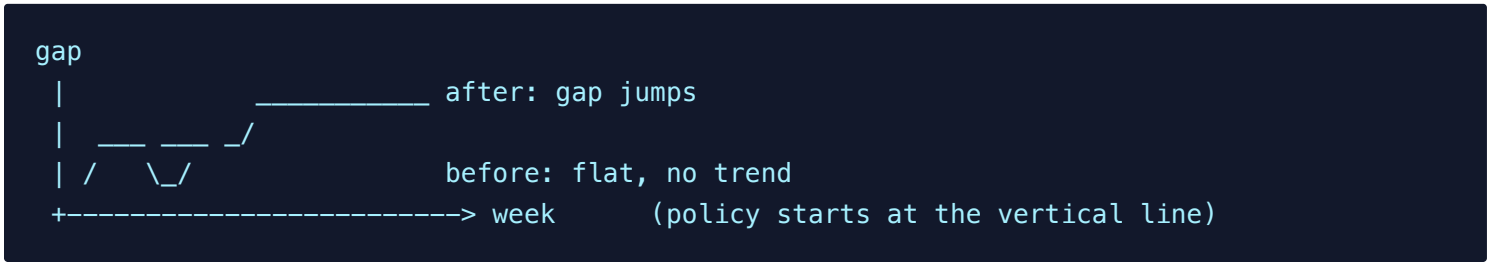
```
post            before (weeks 1–6)   after (weeks 7–12)
control stores         97.58              103.25
treated stores        117.26              127.50
```

**Problem 8.3.1** Compute the DiD and compare with two wrong shortcuts.

| | |
|---|---|
| **Step 1 (naive: after only)** | 127.50 − 103.25 = 24.24. Wrong, because treated stores were already 19.7 higher. |
| **Step 2 (before-after for treated only)** | 127.50 − 117.26 = 10.23. Wrong, because it includes the growth that all stores had (5.68). |
| **Step 3 (DiD)** | Treated change = 127.50 − 117.26 = 10.23. Control change = 103.25 − 97.58 = 5.68. DiD = 10.23 − 5.68 = **4.56**. |
| **Step 4 (regression form)** | `y = a + b·treat + c·post + d·treat×post`. The coefficient `d` = 4.557, the same number. This form lets you add controls and clusters. |
| **Answer** | The policy raised weekly store sales by about 4.6 units. The truth is 6.0. The cluster bootstrap (resample stores, 500 times) gives a 95% CI of 3.04 to 5.98, so the truth sits right at the top edge of the interval. This is ordinary sampling noise with only 10 stores per group, and a good reminder that DiD with few units is imprecise. |
| **Check** | Naive 24.2 and 10.2 are far from 6, while DiD at 4.6 is not. The ordering of errors is what we expected. |
| **Why it matters** | DiD removes both fixed differences between groups (stores differ in size) and common time shocks (everyone grew). |

**Testing parallel trends.** You cannot test the assumption AFTER treatment, but you can look at the pre-period.

- Pre-period slope per week: control 0.786, treated 0.861. Close, which is reassuring.
- **Placebo test:** pretend the policy started in week 4, using only pre-period data. The "effect" should be near 0. We got −0.14. Good.
- An **event study** plots the treated-minus-control gap for every week. Before the change it should hover around a constant, and after the change it should jump.



**Interrupted time series** is the cousin for the case with no control group: fit the pre-trend, project it forward, and treat the difference as the effect. It is weaker because anything else that changed at that moment gets blamed on your policy.

### Connection to the real job

DiD is the workhorse for policy-style launches: store pilots, city-level pricing, a new regulation, a deployment applied to some regions first. Libraries: `linearmodels`, `statsmodels`.

### Applications

- **Retail:** pilot stores vs others.
- **Ride-hailing/food delivery:** city launches.
- **Public policy:** state-level rule changes.
- **Engineering:** a new config rolled out region by region.

| COMMON MISTAKE | Choosing the control group because it "looks similar on the level" rather than "moves similarly over time". And ignoring that treated groups often get picked BECAUSE they were trending up (the manager chose the fastest-growing stores for the pilot), which breaks parallel trends. |
|---|---|

**Check yourself**

1. Treated rose 10, control rose 4. DiD? Answer: 6.
2. What is the key assumption? Answer: parallel trends.
3. What does a placebo test do? Answer: it pretends the policy started earlier; a clear "effect" signals broken assumptions.

## 8.4 Synthetic Control

| INTUITION | You launched in one city (Pune), so DiD has only one treated unit. No single city is a good twin. But a weighted mix of 30% Nagpur, 50% Indore and 20% Jaipur may have followed Pune's sales path almost exactly before the launch. That blend is a "synthetic Pune". After the launch, the gap between real Pune and synthetic Pune is the effect. |
|---|---|

**Definition.** Find non-negative weights `w` over the donor units that add to 1, such that the weighted donor average matches the treated unit's PRE-treatment series as closely as possible. Then forecast the treated unit's no-treatment path with those weights. Effects are the post-treatment gaps.

**Problem 8.4.1** 15 donor cities, 30 months, launch at month 20 in one treated city. By construction, the treated city's no-launch path is a mix of donors 0, 3 and 7 with weights 0.5, 0.3, 0.2, and the launch adds +8. Fit the weights using months 1 to 20 only.

```
from scipy.optimize import minimize
res = minimize(lambda w: np.sum((treated_y[:20] - donors[:20] @ w)**2),
               np.ones(J)/J, bounds=[(0,1)]*J,
               constraints={'type':'eq','fun': lambda w: w.sum()-1}, method='SLSQP')
```

Real output (weights above 0.01 only):

```
donor 0: 0.57   donor 12: 0.17   donor 6: 0.08   donor 7: 0.07   donor 3: 0.04   donor 4:
0.04   donor 13: 0.02
pre-period RMSE      0.431
mean post gap        +7.56   (true effect +8)
pre-period gap sd    0.428
```

- **Step 1** The weights do not recover the built-in recipe exactly (donor 0 got 0.57 instead of 0.5, donors 3 and 7 got 0.04 and 0.07 rather than 0.3 and 0.2). Donors are correlated, so many recipes fit the past equally well. That is normal and fine: we need a good fit, not a unique recipe.
- **Step 2** The pre-period fit error is 0.43, small compared to an 8-unit effect.
- **Step 3** After launch the average gap is +7.56.
- **Answer** The launch added about 7.6 units per month, versus 8 truth.
- **Check (placebo in space)** Pretend each of the 15 donor cities was the treated one and rebuild its synthetic twin from the others. Compare the ratio of post-gap size to pre-gap size (RMSPE ratio). Real treated city: **17.6**. Donor placebos: median 1.2, max 2.1. The real city ranks 1st of 16, giving a permutation p-value of 1/16 = 0.0625, which is the smallest p-value you can get with 16 units.
- **Why it matters** With one treated unit, standard errors do not exist. The placebo ranking is your uncertainty.

**Limits.** You need a good pre-period fit, enough donor units, and no donor affected by the launch. If the pre-period fit is poor, do not use the method.

## Applications

- **Retail/marketplaces:** one-city launches.
- **Marketing:** geo experiments for incrementality of ad spend.
- **Public policy:** a state-level law.
- **Telecom:** a single-region network upgrade.

| COMMON MISTAKE | Reporting a post-period gap without showing the pre-period fit and the placebo tests. A line chart where the two lines overlap before the launch and split after it is the evidence. Without the overlap, it is just a picture of two different cities. |
|---|---|

**Check yourself**

1. What do synthetic control weights have to satisfy? Answer: non-negative and sum to 1.
2. Why a placebo test? Answer: to see whether a gap this big appears for untreated cities too.
3. When is it better than DiD? Answer: when there is one (or few) treated unit and no single good control.

## 8.5 Regression Discontinuity

| INTUITION | A bank gives a top-up offer to everyone with a credit score of 650 or more. A customer at 651 and a customer at 649 are nearly identical in reality, yet one got the offer. Their small difference in score was basically luck. So compare people just above and just below the line: it is a local coin flip made by a rule. |
|---|---|

**Definition. Regression discontinuity (RD)** uses a treatment rule that depends on a cut-off in a "running variable" (the score). Fit a line on each side of the cut-off within a narrow window, and read the jump at the cut-off. Assumption: people cannot precisely control which side they land on, and everything else changes smoothly across the cut-off.

**Problem 8.5.1** 6,000 customers, scores uniform between 550 and 750. Spend = 20 + 0.05(score − 650) + 7·(offer) + noise (sd 6). The offer goes to score ≥ 650. True jump: 7.

| | |
|---|---|
| **Step 1 (naive)** | Mean spend above 650 minus below 650 = **12.0**. Wrong, because higher-score customers spend more anyway (the 0.05 slope). |
| **Step 2 (local linear, window ±20)** | 1,189 customers in the window. Model `spend = a + b·above + c·(score−650) + d·above·(score−650)`. Estimated jump `b` = 6.32, standard error 0.71. |
| **Step 3 (sensitivity to window)** | |

```
window ±5   (n=282)    jump 7.15
window ±10  (n=574)    jump 6.99
window ±20  (n=1189)   jump 6.32 (se 0.71)
window ±40  (n=2431)   jump 7.00
```

- **Answer** The offer raises spend by about 7 (range 6.3 to 7.1 across windows; truth 7.0).
- **Check** The naive 12.0 is far off. The estimates barely move with the window. If the estimate swings a lot with window size, do not trust it.
- **Why it matters** RD is among the most believable non-randomised designs, because the assumption is easy to explain to a non-technical person.

The result is **local**: it applies to customers near 650, not those at 550 or 750. Check for **manipulation**: if staff nudge scores just above the cut-off, there will be a suspicious pile-up of customers right above 650. A histogram of the score shows this.

## Applications

- **Credit:** score cut-offs for approval or limit.
- **Education/government:** eligibility thresholds.
- **Product:** a free-shipping threshold at ₹499.
- **Healthcare:** risk-score thresholds for treatment.

| COMMON MISTAKE | Fitting one high-degree polynomial over all the data. It can invent a jump from curvature far from the cut-off. Prefer local linear in a window and report several window sizes. |
|---|---|

**Check yourself**

1. What does RD estimate? Answer: the effect at the cut-off, for people near it.
2. Why is the naive above-below difference wrong? Answer: customers above the cut-off differ in the score itself, which also affects the outcome.
3. What is manipulation of the running variable? Answer: people or staff steering the score to land on the favoured side.

# 8.6 Instrumental Variables

| INTUITION | You want to know if using auto-pay causes more on-time repayment. But motivated customers both use auto-pay and repay, and motivation is not in your data. Suppose the company randomly sent a reminder SMS encouraging auto-pay sign-up. The SMS changes auto-pay use but has no other way to affect repayment. Then the SMS is an **instrument**: a lever that moves the treatment without touching the outcome except through the treatment. |
|---|---|

**Three conditions** for an instrument `Z`:

1. **Relevance:** Z changes T (testable: the "first stage").
2. **Independence:** Z is as good as random (not related to the hidden confounder).
3. **Exclusion:** Z affects Y only through T (cannot be tested; argue it).

**The Wald estimator:** `effect = (effect of Z on Y) / (effect of Z on T)`. Two-stage least squares (2SLS) generalises this: stage 1 regress `T` on `Z`, stage 2 regress `Y` on the predicted `T`.

**Problem 8.6.1** 30,000 customers. Hidden motivation `u`. SMS `z` is random. Auto-pay `d` depends on `z` and `u`. Repayment score `y = 3 + 2.0·d + 1.5·u + noise`. The true effect of auto-pay is 2.0.

```
ols    = np.polyfit(d, yv, 1)[0]
first  = np.polyfit(z, d, 1)[0]          # effect of SMS on auto-pay
reduced= np.polyfit(z, yv, 1)[0]         # effect of SMS on repayment
wald   = reduced / first
```

Real output:

```
OLS (naive)           3.549
first stage           0.2316   (SMS raises auto-pay uptake by 23.2 points)
reduced form          0.4949
Wald = 0.4949/0.2316  2.136
2SLS by hand          2.136
first-stage F         1,725
```

- **Step 1** OLS gives 3.55, too high, because motivated customers both use auto-pay and repay well.
- **Step 2** SMS moves auto-pay by 0.2316 and repayment by 0.4949. Divide: 2.136.
- **Step 3** A first-stage F-statistic above about 10 is the usual rule against a **weak instrument**; ours is 1,725.
- **Answer** Auto-pay raises the repayment score by about 2.1 (truth 2.0), not 3.5.

- **Check** 2SLS by hand gives the same number as the Wald ratio (2.136). IV estimates are noisier than OLS. That is the price.
- **Why it matters** IV can fix confounding from variables you never measured, but only if the exclusion condition holds. That is an argument, not a test.

IV gives the effect for **compliers**, customers who are moved by the instrument (the **LATE**, local average treatment effect), not for everyone.

### Applications

- **Product:** randomised nudges or encouragement as instruments for feature adoption.
- **Economics:** distance to a facility, draft lotteries.
- **Marketplaces:** random assignment of support agents.

| COMMON MISTAKE | Picking an instrument because it is "correlated with T". That only covers relevance. The hard conditions are independence and exclusion. A weak instrument also blows up the estimate; always report the first-stage F. |
|---|---|

**Check yourself**

1. Name the three instrument conditions. Answer: relevance, independence, exclusion.
2. Z changes Y by 0.5 and T by 0.25. Wald estimate? Answer: 2.
3. Why is IV noisier than OLS? Answer: it uses only the part of T's variation driven by Z.

## 8.7 Uplift Modelling and Heterogeneous Effects

| INTUITION | Marketing teams love to send offers to customers most likely to buy. But some of those would have bought anyway (wasted discount), and a few get annoyed and leave. What you want is the customers whose behaviour the offer CHANGES. That number, the individual's effect, is **uplift**. |
|---|---|

Customers fall into four types:

| Type | Without offer | With offer | Action |
|---|---|---|---|
| Persuadable | no order | order | Target |
| Sure thing | order | order | Do not waste discount |

| Type | Without offer | With offer | Action |
|---|---|---|---|
| Lost cause | no order | no order | Do not target |
| Sleeping dog | order | no order | Never target |

A standard churn or purchase-propensity model finds "sure things" and ignores the rest. Uplift models need randomised (or well-adjusted) data and estimate `τ(x) = P(Y=1 | T=1, x) − P(Y=1 | T=0, x)`.

**Meta-learners** turn any classifier into an uplift model:

- **S-learner:** one model with `T` as a feature; uplift = prediction with T=1 minus T=0. Simple, but can ignore T.
- **T-learner:** one model on treated, one on controls; uplift is the difference.
- **X- and R-learners, causal forests:** more careful versions for small groups or noisy effects (`EconML`, `CausalML`).

**Problem 8.7.1** 40,000 customers, randomised offer. Order probability `sig(−1 + 0.8·x1 + t·1.2·x2)`, so customers with x2 > 0 are helped and x2 < 0 hurt. Train a T-learner on the first half (gradient boosting), score the second half (20,000), and evaluate honestly with the randomised treatment flag.

```
m1 = GradientBoostingClassifier(random_state=0, max_depth=2, n_estimators=100).fit(X[tr&
(t==1)], y[tr&(t==1)])
m0 = GradientBoostingClassifier(random_state=0, max_depth=2, n_estimators=100).fit(X[tr&
(t==0)], y[tr&(t==0)])
uplift = m1.predict_proba(X[te])[:,1] − m0.predict_proba(X[te])[:,1]
```

Real output (customers sorted by predicted uplift, split into 10 equal groups):

```
decile  est_uplift  treated  ctrl   observed_uplift
  1       0.450     0.710   0.268      +0.441
  2       0.281     0.614   0.338      +0.276
  3       0.174     0.498   0.372      +0.126
  4       0.095     0.371   0.294      +0.078
  5       0.032     0.279   0.252      +0.027
  6      −0.013     0.183   0.224      −0.042
  7      −0.061     0.150   0.216      −0.066
  8      −0.113     0.143   0.273      −0.130
  9      −0.178     0.136   0.322      −0.185
 10      −0.303     0.133   0.425      −0.293
```

- **Step 1** The average effect over everyone is only +2.5 points. A simple "is the offer good?" test says "barely".
- **Step 2** Sorted by predicted uplift, the observed uplift (treated rate minus control rate, using the real random flag) falls neatly from +44 points in the top group to −29 points in the bottom group. The model's ranking is

real.

- **Step 3** Treat only the top 40%: observed uplift +22.8 points, versus +2.5 for treating everyone.
- **Answer** Treat only customers in the top 4 deciles (or the top ~50%, where the Qini curve peaks), and never the bottom 40%, who are made worse off.
- **Check** The correlation between estimated and true uplift is 0.978 (we know the truth because we built the data).
- **Why it matters** The average effect hid two opposite groups. Targeting by uplift turns "barely positive" into a large gain, with the discount budget spent on fewer people.

**The Qini curve** is the uplift version of a lift chart: sort customers by predicted uplift and plot the cumulative extra orders (treated orders minus the scaled control orders) as you widen the target group. On our test half, the curve peaks at about 50% of customers with +962 incremental orders, and falls to +249 at 100% (that equals treating everyone). Treating the bottom half destroys 713 orders.

## Connection to the real job

Uplift is the answer to "who should get the retention offer?" (the framing from Chapter 1.3). Combined with costs (expected-value thinking, later in this document), it picks the customers where `uplift × margin − offer cost` is positive.

## Applications

- **Retention and win-back:** target persuadables only.
- **Lending:** which customers benefit from a limit increase.
- **Healthcare:** which patients benefit from a treatment.
- **Pricing:** discount only where it changes the decision.

| COMMON MISTAKE | Evaluating an uplift model with AUC on the outcome. AUC says nothing about the effect. Use uplift-by-decile, Qini curves and a randomised holdout. |
|---|---|

**Check yourself**

1. Which customers should you never target? Answer: sleeping dogs (negative uplift) and, for cost reasons, sure things and lost causes.
2. Why can uplift modelling not be trained on any old data? Answer: you need random or well-adjusted treatment assignment, otherwise confounding contaminates the effect estimates.
3. What does the T-learner use? Answer: separate models for treated and controls; uplift is the difference.

# 8.8 Meta-Learners and Double Machine Learning in Depth

Section 8.7 introduced uplift with one T-learner on randomised data. Real life is harder in two ways. First, the treatment is often NOT randomised, so the model must also handle confounding. Second, you need to know which learner to trust. Here we put four estimators on one dataset where we know the true individual effect, and score each by how close it gets.

**INTUITION**

The average effect answers "does it work?". The conditional effect `τ(x)` answers "for whom?". Each learner is a different way to get `τ(x)` out of ordinary prediction models: S puts treatment inside one model, T fits two models, X patches T's weakness when groups are unequal, and double ML first removes everything that confounding could explain, then fits the leftover.

## The four recipes

| Learner | Recipe | Strength | Weakness |
|---|---|---|---|
| S | One model of `y` on `x` and `t`. Effect = prediction with `t=1` minus `t=0`. | Simple, uses all data | A regularised model can shrink the `t` effect to zero |
| T | Separate models for treated and control. Effect = difference. | Cannot ignore `t` | Noisy when one arm is small; the two errors do not cancel |
| X | Impute each person's missing outcome with the other arm's model, fit models on those imputed gaps, blend using the propensity `e(x)`. | Better when arms are unequal | More steps; needs a good propensity |
| Double ML | Predict `y` from `x` and `t` from `x` using cross-fitting. Regress the `y` residual on the `t` residual. | Robust to confounding that models can capture; has a standard error | Needs enough overlap; the effect form you regress on is a modelling choice |

**Problem 8.8.1** 20,000 customers. Engagement `x1` drives who gets the offer (confounding). The true effect is `τ = 1 + 1.5·x2`, so it depends only on `x2`. Compare naive, S, T, X and double ML.

| | |
|---|---|
| **Setup** | `y = 2·x1 + x2 + τ·t + noise`. Offer chance is `sig(0.8·x1)`. True average effect is 1 because `x2` has mean 0. |
| **Naive** | Treated customers have higher `x1` and so higher `y`. The raw gap will be far above 1. |
| **Cross-fitting** | Split in halves. Fit the nuisance models on one half, predict the other half, and swap. This stops a flexible model from memorising the very rows it residualises. |
| **DML step** | Residuals `ry = y − ŷ(x)` and `rt = t − ê(x)`. Regress `ry` on `rt` and `rt·x2`. The coefficients give `τ(x) = a + b·x2`. |

```
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
rng = np.random.default_rng(8)
n = 20000
x1, x2 = rng.normal(size=n), rng.normal(size=n)
e = 1/(1+np.exp(-(0.8*x1)))                # confounded treatment: x1 drives T
t = (rng.random(n) < e).astype(int)
tau = 1.0 + 1.5*x2                         # true effect depends on x2 only
y = 2*x1 + x2 + tau*t + rng.normal(size=n)
X = np.c_[x1, x2]
print("true ATE      ", round(tau.mean(), 3))
print("naive diff    ", round(y[t==1].mean() - y[t==0].mean(), 3))
def gb(): return GradientBoostingRegressor(n_estimators=150, max_depth=3, random_state=0)
# S-learner
s = gb().fit(np.c_[X, t], y)
tau_s = s.predict(np.c_[X, np.ones(n)]) - s.predict(np.c_[X, np.zeros(n)])
# T-learner
m1 = gb().fit(X[t==1], y[t==1]); m0 = gb().fit(X[t==0], y[t==0])
tau_t = m1.predict(X) - m0.predict(X)
# X-learner
d1 = y[t==1] - m0.predict(X[t==1]); d0 = m1.predict(X[t==0]) - y[t==0]
g1 = gb().fit(X[t==1], d1); g0 = gb().fit(X[t==0], d0)
tau_x = e*g0.predict(X) + (1-e)*g1.predict(X)   # true propensity used for weights
# Double ML (partialling out, 2-fold cross-fitting)
idx = rng.permutation(n); A, B = idx[:n//2], idx[n//2:]
ry = np.zeros(n); rt = np.zeros(n)
for tr, te in [(A, B), (B, A)]:
    ry[te] = y[te] - gb().fit(X[tr], y[tr]).predict(X[te])
    rt[te] = t[te] - gb().fit(X[tr], t[tr]).predict(X[te])
theta = (rt*ry).sum() / (rt*rt).sum()
# CATE by DML: regress ry on rt*[1, x2]
Z = np.c_[rt, rt*x2]
coef = np.linalg.lstsq(Z, ry, rcond=None)[0]
tau_d = coef[0] + coef[1]*x2
print(f"{'method':10s}{'mean':>8s}{'RMSE':>8s}{'corr':>8s}")
for name, v in [("S", tau_s), ("T", tau_t), ("X", tau_x), ("DML", tau_d)]:
    print(f"{name:10s}{v.mean():8.3f}{np.sqrt(((v-tau)**2).mean()):8.3f}{np.corrcoef(v,
tau)[0,1]:8.3f}")
print("DML scalar theta", round(theta, 3), " DML slope on x2", round(coef[1], 3))
```

```
true ATE       0.999
naive diff     2.287
method        mean    RMSE    corr
S            0.969   0.250   0.988
T            0.976   0.280   0.983
X            0.969   0.171   0.994
DML          0.965   0.039   1.000
DML scalar theta 0.961  DML slope on x2 1.481
```

- **Step 1** The naive gap is 2.287, more than double the true 0.999. Confounding inflates it.
- **Step 2** All four learners fix the average (0.965 to 0.976). The average hides the real difference between them.
- **Step 3** On individual effects, RMSE is S 0.250, T 0.280, X 0.171 and DML 0.039. DML wins here because we told it the right shape (effect linear in `x2`). With the wrong shape its advantage would shrink.
- **Step 4** DML recovers the slope: 1.481 against a true 1.5.

**COMMON MISTAKE**

Comparing learners by outcome accuracy (R squared of `y`). All four predict `y` well. What matters is accuracy of the *difference*, which you can only measure on simulated data, or approximately with Qini curves and randomised holdouts on real data. A learner with great R squared can have a useless `τ(x)`.

## Going deeper

- **Why residualise?** The Frisch-Waugh idea: the part of `t` that `x` predicts is exactly the part carrying confounding. Removing it leaves variation in `t` that behaves like a coin flip given `x`. Removing the predictable part of `y` only reduces noise.
- **Neyman orthogonality.** The DML estimating equation is insensitive to small errors in the nuisance models. That is why a rough forest for `ŷ` and `ê` still gives a root-n estimate. A plain "plug in a model" approach does not have this property.
- **Causal forests** (EconML `CausalForestDML`, grf in R) are DML with a forest in the last stage that finds where the effect varies and gives per-person intervals. Not run here, since the library is not installed, but the recipe above is its core.
- **Overlap.** Where propensity is near 0 or 1, residuals of `t` are tiny and the estimate explodes. Trim to a propensity range such as 0.05 to 0.95 and say so.

| | |
|---|---|
| **REAL JOB** | When a stakeholder asks for "who should get the offer", start with the T-learner on a randomised pilot (cheap, explainable). If only observational data exists, use DML or an X-learner with a checked propensity model, and always keep a small randomised holdout to validate the ranking. |

## 8.9 Staggered Rollouts: Why Two-Way Fixed Effects Can Fail

Section 8.3 compared one treated group with one control group at one launch date. Companies rarely do that. Stores get the new checkout in waves, cities get a feature month by month. The habit is to run one regression with a unit fixed effect, a time fixed effect and a 0/1 "treated now" dummy, called two-way fixed effects (TWFE). With one launch date it equals DiD. With several launch dates and effects that change over time, it can be badly wrong, even wrong in sign.

| | |
|---|---|
| **INTUITION** | TWFE averages many 2x2 comparisons. Some of them use units that were treated EARLIER as the "control" for units treated later. Those units are not untreated: their effect is still growing. Subtracting a growing effect from the later group's effect gives a smaller, sometimes negative, number. This is the "forbidden comparison". |

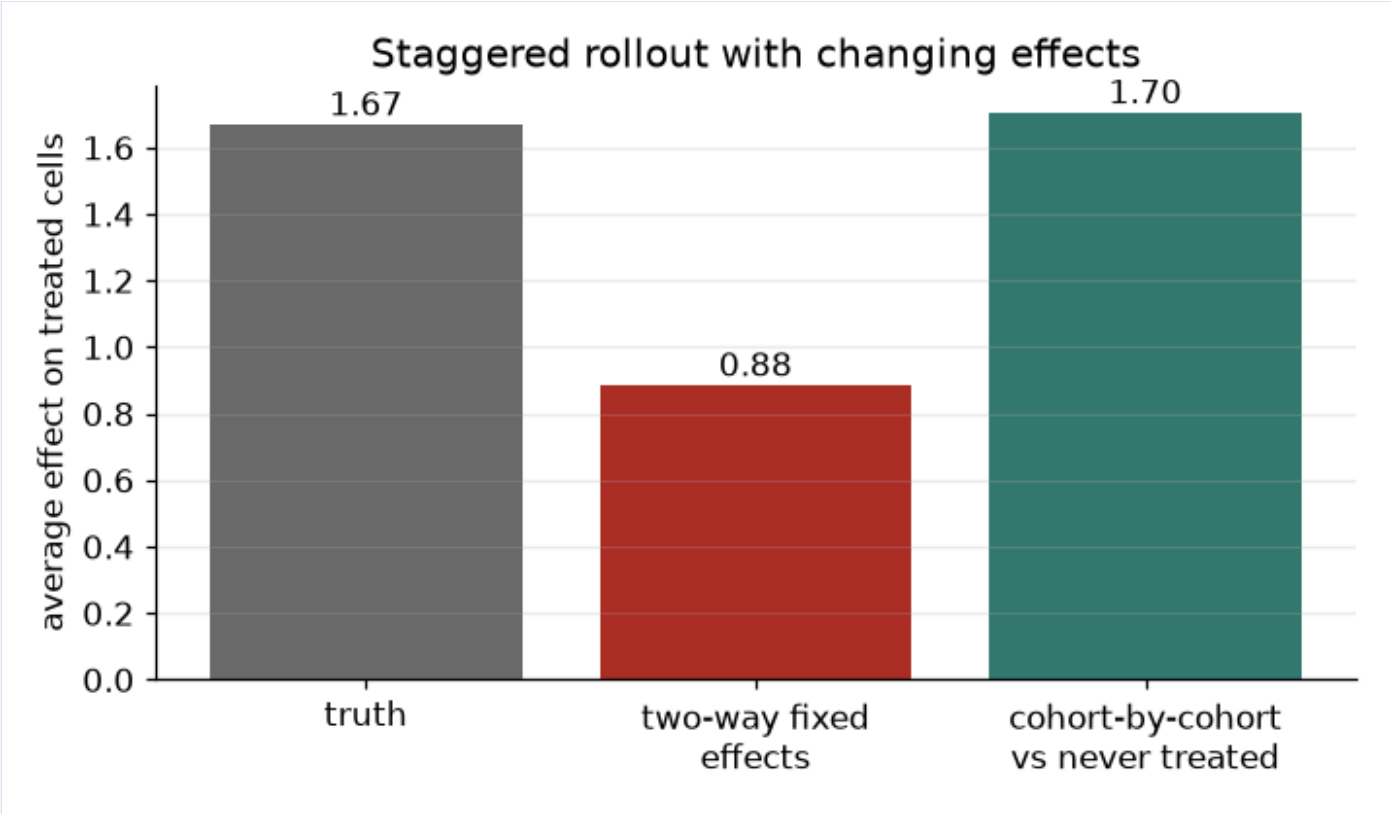
**Problem 8.9.1** 300 units over 12 periods. Cohort A (100 units) is treated from period 4 and its effect grows by 0.5 each period (0.5, 1.0, 1.5, ...). Cohort B (100 units) is treated from period 8 with a flat effect of 0.5. Cohort C (100 units) is never treated. Estimate the average effect on treated cells by TWFE and by cohort-by-cohort DiD against never-treated units.

| | |
|---|---|
| **Truth** | Average over all treated unit-periods. We compute it directly from the effects we planted. |
| **TWFE** | OLS with unit dummies, period dummies and the treated dummy. |
| **Clean DiD** | For each cohort and each period after launch, compare the change since the last pre-period for the cohort with the same change for never-treated units. Average these cells. |

```
import numpy as np
rng = np.random.default_rng(1)
U, T = 300, 12
cohort = np.repeat([4, 8, 99], 100)             # first treated period; 99 = never treated
ufe = rng.normal(0, 1, U); tfe = 0.3*np.arange(T)
def effect(c, t):
    if t < c: return 0.0
    return 0.5*(t - c + 1) if c == 4 else 0.5   # early cohort: grows; late cohort: flat
0.5
Y = np.zeros((U, T)); E = np.zeros((U, T)); D = np.zeros((U, T))
for u in range(U):
    for t in range(T):
        E[u, t] = effect(cohort[u], t); D[u, t] = t >= cohort[u]
        Y[u, t] = ufe[u] + tfe[t] + E[u, t] + rng.normal(0, 0.5)
print("true average effect over all treated cells:", round(E[D == 1].mean(), 3))
uu, tt = np.meshgrid(np.arange(U), np.arange(T), indexing="ij")
Uo = (uu.ravel()[:, None] == np.arange(1, U)[None, :]).astype(float)
To = (tt.ravel()[:, None] == np.arange(1, T)[None, :]).astype(float)
Z = np.c_[D.ravel(), np.ones(U*T), Uo, To]
b = np.linalg.lstsq(Z, Y.ravel(), rcond=None)[0]
print("TWFE estimate:", round(b[0], 3))
nv = cohort == 99; ests = []
for c in (4, 8):
    g = cohort == c
    for t in range(c, T):
        ests.append((Y[g, t] - Y[g, c-1]).mean() - (Y[nv, t] - Y[nv, c-1]).mean())
print("cohort-by-cohort DiD vs never-treated, averaged:", round(np.mean(ests), 3))
```

```
true average effect over all treated cells: 1.667
TWFE estimate: 0.883
cohort-by-cohort DiD vs never-treated, averaged: 1.703
```

- **Step 1** The truth is 1.667.
- **Step 2** TWFE reports 0.883, about half. Cohort B's flat 0.5 effect is compared against cohort A, whose effect is already large and growing, which drags the estimate down.
- **Step 3** The cohort-by-cohort estimator gives 1.703, close to the truth, because it only ever compares against units that are still untreated.




*The same data analysed two ways.*

**How to read it:** the grey bar is the planted truth. The red bar is the habit that many analysts still use. The teal bar uses only clean comparisons. If your rollout is staggered and effects change over time, the red bar can be far off.

## What to do in practice

1. Plot an event study: average outcome gap by periods since launch, for each cohort. Flat lines before launch support parallel trends. Look at the shape after launch too, not just the average.
2. Use an estimator built for staggered timing: Callaway and Sant'Anna, Sun and Abraham, de Chaisemartin and D'Haultfoeuille, or the stacked approach. Not run here, since the packages are not installed, but the code above is the simplest version of the idea.
3. If everyone is eventually treated, "never treated" does not exist. Use not-yet-treated units as controls and say so.

| COMMON MISTAKE | Running one TWFE regression on a rolling launch and quoting the coefficient as "the effect". Always check whether the effect changes over time or differs between cohorts. If it does, TWFE averages with weights that can be negative. |
|---|---|

# 8.10 One Treated Unit Over Time, and How Wrong Could We Be?

## Counterfactual from control series (the CausalImpact idea)

A national campaign launches in one region. You have daily orders for that region and for several untouched regions. The idea: learn how the treated region moves with the controls BEFORE launch, predict what it would have done AFTER launch, and read the gap as the effect. Google's CausalImpact does this with a Bayesian structural time series. The version below uses plain regression, which teaches the same logic and runs here.

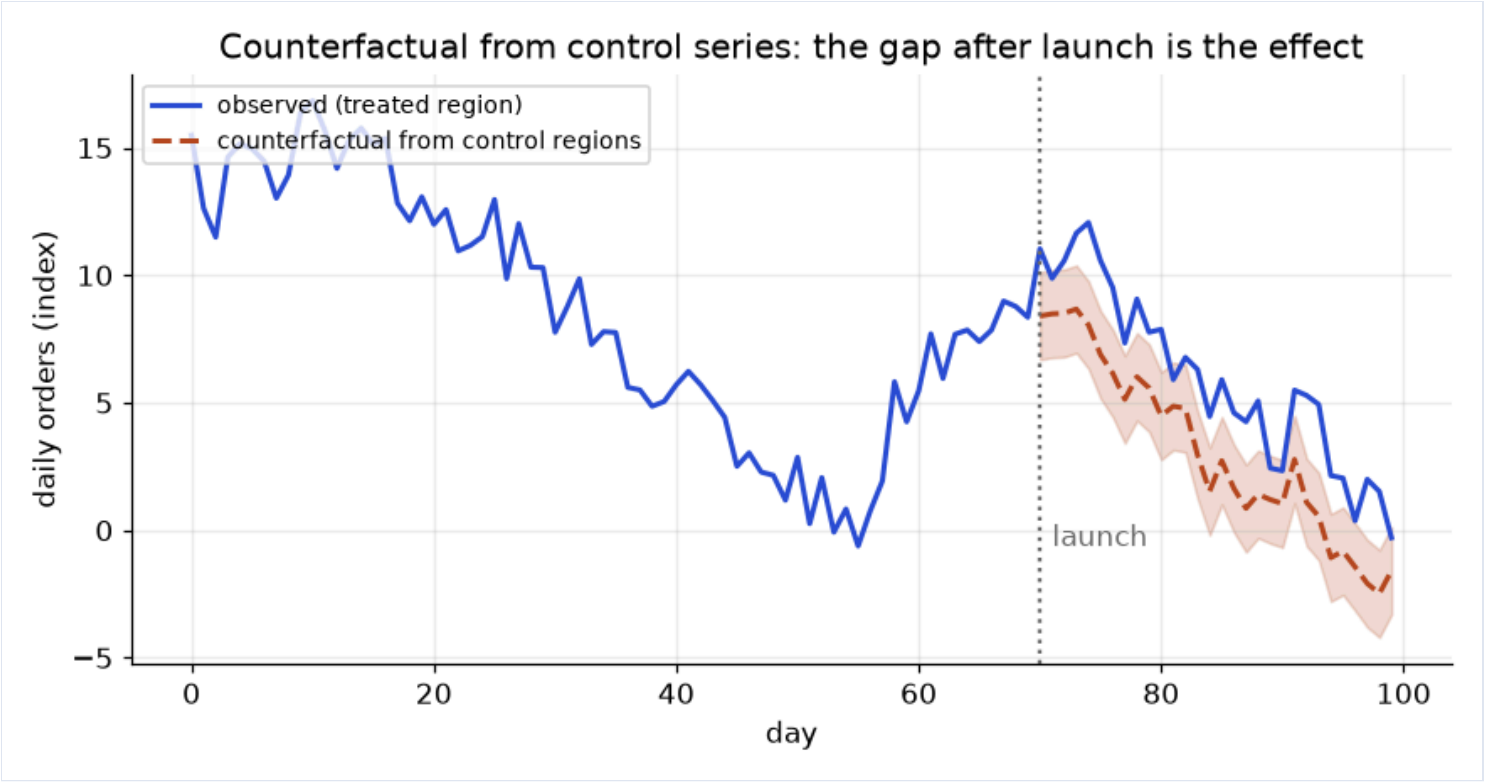
**Problem 8.10.1** 100 days. Launch on day 70. Three control regions share a common trend with the treated region. The planted lift is 3 orders per day. Fit the treated series on the controls over days 0 to 69, predict days 70 to 99, and average the gap. Then do a placebo: pretend the launch was on day 50.

| | |
|---|---|
| **Idea** | Pre-period regression is the counterfactual machine. The controls must not be affected by the launch. |
| **Effect** | Mean of (observed − predicted) after launch. |
| **Placebo** | Same procedure at a fake launch date inside the pre-period. It should give about 0. |
| **Interval** | Rough: pre-period residual sd, doubled to allow for autocorrelation, divided by the root of the post-period length. A real tool uses a proper time-series model. |

```
import numpy as np
rng = np.random.default_rng(5)
T, T0 = 100, 70                                  # launch at day 70
common = np.cumsum(rng.normal(0, 1, T)) + 0.05*np.arange(T)
c1 = common + rng.normal(0, 0.8, T) + 10
c2 = 0.8*common + rng.normal(0, 0.8, T) + 20
c3 = 1.2*common + rng.normal(0, 0.8, T) + 5
treated = 1.0*common + rng.normal(0, 0.8, T) + 15
treated[T0:] += 3.0                              # true lift of 3 per day after launch
C = np.c_[np.ones(T), c1, c2, c3]
beta = np.linalg.lstsq(C[:T0], treated[:T0], rcond=None)[0]
resid = treated[:T0] - C[:T0] @ beta
sd = resid.std(ddof=4)
cf = C[T0:] @ beta
gap = treated[T0:] - cf
print("pre-period fit sd:", round(sd, 3))
print("average daily lift:", round(gap.mean(), 3))
# placebo: pretend launch at day 50 using only days 0..69
beta_p = np.linalg.lstsq(C[:50], treated[:50], rcond=None)[0]
gap_p = treated[50:70] - C[50:70] @ beta_p
print("placebo lift (days 50-69, no launch):", round(gap_p.mean(), 3))
# rough interval: sd of the mean of gap errors, inflated for autocorrelation by 2
se = 2*sd/np.sqrt(T-T0)
print("approx 95% interval:", round(gap.mean()-1.96*se, 2), "to",
round(gap.mean()+1.96*se, 2))
```

```
pre-period fit sd: 0.874
average daily lift: 2.82
placebo lift (days 50-69, no launch): 0.02
approx 95% interval: 2.19 to 3.45
```

- **Step 1** Estimated lift is 2.82 per day against a true 3.0, with a rough interval of 2.19 to 3.45 that covers the truth.
- **Step 2** The placebo effect is 0.02, so the method does not invent effects when none exist.




*Observed region against the counterfactual built from three control regions.*

**How to read it:** before the dotted launch line, the two lines hide each other, which is what a good fit means. After launch the solid blue line lifts away from the dashed orange line. The shaded band is the plausible range for the counterfactual. A gap inside the band is not convincing.

| COMMON MISTAKE | Using controls that the launch also touched (for example a neighbouring region where customers saw the same ads). The model then learns the effect into the counterfactual and reports a smaller lift. Also never fit on the post-period. |
|---|---|

## Sensitivity analysis: how strong would a hidden confounder have to be?

Every observational estimate carries one unprovable assumption: no unmeasured confounding. You cannot prove it, but you can say how strong a hidden confounder would have to be to explain the result away. The **E-value** (VanderWeele and Ding) does that for a risk ratio `RR`:

$$E\text{-value} = RR + \sqrt{( RR \times (RR - 1) )} \text{ (use } 1/RR \text{ first if } RR < 1)$$

**Problem 8.10.2** An observational study finds that customers who used the new feature are 1.8 times as likely to renew (RR = 1.8, 95% interval lower limit 1.3). How strong must a hidden confounder be?

| | |
|---|---|
| **By hand, point** | 1.8 + √(1.8 × 0.8) = 1.8 + √1.44 = 1.8 + 1.2 = 3.0 |
| **By hand, limit** | 1.3 + √(1.3 × 0.3) = 1.3 + 0.6245 = 1.924 |
| **Meaning** | A hidden factor would need to be associated with both feature use and renewal by a risk ratio of at least 3.0 each (beyond what you adjusted for) to fully explain the point estimate, or 1.92 to move the interval to include 1. |

```
import numpy as np
def evalue(rr):
    rr = 1/rr if rr < 1 else rr
    return rr + np.sqrt(rr*(rr-1))
print("E-value for RR 1.8:", round(evalue(1.8), 3))
print("E-value for CI limit 1.3:", round(evalue(1.3), 3))
print("E-value for RR 0.5:", round(evalue(0.5), 3))
```

```
E-value for RR 1.8: 3.0
E-value for CI limit 1.3: 1.924
E-value for RR 0.5: 3.414
```

The code agrees with the hand calculation. For a protective effect, RR = 0.5 is treated as 2, giving 3.414.

| KEY IDEA | Compare the E-value to the strongest confounder you actually measured. If "tenure" has an association of 1.5 with renewal and feature use, a hidden factor of 3.0 is unlikely to be much stronger than tenure. A low E-value (say 1.2) means the finding is fragile whatever else you do. |
|---|---|

## Check yourself

1. When does an S-learner underperform? Answer: when regularisation or tree depth lets the model ignore `t`, so the effect shrinks toward zero.
2. Why does cross-fitting matter? Answer: without it, the nuisance model has seen the rows it residualises and overfitting leaks into the effect estimate.
3. Your controls were also exposed to the campaign. What happens? Answer: the counterfactual absorbs part of the effect and the lift is understated.

# Chapter recap

- Causal questions ask "what if we do X", which prediction cannot answer. Confounding makes naive comparisons wrong, even in sign.
- Draw a DAG first. Adjust for confounders, not for mediators or colliders.
- Propensity score `e(x) = P(T=1|x)` summarises how treatment was assigned. Weighting by `1/e` and `1/(1−e)` rebuilds a fair comparison.
- Check overlap and balance (SMD < 0.1). Compare several methods; doubly robust estimators are safer.
- Difference-in-differences needs parallel trends; check the pre-period and run placebo tests.
- Synthetic control builds a weighted twin of one treated unit; judge it by pre-fit and placebo ranking.
- Regression discontinuity compares customers just above and below a rule; report several window sizes.
- IV needs relevance, independence and exclusion; report the first-stage F; the answer is for compliers.
- Uplift models target the people whose behaviour changes; judge them by observed uplift per decile and Qini, not AUC.
- Every method rests on assumptions you must state. Run sensitivity analysis for hidden confounders.

| Method | Use when | Key assumption | Quick check |
|---|---|---|---|
| Adjustment / IPW / AIPW | All confounders measured | No unmeasured confounding, overlap | SMD, propensity range |
| DiD | Treated and control groups, before and after | Parallel trends | Pre-trends, placebo |
| Synthetic control | One or few treated units | Pre-fit good, donors unaffected | Pre RMSE, placebo rank |
| RD | Rule with a cut-off | No manipulation, smooth otherwise | Histogram, window sweep |
| IV | Hidden confounding plus a random lever | Relevance, independence, exclusion | First-stage F |
| Uplift | Who to target | Random or adjusted assignment | Decile uplift, Qini |

# Practice set

**1. (easy)** Treated customers ordered at 40%, controls at 30%, but treated customers were twice as engaged on average. Can you call the 10-point gap the effect? **Answer:** No. Engagement likely confounds; adjust for it or randomise.

**2. (easy)** IPW weight for a control customer whose propensity is 0.75? **Answer:** $1/(1 - 0.75) = 4$.

**3. (easy)** Treated sales went from 50 to 70; control from 40 to 52. DiD? **Answer:** $(70 - 50) - (52 - 40) = 20 - 12 = 8$.

**4. (medium)** Name a confounder, a mediator and a collider in "discount (T) affects repeat purchase (Y)". **Answer:** confounder: customer loyalty (drives who gets discounts and repeat purchase); mediator: app opens after the discount message; collider: "contacted support" (caused by both a bad experience and a discount confusion).

**5. (medium)** First stage: Z raises T by 0.20. Reduced form: Z raises Y by 0.50. IV estimate and F-statistic rule? **Answer:** $0.50 / 0.20 = 2.5$. Check first-stage F above about 10.

**6. (medium, what is the output)** After IPW, SMD for age is 0.02 and for income is 0.35. Verdict? **Answer:** Age balanced; income not (above 0.1). Improve the propensity model (add income terms), trim weights or use a different method.

**7. (medium, find the bug)** An analyst runs RD with a cubic polynomial over the whole score range and gets a jump of 15 whose sign changes with the polynomial degree. What is wrong? **Answer:** Global polynomials can create a fake jump from curvature far away. Use local linear fits near the cut-off and report several windows.

**8. (hard)** Synthetic control: the pre-period RMSE is 6 and the post-launch gap is 5. Is there an effect? **Answer:** Not convincingly. The pre-fit error is larger than the effect. Improve the donor pool or the fit before claiming anything.

**9. (hard, scenario)** A pilot ran in 4 cities chosen by the regional head because "sales were rising fast there". An analyst uses DiD against other cities. What is the risk? **Answer:** The treated cities were selected on trend, so parallel trends likely fail and DiD overstates the effect. Check the pre-period trend, use matched controls, or synthetic control.

**10. (hard)** Uplift: the average effect of an offer is +1 point, but the top decile shows +30 and the bottom decile −20. Costs: offer ₹50, order margin ₹400. Is it worth targeting the top decile? **Answer:** Expected gain per customer = $0.30 \times 400 - 50 = ₹70$. Yes. In the bottom decile it is $-0.20 \times 400 - 50 = -₹130$, so never target.

**11. (interview)** "How would you measure the impact of a feature launched to everyone at once?" **Answer:** Check whether a control exists (a holdout, a staggered rollout, regions launched later). Use DiD, synthetic control or interrupted time series; state the assumptions and the checks. Recommend a holdout next time.

**12. (interview)** "Your regression says discount users spend more. Does the discount work?" **Answer:** Not necessarily. Customers who receive discounts may differ (confounding). Draw a DAG, adjust with propensity methods or find a quasi-experiment, check balance, and run a sensitivity analysis. Best: a randomised test.

**13. (medium)** Propensity for a treated customer is 0.02. What IPW weight does that customer get and why is it a worry? **Answer:** 1/0.02 = 50. One customer counts as 50, so the estimate swings on a single row. Trim propensities (for example to 0.05 to 0.95), use stabilised or capped weights, or narrow the population.

**14. (medium)** E-value for RR = 2.5. **Answer:** $2.5 + \sqrt{(2.5 \times 1.5)} = 2.5 + \sqrt{3.75} = 2.5 + 1.936 = 4.436$. A hidden confounder would need an association of about 4.4 with both treatment and outcome.

**15. (hard, find the bug)** A team rolled a feature out in three waves, ran a TWFE regression and got an effect of 0.2 that is significant, though every wave's own before-after chart shows an effect near 1.0 that keeps growing. What is the likely cause and the fix? **Answer:** Later waves are compared with earlier, already treated waves whose effect is growing, which pulls the TWFE coefficient down. Use an estimator with clean comparisons (never-treated or not-yet-treated controls), and report an event study.

**16. (hard, code)** In the counterfactual example, what happens to the estimated lift if you add the post-launch days to the regression fit? **Answer:** The model can bend toward the treated series after launch, so the predicted counterfactual moves up and the gap shrinks, often to near zero. Only the pre-period may be used for fitting.

**17. (interview)** "How would you decide between a T-learner and double ML for targeting?" **Answer:** With randomised data and a big sample, a T-learner is fine, easy to explain and checkable on a holdout. With observational data, use double ML or an X-learner that handles confounding through the propensity, check overlap, and validate the resulting ranking on a randomised test before spending budget.

## Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, studying one hour a day. Topic: confounding, DAGs and the backdoor rule in causal inference. Explain with an everyday analogy, then with a retention-offer example where engagement is the confounder. Use small numbers I can verify by hand to show Simpson's paradox flipping a result. Then ask me to draw a DAG for a new scenario and check my answer, including which variables to adjust for and which not to.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: propensity scores, inverse probability weighting and balance checks (SMD). Walk me through a tiny hand-computable example, then a Python example with synthetic data
```

that has a known true effect. Then give me 8 graded exercises without solutions, including overlap problems and extreme weights. After I answer, grade me and explain my errors in simple English.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science. Topic: difference-in-differences and synthetic control for business launches. Quiz me one question at a time using realistic scenarios (store pilots, city launches). After each answer, tell me what I got right, what I missed, and which assumption I should have checked (parallel trends, placebo tests, pre-fit). Start easy and get harder over 10 questions.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science. Topic: choosing between regression discontinuity, instrumental variables and uplift modelling. Here is my business problem: [PASTE HERE]. Tell me which method fits, which assumptions it needs, how to check them, what could go wrong, and what I would say to a non-technical stakeholder. Use plain English and example numbers.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science and preparing for interviews. Topic: causal inference for product and marketing data scientists. Run a mock interview with me, one question at a time, such as "how would you measure the effect of a feature launched to everyone" or "who should get the retention offer". After each answer, score me 1 to 5, give a model answer, and ask a follow-up. Do 8 rounds, then summarise my weak spots.

CHAPTER 9

# FORECASTING AND TIME SERIES FOR DECISIONS

*How to predict demand, orders or traffic honestly: beat the baselines, test with backtests, state the uncertainty, and turn the forecast into a stock or staffing decision.*

Every operations team asks the same question: "How many will we need next week?" Orders to pack, support agents to roster, servers to keep warm, units to stock. A forecast is a number that someone will use to spend money. If it is too low, you lose sales or crash under load. If it is too high, you pay for idle capacity. Anyone who has planned staffing for a busy shop knows the feeling. This chapter teaches the data-science version. You will learn what a time series is made of, the simple baselines that beat many fancy models, the classical models (exponential smoothing, ARIMA, and a Prophet-style regression), machine learning with lag features, the one correct way to validate (walk-forward backtesting), prediction intervals, and finally how to turn a forecast into a decision. By the end you will be able to run a fair model contest and report honestly who won.

**Running example.** Daily orders of a delivery app over two years (730 days, from 1 Jan 2023). We built them: a base level of 200 growing by 0.25 per day, a weekly pattern (weekends are 20 to 25% busier, Mondays 10% quieter), random noise of about 6%, and a five-day festival spike (up to 1.8 times normal) around 12 Nov 2023 and 1 Nov 2024. All values are synthetic with a fixed seed ( `default_rng(4)` ) and all numbers below come from real runs. `statsmodels` , `prophet` and `lightgbm` were not installed here, so we coded Holt-Winters and a Prophet-style regression by hand with `numpy` , and used scikit-learn's `HistGradientBoostingRegressor` for the lag model. Those library calls are shown as "not run here".

## 9.1 Components, Baselines and Error Metrics

**INTUITION**

A time series is a story told in order: today depends on yesterday, last week and last year. Before hiring a clever storyteller (a model), ask a lazy one: "What happened last Sunday? Then assume the same this Sunday." That lazy guess is a baseline. If a model cannot beat it, the model is not worth its complexity.

### Words you need

| Word | Plain meaning | In our orders data |
|---|---|---|
| Level | The average height of the series now | About 390 orders per day in late 2024 |

| Word | Plain meaning | In our orders data |
|---|---|---|
| Trend | Slow, steady rise or fall | +0.25 orders per day each day |
| Seasonality | A pattern that repeats at a fixed period | Weekly: Sunday high, Monday low |
| Holiday / event | Irregular, known-date effect | Festival around 1 Nov |
| Noise | Random wiggle nobody can predict | About 6% either way |
| Horizon | How far ahead we forecast | Next 91 days |
| Decomposition | Splitting a series into those parts | Weekday index by dividing by a 7-day average |

**Decomposition.** A 7-day centred moving average removes the weekly cycle and leaves the trend. Dividing the series by it leaves the seasonal ratios. For our data the weekday indexes, Monday to Sunday, came out as 0.854, 0.897, 0.951, 0.941, 1.014, 1.153, 1.189. Sunday is 19% above the average day. (We built 1.25 with the trend added; the moving average mixes the trend in, so estimates differ a little. STL decomposition in `statsmodels` does this more carefully: not run here.)

## The four baselines

| Baseline | Rule | Use when |
|---|---|---|
| Naive | Tomorrow = today | Random-walk like data (prices) |
| Seasonal naive | Day = same weekday last week | Strong weekly cycle |
| Moving average | Average of last w days | Flat, noisy data |
| Drift | Draw a line from first to last point and extend | Steady trend, no seasonality |

**Problem 9.1.1** (tiny numbers) Orders for 14 days start at Monday. Week 1: 90, 95, 100, 100, 105, 120, 125. Week 2: 99, 104, 110, 110, 115, 132, 138. Forecast week 3 with seasonal naive, and then compare with a simple exponential smoothing on the first four numbers [10, 12, 11, 13] with α = 0.5.

| | |
|---|---|
| **Step 1 (seasonal naive)** | Week 3 forecast = week 2 repeated: 99, 104, 110, 110, 115, 132, 138. |
| **Step 2 (exponential smoothing, SES)** | The rule is `level_new = α · today + (1 − α) · level_old`. It weights recent days more, with weights that shrink geometrically (that is why it is called exponential). Start with level = 10. |
| **Step 3** | After 12: 0.5 × 12 + 0.5 × 10 = 11. After 11: 0.5 × 11 + 0.5 × 11 = 11. After 13: 0.5 × 13 + 0.5 × 11 = 12. |
| **Answer** | The SES forecast for every future day is 12 (flat). Seasonal naive repeats last week's pattern, including the weekend rise. |
| **Check** | 12 is between the recent values 11 and 13, as a weighted average must be. Week 2 is about 1.1 times week 1 in the example, so seasonal naive would be 10% low for week 3 if the growth continues. Baselines are simple, not perfect. |
| **Why it matters** | SES is the building block of all Holt-Winters models: add a trend term and a seasonal term and you have ETS. |

## Error metrics, and why there are so many

For actual values `a` and forecasts `f`:

| Metric | Formula | Good for | Problem |
|---|---|---|---|
| MAE | mean of ` | a − f | ` |
| RMSE | `√mean((a − f)²)` | Punishes big misses | Dominated by outliers |
| MAPE | mean of ` | a − f | / a × 100` |
| WAPE | `Σ | a − f | / Σ |
| MASE | MAE divided by MAE of naive on training data | Compare across series; below 1 beats naive | Needs training history |

| Metric | Formula | Good for | Problem |
|---|---|---|---|
| Bias | mean of `f − a` | Are we systematically high or low? | Cancels in a mean |

**Problem 9.1.2** Actuals `[100, 120, 80, 0.5]`, forecasts `[110, 100, 90, 5]`. Compute MAPE and WAPE.

| | |
|---|---|
| **Step 1 (percent errors)** | 10 / 100 = 10%. 20 / 120 = 16.7%. 10 / 80 = 12.5%. 4.5 / 0.5 = 900%. |
| **Step 2 (MAPE)** | (10 + 16.7 + 12.5 + 900) / 4 = 234.8%. |
| **Step 3 (WAPE)** | Sum of absolute errors = 10 + 20 + 10 + 4.5 = 44.5. Sum of actuals = 300.5. WAPE = 44.5 / 300.5 = 14.8%. |
| **Answer** | MAPE says 235%, which is absurd, because one tiny actual (0.5) dominates. WAPE says 14.8%, which matches our feeling of "we were off by about 15% of volume". |
| **Check** | The three big items were each off by 10 to 20%; WAPE reflects that. |
| **Why it matters** | For slow-moving items, MAPE is a trap. Use WAPE, MASE or MAE. |

## The first contest

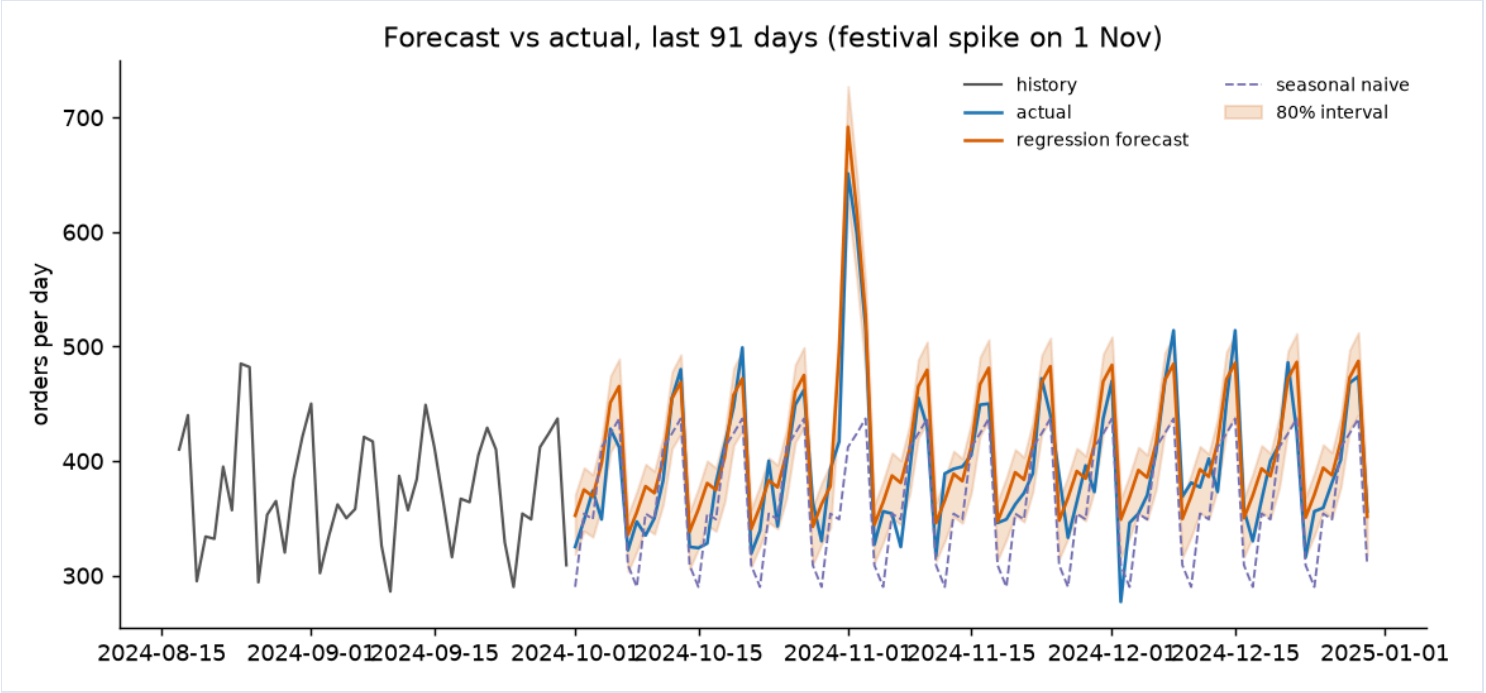
We hold out the last 91 days (1 Oct to 30 Dec 2024, which includes the 1 Nov festival), train on the earlier 639 days, and forecast all 91 days at once.

```
model                          MAE     RMSE    MAPE    WAPE   MASE(7)
naive                        87.59   107.42   20.41   22.13    4.05
drift                        82.81   103.31   19.21   20.92    3.83
moving avg 7                 49.51    69.10   11.53   12.50    2.29
seasonal naive               37.00    50.55    9.03    9.35    1.71
seasonal avg of last 4 wks   33.12    48.58    7.87    8.37    1.53
Holt-Winters (9.2)           24.13    40.84    5.68    6.09    1.12
Regression, Prophet-style    22.88    27.88    6.06    5.78    1.06
```

The MASE here divides each model's test MAE by the average size of a one-week-apart change in the training data (the in-sample seasonal naive error). A value of 1.06 means the test error is 6% larger than that in-sample yardstick; so for this metric lower is better, and values near 1 are good. Note the plain **naive** forecast is the worst, because ignoring the weekly cycle is costly. The **seasonal naive** forecast halves its error (87.6 to 37.0).

That single idea, "use the same weekday", captures most of the signal. Anything more complex must justify itself against 37.0.

```
orders
 650 |               *                    <- festival spike on 1 Nov
     |              ***
 400 |   /\ /\ /\ /\*   \ /\ /\ /\
     |  /  V  V  V  V    V  V  V  V       weekly zig-zag, growing
 250 +-------------------------------------> day
```




*Last 91 days: actual orders, the regression forecast with an 80% interval, and the seasonal naive baseline. The festival spike is partly caught by the regression model.*

How to read the image: the grey line is history, the blue line is what actually happened, and the orange line is the forecast with its shaded 80% band. The dashed purple line is the baseline, which keeps repeating the previous week and misses the growth and the festival.

## Connection to the real job

Baselines are the first slide of every forecasting project: "Our current rule gives WAPE 9.4%. The new model gives 5.8%." That one sentence is how you justify the project.

## Applications

- **Supply chain:** store-level demand for fresh items.
- **Workforce:** call-centre staffing by half-hour.

- **Engineering:** daily active users and capacity planning for servers.
- **Finance:** monthly revenue guidance.

| COMMON MISTAKE | Reporting one error number for a model without the baseline next to it. A MAPE of 9% sounds good or bad depending on whether the naive rule gets 8% or 25%. |
|---|---|

**Check yourself**

1. Which metric breaks when actual values are near zero? Answer: MAPE.
2. What does a MASE below 1 mean? Answer: the model beats the naive forecast (measured on training data).
3. Why is "same weekday last week" so strong for weekly data? Answer: it copies the seasonal pattern and the current level with no fitting.

# 9.2 Stationarity, ACF and the Classical Models

| INTUITION | A stationary series is a calm sea: its average and its wobble stay the same over time. Most real series are not calm: they trend and have seasons. Classical methods either model the trend and season directly (exponential smoothing) or first remove them by subtracting (ARIMA's "differencing"). |
|---|---|

## Stationarity and differencing

**Stationary** means the mean, the spread and the pattern of correlation do not change with time. **Differencing** replaces each value with its change: `d_t = y_t − y_(t−1)`. This removes a steady trend. **Seasonal differencing** uses `y_t − y_(t−7)` and removes a weekly cycle.

The **autocorrelation function (ACF)** at lag k is the correlation between the series and itself k steps earlier. It shows what repeats. **Unit root tests** (ADF, KPSS) check stationarity formally; they live in `statsmodels` (not run here).

**Problem 9.2.1** Read the ACF of our orders data at lags 1, 2, 7, 14.

```
                lag 1   lag 2   lag 7   lag 14
raw series       0.72    0.53    0.86    0.85
after d(1)      −0.18   −0.28    0.67    0.66
after d(7)       0.24    0.13   −0.52    0.02
```

- **Step 1** The raw series has high correlation at lags 7 and 14 (0.86, 0.85). That is the weekly cycle plus trend.
- **Step 2** After ordinary differencing, lag 1 and 2 are negative, but lags 7 and 14 stay high (0.67, 0.66). The weekly cycle survives, so one ordinary difference is not enough.
- **Step 3** After a seasonal difference (lag 7), the lag-14 value falls to 0.02. The weekly pattern is gone. The −0.52 at lag 7 is a typical side effect of differencing noise.
- **Answer** For ARIMA on this series we would use a seasonal difference of 7 (that is the "D = 1, m = 7" of SARIMA).
- **Check** A spike in the ACF at lag 7 on a daily series always signals weekly seasonality; this is quick to confirm on your own data.
- **Why it matters** ACF plots tell you the model structure (how many past terms) before you fit anything.

## ETS / Holt-Winters

**Exponential smoothing with trend and season (ETS, Holt-Winters)** keeps three running numbers: a level, a trend and 7 seasonal factors. Each is updated with its own smoothing weight (α for level, β for trend, γ for season, between 0 and 1). Small weights mean "change slowly, trust the long past". It handles trend and season, needs no stationarity, and is fast. We coded the multiplicative version and chose α, β, γ by minimising the error on the training data from a small grid: it picked α = 0.1, β = 0.01, γ = 0.05.

## ARIMA and SARIMA

**ARIMA(p, d, q)** has three parts: AR (autoregressive, `p` past values), I (integrated, `d` differences), MA (moving average, `q` past forecast errors). **SARIMA** adds a seasonal set. **SARIMAX** adds outside predictors (price, promotion flags). We can show the AR idea by hand: after a log transform and a lag-7 seasonal difference, fitting AR(1) by least squares gave `φ = 0.05`, meaning the seasonally-differenced growth has almost no memory. For this synthetic series, the seasonal difference already removes most structure, so ARIMA would not beat Holt-Winters. In practice use `statsmodels` `SARIMAX` or `pmdarima` / `statsforecast` `AutoARIMA` (not run here). Also see **TBATS** for multiple seasonalities (such as daily and weekly in hourly data).

## The Prophet-style regression

**Prophet** (a library from Meta) is a regression with named pieces: `y = trend + weekly seasonality + yearly seasonality + holidays`. The seasonal pieces are made from sine and cosine waves (a **Fourier series**: a few smooth waves added together can draw any repeating shape). We did the same by hand: `log(orders) = a + b·t + weekly waves (3 pairs) + yearly waves (2 pairs) + five festival-day effects (−2 to +2 days around the festival)`, fitted with ordinary least squares. The festival effects are shared across years, so 2023 teaches the model what 2024 will look like. (The real `prophet` package adds changing trend slopes and Bayesian fitting: not run here.)

**Problem 9.2.2** Which of Holt-Winters or the regression is better on the 91-day test, and is it the same on a fair backtest?

| | |
|---|---|
| **Step 1 (one split)** | From the table: Holt-Winters MAE 24.13, regression MAE 22.88. RMSE: 40.84 against 27.88. The regression's RMSE is much smaller because it caught the festival spike (big errors are punished by RMSE). |
| **Step 2 (rolling backtest, next section)** | Ten origins, 28-day horizon: Holt-Winters 24.68, regression 19.50, seasonal avg 4 weeks 23.85, seasonal naive 33.49. |
| **Answer** | The regression is best in both, mostly because it knows the festival date. Holt-Winters is competitive and needs no holiday list. |
| **Check** | On the one split, the 4-week seasonal average beats seasonal naive (33.1 against 37.0). Averaging a few weeks reduces noise, as the 6% daily noise is large. |
| **Why it matters** | If a model knows the calendar (holidays, paydays, sales), give it that information. Calendar knowledge often beats a better algorithm. |

## Connection to the real job

Capacity and demand planning teams run ETS and SARIMA as the "workhorse" models, because they are cheap, stable and need no GPU. At scale (thousands of series) people use `statsforecast`, which fits them very fast.

## Applications

- **Retail:** weekly seasonality in store sales.
- **Telecom:** traffic with daily and weekly cycles.
- **Finance:** monthly cash flows with quarter-end effects.
- **SRE:** request-rate forecasts for autoscaling.

| | |
|---|---|
| COMMON MISTAKE | Picking ARIMA orders (p, d, q) by staring at the ACF only. Use the ACF as a hint, then compare candidates on a backtest. And never difference data that is already stationary (it adds noise). |

**Check yourself**

1. What does differencing remove? Answer: a trend (ordinary) or a seasonal cycle (seasonal differencing).
2. A daily series has an ACF spike at lag 7. What does it mean? Answer: weekly seasonality.

3. Why Fourier terms? Answer: they let a regression fit smooth repeating shapes with few columns.

## 9.3 Machine Learning with Lags, and How to Validate

| INTUITION | Turn forecasting into a spreadsheet. Each row is a day. Columns are what we knew BEFORE that day: yesterday's orders, orders a week ago, the 28-day average, the weekday. The target column is that day's orders. Now any tabular model can learn it. The only new rule: rows must never see the future. |
|---|---|

### Features

| Feature | Example | Why |
|---|---|---|
| Lag k | orders 1, 7, 14, 28 days ago | Recent level and weekly echo |
| Rolling mean | mean of the previous 7 or 28 days (shifted by 1) | Smooth level |
| Calendar | day of week, day of year, holiday flag, payday | Known future information |
| Known future inputs | planned promotions, price | Only if you will know them at forecast time |

Remember the leakage rule from Chapters 2 and 3: rolling means must be computed on data BEFORE the target day ( `shift(1)` first, then `rolling` ).

**Two ways to forecast many days ahead. Recursive:** predict tomorrow, feed it back as "yesterday" for the day after. Errors compound. **Direct:** train a separate model for each horizon (1 day, 2 days, ...). More work, no compounding. Our run is recursive.

**Global models.** If you have 5,000 products, you can train ONE model on all series together with an "item" column, instead of 5,000 small models. A global model shares what is common (weekly shape, holiday effect) and helps new or short series (cold start). This is usually how LightGBM or neural networks are used for retail forecasting.

### The result, honestly

On the same 91-day test, the recursive gradient boosting model with the lags above scored MAE 35.5, RMSE 49.4, WAPE 8.98%, MASE 1.65. That is **worse** than Holt-Winters (24.1) and the regression (22.9), and about

the same as the seasonal averages.

- **Why:** two years of data with a growing trend. Tree models predict averages of training rows, so they cannot extrapolate above the highest levels they saw in training. Lag features carry the level, but errors compound in the recursive loop. The festival occurs once per year, so two examples is too little to learn from.
- **Takeaway:** ML shines with many series, many rows, many external features (price, promotions, weather). On one short, clean, trending series, classical methods often win. This is "Rule 1" from the skill map in action: compare against the baselines. See the ML document ( `study/ml/` ) for gradient boosting itself.

**Deep and foundation models (awareness only, not run here):** LSTM/GRU, N-BEATS, Temporal Fusion Transformer, PatchTST, and pretrained "foundation" models like Chronos and TimesFM. They can win with large collections of series. They need more data, more tuning and more care, and still must beat the baseline on a backtest.

## Validation: never random k-fold

If rows are shuffled, a row from 15 Dec can be in the training set while its neighbour 14 Dec is in the test set. The model sees the future. Walk-forward (rolling-origin) backtesting copies how the model is used:

```
origin 1:  train |==========|  test [----]
origin 2:  train |==============|  test [----]
origin 3:  train |==================|  test [----]
           time ------------------------------>
```

Each origin: train only on data up to the origin, forecast the next 28 days, record errors, move the origin forward 28 days.

**Problem 9.3.1** Show how much random k-fold flatters the same gradient boosting model on one-step-ahead prediction.

```
# one-step features: lag1, lag7, lag14, lag28, r7, r28, dow, doy
for tri, tei in KFold(5, shuffle=True, random_state=0).split(X): ...   # random folds
m = HistGradientBoostingRegressor(...).fit(X.iloc[:600], np.log(y.iloc[:600]))  # time
split
```

Real output:

```
random 5-fold one-step MAE      20.39
time split (train first 600 rows, test last 102)   29.10
```

- **Step 1** Random folds put days from every period in training, including the neighbours of each test day. The model effectively interpolates.
- **Step 2** The time split trains only on the past, and the same model scores 29.1, which is **43% worse**. (The test sets are not identical; the second one is the last 102 days, which also includes the festival and the highest levels. Both effects are real parts of a real forecast.)
- **Answer** Random k-fold would have told us "MAE 20" when the honest figure is about 29.
- **Check** The honest number is the one in production. If your score drops when you switch to a time split, the earlier score was leaking.
- **Why it matters** Teams that validate with random splits ship models that fail in the first week.

**Problem 9.3.2** A rolling-origin backtest: 10 origins, 28-day horizon, starting at day 450. Compare errors by horizon.

```
model                    mean MAE over 10 backtests   days 1–7   days 22–28
seasonal naive                     33.49                33.91       37.94
seasonal avg 4 weeks               23.85                24.12       27.47
Holt–Winters                       24.68                24.13       31.44
Regression (Prophet–style)         19.50                19.87       20.60
```

- **Step 1** The ranking is stable across ten origins: regression first, then the 4-week average and Holt-Winters, then seasonal naive.
- **Step 2** Error grows with horizon. Holt-Winters worsens from 24.1 (days 1 to 7) to 31.4 (days 22 to 28), because its trend estimate drifts. The regression barely changes (19.9 to 20.6), because its trend is global.
- **Answer** Report error by horizon. A model for next-week staffing and a model for next-quarter planning may be different.
- **Check** 10 origins is small; with real data use many origins (weekly) to get stable estimates.
- **Why it matters** A single split can crown the wrong model by luck.

## Connection to the real job

Interviewers ask: "How do you validate a forecasting model?" Answer: rolling-origin backtest, horizon-specific errors, always against seasonal naive; no random k-fold; no future-known features that will not exist at forecast time.

## Applications

- **E-commerce:** one LightGBM global model for all SKUs.
- **Ride-hailing:** demand per zone per 15 minutes.
- **Energy:** hourly load with temperature forecasts.

- **MLOps:** a backtest is the unit test of a forecasting pipeline.

| COMMON MISTAKE | Using actual future values of a feature (like next week's real price or real weather) in a backtest when, at forecast time, you would only have a forecast of it. The backtest looks great and production does not. |
|---|---|

**Check yourself**

1. Why `shift(1)` before `rolling(7)`? Answer: so the rolling mean uses only days before the target day.
2. Why can tree models struggle with trending series? Answer: they cannot predict values beyond the range seen in training.
3. What is a global model? Answer: one model trained across many series.

## 9.4 Intervals, Special Cases and Forecast to Decision

| INTUITION | A weather report that says "30 degrees" is less useful than "between 27 and 33, 80% sure". You pack differently. A forecast of "300 orders" tells the warehouse nothing about how many extra packers to keep on call. An **interval** says how wrong it might be. |
|---|---|

### Prediction intervals

A **prediction interval** covers a future single value (wider than a confidence interval for a mean, which covers an average; see Chapter 4.4). Three practical ways:

1. **Model-based** (ETS, ARIMA give them), assuming normal errors.
2. **Empirical:** take the relative errors from backtests, and use their 10th and 90th percentiles.
3. **Conformal prediction:** a general recipe that wraps any model: compute errors on held-out windows, take a quantile, and add it to forecasts; it carries a coverage guarantee when data are exchangeable (a strong assumption for time series, so check coverage by backtest anyway).

We use method 2. Relative backtest errors `(actual − forecast) / forecast` from nine earlier origins (252 values) had 10th and 90th percentiles of −9.6% and +5.1%.

**Problem 9.4.1** Apply the 80% interval to the 91 test days and check its coverage.

| | |
|---|---|
| **Step 1** | The interval is `[forecast × (1 − 0.096), forecast × (1 + 0.051)]`. |
| **Step 2** | Count test days whose actual falls inside: 70 of 91 = 76.9%. |
| **Step 3** | Average width = 60.5 orders. |
| **Answer** | The nominal 80% interval covered 76.9% of the test days. Slightly low, because the test period included the festival (rare, big errors) and the backtest windows had mostly calm days. |
| **Check** | A normal-based interval, `forecast × (1 ± 1.2816 × 0.0607)`, covered 74.7%. Both under-cover a little. |
| **Why it matters** | Always measure coverage on held-out data. If "80%" covers 60%, the interval is lying. |

**Quantile (pinball) loss** scores a quantile forecast: for quantile q, loss = `max(q·(a − f), (q − 1)·(a − f))`. Over-forecasting costs `(1 − q)` per unit, under-forecasting costs `q`. For the 90th and 10th percentile bounds on the test set the average pinball losses were 4.08 and 4.18. Other probabilistic scores include CRPS. Many tools (LightGBM with `objective="quantile"`, `statsforecast`) can fit quantiles directly: not run here.

## Special cases (awareness table)

| Situation | What goes wrong | Typical approach |
|---|---|---|
| Hierarchies (SKU → category → total) | Item forecasts do not add up to the total | Hierarchical reconciliation (bottom-up, top-down, MinT) |
| Intermittent demand (many zeros) | MAPE useless; zeros break models | Croston or TSB; judge by WAPE or MASE, forecast quantiles |
| New products (cold start) | No history | Global model with item attributes; analogue products |
| Promotions and prices | Spikes look like anomalies | Add promotion and price as features; record plans |
| Accuracy vs bias | A forecast can be accurate and always low | Track bias separately |
| Outliers, level shifts | Models chase a one-off | Anomaly and change-point detection (`ruptures`, STL residuals) |

Bias example from our test: the seasonal naive forecast averaged 28 orders per day **below** actual (because it ignores growth), while the regression ran 15 per day **above** actual (it over-predicted the festival days). Which hurts more depends on the cost of each type of error, which is the next topic.

## Forecast to decision

A forecast alone does nothing. The decision uses its uncertainty and the costs.

**Newsvendor rule.** You order stock once, then sell. If you order too few, you lose margin per missed sale ( `c_under` ). If too many, you lose cost per leftover unit ( `c_over` ). The best quantity is the forecast quantile at the **critical ratio** `c_under / (c_under + c_over)` .

**Problem 9.4.2** Next week's demand for a fresh item is forecast at mean 300 units with a standard deviation of 8% of the mean (24 units). A missed sale loses ₹40 margin. A leftover unit costs ₹15 (spoilage). How many to order?

| | |
|---|---|
| **Formula** | Critical ratio = c_under / (c_under + c_over). Order = mean + z × sd. |
| **Plug in** | 40 / (40 + 15) = 0.7273. The normal z-value for 0.7273 is 0.6046. |
| **Compute** | Order = 300 + 0.6046 × 24 = 300 + 14.5 = 314.5, so order 315. |
| **Answer** | Order about 315, which is 15 above the mean forecast. The ₹40 loss from a missed sale is larger than the ₹15 loss from a leftover, so it pays to lean high. |
| **Check** | If the costs were equal (critical ratio 0.5, z = 0), you would order the mean, 300. If a stockout cost ₹400, the ratio 0.964 gives z = 1.80 and an order of 343. |
| **Why it matters** | The same forecast produces different orders for different costs. The decision depends on the interval, not only the average. |

**Safety stock** for a service level: `safety stock = z × σ_daily × √lead time` . With a 95% service level (z = 1.645), daily demand standard deviation 25 and lead time 4 days: 1.645 × 25 × √4 = 82.25 units, so keep about 82 units as a buffer. Staffing works the same way: staff to a high quantile, not the mean.

## Connection to the real job

This is the whole point: weekly inventory, rosters, cloud capacity. Everyday capacity planning shows why. Capacity planning is "forecast peak load plus headroom", and headroom is exactly a quantile of the forecast.

## Applications

- **Retail and food delivery:** perishables ordering.

- **Call centres:** staffing at the 90th percentile of call volume.
- **Cloud:** reserve capacity for the forecast 95th percentile.
- **Finance:** cash buffers.

| COMMON MISTAKE | Handing the business a single "most likely" number and forgetting that decisions are asymmetric. Always ask "what does it cost to be 10% too low, and 10% too high?" |
|---|---|

**Check yourself**

1. What do you check after building a prediction interval? Answer: its coverage on held-out data.
2. Costs: missed sale ₹30, leftover ₹10. Critical ratio? Answer: 30 / 40 = 0.75.
3. Why separate bias from error? Answer: a forecast can have small errors but always lean the same way, which costs money systematically.

## 9.5 Hierarchies: Making Forecasts Add Up

A retailer forecasts sales for each store, and also for the whole company. If you build each forecast on its own, the store forecasts will not add up to the company forecast. Finance sees two different totals and loses trust. **Forecast reconciliation** adjusts all the forecasts together so that parts sum to the whole, and, done well, it also improves accuracy.

| INTUITION | Three friends estimate a bill. Two guess their own shares (40 and 55) and the third guesses the total (105). The shares add to 95. Somebody is wrong, but you do not know who. Reconciliation spreads the 10 gap across all three guesses, so that the numbers agree and every guess moves a little. |
|---|---|

### The summing matrix

Write the hierarchy as $y = S \cdot b$, where $b$ is the vector of bottom-level series (stores) and $S$ is a 0/1 matrix that says how the upper levels are sums. For Total = A + B, S has rows Total (1, 1), A (1, 0) and B (0, 1). Given base forecasts $\hat{y}$ for every row, the reconciled forecasts are $\tilde{y} = S \cdot G \cdot \hat{y}$.

- **Bottom-up:** $G$ keeps only the bottom forecasts. Totals are sums of stores. Simple, but total noise grows.
- **Top-down:** $G$ splits the total by historical shares. Good totals, poor details.
- **OLS and MinT:** $G = (S^{T}WS)^{-1}\ S^{T}\ W$ with $W$ a weight matrix. OLS uses identity weights. MinT (minimum trace) uses the covariance of forecast errors and, among all unbiased reconciliations, gives the smallest total error variance.

**Problem 9.5.1** Base forecasts: Total 105, A 40, B 55. Reconcile by bottom-up and by OLS, by hand and in code.

| | |
|---|---|
| **Bottom-up** | A = 40, B = 55, Total = 95. The total forecast of 105 is thrown away. |
| **OLS, step 1** | $S^TS = [[2,1],[1,2]]$. $S^T\hat{y} = [105+40, 105+55] = [145, 160]$. |
| **OLS, step 2** | $(S^TS)^{-1} = (1/3)[[2,-1],[-1,2]]$. $\tilde{b} = (1/3)[2\cdot145 - 160, -145 + 2\cdot160] = (1/3)[130, 175] = [43.33, 58.33]$. |
| **OLS, step 3** | Total = 43.33 + 58.33 = 101.67. The 10 gap is split: the total drops 3.33 and each part rises 3.33. |

```
import numpy as np
# Part 1: tiny hand example. Total = A + B, but base forecasts disagree.
S = np.array([[1, 1], [1, 0], [0, 1]], float)          # rows: Total, A, B
yhat = np.array([105., 40., 55.])
bu = S @ yhat[1:]
G = np.linalg.inv(S.T @ S) @ S.T                        # OLS reconciliation
ols = S @ G @ yhat
print("base       ", yhat, " sum of parts =", yhat[1] + yhat[2])
print("bottom-up  ", bu)
print("OLS recon  ", np.round(ols, 3))
# Part 2: simulation. 6 stores, shared weekly pattern; base forecasts are noisy and
incoherent.
rng = np.random.default_rng(2)
m, T, H = 6, 140, 28
level = rng.uniform(40, 120, m)
t = np.arange(T + H)
dow = np.array([0.9, 0.95, 1.0, 1.0, 1.05, 1.2, 0.9])[t % 7]
Y = level[:, None] * dow * (1 + 0.002*t) + rng.normal(0, 1, (m, T + H)) * np.sqrt(level)
[:, None] * 1.5
tot = Y.sum(0)
def seas_naive(y):                                      # last observed week repeated
    return np.tile(y[T - 7:T], H // 7)
def ma_scaled(y):                                       # smooth level times weekday
shape
    return y[T - 28:T].mean() * dow[T:T + H] / dow[T - 28:T].mean()
base_b = np.array([seas_naive(Y[i]) for i in range(m)])
base_t = ma_scaled(tot)                                 # a different method for the
total, so totals disagree
Sfull = np.vstack([np.ones(m), np.eye(m)])
yh = np.vstack([base_t, base_b])                        # (1+m) x H
G = np.linalg.inv(Sfull.T @ Sfull) @ Sfull.T
rec = Sfull @ G @ yh
bu = Sfull @ base_b
actual = np.vstack([tot[T:], Y[:, T:]])
def mae(f, rows): return np.abs(f[rows] - actual[rows]).mean()
print(f"{'method':12s}{'total MAE':>10s}{'store MAE':>10s}")
for name, f in [("base", yh), ("bottom-up", bu), ("OLS recon", rec)]:
    print(f"{name:12s}{mae(f, [0]):10.2f}{mae(f, list(range(1, m + 1))):10.2f}")
print("gap in base total vs sum of stores (mean abs):", round(np.abs(yh[0] -
yh[1:].sum(0)).mean(), 2))
print("gap after reconciliation:", round(np.abs(rec[0] - rec[1:].sum(0)).max(), 6))
```

```
base        [105.  40.  55.]  sum of parts = 95.0
bottom-up   [95. 40. 55.]
OLS recon   [101.667  43.333  58.333]
method       total MAE store MAE
base             31.58     13.56
bottom-up        43.11     13.56
OLS recon        32.68     12.92
gap in base total vs sum of stores (mean abs): 16.34
gap after reconciliation: 0.0
```

- **Step 1** The hand result [101.667, 43.333, 58.333] matches the code.
- **Step 2** In the six-store simulation, base total MAE is 31.58 and store MAE 13.56. Bottom-up makes the total worse (43.11) because each store's forecast is a noisy copy of last week and the noise adds up. OLS reconciliation cuts the store MAE to 12.92 and holds the total at 32.68, and every forecast now adds up exactly.

| COMMON MISTAKE | Assuming reconciliation always improves every level. Here the total got slightly worse (31.58 to 32.68) while stores improved. You pick the method by the level the decision depends on, and you judge it on a rolling-origin backtest at that level. |
|---|---|

### Going deeper

- MinT needs a covariance matrix of forecast errors that is big for large hierarchies (thousands of SKUs). Practical versions use the diagonal ("WLS") or a shrunk covariance.
- Coherence is not accuracy. Reconciliation only helps when the base forecasts carry independent information at different levels.
- Grouped hierarchies (region by product, with two ways to roll up) and temporal hierarchies (daily, weekly, monthly) use the same `S` idea. In Python see `hierarchicalforecast`, not installed here.

## 9.6 Intermittent Demand: Croston, TSB and the Trap of MAE

Spare parts, slow SKUs and rare events have many days of zero demand and occasional spikes. Standard methods smooth these badly: they forecast a small number every day, and the usual error metric then rewards the wrong answer.

| INTUITION | Split demand into two simple questions: how often does it happen, and how big is it when it does? Croston smooths the size and the gap between demands separately and divides: forecast = size / interval. TSB smooths the chance of demand on every day, so the forecast can decay when demand dries up (Croston cannot). |
|---|---|

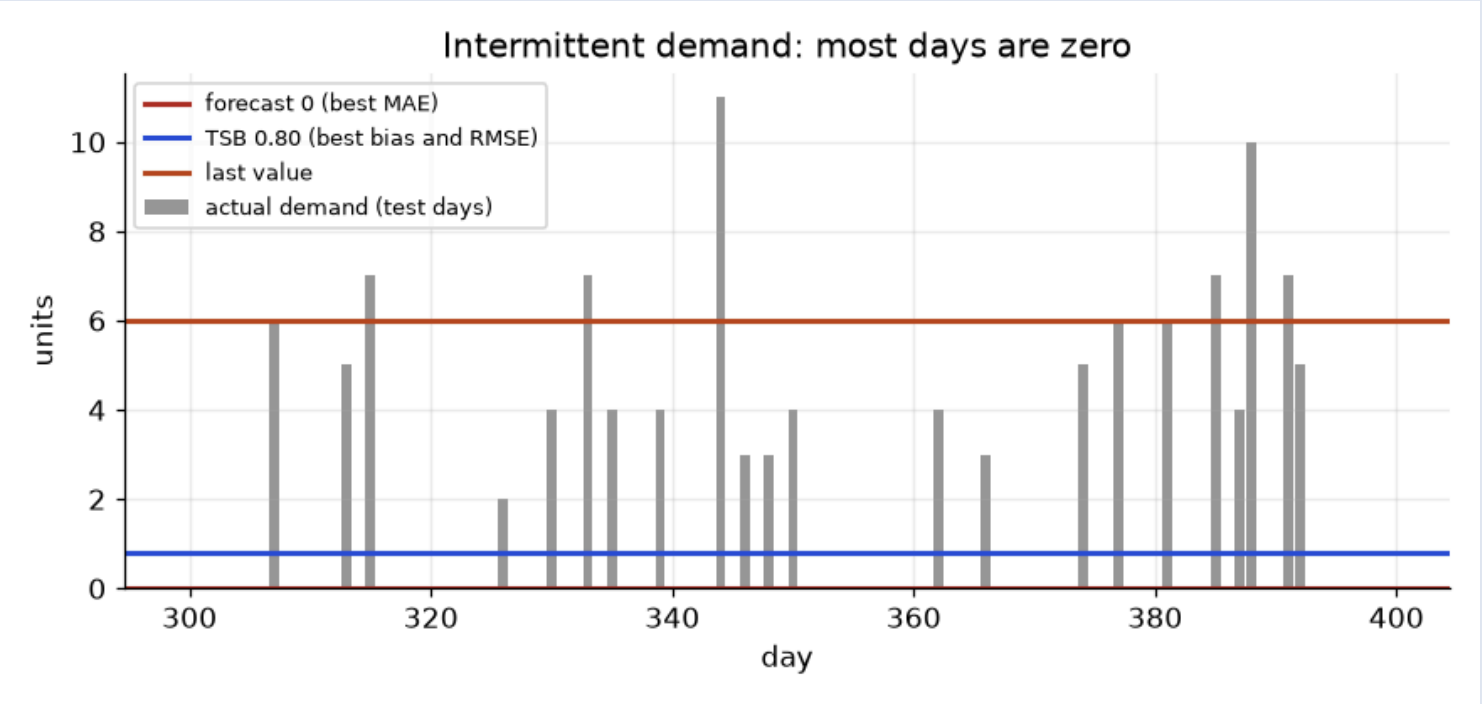
**Problem 9.6.1** 400 days. Demand happens on 20% of days and the size is 1 plus a Poisson(4) draw. Use days 1 to 300 to forecast days 301 to 400 with a flat forecast. Compare zero, last value, mean, 28-day average, Croston, Croston with the SBA bias correction, and TSB.

| | |
|---|---|
| **Croston** | On a demand day, update size `z` and interval `p` by exponential smoothing (alpha 0.1). Forecast = z / p. SBA multiplies by (1 − alpha/2) because Croston is biased upward. |
| **TSB** | Update demand probability every day. On demand days also update size. Forecast = probability × size. |
| **Score** | MAE, RMSE and bias against the 100 test days. |

```
import numpy as np
rng = np.random.default_rng(4)
n, split = 400, 300
occur = rng.random(n) < 0.2                                  # demand happens on 20% of days
size = rng.poisson(4, n) + 1                                 # size when it happens, 1 or
more
d = np.where(occur, size, 0).astype(float)
train, test = d[:split], d[split:]
print("share of zero days:", round((d == 0).mean(), 3), " mean demand/day:",
round(d.mean(), 3))
def croston(y, a=0.1, sba=False):
    z = None; p = None; q = 1
    for v in y:
        if v > 0:
            if z is None: z, p = v, q
            else: z += a*(v - z); p += a*(q - p)
            q = 1
        else: q += 1
    f = z / p
    return f * (1 - a/2) if sba else f
def tsb(y, a=0.1, b=0.1):
    prob = (y > 0).mean(); z = y[y > 0][0]
    for v in y:
        prob += b*((v > 0) - prob)
        if v > 0: z += a*(v - z)
    return prob * z
fc = {"zero (median)": 0.0, "last value": train[-1], "mean of all": train.mean(),
      "moving avg 28": train[-28:].mean(), "Croston": croston(train),
      "Croston SBA": croston(train, sba=True), "TSB": tsb(train)}
print(f"{'method':16s}{'forecast':>9s}{'MAE':>7s}{'RMSE':>7s}{'bias':>8s}")
for k, f in fc.items():
    e = f - test
    print(f"{k:16s}{f:9.3f}{np.abs(e).mean():7.3f}{np.sqrt((e**2).mean()):7.3f}
{e.mean():8.3f}")
print("actual mean in test:", round(test.mean(), 3))
```

```
share of zero days: 0.838  mean demand/day: 0.848
method           forecast    MAE   RMSE    bias
zero (median)       0.000  1.170  2.696  -1.170
last value          6.000  5.090  5.406   4.830
mean of all         0.740  1.584  2.467  -0.430
moving avg 28       0.571  1.490  2.502  -0.599
Croston             0.726  1.576  2.470  -0.444
Croston SBA         0.689  1.556  2.476  -0.481
TSB                 0.796  1.616  2.458  -0.374
actual mean in test: 1.17
```

- **Step 1** A forecast of exactly zero has the best MAE (1.170), because the median of a series that is mostly zeros is zero. A forecast that says "no demand ever" would be useless for stocking.
- **Step 2** RMSE and bias reward honest forecasts of the mean. TSB has the best RMSE (2.458) and the smallest bias (−0.374). The test days happened to average 1.17, above the training mean, so every method sits below.
- **Step 3** "Last value" is terrible (MAE 5.09) because the last day happened to be a 6.




*Test-period demand and three flat forecasts.*

**How to read it:** most bars are missing because demand is zero on most days. The red line at zero touches most of the data, so it wins on average distance. The blue line sits at the average demand, which is what a planner needs to stock correctly.

| | |
|---|---|
| **COMMON MISTAKE** | Judging an intermittent forecast by MAE or MAPE. MAE crowns the zero forecast and MAPE divides by zero. Use RMSE or bias on the mean, or better, a quantile (pinball) loss on the stocking quantile, or the cost of the resulting stock decision. |
| **KEY IDEA** | The right metric depends on what the forecast is used for. A flat mean forecast is correct for expected total demand over a lead time. A high quantile is correct for how much stock to hold. |

# 9.7 Honest Intervals: Conformal Prediction and Pinball Loss

Section 9.4 built intervals from backtest percentiles. **Split conformal prediction** formalises that idea and gives a coverage promise under minimal assumptions. Hold out a calibration set. Compute the absolute errors of the model on it. Take the $(1 - \alpha)(n + 1)/n$ quantile of these errors, call it `q`. The interval is `forecast ± q`. If calibration and future errors are exchangeable, the interval covers the truth at least 90% of the time for $\alpha = 0.1$.

**Problem 9.7.1** 500 days with a trend, weekly cycle, and noise that slowly grows. Train on days 1 to 300, calibrate on 301 to 400, test on 401 to 500. Compare the conformal interval to a normal interval built from training residuals. Then compute the pinball loss.

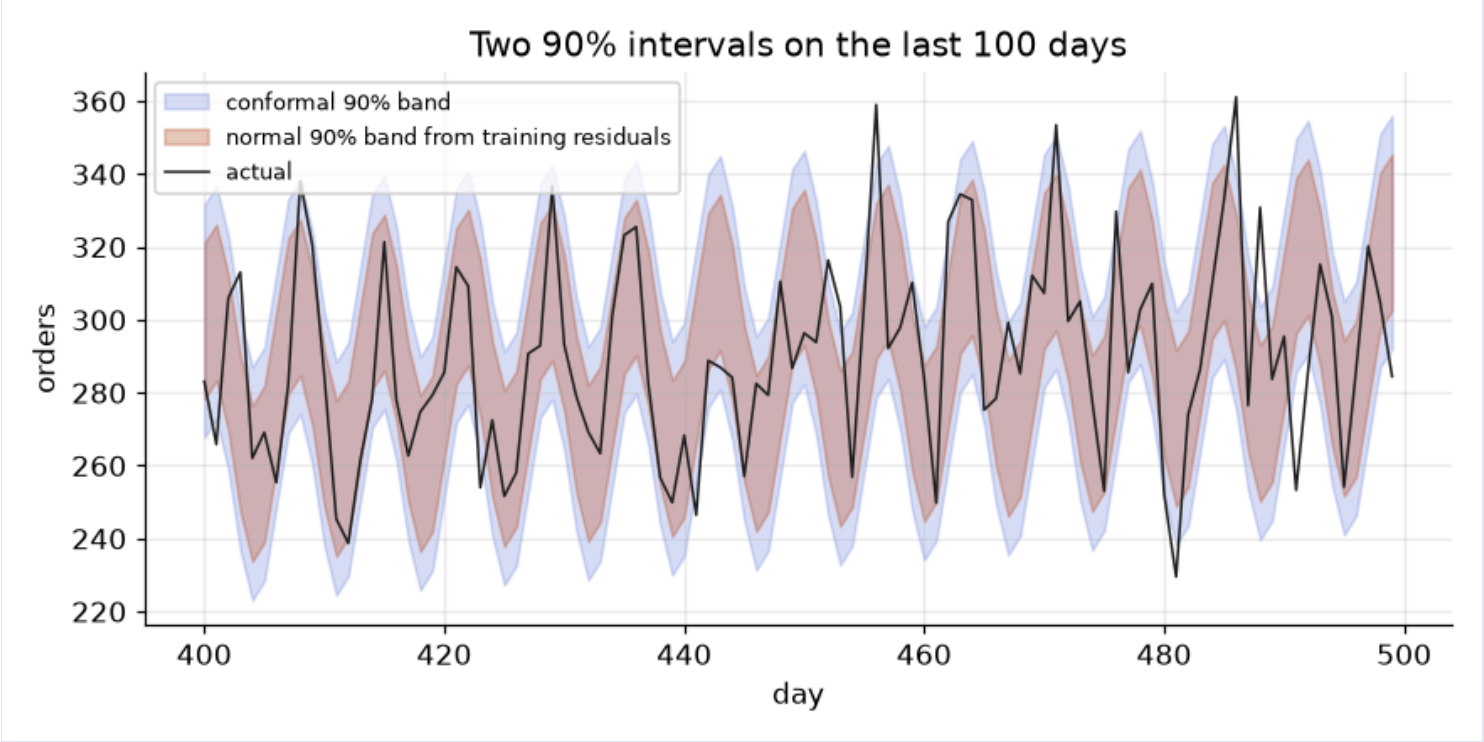
| | |
|---|---|
| **Step 1** | Fit a linear model with trend plus weekly sine and cosine. |
| **Step 2** | Calibration errors give `q` as the $\lceil 101 \times 0.9 \rceil = 91$st smallest error. |
| **Step 3** | Count test coverage for both intervals. |
| **Step 4** | Pinball loss at $q = 0.9$ for the upper bound; hand check on three numbers: errors (actual − forecast) of −2, 8, 18 cost 0.2, 7.2, 16.2, mean 7.867. |

```
import numpy as np
from sklearn.linear_model import LinearRegression
rng = np.random.default_rng(6)
n = 500
t = np.arange(n)
y = 200 + 0.2*t + 25*np.sin(2*np.pi*t/7) + rng.normal(0, 8, n) * (1 + 0.004*t)   # noise
grows a little
X = np.c_[t, np.sin(2*np.pi*t/7), np.cos(2*np.pi*t/7)]
tr, ca, te = slice(0, 300), slice(300, 400), slice(400, 500)           # train,
calibration, test (time order)
m = LinearRegression().fit(X[tr], y[tr])
res = np.abs(y[ca] - m.predict(X[ca]))
k = int(np.ceil((len(res) + 1) * 0.9))
q = np.sort(res)[k - 1]                                  # 90% conformal half-width
pred = m.predict(X[te])
inside = (y[te] >= pred - q) & (y[te] <= pred + q)
print("conformal half-width:", round(q, 2))
print("coverage on test: ", round(inside.mean(), 3), "(target 0.90)")
# naive normal interval from training residual sd
sd = (y[tr] - m.predict(X[tr])).std()
inside_n = np.abs(y[te] - pred) <= 1.645*sd
print("normal interval half-width:", round(1.645*sd, 2), " coverage:",
round(inside_n.mean(), 3))
# pinball loss for the 0.9 quantile
def pinball(a, f, qq): e = a - f; return np.mean(np.maximum(qq*e, (qq-1)*e))
up = pred + q
print("pinball q=0.9 for the upper bound:", round(pinball(y[te], up, 0.9), 3))
print("pinball q=0.9 for the point forecast:", round(pinball(y[te], pred, 0.9), 3))
# hand check with 3 numbers
a = np.array([10., 20., 30.]); f = np.array([12., 12., 12.])
print("hand check pinball q=0.9:", round(pinball(a, f, 0.9), 3))
```

```
conformal half-width: 31.96
coverage on test:  0.84 (target 0.90)
normal interval half-width: 21.37  coverage: 0.65
pinball q=0.9 for the upper bound: 4.169
pinball q=0.9 for the point forecast: 9.616
hand check pinball q=0.9: 7.867
```

- **Step 1** The normal interval is too narrow and covers only 65% instead of 90%, because training residuals are smaller than the future ones (noise grew).
- **Step 2** The conformal half-width of 31.96 lifts coverage to 84%. It is better because it was calibrated on later data, but it still misses 90% because noise keeps growing, which breaks the exchangeability promise.

- **Step 3** The pinball loss of the upper bound is 4.169 and the point forecast scores 9.616 as a 0.9 quantile, so the interval edge is a far better 90th percentile than the plain forecast.




*Conformal band (blue) and normal band (orange) on the last 100 days.*

**How to read it:** the black line is the truth. Whenever it leaves a band, that day is a miss. The narrow orange band lets the line escape often, in particular late on when noise has grown. The blue band is wider and misses less, but the drift in noise shows at the right edge.

| COMMON MISTAKE | Calibrating on random rows of a time series, or on old data, and promising 90%. Calibrate on the most recent window, refresh it regularly, and measure live coverage. When error size changes with time or level, use a scaled error (error divided by a local scale) or a rolling calibration window. |
|---|---|

## Going deeper

- **Adaptive conformal** (ACI) nudges the target level after every miss, restoring long-run coverage when the data drift.
- **Pinball loss** is the proper scoring rule for a quantile. Averaging it over several quantiles approximates CRPS, the standard score for a full predictive distribution. A model that gives only the median gets only one pinball loss.
- **Marginal versus conditional coverage.** Conformal gives 90% on average over days, not 90% on each day. Weekends can be under-covered while weekdays are over-covered. Check coverage by weekday and by size bucket.

# Chapter recap

- A series has level, trend, seasonality, events and noise. Decompose it to see them.
- Always run baselines first: naive, seasonal naive, moving average, drift. Seasonal naive halved our error (87.6 to 37.0).
- Pick metrics that suit the data: MAE and WAPE for volumes, MASE across series, avoid MAPE near zero, track bias.
- Differencing and the ACF reveal trend and seasonality; use seasonal differencing for weekly cycles.
- ETS (Holt-Winters), SARIMA and Prophet-style regressions are strong, cheap workhorses. Give models the calendar.
- ML with lag features needs `shift` before `rolling`, and shines with many series and external features, not necessarily on one short series.
- Validate with walk-forward backtests over many origins, report error by horizon. Never random k-fold.
- Give a prediction interval, and check its coverage; ours claimed 80% and covered 77%.
- Decisions need quantiles: newsvendor critical ratio and safety stock turn a forecast into an order.
- Anomaly detection, hierarchies, intermittent demand and cold start have their own tools.

| Item | Formula or pattern |
|---|---|
| Seasonal naive | `f(t+h) = y(t+h−m)` |
| SES | `level = α·y + (1−α)·level` |
| WAPE | `Σ |
| MASE | `MAE / MAE(naive on train)` |
| Difference | `y(t) − y(t−1)`, seasonal `y(t) − y(t−m)` |
| Lag features | `shift(k)`, `shift(1).rolling(w).mean()` |
| Backtest | Train to origin, forecast h, roll origin forward |
| Empirical interval | Percentiles of backtest relative errors |
| Newsvendor | Order = quantile at `c_under / (c_under + c_over)` |
| Safety stock | `z · σ_daily · √lead time` |

## Practice set

**1. (easy)** Seasonal naive on daily data with m = 7: forecast for next Tuesday if last Tuesday was 120? **Answer:** 120.

**2. (easy)** SES with α = 0.3, previous level 50, new observation 60. New level? **Answer:** 0.3 × 60 + 0.7 × 50 = 18 + 35 = 53.

**3. (easy)** Actuals 50 and 150, forecasts 60 and 120. Compute the WAPE. **Answer:** (10 + 30) / 200 = 20%.

**4. (medium)** MASE: model MAE is 12, and the naive forecast's MAE on the training data is 15. MASE and meaning? **Answer:** 12 / 15 = 0.8. Beats the naive by 20%.

**5. (medium, what is the output)** A model is evaluated with `KFold(shuffle=True)` on daily data and shows MAE 20. With a time split it shows 29. Which do you trust? **Answer:** The time split; random folds leak neighbouring days.

**6. (medium, find the bug)** `df["r7"] = df.y.rolling(7).mean()` is used to predict `y` on the same row. What is wrong? **Answer:** The window includes today's `y`, so the target leaks into the feature. Use `df.y.shift(1).rolling(7).mean()`.

**7. (medium)** Costs: stockout ₹60, leftover ₹20, demand ~ Normal(mean 100, sd 10). Order quantity? (z for 0.75 is 0.674.) **Answer:** Ratio 60/80 = 0.75, z = 0.674, order = 100 + 6.7 = 106.7, so about 107.

**8. (hard)** Your 90% interval covers only 65% of backtest days. List two likely causes and fixes. **Answer:** Intervals too narrow because errors are not normal or the variance grows with level; use empirical or relative-error quantiles, or conformal calibration. Also possible: the backtest windows contain events (promotions) not in the calibration data; add those features or widen.

**9. (hard, scenario)** A new item has 3 weeks of sales. Which approach? **Answer:** A global model trained on similar items using item attributes (category, price), or an analogue product's profile scaled to the first weeks; update quickly as data arrive; give wide intervals.

**10. (hard)** A model has MAPE 8% but a bias of +12% (always over-forecasts). Is it fine? **Answer:** No. Errors of 8% that always lean 12% high mean systematic over-stocking. Investigate (maybe it learns promotion periods wrongly); fix or de-bias, and measure bias separately.

**11. (interview)** "Your LightGBM forecast loses to seasonal naive on the backtest. What do you do?" **Answer:** Report it honestly. Check leakage in features, whether trend or level shifts break the tree model, try differencing the target or predicting the ratio to last week's value, add calendar and promotion features, then compare again. If still behind, ship the baseline.

**12. (interview)** "How do you turn a demand forecast into an inventory decision?" **Answer:** Get a probabilistic forecast, estimate the costs of stockout and overstock, set order quantity at the critical-ratio quantile, add safety

stock for lead time uncertainty, and monitor realised service level and bias.

**13. (easy)** Store A forecast 70, store B forecast 30, total forecast 110. Bottom-up total? **Answer:** 70 + 30 = 100. The base total of 110 is discarded.

**14. (medium)** Pinball loss at q = 0.8 for actual 50 and forecast 40, then forecast 60. **Answer:** 40 is below actual, error +10, loss 0.8 × 10 = 8. 60 is above actual, error −10, loss (0.8 − 1)(−10) = 2. Under-forecasting is four times as costly, matching a 0.8 quantile.

**15. (medium, find the bug)** A team reports a MAPE of 12% on a spare-parts series where 85% of days are zero. What is wrong? **Answer:** MAPE divides by the actual, which is zero on most days, so it is undefined or has been computed only on non-zero days, which hides the stock-out problem. Use RMSE, bias or a stocking-cost metric.

**16. (hard)** With 49 calibration errors, what rank gives the 80% split-conformal half-width? **Answer:** $\lceil (49 + 1) \times 0.8 \rceil = \lceil 40 \rceil = 40$, so the 40th smallest absolute error.

**17. (interview)** "Your 95% intervals cover only 80% in production. What do you check?" **Answer:** Whether calibration data are recent and the same distribution as production (drift, new promotions, a new product mix), whether errors grow with the level (use scaled errors), whether coverage differs by segment or weekday, and whether the model changed after calibration. Recalibrate on a recent window and monitor live coverage.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC moving into data science, studying one hour a day. Topic: forecasting baselines (naive, seasonal naive, moving average, drift) and error metrics (MAE, RMSE, MAPE, WAPE, MASE, bias). Explain each with a daily orders example and small numbers, show when MAPE fails, and then give me 8 graded exercises without solutions. Wait for my answers, then grade them and explain every mistake in simple English.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data science. Topic: exponential smoothing (SES, Holt, Holt-Winters), differencing and the ACF for daily data with weekly seasonality. Explain the idea with an everyday analogy, walk through one SES update by hand, show how to read an ACF, then quiz me one question at a time. Use simple English and check my understanding before moving on.
```

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science. Topic: building lag and rolling features for ML forecasting and validating with walk-forward backtesting. Here is my feature code: [PASTE HERE]. Review it for leakage (shift before rolling, future-known features), explain every problem in plain English, show the corrected code, and design a rolling-origin backtest for it with horizon-specific errors.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science. Topic: prediction intervals, quantile loss and turning a forecast into an inventory or staffing decision (newsvendor rule, safety stock). Teach me with a delivery-app example using real rupee numbers, explain the critical ratio, and give me 6 numeric problems of rising difficulty. Do not give solutions until I answer, and then check every number.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science and preparing for interviews. Topic: time series forecasting for business decisions. Run a mock interview, one question at a time, such as "how do you validate a forecasting model", "your ML model loses to seasonal naive, what now", or "how do you forecast for a new product". Score each answer 1 to 5, give a model answer, and ask a follow-up. Do 8 rounds, then list my weak spots.

CHAPTER 10

# RECOMMENDATION, RANKING AND SEARCH

*How to decide what to show each user: baselines, similarity, matrix factorization, the two-stage design used in industry, the right metrics, cold start and search relevance.*

Open any shopping, music or video app. Hundreds of thousands of items exist, the screen shows ten. Someone, or rather some system, chose those ten for you, and that choice drives a large part of the company's revenue. Building such a system is a mix of data science, engineering and product thinking. The hard parts are not the algorithms. They are the data (you only see what was shown), the evaluation (offline numbers often disagree with online results) and the scale (you cannot score 10 million items for every request). By the end of this chapter you will be able to build a working recommender from scratch, compare it with honest baselines using ranking metrics (Precision@K, Recall@K, NDCG, MRR), explain the two-stage "candidate then rank" architecture with numbers, handle new items (cold start), spot position bias, and score documents for a search query with BM25. The algorithms behind gradient boosting and neural nets live in the ML document (`study/ml/`); here we only use them.

**Running example.** A shopping app with 2,000 users and 600 items. We generated interactions ("user bought or clicked item") from hidden taste factors plus an item popularity effect, with a fixed seed (`default_rng(10)`). The result is 37,347 interactions: 18.7 per user, a density of 3.1%. Each interaction has a timestamp in [0, 1). We **train on the first 80% of time** (28,638 interactions) and **test on the last 20%** (8,709). The last 30 items launch only at t = 0.8, so they are brand new in the test period. All outputs below are real. Libraries `implicit`, `LightFM`, `Surprise`, `faiss` and `lightgbm` were not installed, so we wrote ALS and similarity in `numpy`, and used scikit-learn's `HistGradientBoostingClassifier` as the ranker.

## 10.1 Problem Types, Feedback Data and the Baselines

**INTUITION**

A good shop assistant does three things. They know what is popular. They know what people like you bought. And they notice what you looked at today. A recommender automates those same three ideas, but it only ever sees what customers did, not what they would have liked.

## Problem types

| Type | Question | Example |
|---|---|---|
| Top-N recommendation | Which N items for this user? | "Recommended for you" row |
| Rating prediction | What star rating would the user give? | Old Netflix prize style |
| Next-item / sequence | What comes next after these clicks? | "Up next" video |
| Similar items | Items like this one | "Customers also viewed" |
| Search ranking | Order results for a query | Product search |
| Feed ranking | Order a mixed feed | Social feed |

Today most work is top-N and ranking, not rating prediction, because users rarely rate.

## Feedback data

**Explicit feedback** means the user says what they like (a 5-star rating). It is clear but rare. **Implicit feedback** is behaviour: click, view, purchase, watch time. It is plentiful but noisy: a click is not a like, and "no interaction" does not mean dislike (the user may never have seen the item).

Three biases to know now:

- **Exposure bias:** users can only interact with what the system showed. Items never shown look "disliked".
- **Popularity bias:** popular items get more data and more exposure, so they get recommended even more.
- **Position bias:** items shown at the top get more clicks just because they are at the top (Section 10.4).

In our data, the top 10% of items (60 items) hold 33.7% of training interactions.

## The baselines

**Problem 10.1.1** (tiny numbers) Three users bought: Asha {shoes, socks}, Ravi {shoes, bag}, Meena {shoes, socks, hat}. Recommend one item to a new user who just bought shoes, with (a) popularity and (b) item-item co-occurrence.

| | |
|---|---|
| **Step 1 (popularity)** | Counts: shoes 3, socks 2, bag 1, hat 1. Excluding items the user already has, the top is socks (2). |
| **Step 2 (co-occurrence with shoes)** | Customers who bought shoes also bought: socks 2 times (Asha, Meena), bag 1, hat 1. The top is socks. |
| **Step 3 (cosine similarity)** | The normalised version divides by the geometric mean of the two popularity counts. `sim(shoes, socks) = 2 / √(3 × 2) = 0.816`, `sim(shoes, bag) = 1 / √(3 × 1) = 0.577`. |
| **Answer** | Both recommend socks here; cosine does not just reward the most popular item (shoes), it also adjusts for popularity. |
| **Check** | Cosine is between 0 and 1; 1 would mean the two items are always bought together. |
| **Why it matters** | "Customers who bought this also bought" is item-item co-occurrence. It is simple, fast and strong. |

Cosine similarity of two item columns `a`, `b` of the 0/1 user-item matrix: `sim(a, b) = (a · b) / (‖a‖ ‖b‖)`. For a user, the score of an unseen item is the sum of its similarities to the items the user already has: `score = X · Sim`, where `X` is the user-item matrix.

## Content-based and kNN

- **Content-based:** describe items by features (text TF-IDF, category, embeddings) and recommend items similar to those the user liked. It needs no other users, so it works for new items.
- **User-user kNN:** find the 50 users most similar to you and recommend what they bought.
- **Item-item kNN:** as above; usually the more stable.

## Metrics for ranking

For one user with a recommended list of K items and a set of items they really interacted with later ("relevant" items):

| Metric | Meaning |
|---|---|
| Precision@K | share of the K shown that are relevant |
| Recall@K | share of all relevant items that appear in the K |
| Hit rate@K | 1 if at least one relevant item is in the K, else 0 |
| MRR | `1 / rank` of the first relevant item |
| NDCG@K | a score that gives more credit to relevant items near the top |
| Coverage | share of the catalogue that is ever recommended |
| Diversity, novelty, serendipity | how different, unfamiliar and pleasantly surprising the list is |

**NDCG** (normalised discounted cumulative gain): each relevant item at position `r` earns `1 / log₂(r + 1)`. Add them (DCG), then divide by the best possible DCG (IDCG) so the score is between 0 and 1.

**Problem 10.1.2** A list of 5 items. Relevant flags by position: `[0, 1, 0, 1, 0]`. The user has 2 relevant items in total. Compute NDCG@5, Precision@5 and MRR.

| | |
|---|---|
| **Formula** | DCG = Σ rel / $\log_2$(position + 1) |
| **Plug in** | Position 2: 1 / $\log_2(3)$ = 0.6309. Position 4: 1 / $\log_2(5)$ = 0.4307. |
| **Compute** | DCG = 0.6309 + 0.4307 = 1.0616. The ideal list has both relevant items at positions 1 and 2: IDCG = 1 + 0.6309 = 1.6309. NDCG = 1.0616 / 1.6309 = 0.651. |
| **Answer** | NDCG@5 = 0.651, Precision@5 = 2/5 = 0.40, MRR = 1/2 = 0.5. |
| **Check** | If the relevant items were at positions 1 and 2, NDCG would be exactly 1. |
| **Why it matters** | Precision does not care where the hits are. NDCG does, and users mostly look at the top. |

## The first contest

We score every item for every user, hide the items already in training, take the top 10 per user, and compare with test-period interactions.

```
model                  P@10    R@10    NDCG@10   MRR     Hit@10   Coverage
random                 0.0074  0.0157  0.0107    0.0189  0.0716   1.000
popularity             0.0365  0.0873  0.0651    0.1124  0.3195   0.028
recent popularity      0.0350  0.0824  0.0633    0.1112  0.3029   0.028
item-item cosine       0.0904  0.1834  0.1674    0.2997  0.5467   0.455
user-user kNN (50)     0.0830  0.1693  0.1496    0.2661  0.5187   0.498
SVD rank 5             0.0701  0.1502  0.1326    0.2429  0.4886   0.177
SVD rank 20            0.0749  0.1483  0.1342    0.2494  0.4876   0.342
ALS implicit, f = 16   0.0751  0.1505  0.1323    0.2379  0.4767   0.627
```

- **Step 1** Random: NDCG 0.011. This is the floor.
- **Step 2** Popularity: 0.065, six times better, but it recommends the same 17 items to everyone (coverage 2.8% means 17 of 600 items ever appear).
- **Step 3** Personalised methods double or triple it (0.13 to 0.17).
- **Answer** The best model here, by every metric, is **item-item cosine**, a one-line similarity. Matrix factorization (next section) did not beat it on this small, sparse dataset, though it covers more of the catalogue.
- **Check** Recall@10 is 0.18 for the winner: of a user's future items, about 18% appear in the first 10 slots.
- **Why it matters** In real projects, item-item or popularity baselines are often hard to beat, and they cost almost nothing. Beat them first before trusting anything fancier.

## Connection to the real job

The first deliverable of a recommender project is a table like this, with baselines in the top rows.

## Applications

- **E-commerce:** "frequently bought together".
- **Streaming:** "because you watched".
- **Fintech:** suggesting a product (insurance, mutual fund) from past usage.
- **Food delivery:** reorder suggestions.

| COMMON MISTAKE | Treating "no interaction" as "dislike". The user may never have seen the item. This is why implicit-feedback methods weight observed interactions more heavily, and why sampling negatives needs care. |
|---|---|

**Check yourself**

1. What is the difference between Precision@K and NDCG@K? Answer: NDCG rewards hits near the top; precision ignores the positions.

2. Why can cosine beat raw co-occurrence? Answer: it divides by item popularity, so popular items are not always on top.
3. Is a click an explicit or implicit signal? Answer: implicit.

## 10.2 Matrix Factorization

**INTUITION**

Imagine each customer has a hidden taste profile (loves sport, hates formal wear) and each item has a hidden style profile. If the two profiles point the same way, the customer will probably like the item. We never see profiles, but we can learn them from who bought what. **Matrix factorization (MF)** learns a short list of numbers (a **latent factor** vector) for each user and each item, so that the dot product of a user's vector and an item's vector predicts the interaction.

### How it works

The user-item matrix `X` has 2,000 rows and 600 columns, almost all zeros. MF finds a user matrix `P` (2,000 × f) and an item matrix `Q` (600 × f) with `f` small (say 16) so that $X \approx P \cdot Q^T$. The predicted score of item `i` for user `u` is `P[u] · Q[i]`.

Methods:

- **SVD** (singular value decomposition): the exact best low-rank approximation of the matrix. It treats every empty cell as a real zero.
- **ALS** (alternating least squares) for implicit feedback: gives each observed interaction a higher **confidence** $c = 1 + \alpha \cdot x$ and solves for `P` with `Q` fixed, then `Q` with `P` fixed, again and again. Each solve is a small linear system: $p_u = (Q^T C_u Q + \lambda I)^{-1} Q^T C_u x_u$. $\lambda$ (regularisation) keeps the vectors small.
- **BPR** (Bayesian personalised ranking): learns by pairs: "this user prefers the item they bought over a random item they did not". It optimises ranking directly. Not run here.
- **Factorization machines**: extend the dot product to include side information such as category and device. Not run here.

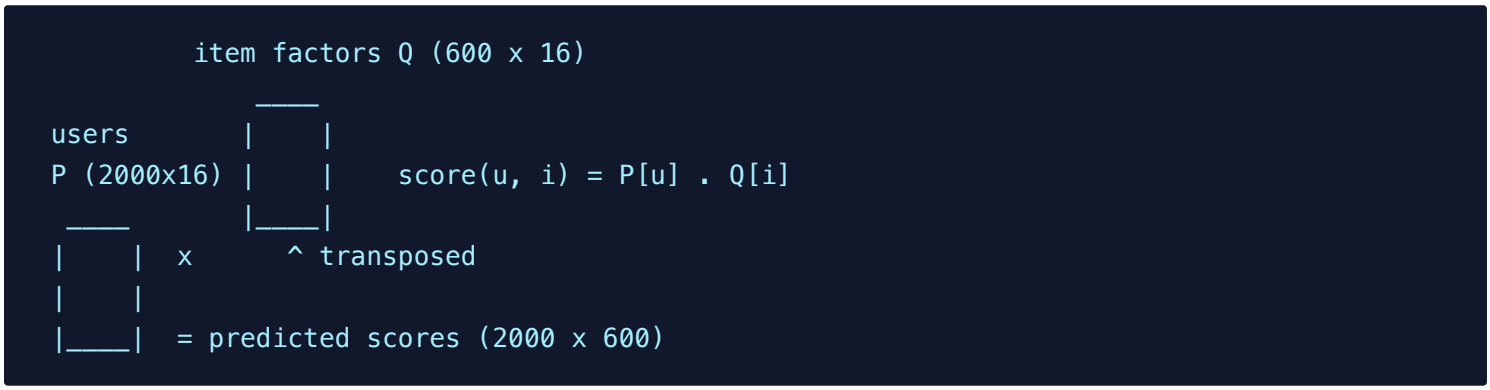
**Problem 10.2.1** (tiny numbers, by hand) User vector `[1.0, 0.5]`. Item A vector `[0.8, 0.2]`, item B vector `[−0.3, 0.9]`. Which item scores higher?

| | |
|---|---|
| **Formula** | score = Σ (user factor × item factor) |
| **Plug in** | A: 1.0 × 0.8 + 0.5 × 0.2. B: 1.0 × (−0.3) + 0.5 × 0.9. |
| **Compute** | A = 0.8 + 0.1 = 0.90. B = −0.3 + 0.45 = 0.15. |
| **Answer** | Item A (0.90) beats item B (0.15) for this user. |
| **Check** | The user's first factor is large and A's first factor is large, so the dot product is large. That is what "pointing the same way" means. |
| **Why it matters** | Scoring every item for one user is a matrix-vector product; at serving time a vector index finds the top items without scoring all of them (Section 10.3). |

**Problem 10.2.2** Run ALS and SVD, and see what choices matter (results from the contest table).

| | |
|---|---|
| **Step 1** | SVD with rank 5: NDCG 0.1326. Rank 20: 0.1342. A bigger rank barely helped. |
| **Step 2** | ALS (16 factors, λ = 10, α = 20, 12 rounds): NDCG 0.1323, but coverage 62.7% against 17.7% for SVD rank 5. ALS recommends a much wider range of items. |
| **Step 3** | Item-item cosine: 0.1674, better than all MF variants. |
| **Answer** | On our small data, MF is a good model, not the best. We did no tuning of the factor count, regularisation or α. With 10 to 100 times more interactions and tuning, MF usually wins; you must test. |
| **Check** | ALS recall@10 = 0.1505 and SVD rank-5 = 0.1502. The two models agree on the broad structure. |
| **Why it matters** | Do not assume the sophisticated model wins. Try it, tune it, and compare to the simple baseline. |

## The picture




Read it as: row `u` of `P` times column `i` of `Qᵀ` gives one cell of the score matrix.

## Connection to the real job

MF is the classic "model-based collaborative filtering". Its learned item vectors (embeddings) are also used for "similar items" lists and as input features to a ranking model.

## Applications

- **Music and video:** user and item embeddings for "similar to".
- **Retail:** next-basket ideas.
- **Ads:** click-through prediction with factorization machines.
- **Banking:** product propensity.

| | |
|---|---|
| **COMMON MISTAKE** | Choosing a factor count or regularisation on the test set. Tune on a validation period earlier than the test period, then evaluate once on the test. |

### Check yourself

1. What is a latent factor? Answer: a learned hidden number describing a taste or style.
2. What does ALS alternate between? Answer: solving for the user vectors with items fixed, then item vectors with users fixed.
3. Why `λ`? Answer: to prevent overfitting by keeping the vectors small.

# 10.3 The Two-Stage Architecture, Learning to Rank and Evaluation Traps

**INTUITION**

A recruiter with 50,000 CVs does not read each one in depth. First a quick filter shortlists 100 who look plausible. Then a careful interview ranks those 100. Recommenders work the same way: a cheap stage must not miss the good items, and an expensive stage orders them well.

## The stages

```
all items (millions)
       |
       v
1. CANDIDATE GENERATION   fast, recall-focused, vector search (ANN)   -> ~100-1000 items
       |
       v
2. RANKING                precise, many features, ML model           -> ordered list
       |
       v
3. RE-RANKING             diversity, business rules, de-duplication   -> final ~10 items
```

- **Candidate generation** can use several sources at once (popular items, item-item neighbours, MF top-k, recently viewed). Vector retrieval uses **approximate nearest neighbour (ANN)** search (libraries like `faiss`): not run here, because we only have 600 items, so we score all of them by brute force. The key metric of this stage is **recall@N**: "is the item the user wants inside the shortlist?"
- **Ranking** uses a model that sees features of the user, the item and the pair (history, price, time of day, the MF score).
- **Re-ranking** adds rules: do not show five near-identical items, boost new sellers, remove items out of stock.

**Learning to rank** trains the ranker on lists. **Pointwise** predicts one score per item (like our classifier). **Pairwise** learns "A should rank above B" (RankNet). **Listwise** optimises a list metric directly (LambdaMART in LightGBM's ranker is the industry classic: not run here). **Two-tower neural networks** learn a user vector and an item vector separately and take their dot product, which allows precomputed item vectors and ANN retrieval. **Sequence models** (GRU4Rec, SASRec/transformers) use the order of recent clicks.

**Problem 10.3.1** How large must the candidate list be?

```
ALS recall@10   0.150
ALS recall@50   0.365
ALS recall@100  0.490
ALS recall@200  0.629
```

- **Step 1** The ranker can only choose among candidates. If the shortlist has 100 items, it can reach at most recall 0.490 (49% of future items).
- **Step 2** At 200 candidates the ceiling is 0.629, but the ranker has to score twice as many.
- **Answer** The candidate stage sets the ceiling. Choose N from the recall curve and your latency budget (here, 100 is a reasonable compromise).
- **Check** Recall must rise with N; it does (0.15, 0.37, 0.49, 0.63).
- **Why it matters** Engineers often tune only the ranker. If the candidate stage misses the item, no ranker can recover it.

**Problem 10.3.2** Add a ranker on top of the ALS top-100.

| | |
|---|---|
| **Step 1 (training data)** | To avoid leakage, we fit a fresh ALS on only the first 60% of time, took its top 100 for each user (200,000 rows), and labelled a candidate 1 if the user interacted with it between 60% and 80% of the time (positive rate 2.2%). |
| **Step 2 (features)** | ALS score, ALS rank, item popularity, user activity, and their interaction. A gradient boosting classifier predicts the click probability. |
| **Step 3 (test)** | Re-score the top 100 of the full ALS using the model, and evaluate on the test period. |

```
                        P@10    R@10    NDCG@10  MRR     Hit@10
ALS only (one stage)    0.0751  0.1505  0.1323   0.2379  0.4767
ALS top-100 + GBM ranker  0.0798  0.1663  0.1393   0.2381  0.5119
```

- **Answer** The ranker improved recall@10 from 0.150 to 0.166 and hit rate from 47.7% to 51.2%, a modest gain (NDCG +5%). It still did not beat item-item cosine (0.1674), because our ranker has only five simple features. Real rankers have hundreds.
- **Check** The ranker was trained on an earlier window than the test window, so no leakage.
- **Why it matters** Two-stage architecture is what you describe in a system-design interview; the improvement comes mostly from adding rich features in stage two.

### Evaluation traps

**Random vs time split.** Random splits let the model see future interactions. With item-item cosine, switching from a time split to a random 80/20 split on the same data inflated Recall@10 from 0.183 to 0.224 (+22%) and NDCG@10 from 0.167 to 0.190. A model must be tested on the future, like in forecasting (Chapter 9).

**Offline versus online.** Offline metrics measure how well you predict the past behaviour that the OLD system caused. The new system can differ. Use offline metrics to filter ideas, then **A/B test** (Chapter 7). Other tools: **interleaving** (mix the results of two rankers in one list and see which gets the clicks; needs far fewer users than an A/B test) and **counterfactual evaluation** with **inverse propensity scoring** (IPS, Chapter 8).

### Connection to the real job

"Design a recommendation system" is a standard interview. Draw the three stages, name the metric of each stage, state the baseline, the data split and the online test.

### Applications

- **E-commerce:** home page and cart recommendations.
- **Video/music:** multi-source candidates then a neural ranker.
- **Job portals:** candidate-to-job matching.
- **Ads:** candidate ads then click-through-rate ranking.

| COMMON MISTAKE | Leaking the future in the ranker's training: using item popularity computed over ALL time, including the test period. Compute every feature as of the prediction time. |
|---|---|

**Check yourself**

1. What is the metric of the candidate stage? Answer: recall@N.
2. Why can the ranker not fix a bad candidate list? Answer: it only reorders the candidates it receives.
3. What is interleaving? Answer: showing a mixed list from two rankers and counting which one gets more clicks.

## 10.4 Cold Start, Position Bias and Search

| INTUITION | A new employee has no history and nobody knows their strengths. You can read the CV (content), ask the manager for a reference (similar people), or give them a trial project (explore). New users and new items need the same three tricks. |
|---|---|

## Cold start

**Cold start** means a new user or new item with no interactions. Item-item and MF models assign it no score, so it is never shown, and so it never gets data. A vicious circle.

Remedies:

- **Content features:** learn a mapping from item attributes to the factor vector.
- **Popularity or recent popularity** as a default for new users.
- **Onboarding questions:** ask three taste questions at signup.
- **Exploration or bandits** (Chapter 7): show new items to a small share of users to collect data.

**Problem 10.4.1** In our data, 20% of test interactions are on the 30 brand-new items (about 0.86 of 4.35 test interactions per user). For those items the history models have nothing. We learn a ridge regression that maps each item's (noisy) content features to its ALS vector, trained on the 570 warm items, and predict vectors for the 30 cold items. We evaluate only on the cold items (ranking the 30).

```
cold items only         Precision@10   Recall@10
random order            0.054          0.351
content-mapped factors  0.119          0.747
```

- **Step 1** A random order among 30 items puts 10 on the list and finds 35% of relevant items.
- **Step 2** Content-mapped factors find 75% and double the precision.
- **Answer** Mapping from item attributes to the model's space gives new items sensible scores on day one, before any interaction exists.
- **Check** Random recall of 0.351 is close to 10/30 = 0.333, as expected by chance.
- **Why it matters** Cold start is where the content model earns its keep. In production you blend both: content-based for new items, collaborative for items with history.

## Position bias and counterfactuals

Users look at the top more. A click at position 1 is partly "it was at the top". Our simulation: item A has a true click rate of 10% when examined. The chance of being examined is 1.0 at position 1 and 0.4 at position 2. Item A is placed at either position with equal chance (200,000 impressions).

- Observed click rate at position 1: 9.8%. At position 2: 3.9%.
- The simple overall average: 6.9%, which understates the item's true appeal of 10%.
- **IPS correction:** weight each click by `1 / examination probability`. Corrected rate: 9.8%, near the true 10%.

Position bias is a feedback loop: whatever you put on top collects clicks, and then the model learns to put it on top. Solutions: randomise a small share of positions, include position as a training feature (and set it to a fixed value at serving), or use IPS weighting. Other ethical points: filter bubbles (users see only more of the same), fairness to sellers and creators, and manipulation by fake clicks.

## Search: BM25 and relevance

**Search ranking** has a query instead of a user. The classic first stage is **BM25**, a keyword score. For a query word `w` in document `d`:

```
idf(w) = ln(1 + (N − df + 0.5) / (df + 0.5))
term score = idf × tf × (k1 + 1) / (tf + k1 × (1 − b + b × len(d) / avglen))
BM25(q, d) = sum of term scores over query words
```

`N` is the number of documents, `df` the number of documents containing the word, `tf` the count of the word in this document. Rare words count more (`idf`), repeated words help but with diminishing returns (`k1`), and long documents are discounted (`b`). Typical values are `k1 = 1.5`, `b = 0.75`.

**Problem 10.4.2** Four product titles: (1) "cheap red running shoes", (2) "blue running shoes for men", (3) "red leather office bag", (4) "running shorts red". Query: "red running". Score document 4.

| | |
|---|---|
| **Step 1** | `N = 4`. "red" appears in 3 documents, "running" in 3. `idf = ln(1 + (4 − 3 + 0.5) / (3 + 0.5)) = ln(1.4286) = 0.3567`. |
| **Step 2** | The titles have 4, 5, 4 and 3 words, so the average length is (4 + 5 + 4 + 3) / 4 = 4.0 (our code printed the same). |
| **Step 3** | Document 4 has 3 words. Each term: tf = 1. `1 − b + b × 3/4 = 0.25 + 0.5625 = 0.8125`. Denominator: `1 + 1.5 × 0.8125 = 2.21875`. Term score = 0.3567 × 2.5 / 2.21875 = 0.4019. |
| **Step 4** | Two query words: 2 × 0.4019 = 0.8038. |
| **Answer** | Real output: doc 4 = 0.804, doc 1 = 0.713, doc 3 = 0.357, doc 2 = 0.321. The shortest title with both words ranks first. |
| **Check** | Document 3 contains only "red" (0.357), doc 2 only "running" (0.321, longer and so discounted more). |
| **Why it matters** | BM25 is the strong, cheap baseline that modern search still uses as a first stage. |

A modern search stack: **query understanding** (spelling, synonyms, intent) → **retrieval** (BM25 plus **vector search** on text embeddings, combined as **hybrid search**) → **learning-to-rank** model using click logs and **relevance judgments** (human labels such as "perfect / good / bad") → evaluation with NDCG. Embedding models and vector databases are covered in the GenAI part of the plan.

### Connection to the real job

Search teams spend as much time on judgments and query logs as on models. Measuring search quality means fixed query sets with graded labels, NDCG, and online tests on click and conversion.

### Applications

- **E-commerce search:** BM25 plus embedding retrieval, ranker with price and popularity.
- **Job and property portals:** query plus filters plus personalisation.
- **Customer support:** retrieval of the best help article.
- **Enterprise:** document search with hybrid retrieval.

| COMMON MISTAKE | Judging a new ranker only by offline NDCG against labels collected from the old ranker's top results. Items the old system never showed have no labels, so the new system is punished for being different. Add fresh judgments, interleaving or an A/B test. |
|---|---|

**Check yourself**

1. Name two ways to handle a new item. Answer: content-based features, and exploration (showing it to a small share of users).
2. What does IPS do? Answer: weights observations by 1 / probability of being seen, to undo position (exposure) bias.
3. What does a high `idf` mean? Answer: the word is rare, so it is informative.

## Chapter recap

- Recommenders choose a few items from many. Most work today is top-N ranking from implicit feedback.
- Implicit feedback has exposure, popularity and position bias. "No interaction" is not "dislike".
- Start with baselines: popularity and item-item cosine. In our test item-item (NDCG 0.167) beat popularity (0.065) and MF (0.132).
- Matrix factorization learns user and item vectors; the score is their dot product. Tune it and compare it to the baselines.

- Two-stage systems: candidate generation (recall, ANN), ranking (precision, features), re-ranking (rules). Recall@N of the first stage caps everything after.
- Use ranking metrics (NDCG, MRR, Recall@K), plus coverage and diversity. Evaluate on a time split; random splits inflated recall by 22%.
- Offline metrics filter ideas; A/B tests decide. Interleaving and IPS help with cheaper or counterfactual evaluation.
- Cold start: content features, popularity, onboarding questions and exploration. Mapping content to factors doubled precision on new items.
- Position bias creates feedback loops; correct with IPS or randomisation.
- Search starts with BM25 (rare words matter, long documents are discounted), then adds vector retrieval and learning to rank.

| Item | Formula or pattern |
|---|---|
| Cosine similarity | `a·b / (‖a‖‖b‖)` |
| Item-item score | `X · Sim` |
| Precision@K, Recall@K | hits/K, hits/total relevant |
| NDCG | `Σ rel/log₂(pos+1)` divided by the ideal |
| MRR | `1 / rank of first hit` |
| MF score | `P[u] · Q[i]` |
| ALS confidence | `c = 1 + α·x` |
| IPS | weight `1 / P(examined)` |
| BM25 | `idf × tf(k1+1) / (tf + k1(1−b+b·len/avglen))` |

# Practice set

**1. (easy)** Recommended list of 10, 3 relevant. Precision@10? **Answer:** 3/10 = 0.30.

**2. (easy)** First relevant item at position 4. Reciprocal rank? **Answer:** 1/4 = 0.25.

**3. (easy)** User vector `[2, 1]`, item vector `[0.5, 3]`. Score? **Answer:** 2 × 0.5 + 1 × 3 = 4.

**4. (medium)** User has 4 relevant future items. The top-10 list contains 2 of them. Recall@10 and Precision@10? **Answer:** Recall 2/4 = 0.50; Precision 2/10 = 0.20.

**5. (medium)** Cosine between item A (bought by users {1, 2, 3, 4}) and item B (bought by {3, 4, 5}). **Answer:** Overlap = 2. `2 / √(4 × 3) = 2 / 3.464 = 0.577`.

**6. (medium, what is the output)** Relevance flags `[1, 0, 0]` with 1 relevant item in total. NDCG@3? **Answer:** DCG = 1, IDCG = 1, so NDCG = 1.0.

**7. (medium, find the bug)** A team evaluates with a random 80/20 split of interactions, and the model gets Recall@10 of 0.45. After launch, the online hit rate is far lower. Why? **Answer:** The random split leaks the future (and includes items that had no history online), inflating the offline score; use a time-based split and an A/B test.

**8. (hard)** The candidate generator has recall@100 of 0.40 and the ranker has a perfect ordering. What is the maximum possible recall@10? **Answer:** At most 0.40, and it could be lower if there are more than 10 relevant candidates; the ranker cannot recover items the first stage dropped.

**9. (hard)** Position 3 has examination probability 0.2 and an item got 30 clicks from 1,000 impressions at that position. IPS-corrected click rate? **Answer:** Raw rate 3.0%. Divide by 0.2: 15%.

**10. (hard, scenario)** A new streaming app launches with 5,000 users and 300 titles, with 3 interactions per user. What approach? **Answer:** Start with popularity (recent) and content-based similarity, ask taste questions at signup, use item-item cosine once data accumulate, add exploration for new titles, and move to MF or two-stage later when there are enough interactions.

**11. (interview)** "Design a recommender for an e-commerce home page." **Answer:** Define goal and metrics (conversion, revenue, guardrails), data (clicks, purchases, with time), baselines (popularity, item-item), candidate generation from multiple sources with recall@N, a ranker with user-item-context features, a re-ranker for diversity and stock, time-split offline evaluation, cold-start plan, then an A/B test with long-term metrics.

**12. (interview)** "Offline NDCG improved 10% but the A/B test is flat. Why?" **Answer:** Offline data come from the old system's exposure, so improvements may be on items the old system already showed; position and popularity bias; the metric (NDCG on clicks) may not match the business metric (revenue, retention); effect may be too small for the test's power; or novelty effects. Check by interleaving, counterfactual evaluation, and metric alignment.

## Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional at an Indian MNC
moving into data science, studying one hour a day. Topic: recommendation baselines
(popularity, item-item cosine) and ranking metrics (Precision@K, Recall@K, MRR, NDCG,
coverage). Explain each with a tiny shopping example I can calculate by hand, then give me
```

8 graded exercises without solutions. After I answer, check every number and explain my mistakes in simple English.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science. Topic: matrix factorization for implicit feedback (SVD, ALS, BPR). Explain latent factors with an everyday analogy, show a two-factor example with hand-computed dot products, explain what confidence weighting and regularisation do, and then ask me five questions one at a time. Give feedback after each answer and keep the English simple.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science. Topic: the two-stage recommender architecture (candidate generation, ranking, re-ranking) and offline evaluation traps. Here is my system design: [PASTE HERE]. Review it: is recall@N measured, is the split time-based, is there feature leakage, is there position bias, what is the baseline, and how will an A/B test confirm the gain? Give fixes with small numeric examples.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science. Topic: cold start, position bias with inverse propensity scoring, and BM25 search ranking. Teach each using a shopping app, with a hand-computed example for IPS and for one BM25 score. Then give me six mixed problems, wait for my answers, and grade them honestly.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data science and preparing for interviews. Topic: recommendation, ranking and search systems. Run a mock interview with me, one question at a time, such as "design a recommender for an e-commerce home page", "offline NDCG improved but the A/B test is flat, why", or "how do you handle new items". Score each answer 1 to 5, show a model answer, and ask a follow-up. Do 8 rounds, then summarise my weak spots.

CHAPTER 11

# APPLIED DOMAINS: CHURN, CREDIT RISK, FRAUD, MARKETING, DEMAND, HEALTHCARE AND OPERATIONS

*The same model skills, six different businesses: what each domain asks, what its special metrics mean, and how to start in each.*

Two candidates with equal skill are not equal in an interview. The one who says "for credit risk we check KS, Gini and PSI, and the label arrives 12 months late" sounds like a colleague. The one who says "I would train XGBoost" sounds like a student. This chapter gives you the vocabulary, the special metrics and one runnable example for each major domain: customer churn and lifetime value, credit scorecards, fraud, marketing mix and pricing, demand and supply chain, healthcare, and operations analytics (queues, delivery times, capacity). By the end you will be able to build a small scorecard by hand, set a fraud threshold from costs, fit an adstock curve, and predict when a warehouse hits its capacity limit. Pick one or two domains to go deep. Skim the rest.

Code uses the shop dataset from Chapter 1 ( `from ds_common import *` , seed 42) for churn, and small synthetic generators (fixed seeds) for the other domains. All outputs below were produced by running the code with numpy, pandas, scikit-learn and scipy. Where I name a library we did not run (BG/NBD, PyMC-Marketing, OR-Tools), I say "not run here".

## 11.1 Churn and Customer Lifetime Value

**INTUITION**

A gym knows exactly when you leave, because you cancel. A supermarket never knows: you just stop coming. Churn work starts with that one question: *can I see the exit?* If yes it is "contractual" churn. If no, you must invent a rule for "gone" (for example, no order for 90 days), and that rule is a business decision, not a maths result.

**Words you need.**

| Word | Plain meaning | Example in the shop data |
|---|---|---|
| Contractual churn | Customer must cancel; you see it | A pro plan cancelled |
| Non-contractual churn | Customer just goes silent | No order for 90 days |

| Word | Plain meaning | Example in the shop data |
|---|---|---|
| Censored | Still active when data ends, so end time unknown | Customer still ordering on 30 Apr 2025 |
| Survival curve S(t) | Share still active after t days | S(180) = 0.47 |
| CLV | Customer lifetime value: total future profit from one customer | 12-month profit per customer |
| RFM | Recency, Frequency, Monetary: simple behaviour scores | Days since last order, order count |

**Survival analysis** answers "how long until churn?" instead of "will churn happen in 60 days?". Its key tool is the **Kaplan-Meier estimator**. At each day t where someone churned, multiply the running survival by `(1 − d/n)`, where `d` is churners that day and `n` is customers still at risk. Censored customers leave the "at risk" count but never count as churners. This uses customers who are still active, which a plain "churned or not" table wastes.

**Problem 11.1.1** Five customers have tenure (days) 2, 3, 3, 5, 8. Events: churned, churned, censored, churned, censored. Compute the Kaplan-Meier survival.

| | |
|---|---|
| **Formula** | S(t) = S(previous) × (1 − d/n) |
| **Step 1** | Day 2: n = 5 at risk, d = 1 churn. S = 1 × (1 − 1/5) = 0.80. |
| **Step 2** | Day 3: one churner and one censored. Everyone with tenure ≥ 3 is at risk: n = 4. d = 1. S = 0.80 × (1 − 1/4) = 0.60. |
| **Step 3** | Day 5: at risk are tenure 5 and 8, so n = 2, d = 1. S = 0.60 × (1 − 1/2) = 0.30. |
| **Step 4** | Day 8 is censored, no churn, S stays 0.30. |
| **Answer** | S = 0.80, 0.60, 0.30, 0.30 at days 2, 3, 5, 8. |
| **Check** | The code below prints the same four numbers. |

**Problem 11.1.2** On the shop data define churn as "last order more than 90 days before 30 April 2025". Compute S(90), S(180), S(365) by plan.

```
from ds_common import *
customers, orders = make_data()
last = orders.groupby("customer_id").order_ts.max()
c = customers.set_index("customer_id").join(last.rename("last")).dropna(subset=["last"])
c["event"] = ((END_DATE - c["last"]).dt.days > 90).astype(int)
c["tenure"] = np.where(c.event == 1, (c["last"] - c.signup_date).dt.days,
                       (END_DATE - c.signup_date).dt.days)

def km(t, e, points):
    surv, s = {}, 1.0
    for u in np.sort(np.unique(t[e == 1])):
        at = (t >= u).sum(); d = ((t == u) & (e == 1)).sum()
        s *= 1 - d / at; surv[u] = s
    surv = pd.Series(surv)
    return {p: round(float(surv[surv.index <= p].iloc[-1]), 3) for p in points}

print(km(np.array([2,3,3,5,8]), np.array([1,1,0,1,0]), [2,3,5,8]))
for pl in ["free", "pro"]:
    m = c[c.plan == pl]
    print(pl, len(m), km(m.tenure.values, m.event.values, [90, 180, 365]))
```

```
{2: 0.8, 3: 0.6, 5: 0.3, 8: 0.3}
free 3808 {90: 0.662, 180: 0.467, 365: 0.262}
pro 1688 {90: 0.852, 180: 0.734, 365: 0.57}
```

- **Answer** After one year only 26.2% of free customers are still active, against 57.0% of pro customers.
- **Check** Pro should survive longer because we built a lower churn hazard into pro (Chapter 1). It does.
- **Why it matters** "Pro customers are worth more" is now a number you can put in a retention budget.

**CLV, the practical way.** The fancy models (BG/NBD for how often a customer buys while alive, Gamma-Gamma for how much they spend, not run here) are useful, but start with a cohort table: for customers who signed up long enough ago to be fully observed, add up revenue by month since signup and multiply by margin. Restrict to customers with a full window, otherwise recent customers drag the average down (a censoring error).

```
o = orders.merge(customers[["customer_id", "signup_date", "plan"]], on="customer_id")
o["month"] = (o.order_ts - o.signup_date).dt.days // 30
old = customers[customers.signup_date < "2024-04-01"]       # 12 full months observed
for pl in ["free", "pro"]:
    ids = old[old.plan == pl].customer_id
    x = o[o.customer_id.isin(ids) & (o.month < 12)]
    rev = x.groupby("month").amount.sum() / len(ids)
    print(pl, len(ids), rev.round(0).astype(int).tolist(), round(rev.sum()),
round(rev.sum() * 0.20))
```

```
free 2676 [1085, 969, 866, 793, 670, 617, 549, 518, 417, 392, 386, 347] 7610 1522
pro 1109 [2495, 2388, 2301, 2305, 2111, 2123, 2004, 1906, 1873, 1772, 1718, 1707] 24702
4940
```

The last two numbers on each line are 12-month revenue and 12-month profit (revenue × 0.20). Notice the monthly revenue per customer falls from 1,085 to 347 for free customers: that fall is churn at work.

- **Answer** First-year revenue per customer is about 7,610 rupees (free) and 24,702 rupees (pro). With a 20% margin, 12-month CLV is 1,522 and 4,940 rupees.
- **Check** 24,702 × 0.20 = 4,940.4 and 7,610 × 0.20 = 1,522.
- **Why it matters** CLV caps what you may spend to win a customer. Paying 2,000 rupees to acquire a free customer loses money in year one.

**RFM segmentation** gives action groups without any model. We cut recency and frequency into thirds among active customers at T and check the churn label of Chapter 1.

```
d = make_label_table(customers, orders)
past = orders[orders.order_ts < PRED_DATE]
g = past.groupby("customer_id").agg(last=("order_ts", "max"), f=("order_id", "count"))
d = d.merge(g, left_on="customer_id", right_index=True)
d["rec"] = (PRED_DATE - d["last"]).dt.days
d["R"] = pd.qcut(d.rec, 3, labels=[3, 2, 1]).astype(int)          # 3 = most recent
d["F"] = pd.qcut(d.f.rank(method="first"), 3, labels=[1, 2, 3]).astype(int)
d["seg"] = np.select([(d.R == 3) & (d.F == 3), (d.R == 1) & (d.F == 1)],
                     ["champions", "slipping"], "middle")
print(d.groupby("seg").churn.agg(n="size", churn_rate="mean").round(3))
```

```
             n  churn_rate
seg
champions   447       0.027
middle     1719       0.235
slipping    396       0.593
```

The "slipping" group churns 22 times more often than "champions" (0.593 / 0.027 = 22). That is a free, explainable model.

**The picture.** Survival curve: x axis is days since signup, y axis is the share still active, starting at 1.0 and stepping down at each churn day. The pro curve sits above the free curve. The gap at day 365 (0.57 vs 0.26) is the area where retention offers pay.

**Churn is not the end: uplift.** Predicting who will leave does not tell you who will *respond to an offer*. Some "likely churners" leave no matter what; some would have stayed anyway. Uplift modelling (Chapter 8) estimates the *change* in churn caused by the offer. Target by uplift times value, not by churn score alone.

**Connection to the job.** Telecom, SaaS, banks and e-commerce all run churn teams. Next-best-action systems combine churn score, CLV and offer cost (Chapter 12).

**Applications**

- Data Analytics: retention cohorts and RFM dashboards.
- Data Science: survival models, uplift targeting.
- ML Engineering: nightly scoring of "slipping" customers to the CRM.

| COMMON MISTAKE | Computing average lifetime as `1 / churn rate` or averaging tenure of churned customers only. Active customers are censored, and dropping them makes lifetimes look shorter than they are. Use survival curves. |
|---|---|

**Check yourself**

1. In Kaplan-Meier, does a censored customer count as a churner? Answer: No. They leave the at-risk count but are never counted as a churn event.
2. Why restrict the CLV cohort to old signups? Answer: Recent signups have not had 12 months, so their revenue is cut short and CLV looks falsely low.
3. A churn model ranks the top 10% by risk. Why might an offer to them still waste money? Answer: Risk is not responsiveness; many will leave anyway. You need uplift.

## 11.2 Credit Risk and Scorecards

| INTUITION | A bank clerk used to ask: "Do you have a salary slip? Any past defaults?" and add points in her head. A scorecard is that clerk written down: each answer earns points, the total is the score, and a regulator can read every line. That is why banks still love logistic regression over black boxes. |
|---|---|

**Words you need.**

| Word | Meaning |
|---|---|
| PD | Probability of default: chance the borrower fails to pay in a fixed window |
| LGD | Loss given default: share of the loan not recovered |
| EAD | Exposure at default: amount owed when default happens |
| Expected loss | `PD × LGD × EAD` |
| WoE | Weight of evidence of a bin: `ln(share of goods / share of bads)` |
| IV | Information value: how well a variable separates good from bad; `Σ (pg − pb) × WoE` |
| KS | Largest gap between cumulative bad rate and cumulative good rate across the score |
| Gini | `2 × AUC − 1` |
| PSI | Population stability index: how much a distribution moved |
| Good/bad | Bad = defaulted; good = paid |

**Problem 11.2.1** A bank has 1,000 good and 100 bad customers. In the bin "utilisation above 60%" there are 100 goods and 20 bads. Find WoE and the IV contribution.

| | |
|---|---|
| **Formula** | pg = goods in bin / all goods; pb = bads in bin / all bads; WoE = ln(pg / pb); IV part = (pg − pb) × WoE |
| **Plug in** | pg = 100 / 1,000 = 0.10; pb = 20 / 100 = 0.20 |
| **Compute** | WoE = ln(0.10 / 0.20) = ln(0.5) = −0.693. IV part = (0.10 − 0.20) × (−0.693) = 0.0693 |
| **Answer** | WoE is −0.693 (negative: this bin has more bads than its share of goods). It adds 0.069 to the IV. |
| **Check** | A bin with pg = pb has WoE = ln(1) = 0 and contributes nothing. Correct, since it separates nothing. |
| **Why it matters** | WoE puts every bin on one log-odds scale, so any variable (even a category) becomes a clean numeric input. |

**Rule of thumb for total IV:** below 0.02 useless, 0.02 to 0.1 weak, 0.1 to 0.3 medium, above 0.3 strong, above 0.5 suspicious (check leakage). These are conventions, not laws.

**Build a scorecard.** We simulate 20,000 applicants with utilisation, debt-to-income (dti), age and past delinquencies, and a 5.45% default rate. We bin on the first 14,000 (train), compute WoE and IV, fit logistic regression on WoE, and test on the last 6,000.

```
import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
rng = np.random.default_rng(7); n = 20000
util = np.clip(rng.beta(2, 3, n), 0, 1); dti = np.clip(rng.normal(0.35, 0.12, n), 0.02,
0.9)
age = np.clip(rng.normal(36, 9, n), 21, 65).round(); delinq = rng.poisson(0.4, n)
z = -3.2 + 2.2*(util-0.4) + 3.0*(dti-0.35) - 0.02*(age-36) + 0.45*delinq
y = (rng.random(n) < 1/(1+np.exp(-z))).astype(int)
df = pd.DataFrame(dict(util=util, dti=dti, age=age, delinq=delinq, y=y))
tr, te = df.iloc[:14000], df.iloc[14000:]

def woe_table(x, y, bins):
    t = pd.DataFrame({"b": pd.cut(x, bins, include_lowest=True), "y": y}).groupby("b",
observed=True).y.agg(["size", "sum"])
    t.columns = ["n", "bad"]; t["good"] = t.n - t.bad
    t["pg"] = t.good / t.good.sum(); t["pb"] = t.bad / t.bad.sum()
    t["woe"] = np.log(t.pg / t.pb); t["iv"] = (t.pg - t.pb) * t.woe
    return t
specs = {"util": [0, .2, .4, .6, 1.0], "dti": [0, .25, .35, .45, 1.0],
         "age": [20, 30, 40, 50, 70], "delinq": [-1, 0, 1, 2, 20]}
tabs = {k: woe_table(tr[k], tr.y, b) for k, b in specs.items()}
print(tabs["util"].round(3))
print({k: round(float(t.iv.sum()), 3) for k, t in tabs.items()})
```

```
                n  bad  good     pg     pb    woe     iv
b
(-0.001, 0.2]  2527   70  2457  0.186  0.087  0.765  0.076
(0.2, 0.4]     4800  223  4577  0.347  0.276  0.229  0.016
(0.4, 0.6]     4159  274  3885  0.294  0.339 -0.141  0.006
(0.6, 1.0]     2514  241  2273  0.172  0.298 -0.549  0.069
{'util': 0.168, 'dti': 0.115, 'age': 0.03, 'delinq': 0.117}
```

Read the table: low utilisation (below 20%) has WoE +0.765, so these customers are safer. High utilisation has WoE −0.549. WoE falls steadily as utilisation rises, which is a *monotonic* pattern, exactly what regulators like to see. Utilisation (IV 0.168) and delinquency and dti (about 0.115) are medium; age (0.03) is weak.

Next, fit the logistic model on WoE values and measure ranking power.

```
def apply_woe(d):
    X = pd.DataFrame(index=d.index)
    for k, b in specs.items():
        X[k] = pd.cut(d[k], b,
include_lowest=True).map(tabs[k].woe.to_dict()).astype(float)
    return X
Xtr, Xte = apply_woe(tr), apply_woe(te)
lr = LogisticRegression(C=1e6, max_iter=1000).fit(Xtr, tr.y)
print(dict(zip(Xtr.columns, lr.coef_[0].round(3))), lr.intercept_.round(3))

def ks(y, s):
    d = pd.DataFrame({"y": y, "s": s}).sort_values("s", ascending=False)
    return ((d.y.cumsum()/d.y.sum()) - ((1-d.y).cumsum()/(1-d.y).sum())).abs().max()
for nm, X, d in [("train", Xtr, tr), ("test", Xte, te)]:
    pr = lr.predict_proba(X)[:, 1]; a = roc_auc_score(d.y, pr)
    print(nm, "AUC", round(a, 4), "Gini", round(2*a-1, 4), "KS", round(ks(d.y.values, pr),
4))
```

```
{'util': -1.022, 'dti': -1.01, 'age': -1.05, 'delinq': -1.045} [-2.793]
train AUC 0.6785 Gini 0.3571 KS 0.2591
test AUC 0.6853 Gini 0.3706 KS 0.2949
```

All four coefficients are close to −1. That is a good sign: WoE is already "log-odds of good", and we model "bad", so the sign is negative and the size is near one when WoE is a faithful summary. Coefficients far from −1, or with a positive sign, signal correlated variables or bad bins.

**Problem 11.2.2** Compute the Gini and read the result. Test AUC is 0.6853.

| | |
|---|---|
| **Formula** | Gini = 2 × AUC − 1 |
| **Compute** | 2 × 0.6853 − 1 = 0.3706 |
| **Answer** | Gini is 0.37 and KS is 0.29 on the test set. |
| **Check** | Train and test are close (0.357 vs 0.371), so the model is not overfitted. A coin flip has AUC 0.5 and Gini 0. |
| **Why it matters** | A very high Gini on loan data (say 0.9) would make a risk head suspect leakage before celebrating. (Our data is synthetic with only four weak inputs; do not compare it to a real bank, and real ranges depend on the product and the data.) |

**Points per bin.** A scorecard converts log-odds to points. Choose a base score (600 points at odds 50 good per bad) and "points to double the odds" (PDO = 20). Then `factor = PDO / ln 2 = 28.854` and `offset = 600`

`− factor × ln 50 = 487.123`. Each bin's points are `−(WoE × coefficient + intercept/4) × factor + offset/4`, rounded.

```
util bin          points
(-0.001, 0.2]      164
(0.2, 0.4]         149
(0.4, 0.6]         138
(0.6, 1.0]         126
```

An applicant with utilisation 0.55, dti 0.30, age 45, one delinquency scores **566** (sum of four bin points, from the run). Any underwriter can recompute that by hand, which is the point.

**PSI, the monitoring metric.** `PSI = Σ (actual% − expected%) × ln(actual% / expected%)` over score bins. Conventions: below 0.1 stable, 0.1 to 0.25 watch, above 0.25 significant shift.

**Problem 11.2.3** Expected shares in two bins are 50% and 50%. This month they are 40% and 60%. Find PSI.

| | |
|---|---|
| **Plug in** | (0.40 − 0.50) × ln(0.40/0.50) + (0.60 − 0.50) × ln(0.60/0.50) |
| **Compute** | (−0.10)(−0.2231) + (0.10)(0.1823) = 0.02231 + 0.01823 = 0.0405 |
| **Answer** | PSI = 0.04, stable. |

On the scorecard: PSI of test scores against train scores (same population) was **0.0006**. We then simulated a recession: utilisation up by 0.15 and dti up by 0.05. PSI became **0.4559**, far above 0.25. The score distribution moved, so we investigate before trusting decisions.

**Other credit ideas.** Expected loss `= PD × LGD × EAD`. For example PD 3%, LGD 45%, EAD 200,000 rupees gives `0.03 × 0.45 × 200000 = 2,700` rupees. **Reject inference**: you only see repayment for loans you approved, so training only on approved customers is a selection bias (Chapter 4, and Chapter 2 on selective labels). Methods such as parcelling and augmentation guess outcomes for rejects; they need strong assumptions and are not run here. **Explainability and regulation**: Indian lenders follow RBI guidance on model risk and fair practice codes; the exact circulars change, so read the current ones. In practice you need reason codes (the points that pulled a score down) for each rejected applicant.

**Connection to the job.** BFSI is the largest hiring domain for data scientists in India. Scorecards, collections models and early-warning systems all use these metrics.

### Applications

- Banks: application scorecards, behaviour scorecards, limit assignment.
- Fintech and UPI lenders: thin-file scoring with alternative data (with consent).
- Insurance: frequency times severity models for premiums (awareness).

| COMMON MISTAKE | Quoting AUC, KS and Gini on the *training* data, or choosing bins by looking at test data. Bin and fit on train, report on out-of-time test, and monitor PSI after launch. |
|---|---|

**Check yourself**

1. WoE of a bin is 0. What does it say? Answer: Goods and bads are in the bin in the same proportion as overall; it separates nothing.
2. AUC 0.80. Gini? Answer: 2 × 0.80 − 1 = 0.60.
3. PSI = 0.46 on the score. First action? Answer: Investigate which inputs shifted (PSI per variable), then decide whether to recalibrate or rebuild.

## 11.3 Fraud Detection

| INTUITION | Fraud is the needle in the haystack with a fire alarm attached. Fraud is rare, every alert costs an analyst's time, and fraudsters change tactics when you catch them. So accuracy is useless, and the real question becomes "with 500 reviews a day, how many frauds can I catch?" |
|---|---|

**What makes fraud special.**

| Feature | Consequence |
|---|---|
| Extreme imbalance (about 0.4% fraud here) | Accuracy is meaningless; use precision, recall, PR-AUC |
| Fixed review capacity | Judge *precision at K* (best K cases) |
| Unequal costs | Choose the threshold from money, not 0.5 |
| Adversaries adapt | Concept drift; retrain, monitor, add rules |
| Real-time scoring | Latency budget (tens of milliseconds); simple features at serving time |
| Network structure | Shared devices or addresses link fraud rings (graph features, awareness) |

We simulate 200,000 transactions with 0.416% fraud (831 cases), train gradient boosting on the first 140,000, and test on the last 60,000 (253 frauds).

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score, average_precision_score
rng = np.random.default_rng(11); n = 200000
amt = rng.lognormal(6, 1, n); night = (rng.random(n) < 0.2).astype(int)
newdev = (rng.random(n) < 0.1).astype(int); vel = rng.poisson(1, n)
z = -12.8 + 0.8*np.log(amt) + 1.5*night + 2.5*newdev + 0.7*vel
y = (rng.random(n) < 1/(1+np.exp(-z))).astype(int)
X = np.c_[np.log(amt), night, newdev, vel]
m = HistGradientBoostingClassifier(random_state=0).fit(X[:140000], y[:140000])
p = m.predict_proba(X[140000:])[:, 1]; yt = y[140000:]
print("test frauds", yt.sum(), "AUC", round(roc_auc_score(yt, p), 3),
      "AP", round(average_precision_score(yt, p), 3), "base AP", yt.mean().round(5))
print("accuracy of always-no", round(1 - yt.mean(), 5))
top = yt[np.argsort(-p)[:500]]
print("caught in top 500:", top.sum(), "precision@500", round(top.mean(), 3), "recall",
round(top.sum()/yt.sum(), 3))
```

```
fraud rate 0.00416 831
test frauds 253 AUC 0.865 AP 0.061 base AP 0.00422
accuracy of always-no 0.99578
review cap 500 caught 45 precision@cap 0.09 recall 0.178
```

Read it: "always say no fraud" is 99.58% accurate and catches nothing. AUC 0.865 looks good, but the average precision (area under the precision-recall curve) is only 0.061, about 14 times the random baseline of 0.0042, yet still low. With 500 reviews we catch 45 of 253 frauds (recall 17.8%), precision 9%.

**Problem 11.3.1** A missed fraud loses 5,000 rupees on average. Reviewing an alert costs 100 rupees. At what fraud probability is it worth reviewing?

| | |
|---|---|
| **Formula** | Compare two costs for one transaction with fraud probability p. If we ignore it we expect to lose `p × c_miss`. If we review it we pay `c_review` on the innocent ones, an expected `(1 − p) × c_review`. Setting them equal gives `p* = c_review / (c_review + c_miss)` (we assume a reviewed fraud is caught for sure). |
| **Plug in** | p* = 100 / (100 + 5,000) |
| **Compute** | 100 / 5,100 = 0.0196 |
| **Answer** | Review any transaction with a fraud probability above about 2%, not above 50%. |
| **Check** | At p = 0.0196, expected cost of ignoring is 0.0196 × 5,000 = 98; expected cost of reviewing is 0.9804 × 100 = 98. Equal, so that is the break-even. |

The same logic on the test set (cost of FP = 100, cost of FN = 5,000):

```
threshold  flagged  TP    FP    FN   total cost
0.5            26      4    22   249   1,247,200
0.1           334     43   291   210   1,079,100
0.0196       1803     97  1706   156     950,600
0.005        7738    176  7562    77   1,141,200
```

- **Answer** The cost-based threshold (0.0196) has the lowest cost, 950,600 rupees. Doing nothing costs 253 × 5,000 = 1,265,000. Saving = 314,400, or 24.9%.
- **Check** 950,600 = 1,706 × 100 + 156 × 5,000 = 170,600 + 780,000. Correct.
- **Why it matters** This is a model that "only" has precision 5% (97 of 1,803) yet saves a quarter of fraud cost. Judge by money, with capacity as a constraint. If you can only review 500, take the top 500.

(These probabilities assume reasonable calibration. Boosted models can be miscalibrated; check calibration before using cost-based thresholds, see 11.5.)

**Connection to the job.** UPI and card fraud teams run rules plus models in under 100 ms. Features like "transactions from this device in the last 10 minutes" must be computed identically in training and serving (Chapter 13).

### Applications

- Payments: card, UPI, wallet fraud.
- E-commerce: fake returns, promo abuse, fake reviews.
- Insurance: claim fraud (graph features).
- AML (awareness): unusual transaction patterns sent to human investigators.

| COMMON MISTAKE | Random splits on fraud data. Fraudsters reuse patterns, so a random split leaks. Split by time, and also hold out whole merchants or devices when you can. |
|---|---|

### Check yourself

1. Why is PR-AUC better than ROC-AUC for 0.4% fraud? Answer: ROC-AUC counts the huge number of true negatives and looks good; PR-AUC focuses on the rare positives.
2. Review cost 50, missed-fraud loss 2,500. Break-even probability? Answer: 50 / 2,550 = 0.0196.
3. Fraud pattern changes next month. What do you watch? Answer: Score PSI, recall on confirmed frauds, and the share of alerts that are false.

# 11.4 Marketing Mix and Pricing

| INTUITION | You spent money on TV, search ads and discounts. Sales went up. Which one worked? A marketing mix model (MMM) is a regression that adds two realistic twists: an ad keeps working for days after it airs (carry-over), and each extra rupee works a bit less than the one before (saturation). |
|---|---|

**Words you need.**

| Word | Meaning | Example |
|---|---|---|
| Attribution | Giving credit for a sale to touchpoints | Last-touch: all credit to the last click |
| Incrementality | Sales *caused* by the campaign, beyond what would happen anyway | Geo experiment: ads in some cities only |
| Adstock | Carry-over: `adstock_t = spend_t + a × adstock_{t−1}` | a = 0.5 halves the effect each period |
| Saturation | Diminishing returns: `x / (x + k)` | k = half-saturation point |
| Price elasticity | % change in demand for 1% change in price | −1.8 means a 1% price rise loses 1.8% of units |

Attribution rules (last-touch, multi-touch, data-driven) are fast but only *correlational*; they love channels that sit at the end of the journey (brand search). Incrementality tests (holdout groups, geo experiments, see Chapters 7 and 8) are the truth check. MMM uses aggregate weekly data, needs no user tracking, and fits well under privacy rules.

**Problem 11.4.1** Spend is 100 in week 1 and 0 afterwards. With a = 0.5, find adstock for weeks 1 to 3. Then find the saturation `x/(x+150)` for x = 50, 150, 450.

| | |
|---|---|
| **Formula** | adstock_t = spend_t + 0.5 × adstock_{t−1}; saturation = x / (x + k) |
| **Compute** | Week 1: 100. Week 2: 0 + 0.5 × 100 = 50. Week 3: 0 + 0.5 × 50 = 25. Saturation: 50/200 = 0.25; 150/300 = 0.50; 450/600 = 0.75. |
| **Answer** | Adstock is 100, 50, 25. Saturation is 0.25, 0.50, 0.75: tripling x from 150 to 450 raises response only from 0.50 to 0.75. |
| **Check** | The code prints `[100.0, 50.0, 25.0]` and `[0.25, 0.5, 0.75]`. |
| **Why it matters** | Saturation is why you cannot just "double the budget"; the second rupee buys less than the first. |

**Fit an MMM.** We simulate 104 weeks with true adstock a = 0.6, half-saturation k = 150 and effect b = 200 on top of a base of 500 plus yearly seasonality, plus noise. Then we fit by least squares.

```
from scipy.optimize import least_squares
rng = np.random.default_rng(5); T = 104
spend = np.abs(rng.normal(100, 40, T)) * (1 + 0.5*(np.arange(T) % 13 == 0))
def adstock(x, a):
    out = np.zeros_like(x); c = 0
    for i, v in enumerate(x):
        c = v + a*c; out[i] = c
    return out
sat = lambda x, k: x / (x + k)
t = np.arange(T)
sales = 500 + 10*np.sin(2*np.pi*t/52) + 200*sat(adstock(spend, 0.6), 150) + rng.normal(0,
5, T)
def resid(p):
    a, k, b, base, s1, s2 = p
    return (base + s1*np.sin(2*np.pi*t/52) + s2*np.cos(2*np.pi*t/52)
            + b*sat(adstock(spend, a), k) - sales)
f = least_squares(resid, [0.3, 100, 100, 400, 0, 0],
                  bounds=([0, 10, 0, 0, -50, -50], [0.95, 1000, 1000, 1000, 50, 50]))
print("fit a,k,b,base:", f.x[:4].round(2))
```

```
fit a,k,b,base: [  0.58 326.19 198.68 541.63]
```

- **Answer** The carry-over is recovered well (0.58 vs 0.6) and the effect size 198.7 (true 200). But k = 326 (true 150) and base = 542 (true 500) are off.

- **Check** Base, k and b trade against each other: a larger k with a larger base can mimic a smaller k with a smaller base over the observed spend range.
- **Why it matters** This is the honest limit of MMM. Parameters are weakly identified when spend varies little. Bayesian MMM (PyMC-Marketing, Robyn; not run here) adds priors from earlier experiments, and you should calibrate the model with incrementality tests.

**Pricing.** Fit `ln(units) = c + e × ln(price)`; the slope `e` is the elasticity. We continue the same random generator and simulate 300 prices between 80 and 120 with true elasticity −1.8.

```
p = rng.uniform(80, 120, 300)
q = np.exp(8 - 1.8*np.log(p) + rng.normal(0, 0.1, 300))
e = np.polyfit(np.log(p), np.log(q), 1)[0]
print("elasticity", round(e, 3))
```

```
elasticity -1.778
```

**Problem 11.4.2** Using e = −1.778, what happens to units and revenue if price rises 10%? What price maximises profit if unit cost is 60?

| | |
|---|---|
| **Plug in** | units factor = 1.1^(−1.778); revenue factor = 1.1^(1 − 1.778) |
| **Compute** | 1.1^(−1.778) = 0.844, so units fall 15.6%. Revenue factor = 1.1^(−0.778) = 0.9285, so revenue falls 7.15%. Profit-maximising price for constant elasticity: `p* = c × e / (1 + e)` = 60 × (−1.778) / (−0.778) = 137.1 |
| **Answer** | A 10% rise loses 15.6% of units and 7.15% of revenue. The formula says 137. |
| **Check** | If \|e\| > 1, raising price lowers revenue. That matches. But our data only has prices from 80 to 120. A price of 137 is *extrapolation*, outside what we have seen, so do not trust it; test 110 to 125 in a real experiment first. |
| **Why it matters** | Elasticity from observational data is biased when prices were set in response to demand (confounding, Chapter 4). Randomised price tests are safer. |

Other pricing words: dynamic pricing (change price by time and stock), promotions (they steal sales from the future; model the dip after), markdown optimisation (clearing stock before it expires).

**Applications**

- FMCG and retail: media budget split, promo planning.
- Quick commerce and e-commerce: price tests, delivery-fee experiments.
- Telecom: plan pricing and bundle design.

| COMMON MISTAKE | Treating MMM coefficients as causal truth. Spend is often higher when sales are already expected to be high (festival season). Include seasonality and events, calibrate with experiments, and give ranges. |
|---|---|

**Check yourself**

1. Adstock a = 0.8: what share of this week's effect remains next week? Answer: 80%.
2. Elasticity −0.6 (inelastic). Price up 5%: revenue? Answer: Up. Units fall about 3% and price rises 5%, so revenue rises (1.05^0.4 ≈ +2%).
3. Why prefer a geo experiment over last-touch for a TV campaign? Answer: It measures the extra sales caused by the campaign; last-touch only counts who clicked last.

## 11.5 Demand, Supply Chain, Healthcare and Other Domains

| INTUITION | A kirana owner stocks rice for the week. Too little and customers leave. Too much and the money is stuck on the shelf. Forecasting gives a number, but *stock* is a decision about the risk around that number. Healthcare has the same shape: a prediction is only useful if it leads to the right treatment decision. |
|---|---|

**Demand and inventory (builds on forecasting, Chapter 6).** Two simple formulas carry most practical work.

- **Safety stock** covers demand uncertainty during the lead time `L` (days between ordering and delivery): `SS = z × σ_daily × √L`, where `z` is the normal quantile for your service level (1.645 for 95%).
- **Newsvendor**: for one-shot stock, order the quantity at the *critical ratio* `CR = c_under / (c_under + c_over)`, where `c_under` is the profit lost per unit short and `c_over` the loss per unit left over.

**Problem 11.5.1** Daily demand has mean 100 and standard deviation 30. Lead time is 4 days. Target service 95%. Find safety stock and reorder point.

| | |
|---|---|
| **Formula** | SS = z × σ × √L; ROP = mean × L + SS |
| **Plug in** | z = 1.645; √4 = 2 |
| **Compute** | SS = 1.645 × 30 × 2 = 98.7. ROP = 100 × 4 + 98.7 = 498.7 |
| **Answer** | Keep about 99 units of safety stock and reorder when stock falls to about 499. |
| **Check** | The code printed `z95 1.645 safety stock 98.7 ROP 498.7`. |
| **Why it matters** | Doubling lead time raises safety stock by only √2, not 2, because random errors partly cancel. |

**Problem 11.5.2** Festival item. Each unit short loses 40 rupees profit; each unit left over costs 10. Demand is Normal(mean 500, sd 100). How much to stock?

| | |
|---|---|
| **Compute** | CR = 40 / (40 + 10) = 0.80. Normal quantile at 0.80 is +0.8416. Q* = 500 + 0.8416 × 100 = 584.2 |
| **Answer** | Stock about 584 units, well above the mean, because shortages cost four times more than leftovers. |
| **Check** | Code printed `critical ratio 0.8 Q* 584.2`. |

Other supply-chain tools (OR-Tools for routing and scheduling is covered in Chapter 12; not run here): ETA prediction for logistics is a regression with heavy tails (report quantiles, not just the mean), and intermittent demand uses Croston-type methods.

**Healthcare.** Health data has rare outcomes, strict privacy (identifiers, consent, India's DPDP Act), long label delays and high costs for errors. Two ideas matter most. **Calibration**: when the model says 20% risk, about 20 of 100 such patients should have the event. **Clinical utility** (net benefit): at a threshold probability `p_t`, `net benefit = TP/N − FP/N × p_t/(1 − p_t)`. It compares a model with "treat everyone" and "treat nobody" (net benefit 0). Survival analysis (11.1) is also standard.

We simulated 20,000 patients (8.9% event rate overall, 8.76% in the test part), trained on 12,000, and compared logistic regression with a random forest on the other 8,000 by quintiles of predicted risk.

```
rf                      pred    obs     n
(-0.001, 0.005]        0.002  0.019  1862
(0.005, 0.025]         0.016  0.038  1642
(0.025, 0.06]          0.043  0.052  1353
(0.06, 0.15]           0.101  0.111  1583
(0.15, 0.915]          0.313  0.229  1560
lr                      pred    obs     n
(-0.00084, 0.016]      0.009  0.008  1600
(0.016, 0.0352]        0.025  0.022  1600
(0.0352, 0.0683]       0.050  0.056  1600
(0.0683, 0.141]        0.098  0.094  1600
(0.141, 0.923]         0.271  0.258  1600
```

- **Read it** Logistic regression's predictions match the observed rates in every bin (0.271 predicted vs 0.258 observed at the top). The random forest says 0.2% for patients whose real rate is 1.9%, and 31% for patients whose real rate is 23%. Ranking may be fine, but the numbers are not probabilities. Recalibrate (isotonic or Platt, Chapter on ML evaluation) before using thresholds.

```
p_t   model NB   treat-all NB   treat-none
0.05  0.0547     0.0396         0
0.10  0.0354    -0.0138         0
0.20  0.0201    -0.1405         0
0.30  0.0113    -0.3034         0
```

**Problem 11.5.3** Compute the net benefit of "treat all" at threshold 0.10. The test-set event rate is 0.0876.

| | |
|---|---|
| **Plug in** | prevalence − (1 − prevalence) × 0.10/0.90 |
| **Compute** | 0.0876 − 0.9124 × 0.1111 = 0.0876 − 0.1014 = −0.0138, which matches the table |
| **Answer** | Treating everybody at a 10% threshold does harm (negative net benefit), while the model has +0.035. |
| **Check** | At threshold 0.05, treat-all (0.0396) is positive but the model (0.0547) is still better. |
| **Why it matters** | A doctor who treats at a 10% risk accepts nine false alarms per true case. Net benefit uses exactly that trade-off. |

**Other domains at a glance.**

| Domain | Typical problems | Special care |
|---|---|---|
| Telecom | Churn, ARPU, network anomalies, usage forecasts | Huge data, billing definitions |
| Retail / e-commerce | Assortment, search, recommendations, returns | Seasonality, cold-start |
| SaaS | MRR/ARR, activation, retention funnels | Cohorts, product events |
| Manufacturing | Predictive maintenance, quality, time-to-failure | Few failures, sensor drift |
| Logistics | ETA, routing, hub capacity | Heavy-tailed delays |
| Geospatial | Distance features, delivery zones, spatial joins | Use `geopandas`, `shapely`, `h3` (not run here) |
| NLP and vision in business | Ticket routing, document extraction, review mining | Details in the deep learning and GenAI material |

**Learning a domain fast.** Read one annual report and one earnings call, write a glossary of 50 terms, ask an expert for 30 minutes, join a business meeting, and build one small domain project. India-specific strengths worth knowing: UPI-scale payments, quick commerce, ONDC (open commerce networks), agritech, Indian-language NLP, global capability centres (GCC), and DPDP and data localisation rules.

**COMMON MISTAKE**

Reporting "95% service level" without saying how demand uncertainty was measured. If the forecast errors are skewed or intermittent, the normal formula under-covers; use empirical quantiles of past errors instead.

**Check yourself**

1. Lead time goes from 4 to 16 days. Safety stock changes by what factor? Answer: $\sqrt{(16/4)} = 2$.
2. What is the critical ratio if shortage costs 20 and surplus costs 30? Answer: 20/50 = 0.40.
3. Why does a model with good AUC still need calibration for clinical use? Answer: Doctors act on the probability value, and AUC only tests ranking.

# 11.6 Operations Analytics: Queues, Delivery Times and Capacity

| INTUITION | Think of a bank with one counter. When few customers arrive, nobody waits. When the counter is busy 95% of the time, the line grows and grows, even though the counter is "only" 5% idle. Operations analytics (warehouses, hospitals, call centres, delivery fleets) is about reading such waiting and flow numbers, spotting when something goes wrong, and predicting when capacity runs out. |
|---|---|

**Two laws that explain most waiting problems.**

- **Little's law**: `L = λ × W`. The average number of items in a system (L) equals the arrival rate (λ) times the average time each item spends in the system (W). Patients in a clinic, orders in a warehouse and customers in a bank all obey it.
- **M/M/1 queue** (one server, random arrivals and random service times): `R = S / (1 − ρ)`, where `R` is the average time in the system, `S` is the average service time and `ρ` (rho) is utilisation, the share of time the server is busy.

**Problem 11.6.1** A clinic admits 50 patients per hour and each stays 0.4 hours on average. How many patients are in the clinic? Then a bank counter serves a customer in S = 0.1 hour (6 minutes) on average. What is the average time in the bank at utilisation 50% to 95%?

| | |
|---|---|
| **Formula** | L = λ × W; R = S / (1 − ρ) |
| **Compute** | L = 50 × 0.4 = 20 patients. R: ρ = 0.5 gives 0.1/0.5 = 0.200 h; 0.7 gives 0.333 h; 0.8 gives 0.500 h; 0.9 gives 1.000 h; 0.95 gives 2.000 h |
| **Answer** | About 20 patients are in the clinic at any time. Going from 50% to 95% utilisation makes the time in the bank 10 times longer (0.2 h to 2.0 h, that is 12 minutes to 2 hours). |
| **Check** | At ρ = 0 the formula gives R = S = 0.1 h, a customer who never waits. Correct. |
| **Why it matters** | "The counter is busy only 70% of the time, so it is fine" is not a safe sentence. The last 20% of utilisation costs more waiting than the first 70%. |

**Anomaly detection on delivery times.** A warehouse records the average order-to-dispatch time (in seconds) every minute. Orders arrive in a daily cycle, so the busy hour is slower, and that is normal. A global rule (flag if more than 3 standard deviations from the overall mean) mixes up "busy" and "broken". Compare each minute with its own recent past instead: use a rolling median and a rolling MAD (median absolute deviation, a robust spread), and a robust z-score `0.6745 × (x − median) / MAD`.

We simulate 7 days of per-minute readings with a daily order cycle, plus one incident (minutes 5000 to 5029, +150 seconds).

```
rng = np.random.default_rng(3); T = 7*24*60; t = np.arange(T)
orders_per_hr = 100 + 40*np.sin(2*np.pi*t/1440 - 1.5) + 0.001*t + rng.normal(0, 5, T)
dispatch = 80 + 0.8*np.maximum(orders_per_hr - 100, 0)**1.2 + rng.normal(0, 6, T)
dispatch[5000:5030] += 150
s = pd.Series(dispatch)
med = s.rolling(60, min_periods=30).median()
mad = (s - med).abs().rolling(60, min_periods=30).median()
flag = (0.6745*(s - med)/mad) > 6
idx = np.where(flag)[0]
print("flagged", len(idx), "first", idx.min(), "last", idx.max(),
      "inside incident", int(((idx >= 5000) & (idx < 5030)).sum()))
print("global z>3 count", int(((s - s.mean())/s.std() > 3).sum()))
```

```
flagged minutes 24 first 5000 last 5023
flagged inside 5000–5029: 24
global z>3 count 30
```

- **Answer** The rolling rule flagged 24 minutes, all inside the incident, starting at the first bad minute (5000). It missed the last 6, because the rolling median began to adapt to the new level. The global rule caught all 30 (we checked), with no false alarms.
- **Check** So on a big incident (+150 seconds) the simple global rule wins. Do not assume the fancy method is better.
- **Why it matters** The rolling rule earns its place on *small* incidents in *quiet* hours. We repeat the test with a +40 second incident of 30 minutes at the low-order time of day:

```
incident 7169 7198 rolling flagged 5 inside 5 | global flagged 1 inside 0
```

At the quiet hour the global threshold (set by busy-hour variation) never fires inside the incident, and its single flag is a false alarm. The rolling rule catches 5 of 30 minutes, which is still a late and partial warning. A rolling window adapts, which is both strength (it follows the daily cycle) and weakness (a long incident becomes "normal"). Pair it with a fixed service rule such as "average dispatch time above 300 seconds for 5 minutes in a row".

**Capacity forecast.** The warehouse's daily peak load rose from about 200 orders with a slope of 0.8 per day plus noise. The packing limit is 300 orders. Fit a line, find when it crosses, and bootstrap the date.

```
days = 90
peak = 200 + 0.8*np.arange(days) + rng.normal(0, 6, days)
b = np.polyfit(np.arange(days), peak, 1)
d = (300 - b[1]) / b[0]
print("slope", b.round(3), "day hits limit", round(d, 1), "from day 90 =", round(d - 90,
1))
```

```
slope [  0.814 200.115]
day hits limit 122.7 from day 90 = 32.7
90% interval [119.3 126.3]
```

(The 90% interval came from 1,000 bootstrap refits of the line on resampled days.)

- **Answer** The fitted slope is 0.814 orders per day, so the limit of 300 is hit on about day 123, roughly 33 days from now, with a 90% interval of day 119 to 126.
- **Check** (300 − 200.115) / 0.814 = 122.7. Correct.
- **Why it matters** Say "between 29 and 36 days from now", not "day 123". Hiring staff or leasing space takes lead time; plan with the early end of the interval. And remember that load rarely stays linear (sales campaigns, festivals).

**Your portfolio piece.** Take any public operations dataset (taxi trips, hospital appointments, delivery logs, bike-share) and produce: percentiles of waiting time by hour (p50, p90, p99), a Little's law consistency check, the utilisation level where waiting explodes, anomaly flags checked against known events, and a capacity date with an interval.

### Applications

- Logistics and e-commerce: delivery-time anomaly detection, hub capacity planning.
- Healthcare operations: appointment waiting times, bed occupancy forecasts.
- Banks and call centres: staffing to a waiting-time target.
- Technology operations: the same laws apply to servers handling requests.

| COMMON MISTAKE | Averaging waiting times. Averages hide the tail: a few customers who wait two hours matter more than a mean of 12 minutes. Report p50, p90, p99, and when you combine groups, combine the raw records, not the percentiles of percentiles. |
|---|---|

### Check yourself

1. 200 patients are in a hospital and each stays 0.5 hours on average. What is the admission rate? Answer: λ = L / W = 200 / 0.5 = 400 patients per hour.

2. Why does a rolling median beat a global threshold for a daily-cycle metric? Answer: It compares each point to its own recent level, so the daily peak is not flagged.
3. Why give a date interval for the capacity limit? Answer: The trend line has uncertainty; the bootstrap shows a range so planning can use the early end.

## 11.7 Survival Regression: Who Churns Sooner, and by How Much

Kaplan-Meier (Section 11.1) gives one curve per group. It cannot say "how much does being on the pro plan change the risk, holding region and age fixed?". For that you need a model of the **hazard**: the chance that a customer who is still active at the start of month `t` churns during month `t`. Two practical routes are the discrete-time model (a logistic regression on a stacked table) and the Cox model (a regression on the hazard ratio that never specifies the baseline shape).

**INTUITION**

Think of a leaky bucket per customer. Each month the bucket either drains completely (churn) or survives. The hazard is the chance it drains this month, given it has not yet. A covariate such as "pro plan" makes the hole smaller or bigger for everyone in that group by the same *factor*. That factor is the hazard ratio.

### Route 1: the discrete-time model (person-period table)

Turn each customer into one row per month they were observed. The row says: this customer, this month, did they churn? A customer who churned in month 3 gives rows for months 1, 2 (outcome 0) and month 3 (outcome 1). A customer censored after month 2 gives months 1 and 2 (both 0) and nothing else. Then fit an ordinary logistic regression with `month` and the covariates as features.

**Problem 11.7.1** Four customers: A churned in month 3, B was censored after month 2, C churned in month 1, D was censored after month 4. Build the person-period table and count rows and events.

| | |
|---|---|
| **Idea** | One row per customer per month observed. Outcome is 1 only on the churn month. |
| **A** | (A,1,0), (A,2,0), (A,3,1): 3 rows, 1 event. |
| **B** | (B,1,0), (B,2,0): 2 rows, 0 events (censored customers never get a 1). |
| **C** | (C,1,1): 1 row, 1 event. |
| **D** | (D,1,0), (D,2,0), (D,3,0), (D,4,0): 4 rows, 0 events. |
| **Answer** | 10 rows, 2 events. The logistic model sees 2 positives out of 10 rows, but the right reading is a monthly hazard: month 1 has 4 at risk and 1 event (hazard 0.25), month 2 has 3 at risk and 0 events, month 3 has 2 at risk and 1 event (0.5), month 4 has 1 at risk and 0 events. |

**Problem 11.7.2** Simulate 4,000 customers, half on the pro plan. The true monthly hazard on the logit scale is `-2.2` for free and `-2.2 - 0.8` for pro, constant over time, observed for at most 12 months. Fit the model and check that it recovers the plan effect and the 12-month survival.

```
import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(7)
n = 4000
pro = rng.integers(0, 2, n)
h = 1 / (1 + np.exp(-(-2.2 - 0.8 * pro)))      # true monthly hazard
T = rng.geometric(h)                            # month of churn
event = (T <= 12).astype(int)                   # churn seen inside the window
tenure = np.minimum(T, 12)                      # censored at 12

rows = []
for i in range(n):
    for m in range(1, tenure[i] + 1):
        rows.append((m, pro[i], int(event[i] == 1 and m == tenure[i])))
pp = pd.DataFrame(rows, columns=["month", "pro", "churn"])
print("rows", len(pp), "events", pp.churn.sum())

lr = LogisticRegression(C=1e6).fit(pp[["month", "pro"]], pp.churn)
print("month coef", lr.coef_[0][0].round(3), "pro coef", lr.coef_[0][1].round(3),
      "odds ratio", np.exp(lr.coef_[0][1]).round(3))

for p in (0, 1):
    hz = lr.predict_proba(pd.DataFrame({"month": range(1, 13), "pro": p}))[:, 1]
    print("plan", p, "model S(12)", np.prod(1 - hz).round(3),
          "empirical share still active", (T[pro == p] > 12).mean().round(3))
```

```
rows 32786 events 2361
month coef 0.005 pro coef -0.764 odds ratio 0.466
plan 0 model S(12) 0.278 empirical share still active 0.278
plan 1 model S(12) 0.541 empirical share still active 0.541
```

**How to read it.** The pro coefficient is close to the true value of -0.8, so the odds of churning in any given month for pro customers are about 0.47 times those of free customers. The month coefficient is near zero because we built a constant hazard. On real data it is usually not zero (hazards fall or rise with tenure), and that is the point of including `month`. The model survival at 12 months matches the share who really were still active.

| COMMON MISTAKE | Dropping censored customers, or treating "still active" as "never churns". Both bias survival upward or downward. The person-period table includes censored customers for exactly the months they were observed and no more. Another frequent error is a random row split: one customer then lands in both train and test. Split by customer id. |
|---|---|

## Route 2: the Cox model

The Cox proportional hazards model writes the hazard for a customer with covariates `x` as `h(t | x) = h0(t) · exp(β·x)`. The baseline `h0(t)` can be any shape and is never estimated; only `β` is. `exp(β)` is the hazard ratio: 0.5 means half the churn risk at every moment. The fit maximises the **partial likelihood**: at each churn time, the probability that it was *this* customer who churned, among everyone still at risk.

**Problem 11.7.3** Three customers. A (free, x = 1) churns on day 2. B (pro, x = 0) churns on day 5. C (free, x = 1) is censored on day 7. Find the hazard ratio of free versus pro by hand.

| | |
|---|---|
| **Setup** | Let u = exp(β), the hazard ratio of free (x = 1) against pro (x = 0). Risk weights: free = u, pro = 1. |
| **Day 2** | Everyone is at risk: weights u + 1 + u = 2u + 1. Chance it was A: u / (2u + 1). |
| **Day 5** | At risk are B and C: weights 1 + u. Chance it was B: 1 / (1 + u). |
| **Log-likelihood** | l = ln u - ln(2u + 1) - ln(1 + u). Differentiate with respect to u and set to zero: 1/u - 2/(2u+1) - 1/(1+u) = 0. |
| **Solve** | 1/u - 2/(2u+1) = 1/(u(2u+1)), so 1/(u(2u+1)) = 1/(1+u). That gives 1 + u = 2u² + u, so u² = 1/2 and u = 0.7071. |
| **Meaning** | Hazard ratio 0.707 for free against pro, so β = ln 0.7071 = -0.3466. With three customers this is noise, not evidence; it only shows the mechanics. |

```
import numpy as np
from scipy.optimize import minimize_scalar

def neg_log_lik(b):
    return -(b - np.log(2 * np.exp(b) + 1) - np.log(1 + np.exp(b)))

r = minimize_scalar(neg_log_lik, bounds=(-3, 3), method="bounded")
print("beta", round(r.x, 4), "hazard ratio", round(float(np.exp(r.x)), 4))
```

```
beta -0.3466 hazard ratio 0.7071
```

| KEY IDEA | Discrete-time logistic regression and Cox regression answer the same question. The logistic route is easier to build with scikit-learn and lets you use any model (even gradient boosting) on the stacked table. The Cox route needs a survival library (such as lifelines or scikit-survival; not run here) but gives hazard ratios with standard errors and does not need you to bin time. |
|---|---|

### Going deeper

The proportional hazards assumption says the ratio is the same at every tenure. Check it: fit the model separately on early months (1 to 3) and later months (4 to 12). If the plan coefficient changes sign or halves, add an interaction `pro × month` or stratify. Another expert point is the **competing risk**: a customer who moves country can never churn for price reasons, so treating those as ordinary censoring overstates how long the rest would last. Fine and Gray models or cause-specific hazards handle this; the stats document covers the formal theory.

## 11.8 Expected Purchases and Spend: BG/NBD and Gamma-Gamma

Section 11.1 said BG/NBD was "not run here". We now build it, because it is the clearest example of a *probabilistic customer model*: a few interpretable assumptions, a likelihood you can code in 20 lines, and a forecast of future purchases for each customer. It suits a non-contractual retailer where you never see the customer leave.

| INTUITION | A customer is a coin-and-clock pair. While alive, they buy at their own steady rate (some often, some rarely). After every purchase there is a small chance they quietly stop forever. If a customer who used to buy every week has been silent for 20 weeks, they are probably gone. If a customer who buys twice a year has been silent for 20 weeks, they are probably fine. The model does this arithmetic for you. |
|---|---|

### The assumptions

- While alive, purchases for customer `i` arrive at random with rate `λ` per week (a Poisson process).
- After each purchase the customer drops out with probability `p`, and never returns.
- `λ` varies across customers as a Gamma distribution with shape `r` and rate `α`.
- `p` varies across customers as a Beta distribution with parameters `a` and `b`.

- Each customer is summarised by three numbers: `x` repeat purchases, `t_x` the week of the last one, and `T` the weeks observed (the "recency, frequency, T" summary).

**Problem 11.8.1** By hand: customer 1 has x = 0 (bought once, then nothing for T weeks). Customer 2 has x = 5, t_x = 40, T = 52. Which one does the model call "alive for sure"?

| | |
|---|---|
| **Idea** | Dropout can only happen right after a purchase. With x = 0 repeat purchases, there has been no repeat event after which a dropout could have been noticed. |
| **Customer 1** | P(alive) = 1 in BG/NBD. (This is a known quirk: customers with no repeat purchases are treated as alive, but with a low expected rate because λ is shrunk towards the population.) |
| **Customer 2** | P(alive) is below 1 because 12 silent weeks (52 - 40) is long for someone buying about every 8 weeks (40 / 5). The formula in the code below gives the exact number. |

```
import numpy as np
from scipy.special import gammaln, hyp2f1
from scipy.optimize import minimize

rng = np.random.default_rng(11)
N, T, H = 3000, 52.0, 26.0                      # customers, weeks observed, weeks held
out
r0, al0, a0, b0 = 0.8, 8.0, 1.2, 3.0            # true population parameters
lam = rng.gamma(r0, 1 / al0, N)
p = rng.beta(a0, b0, N)

x = np.zeros(N); tx = np.zeros(N); fut = np.zeros(N)
for i in range(N):
    t, times = 0.0, []
    while True:
        t += rng.exponential(1 / lam[i])
        if t > T + H: break
        times.append(t)
        if rng.random() < p[i]: break           # dropped out after this purchase
    cal = [u for u in times if u <= T]
    x[i] = len(cal); tx[i] = cal[-1] if cal else 0.0
    fut[i] = len(times) - len(cal)              # purchases in the held-out weeks

def nll(th):
    r, al, a, b = np.exp(th)
    A = gammaln(r + x) - gammaln(r) + r * np.log(al)
    B = gammaln(a + b) + gammaln(b + x) - gammaln(b) - gammaln(a + b + x)
    t1 = -(r + x) * np.log(al + T)
    t2 = np.where(x > 0, np.log(a) - np.log(np.maximum(b + x - 1, 1e-9)) - (r + x) *
np.log(al + tx), -np.inf)
    return -np.sum(A + B + np.logaddexp(t1, t2))

res = minimize(nll, np.log([1, 1, 1, 1]), method="Nelder-Mead",
               options={"maxiter": 4000, "xatol": 1e-6, "fatol": 1e-8})
r, al, a, b = np.exp(res.x)
print("fitted r, alpha, a, b:", np.round([r, al, a, b], 3), " true:", [r0, al0, a0, b0])

def p_alive(x, tx, T):
    d = 1.0 if x > 0 else 0.0
    return 1 / (1 + d * a / (b + max(x, 1) - 1) * ((al + T) / (al + tx)) ** (r + x))

def expected(x, tx, T, t):
    d = 1.0 if x > 0 else 0.0
    core = (a + b + x - 1) / (a - 1) * (1 - ((al + T) / (al + T + t)) ** (r + x)
                                        * hyp2f1(r + x, b + x, a + b + x - 1, t / (al +
T + t)))
    return core * p_alive(x, tx, T)
```

```
print("customer 2 P(alive):", round(p_alive(5, 40, 52), 3), " expected next 26 weeks:",
round(expected(5, 40, 52, 26), 3))
ex = np.array([expected(x[i], tx[i], T, H) for i in range(N)])
print("held-out purchases per customer: model", ex.mean().round(3), " actual",
fut.mean().round(3))
for lo, hi in [(0, 0), (1, 1), (2, 3), (4, 99)]:
    m = (x >= lo) & (x <= hi)
    print("x in", (lo, hi), "n", int(m.sum()), "model", ex[m].mean().round(3), "actual",
fut[m].mean().round(3))
```

```
fitted r, alpha, a, b: [0.835 8.333 0.971 2.355]  true: [0.8, 8.0, 1.2, 3.0]
customer 2 P(alive): 0.642  expected next 26 weeks: 1.389
held-out purchases per customer: model 0.613  actual 0.576
x in (0, 0) n 578 model 0.325 actual 0.324
x in (1, 1) n 996 model 0.205 actual 0.19
x in (2, 3) n 782 model 0.443 actual 0.4
x in (4, 99) n 644 model 1.709 actual 1.613
```

**How to read it.** The fit recovers the population parameters only roughly (0.97 against a true 1.2 for `a`; the customer-level parameters are never seen, so some drift is normal). The key check is the last table: for each frequency group the model's average forecast for the 26 held-out weeks lands within about 10% of what really happened in every frequency group (for example 1.71 against 1.61 for the most frequent buyers), and the overall average is 0.61 against 0.58. A model is never exact; you are checking for large, systematic gaps. Always run this holdout calibration check before you trust a CLV number.

**Adding money: Gamma-Gamma.** Average spend per order varies across customers too, and a customer with only 2 orders has a noisy average. Gamma-Gamma shrinks the observed average $\bar{m}$ towards the population mean. With $x$ orders, the expected spend is a weighted average: $E[\text{spend}] = w \cdot \bar{m} + (1 - w) \cdot \mu$ where $w = p \cdot x / (p \cdot x + q - 1)$ and the population mean is $\mu = \gamma \cdot p / (q - 1)$ ($p$, $q$, $\gamma$ are the model's three fitted parameters).

**Problem 11.8.2** Fitted p = 6, q = 4, γ = 15. A customer has 3 orders averaging 50 rupees. What spend does the model expect for the next order, and what would it be with 30 orders at the same average?

```
p_, q_, g_ = 6.0, 4.0, 15.0
mu = g_ * p_ / (q_ - 1)
for x_, mbar in [(3, 50.0), (30, 50.0)]:
    w = p_ * x_ / (p_ * x_ + q_ - 1)
    print("orders", x_, "weight on own average", round(w, 3), "expected spend", round(w *
mbar + (1 - w) * mu, 2))
print("population mean", mu)
```

```
orders 3 weight on own average 0.857 expected spend 47.14
orders 30 weight on own average 0.984 expected spend 49.67
population mean 30.0
```

By hand for 3 orders: `w = 18 / (18 + 3) = 0.857`, so the expectation is `0.857 × 50 + 0.143 × 30 = 47.14`. With 30 orders, the weight rises to 0.984 and the answer to 49.67. More evidence, less shrinkage.

Then **CLV over 26 weeks = expected purchases × expected spend per order × margin**, optionally discounted. Multiply the two models only if spend and frequency are roughly uncorrelated (check this: `corr(x, m̄)` on repeat buyers).

| COMMON MISTAKE | Validating CLV on the same weeks used for fitting. Hold out the last 26 weeks as in the code, compare predicted and actual totals per frequency group, and only then trust the model. A second mistake: fitting on customers who joined at different dates without aligning each customer's clock (T is weeks since *their own* first purchase). |
|---|---|

### Going deeper

BG/NBD assumes the dropout is a constant per-purchase probability, purchase rate is constant while alive, and customers are independent. Trends (promotions, seasons) break the second assumption; modern variants add covariates (PyMC-Marketing, not run here) or use a gradient-boosted model on RFM features to predict next-quarter spend directly. In practice many teams run both: BG/NBD as a transparent baseline and a GBM when there are rich features. The baseline is hard to beat for customers with very few orders.

## 11.9 Fraud: Rare Events, Cost Curves and Graph Features

Section 11.3 showed precision, recall and thresholds on a fraud problem. This section adds the two ideas that separate a junior from a senior fraud analyst: choose the threshold from *money*, not from F1, and use *relationships* between accounts as features.

### Cost curves: choosing the threshold in rupees

Let a missed fraud (false negative) cost the transaction amount, and let each flagged transaction (true or false positive) cost a 5 rupee manual review. Total cost at a threshold is `5 × flagged + sum of amounts of frauds not flagged`. Plot or tabulate it across thresholds and take the minimum. Two baselines belong on every cost table: "flag nothing" (pay every fraud) and "flag everything" (pay for every review).

## Graph features: accounts that share a device

Fraud rings reuse devices, cards or addresses. A simple graph feature is the number of accounts seen on the same device. Honest families sometimes share a device too, so the feature is strong but not perfect. The synthetic data below has 60,000 transactions with 1% fraud; 60% of fraud rows come from a ring that shares 60 devices.

**Problem 11.9.1** Train a gradient-boosted model with and without `acc_on_device`. Compare average precision (area under the precision-recall curve, the right summary for rare events), then find the cost-minimising threshold.

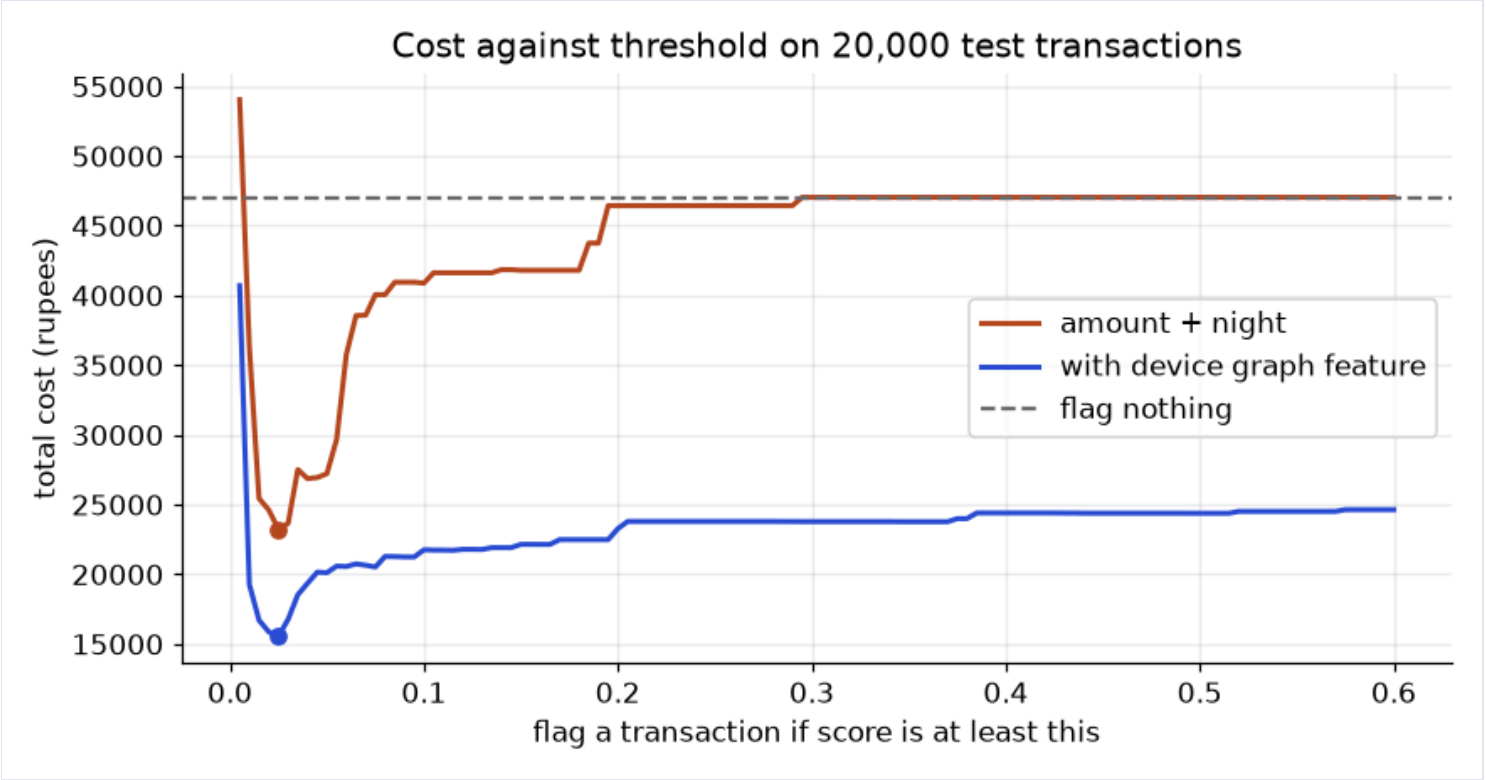
```
import numpy as np, pandas as pd
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import average_precision_score, roc_auc_score

rng = np.random.default_rng(5)
n = 60000
fraud = (rng.random(n) < 0.01).astype(int)
amount = np.where(fraud == 1, rng.lognormal(5.0, 0.8, n), rng.lognormal(4.2, 0.8,
n)).round(2)
night = np.where(fraud == 1, rng.random(n) < 0.45, rng.random(n) < 0.2).astype(int)
ring = (fraud == 1) & (rng.random(n) < 0.6)
device = np.where(ring, rng.integers(0, 60, n), 1000 + np.arange(n))
family = rng.random(n) < 0.15
device = np.where((fraud == 0) & family, 1000 + rng.integers(0, 20000, n), device)
df = pd.DataFrame(dict(amount=amount, night=night, device=device, fraud=fraud))
df["acc_on_device"] = df.groupby("device").device.transform("size")

train = np.arange(n) < 40000
test = ~train
scores = {}
for name, cols in [("amount+night", ["amount", "night"]), ("with graph", ["amount",
"night", "acc_on_device"])]:
    m = HistGradientBoostingClassifier(random_state=0).fit(df.loc[train, cols],
df.fraud[train])
    s = m.predict_proba(df.loc[test, cols])[:, 1]
    scores[name] = s
    print(name, "AUC", round(roc_auc_score(df.fraud[test], s), 3), "AP",
round(average_precision_score(df.fraud[test], s), 3))

y = df.fraud[test].values; amt = df.amount[test].values; review = 5.0
print("flag nothing:", round(amt[y == 1].sum()), " flag everything:", round(review *
len(y)))
best = None
for th in [0.01, 0.02, 0.03, 0.05, 0.1, 0.2, 0.3, 0.5]:
    f = scores["with graph"] >= th
    cost = review * f.sum() + amt[(~f) & (y == 1)].sum()
    print("threshold", th, "flagged", int(f.sum()), "caught", int((f & (y == 1)).sum()),
"of", int(y.sum()), "cost", round(cost))
```

```
amount+night AUC 0.771 AP 0.053
with graph AUC 0.867 AP 0.542
flag nothing: 47034  flag everything: 100000
threshold 0.01 flagged 2756 caught 158 of 212 cost 19268
threshold 0.02 flagged 1443 caught 143 of 212 cost 15864
threshold 0.03 flagged 956 caught 131 of 212 cost 16762
threshold 0.05 flagged 331 caught 114 of 212 cost 20109
threshold 0.1 flagged 144 caught 107 of 212 cost 21749
threshold 0.2 flagged 124 caught 102 of 212 cost 23265
threshold 0.3 flagged 119 caught 100 of 212 cost 23772
threshold 0.5 flagged 109 caught 97 of 212 cost 24374
```




*Total cost (5 rupees per review plus the amounts of missed frauds) against the flag threshold, with and without the device feature. Dots mark each minimum.*

**How to read the figure.** Lower is better. Both curves start high on the left (flagging nearly everything costs reviews), fall to a minimum at a low threshold near 0.02, then rise to the dashed "flag nothing" line as the threshold climbs. The blue curve sits well below the orange one at every threshold, and its minimum is the lowest point on the chart. A threshold of 0.5 is nowhere near optimal for either model.

**How to read it.** Average precision jumps when the graph feature is added, while AUC rises less, a typical sign that AUC hides rare-event quality. In the cost table the best threshold is 0.02 (cost 15,864, against 47,034 for flagging nothing and 24,374 at 0.5), which is far from 0.5 because a missed fraud costs much more than a review. Note the leak risk: `acc_on_device` must be computed only from transactions *before* the one being

scored. Here we counted over the whole table to keep the example short, which flatters the result; in production compute it point-in-time (Chapter 2).

| COMMON MISTAKE | Reporting accuracy or AUC on a 1% problem and shipping at threshold 0.5. A model that flags nothing is 99% accurate. Use average precision, recall at a fixed review capacity, and the cost table above. |
|---|---|

## 11.10 Scorecards: Reject Inference

A credit model is trained on applicants you *approved*, because only they have a repayment outcome. But it will score *everyone* who applies. The rejected group is missing from training: this is **selection bias**, and fixing it is called **reject inference**.

| INTUITION | Imagine learning which students pass an exam by only observing students the teacher let sit it. If the teacher chose using marks you also have, you can still learn the rule. If the teacher chose using a gut feeling you never recorded, what you learn is bent by that hidden feeling. |
|---|---|

**Problem 11.10.1** Simulate 60,000 applicants, default driven by income, utilisation and a hidden officer impression. Compare two approval policies: A uses only recorded variables, B also uses the hidden impression. Fit on approved applicants only, and compare to a model fitted on everyone (which you can never do in real life).

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

rng = np.random.default_rng(3)
n = 60000
inc = rng.normal(0, 1, n); util = rng.normal(0, 1, n); hid = rng.normal(0, 1, n)
z = -2.0 - 0.9 * inc + 0.6 * util + 1.0 * hid
d = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
X = np.c_[inc, util]
tr = np.arange(n) < 40000; te = ~tr

def study(label, score):
    appr = score > 0
    mA = LogisticRegression(C=1e6).fit(X[tr & appr], d[tr & appr])
    mAll = LogisticRegression(C=1e6).fit(X[tr], d[tr])
    print(label, "approved", appr.mean().round(2), "bad rate approved",
d[appr].mean().round(3), "rejected", d[~appr].mean().round(3))
    for nm, m in (("approved only", mA), ("all applicants", mAll)):
        print("  ", nm, "coefs", m.coef_[0].round(2), "AUC", round(roc_auc_score(d[te],
m.predict_proba(X[te])[:, 1]), 3),
              "mean predicted PD", m.predict_proba(X[te])[:, 1].mean().round(3), "actual",
d[te].mean().round(3))

study("Policy A (recorded only):", 0.8 * inc - 0.5 * util + rng.normal(0, 0.3, n))
study("Policy B (uses hidden view):", 0.5 * inc - 0.5 * util - 1.5 * hid + rng.normal(0,
0.3, n))
```

```
Policy A (recorded only): approved 0.5 bad rate approved 0.091 rejected 0.281
   approved only coefs [-0.8   0.58] AUC 0.738 mean predicted PD 0.192 actual 0.187
   all applicants coefs [-0.79  0.52] AUC 0.738 mean predicted PD 0.185 actual 0.187
Policy B (uses hidden view): approved 0.5 bad rate approved 0.062 rejected 0.308
   approved only coefs [-0.69  0.34] AUC 0.736 mean predicted PD 0.077 actual 0.187
   all applicants coefs [-0.79  0.52] AUC 0.738 mean predicted PD 0.185 actual 0.187
```

**How to read it.** Under Policy A the approved-only coefficients match the all-applicants ones: when selection depends only on variables you recorded and model, there is no bias to fix. Under Policy B the utilisation coefficient shrinks (0.34 against 0.52) and the predicted default rate for the full applicant pool is far too low (mean predicted PD 0.077 against an actual 0.187). Ranking power (AUC) hardly moves, which is why this bias hides: the scorecard still ranks well but under-prices risk. Practical remedies: (1) keep a small random-approve

test cell so you observe outcomes for some who would be rejected; (2) use the bounded-exposure data of past policies; (3) apply *parcelling or fuzzy augmentation* (label rejects with the model's own probabilities, weighted); this helps only when selection is on recorded variables, so it is a mitigation, not a cure.

| KEY IDEA | Selection on observed variables is fixable by modelling; selection on hidden variables is fixable only by new data (a randomised test cell). Say which case you are in when you present a scorecard. |
|---|---|

## Chapter recap

- Churn needs a definition first (contractual vs silent). Kaplan-Meier survival uses censored customers properly: 26% of free vs 57% of pro customers survive a year.
- CLV from cohort revenue times margin: 1,522 (free) vs 4,940 (pro) rupees in year one; RFM gave churn rates from 2.7% to 59.3%.
- Scorecards: WoE bins, IV to select, logistic regression on WoE, points from PDO. Test AUC 0.685, Gini 0.371, KS 0.295.
- PSI under 0.1 is stable. A simulated shift gave PSI 0.456.
- Fraud: accuracy is useless at 0.4% fraud; use precision at capacity and the cost threshold `c_review / (c_review + c_miss)` = 2%, which saved 24.9%.
- MMM uses adstock and saturation, but parameters can be weakly identified. Pricing elasticity from observation is not causal and should not be extrapolated.
- Safety stock `z × σ × √L` and newsvendor critical ratio turn forecasts into stock.
- Healthcare needs calibration and net benefit. Operations data uses Little's law, the M/M/1 knee, rolling robust anomaly detection and capacity dates with intervals.

| Item | Formula | Example |
|---|---|---|
| Kaplan-Meier | `S × (1 − d/n)` | 0.8 → 0.6 → 0.3 |
| WoE | `ln(pg / pb)` | ln(0.5) = −0.693 |
| IV | `Σ (pg − pb) × WoE` | 0.168 for utilisation |
| Gini | `2 × AUC − 1` | 0.3706 |
| PSI | `Σ (a − e) × ln(a / e)` | 0.0405 |
| Expected loss | `PD × LGD × EAD` | 0.03 × 0.45 × 200,000 = 2,700 |

| Item | Formula | Example |
|---|---|---|
| Cost threshold | `c_fp / (c_fp + c_fn)` | 100 / 5,100 = 0.0196 |
| Adstock, saturation | `a × prev + x`, `x / (x + k)` | 100, 50, 25 |
| Safety stock | `z × σ × √L` | 98.7 |
| Newsvendor | `c_u / (c_u + c_o)` | 0.80 → Q = 584 |
| Little's law, M/M/1 | `L = λW`, `R = S/(1−ρ)` | 20 patients, 2.0 h at 95% |

## Practice set

**1.** (Easy) Bin A has 40% of goods and 10% of bads. Find its WoE. **Answer:** ln(0.40/0.10) = ln 4 = 1.386. Very safe bin.

**2.** (Easy) PD 2%, LGD 50%, EAD 100,000 rupees. Expected loss? **Answer:** 0.02 × 0.5 × 100,000 = 1,000 rupees.

**3.** (Easy) A shop serves 120 customers per hour and each spends 0.25 hours inside. How many are inside on average? **Answer:** 120 × 0.25 = 30.

**4.** (Medium) PSI for three bins, expected (0.5, 0.3, 0.2), actual (0.4, 0.3, 0.3). Is it stable? **Answer:** (−0.1)ln(0.8) + 0 + (0.1)ln(1.5) = 0.0223 + 0.0405 = 0.0629. Below 0.1, stable.

**5.** (Medium) Find the bug: "Churn rate = 65% so average customer lifetime is 1/0.65 = 1.5 periods." The 65% came from all customers including still-active ones, treated as non-churned. **Answer:** Mixing a share of customers (who churned by a date) with a per-period hazard. The 65% depends on observation length, and active customers are censored. Use a survival curve or a per-period hazard.

**6.** (Medium) What is printed? `adstock(np.array([200, 0, 0.]), 0.5)` **Answer:** `[200., 100., 50.]`, since each week keeps half of the previous adstock.

**7.** (Medium) Missed fraud costs 3,000, review costs 60. Capacity allows 200 reviews. Threshold from cost is about 2%, but 2% flags 900 cases a day. What do you do? **Answer:** Capacity is a constraint: review the top 200 by score (precision at K), then argue for more reviewers or a cheap automated step if the expected savings of the next 700 reviews exceed their cost.

**8.** (Hard) Elasticity −2.5, unit cost 40. Optimal price and warning? **Answer:** p* = 40 × (−2.5) / (−1.5) = 66.7. Warn that this assumes constant elasticity and must be tested inside the observed price range.

**9.** (Interview) "Your credit model has Gini 0.85 on test. Is that great?" **Answer:** Suspicious. Check for leakage (variables created after the decision, such as collection status), a time-ordered split, IV above 0.5 for any variable, and that the label window is respected. Real application scorecards are usually far below such a number.

**10.** (Interview) "How would you detect a slow capacity problem versus a sudden breakdown in delivery-time data?" **Answer:** Sudden: rolling robust z-score plus a fixed service rule. Slow: weekly peaks regressed on time (trend with interval), plus a look at where waiting time explodes as utilisation rises. Report the date range at which a limit is hit.

**11.** (Hard) A random forest has AUC 0.87 and its 5% to 10% predicted-risk bin has an observed rate of 3%. A doctor will treat at 10%. Problem? **Answer:** Poor calibration. The probabilities are not real, so the threshold means something different. Calibrate on a held-out set, then re-check net benefit.

**12.** (Medium) Four customers: tenures 2 (churned), 4 (censored), 3 (churned), 1 (churned). How many person-month rows, and what is the month-1 hazard? **Answer:** Rows: 2 + 4 + 3 + 1 = 10. Month 1: 4 at risk, 1 churn (the tenure-1 customer), so hazard = 1/4 = 0.25.

**13.** (Medium) A Cox model gives a coefficient of -0.69 for the pro plan against free. State it as a hazard ratio and in words. **Answer:** exp(-0.69) = 0.50. At any moment a pro customer has half the churn risk of a comparable free customer (not "half the churn rate overall", and not "lives twice as long", unless the hazard is exactly constant).

**14.** (Medium) Gamma-Gamma with p = 6, q = 4, γ = 15 (population mean 30). A customer has 1 order of 90 rupees. Expected spend? **Answer:** w = 6 / (6 + 3) = 0.667. Expected = 0.667 × 90 + 0.333 × 30 = 60 + 10 = 70 rupees. One order is weak evidence, so it is pulled a long way towards 30.

**15.** (Hard) Fraud: a missed fraud costs its amount (average 200), a review costs 5. At what fraud probability is flagging worth it, for an average transaction? **Answer:** Flag if p × 200 > 5, so p > 0.025. A threshold of 0.5 would miss most worthwhile flags. For a 1,000 rupee transaction the threshold is 0.005.

**16.** (Hard) (find the flaw) A bank trains a scorecard only on approved loans, reports AUC 0.78, and claims the portfolio default rate will be 4% on all applicants. What is wrong? **Answer:** The 4% is estimated on approved applicants only and the model never saw rejects, so the pool-wide PD is probably higher. AUC on approved loans is also an optimistic proxy for ranking quality on the full pool. Ask for a random test cell or a reject-inference check, and report predicted versus actual default for the full applicant population only after it is available.

# Prompt pack

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me credit-risk scorecards step by step: PD, LGD, EAD, WoE binning, Information Value, logistic regression on WoE, converting to points (base score and PDO), KS, Gini and PSI. Use a tiny made-up table of 10 loans with real numbers I can compute by hand. After each step, ask me to compute the next number and check my answer before moving on.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles. Topic: churn and customer lifetime value, covering contractual vs non-contractual churn, censoring, Kaplan-Meier survival, cohort CLV, RFM segments and why uplift beats churn scores for retention offers. Give me 8 graded exercises with small numbers and no solutions. When I send my answers, grade them, show every calculation step, and explain the business decision that each answer changes.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles. Topic: fraud detection with extreme class imbalance: why accuracy fails, precision at fixed review capacity, PR-AUC, cost-based thresholds and concept drift. Quiz me one question at a time (10 questions, easy to hard), using numbers like 0.4% fraud rate and costs in rupees. Wait for my answer each time, then correct me and explain the intuition.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles. Topic: marketing mix modelling (adstock, saturation, identifiability limits, incrementality tests) and price elasticity (log-log regression, optimal price formula, extrapolation danger). Review my work below: [PASTE HERE MY CODE OR MY EXPLANATION]. Point out statistical mistakes, causal mistakes and unclear wording, then rewrite my explanation in simple English for a marketing manager.

✨ **AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles and English is my second language. Topic: operations analytics: Little's law, the M/M/1 utilisation curve, robust rolling anomaly detection on delivery times, and capacity forecasting with uncertainty intervals. Run a mock interview for a Data Scientist role at a logistics or healthcare-operations company: ask 8 questions, one at a time, with follow-ups. Score each answer out of 5 and show a model answer.

CHAPTER 12

# FROM PREDICTION TO DECISION: COST MATRICES, THRESHOLDS, OPTIMIZATION AND SIMULATION

*A probability is not an action. This chapter turns scores into money, budgets into allocations, and uncertainty into ranges you can defend.*

Your churn model says customer 4,017 has a 31% chance of leaving. So what? Do we send a 100 rupee offer, call them, or do nothing? Nobody pays a data scientist for the 31%. They pay for the decision, and a good decision needs three more ingredients: what each outcome costs, what limits we face (budget, call-centre minutes, stock), and how sure we are about our inputs. By the end of this chapter you will be able to choose a threshold from a cost matrix, allocate a budget with linear and integer programming, simulate a campaign to get a profit range, and explain bandits and feedback loops in plain words. We reuse the shop churn table from Chapter 1 (`from ds_common import *`) and the fraud and demand ideas from Chapter 11. All outputs are real runs with fixed seeds (numpy, pandas, scikit-learn, scipy).

## 12.1 Cost Matrices, Expected Value and Choosing the Threshold

**INTUITION**

A smoke alarm that rings at every toast is switched off by the family. One that stays silent in a real fire is worse. The *right* sensitivity is not "balanced"; it depends on what a false alarm costs against what a miss costs. Choosing a threshold is the same question for any model.

**Words you need.**

| Word | Meaning | Example |
|---|---|---|
| Threshold | The score above which we act | Send an offer if churn probability ≥ 0.22 |
| Cost matrix | A 2×2 table of money for TP, FP, FN, TN | TP = +350, FP = −100 |
| Expected value (EV) | Probability-weighted average outcome | `p × 450 − 100` |

| Word | Meaning | Example |
|---|---|---|
| Operating point | The chosen threshold (or number of cases to act on) | Top 273 customers |
| Break-even probability | Probability where acting and not acting have equal EV | 22.2% |

**The retention-offer decision.** We offer a discount that costs 100 rupees to send. If the customer was going to churn, the offer saves them 30% of the time, and a saved customer is worth 1,500 rupees of future profit. If they would have stayed, the offer is just a cost. (This simple story treats the offer as acting only on churners. Real offers also have effects on non-churners; see uplift below.)

**Problem 12.1.1** Using the numbers above, find the expected gain of one offer to a customer with churn probability p, and the break-even p.

| | |
|---|---|
| **Formula** | EV(send) = p × (0.30 × 1,500) − 100. EV(do nothing) = 0 |
| **Plug in** | 0.30 × 1,500 = 450, so EV(send) = 450p − 100 |
| **Compute** | Set 450p − 100 = 0, so p = 100 / 450 = 0.2222 |
| **Answer** | Send the offer when churn probability is above 22.2%. Customer 4,017 at 31% has EV = 450 × 0.31 − 100 = +39.5 rupees, so yes, send. |
| **Check** | At p = 0.2222, EV = 450 × 0.2222 − 100 = 0.0. At p = 0.5, EV = +125, which is positive, as expected. |
| **Why it matters** | The threshold came from the business numbers, not from 0.5. If the offer cost doubles to 200, the break-even moves to 44% and fewer customers get it. |

**Problem 12.1.2** Now run it on data. We train logistic regression on 1,700 of the 2,562 active shop customers (features: days since last order, log of order count, pro plan, ads channel), and evaluate on the other 862.

```
from ds_common import *
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
customers, orders = make_data()
d = make_label_table(customers, orders)
past = orders[orders.order_ts < PRED_DATE]
g = past.groupby("customer_id").agg(last=("order_ts", "max"), f=("order_id", "count"))
d = d.merge(g, left_on="customer_id", right_index=True)
d["rec"] = (PRED_DATE - d["last"]).dt.days
d["pro"] = (d.plan == "pro").astype(int); d["ads"] = (d.channel == "ads").astype(int)
d["logf"] = np.log1p(d.f)
X = d[["rec", "logf", "pro", "ads"]].values; y = d.churn.values
idx = np.random.default_rng(0).permutation(len(d)); tr, te = idx[:1700], idx[1700:]
mu, sd = X[tr].mean(0), X[tr].std(0)
lr = LogisticRegression().fit((X[tr] - mu) / sd, y[tr])
p = lr.predict_proba((X[te] - mu) / sd)[:, 1]; yt = y[te]
print(len(tr), len(te), "churn rate test", yt.mean().round(3), "AUC",
round(roc_auc_score(yt, p), 3))

cost, save, val = 100, 0.3, 1500
def profit(send):
    return (send & (yt == 1)).sum() * save * val - send.sum() * cost
for t in [0.0, 0.1, cost/(save*val), 0.4, 0.6, 0.8]:
    s = p >= t; print(round(t, 3), "sent", s.sum(), "profit", round(profit(s)))
```

Before you run it: which threshold do you expect to win, 0.0 (send to everyone) or the break-even 0.22?

```
1700 862 churn rate test 0.265 AUC 0.878
0.0 sent 862 profit 16400
0.1 sent 449 profit 48250
0.222 sent 273 profit 53250
0.4 sent 204 profit 46200
0.6 sent 160 profit 39350
0.8 sent 112 profit 31100
```

- **Answer** Profit peaks near the break-even threshold: 53,250 rupees at 0.222, against 16,400 if we send to everyone and 0 if we send to no one. A sweep over 101 thresholds found the best at 0.23 with 53,450, almost the same.
- **Check** At 0.222 the model reaches 179 of the 228 test churners and wastes 94 offers on stayers. The cost matrix is: TP = 0.30 × 1,500 − 100 = +350, FP = −100, FN and TN = 0. Then 179 × 350 − 94 × 100 = 62,650 − 9,400 = 53,250. It matches.

- **Why it matters** An AUC of 0.878 sounds like a score; 53,250 rupees on 862 customers is a result. The first one cannot go in a budget meeting.

**The picture.** Draw profit on the y axis against threshold on the x axis. At threshold 0 you send to all: many wasted offers, profit 16,400. As the threshold rises you drop the customers whose EV is negative and profit climbs. Past 0.22 you start dropping customers whose EV is still positive, and profit falls slowly. The curve is a hill with a *flat top*: anything between 0.15 and 0.30 is almost as good. That flatness is useful; do not argue about the third decimal.

**Fixed budget instead of a threshold.** If marketing gives you a budget, take the top K by score (like precision at K in fraud, Chapter 11):

```
K = 100  profit 27,800  (spend 10,000)
K = 200  profit 46,150  (spend 20,000)
K = 400  profit 50,450  (spend 40,000)
```

At K = 273 you would match the threshold rule (about 53,250). The value of the last rupee spent falls: going from 200 to 400 offers doubles the spend and adds only 4,300 profit.

**Calibration must hold.** The expected-value formula multiplies a *probability* by a value. If the model says 0.84 and the real rate is 0.75, you over-pay. Quintiles on the test set:

```
predicted   observed
0.040       0.029
0.069       0.058
0.110       0.128
0.283       0.360
0.840       0.746
```

The model is roughly calibrated but the middle is under-confident (0.283 predicted, 0.360 observed) and the top is over-confident. With 172 customers per bin, a few points of noise are normal. If it were far off, recalibrate (Platt or isotonic) before any money calculation.

**Words that look similar and are not.**

| Idea | Question it answers |
|---|---|
| Churn probability | Who will leave? |
| Uplift | Who will leave *less* if we act? (needs an experiment or causal model, Chapter 8) |
| Value at risk | How much is it worth if we save them? |

The EV above used a single "save rate" of 30% for all churners. If an A/B test shows the offer only works on some groups, the threshold should use *uplift* × *value*, and "sure things" and "lost causes" drop out. Use the cost matrix thinking; just swap in the right quantity.

**Connection to the job.** Every ML deployment has this step, from loan approval cut-offs to fraud queues. Writing the cost matrix with the business owner is often the most valuable meeting of the project, because it forces them to say what a mistake costs.

**Applications**

- Data Science: threshold choice for churn, fraud, collections, lead scoring.
- Data Analytics: ROI of campaigns by score decile.
- MLOps: threshold is a config value; review it when costs change, not only when the model changes.

| COMMON MISTAKE | Using 0.5 as the threshold, or "the threshold that maximises F1". Neither knows your costs. Also, do not forget costs that depend on action volume (agent time) or interact (offer fatigue). |
|---|---|

**Check yourself**

1. Offer costs 60, saves 20% of churners, a saved customer is worth 900. Break-even probability? Answer: 60 / (0.20 × 900) = 60 / 180 = 0.333.
2. Why can profit be higher at threshold 0.222 than at 0.0, even though 0.0 reaches all 228 churners? Answer: It also sends 634 wasted offers, costing more than the extra churners are worth.
3. A model is badly over-confident. Which part of the EV formula breaks? Answer: The probability `p`; the multiplication by value then overstates the gain.

## 12.2 Optimization: Allocating Scarce Resources

| INTUITION | A shopkeeper has 100 rupees of ad budget and three places to spend it. Each place pays back differently and has a cap. She will put money into the best place until it is full, then the next best. That is a linear program: a goal (maximise profit), choices (how much per channel), and rules (budget, caps). Optimisation is just doing that carefully when there are hundreds of choices. |
|---|---|

**Words you need.**

| Word | Meaning |
|---|---|
| Decision variable | A number we choose (rupees per channel) |
| Objective | The single number we maximise or minimise (profit, cost) |
| Constraint | A rule the answer must obey ( `total ≤ budget` ) |
| Linear program (LP) | Objective and constraints are straight-line formulas; variables can be fractions |
| Integer program (MIP/MILP) | Some variables must be whole numbers, often 0 or 1 (choose or not) |
| Shadow price | How much the objective improves if a constraint loosens by 1 unit |

**Problem 12.2.1** Three channels return 3.0, 2.2 and 1.5 rupees of profit per rupee. Budget is 100 (thousand). Caps: channel 1 up to 50, channel 2 up to 40, channel 3 up to 60. Contract rule: at least 10 must go to channel 3. Maximise profit.

| | |
|---|---|
| **Step 1** | Spend the minimum on channel 3: 10. Remaining 90. |
| **Step 2** | Fill the best channel first: channel 1 gets its cap, 50. Remaining 40. |
| **Step 3** | Next best, channel 2: its cap is 40, and 40 remains. Channel 2 gets 40. Remaining 0. |
| **Compute** | Profit = 3.0 × 50 + 2.2 × 40 + 1.5 × 10 = 150 + 88 + 15 = 253 |
| **Answer** | Allocate (50, 40, 10) for a profit of 253 thousand. The solver agrees: |

```
from scipy.optimize import linprog
c = -np.array([3.0, 2.2, 1.5])                  # linprog minimises, so negate
res = linprog(c, A_ub=[[1, 1, 1]], b_ub=[100], bounds=[(0, 50), (0, 40), (10, 60)])
print("alloc", res.x.round(1), "profit", round(-res.fun, 1), "shadow price", round(-
res.ineqlin.marginals[0], 3))
res2 = linprog(c, A_ub=[[1, 1, 1]], b_ub=[101], bounds=[(0, 50), (0, 40), (10, 60)])
print("budget + 1 ->", round(-res2.fun, 2))
```

```
alloc [50. 40. 10.] profit 253.0 shadow price 1.5
budget + 1 -> 254.5
```

- **Check** The shadow price 1.5 says one extra thousand of budget adds 1.5 thousand profit. We confirm: 254.5 − 253 = 1.5. Why 1.5 and not 3.0? Channels 1 and 2 are full, so the extra rupee can only go to channel 3, which pays 1.5.

- **Why it matters** The shadow price is the answer to "should I ask finance for more budget?". If a new channel can be bought below 1.5 return, ask. Also, the rule "at least 10 on channel 3" costs us: forced money pays 1.5, but we could earn 2.2 if channel 2 had room.

**Integer choices: who gets a call?** Now the decision is yes or no per customer. We have 12 at-risk customers with expected values (rupees) and call times (minutes), and 60 minutes of agent time. Choose a set that maximises total value. This is the **0/1 knapsack problem**: a bag with limited capacity, items with value and weight.

```
from scipy.optimize import milp, LinearConstraint, Bounds
rng = np.random.default_rng(1); n = 12
value = np.round(rng.uniform(100, 900, n)); w = rng.integers(5, 30, n)
print("values ", value.astype(int).tolist()); print("minutes", w.tolist())
r = milp(-value, constraints=LinearConstraint(w[None, :], 0, 60), integrality=np.ones(n),
bounds=Bounds(0, 1))
sel = np.where(r.x > 0.5)[0]
print("optimal", sel.tolist(), int(value[sel].sum()), int(w[sel].sum()))
order = np.argsort(-value / w); tot = wt = 0; gs = []          # greedy by value per
minute
for i in order:
    if wt + w[i] <= 60: gs.append(int(i)); wt += w[i]; tot += value[i]
print("greedy ", sorted(gs), int(tot), wt)
```

```
values  [509, 860, 215, 859, 349, 439, 762, 427, 540, 122, 703, 531]
minutes [25, 13, 16, 24, 8, 12, 8, 16, 29, 8, 14, 15]
optimal [1, 4, 6, 10, 11] 3205 58
greedy  [1, 4, 5, 6, 10] 3113 55
```

- **Answer** The best set (customers 1, 4, 6, 10, 11) is worth 3,205 rupees and uses 58 of the 60 minutes. The greedy rule "best value per minute first" gets 3,113, which is 2.9% lower.
- **Check** 860 + 349 + 762 + 703 + 531 = 3,205 and 13 + 8 + 8 + 14 + 15 = 58. Both add up and 58 ≤ 60.
- **Why it matters** Greedy is a good first answer and often fine. When the gap pays for the effort, use a solver. Never ship a heuristic without measuring its gap to a proper optimum, as we just did.

**Assignment: matching people to tasks.** Four agents, four tasks, and a cost (minutes) matrix. Each agent gets exactly one task. `scipy.optimize.linear_sum_assignment` solves it exactly.

```
from scipy.optimize import linear_sum_assignment
C = np.array([[9, 2, 7, 8], [6, 4, 3, 7], [5, 8, 1, 8], [7, 6, 9, 4]])
ri, ci = linear_sum_assignment(C); print(list(zip(ri.tolist(), ci.tolist())), int(C[ri,
ci].sum()))
```

```
[(0, 1), (1, 0), (2, 2), (3, 3)] 13
```

Agent 0 takes task 1 (cost 2), agent 1 takes task 0 (6), agent 2 takes task 2 (1), agent 3 takes task 3 (4). Total 2 + 6 + 1 + 4 = 13. Note that agent 1's cheapest task is 2 (cost 3), but giving it away lets agent 2 take it at cost 1 and the total is lower. Local choices are not global optimum.

**The picture.** For two variables, imagine a polygon on a graph where each side is a constraint. Every allowed answer lies inside. The objective is a straight line sliding across; the last point it touches is the optimum, and it is always at a *corner*. That is why LP answers are "channel full or at zero", just like (50, 40, 10).

**The wider OR toolbox (not run here, standard tools).** Network flow and transportation problems (move goods from warehouses to stores at least cost). Routing (TSP, vehicle routing VRP) with Google OR-Tools. Scheduling with constraint programming. Modelling libraries: PuLP, Pyomo, cvxpy, and commercial solvers such as Gurobi (awareness). Knapsack-like problems stay hard as they grow, so large routing uses heuristics and metaheuristics (local search, simulated annealing, genetic algorithms) that give good, not guaranteed-optimal, answers. Queueing theory (Chapter 11.6) tells you the staff level at which waiting time explodes, an optimisation constraint: keep utilisation below the point where the line starts to explode.

**Predict, then optimise.** The values in the knapsack came from models (probability × value). Errors in predictions flow into the plan: the optimiser is an "error maximiser", it loves the customers whose value was over-estimated. Mitigate with calibrated inputs, caps, and the simulation in 12.3.

**Connection to the job.** Budget allocation in marketing, route and slot planning in logistics, shift rosters, ad bidding, portfolio selection, and hospital bed planning all use this machinery. "Predict then optimise" is a very hireable skill pair.

### Applications

- Marketing: budget split with caps and minimums.
- Logistics: delivery assignment, routing, load planning.
- Operations: shift scheduling, call-centre staffing.
- Finance: portfolio and limit allocation.
- Healthcare and call centres: choosing the number of staff or beds to meet a waiting-time target at minimum cost.

| COMMON MISTAKE | Writing the objective without the real constraints (agent minutes, legal minimums, fairness limits), then being surprised that the "optimal" plan is unusable. Review every constraint with the owner and check that the solver result is feasible by hand. |
|---|---|

### Check yourself

1. What is a shadow price of 0 for a constraint? Answer: The constraint is not binding; extra of that resource gives no gain.
2. Why does an LP optimum sit at a corner? Answer: A straight-line objective over a polygon is maximised at an extreme point.
3. Greedy by value per minute gave 3,113, optimal 3,205. Is greedy wrong? Answer: Not wrong, just 2.9% below the optimum; whether that matters depends on the stakes.

## 12.3 Simulation: Monte Carlo, What-If and Sensitivity

| INTUITION | A weather service does not solve one exact equation for tomorrow; it runs the model many times with slightly different starting readings and reports how often it rains. Monte Carlo simulation does the same for business numbers. If you cannot solve a formula because inputs are uncertain, draw random inputs many times and look at the spread of outcomes. |
|---|---|

**Words you need.**

| Word | Meaning |
|---|---|
| Monte Carlo | Repeat a calculation with random inputs thousands of times |
| Scenario / what-if | One specific set of assumptions ("if the save rate is 20%") |
| Sensitivity analysis | Change one input at a time and see how much the answer moves |
| Risk | The probability and size of bad outcomes (P(loss), 5th percentile) |
| Discrete-event simulation | Simulate events (arrivals, services) in time order, as in a queue |

**Problem 12.3.1** Our campaign sends 273 offers at 100 each. We are not sure of three things: the true churner share among those sent (about 60%, Beta(60, 40)), the save rate (about 30%, Beta(3, 7)), and the value of a saved customer (about 1,400 rupees, lognormal with spread 0.3). Find the profit range.

| | |
|---|---|
| **Formula** | profit = 273 × churner_share × save_rate × value − 273 × 100 |
| **Step 1** | Draw 100,000 values for each input from its distribution |
| **Step 2** | Compute profit for each draw |
| **Step 3** | Summarise: mean, percentiles, and the chance of loss |

```
rng = np.random.default_rng(42); N = 100000; n_send, cost = 273, 100
cs = rng.beta(60, 40, N); sv = rng.beta(3, 7, N); vl = rng.lognormal(np.log(1400), 0.3, N)
profit = n_send * cs * sv * vl - n_send * cost
print("mean profit", profit.mean().round(0), "P(loss)", (profit < 0).mean().round(3))
print("5%, 50%, 95%", np.percentile(profit, [5, 50, 95]).round(0))
```

```
mean profit 44614.0 P(loss) 0.104
5%, 50%, 95% [ -7266.  36817. 123503.]
```

- **Answer** The average is 44,614 rupees, but there is a 10.4% chance of losing money, and the 90% range runs from −7,266 to +123,503. The median (36,817) is lower than the mean because a few lucky draws of value and save rate pull the mean up (a right-skewed result).
- **Check** The inputs are independent, so the mean of the product is the product of the means. The mean of a lognormal with median 1,400 and spread 0.3 is 1,400 × e^(0.3²/2) = 1,400 × 1.046 = 1,464.5. Then 273 × 0.6 × 0.3 × 1,464.5 − 27,300 = 71,967 − 27,300 = 44,667, close to the simulated 44,614. If we plug in the *medians* instead we get 38,318 (the "base" line below), about 6,000 lower, because these inputs are skewed (mean above median). So a single plugged-in number depends on which "typical" value you pick. This is a small version of the *flaw of averages*.
- **Why it matters** "Expected profit 45k" and "one chance in ten of losing" are very different messages from "profit 53k". Executives can decide how much risk they accept.

**Sensitivity: which input matters?** Fix the other two at their medians, move one input from its 10th to its 90th percentile, and record profit (a *tornado analysis*):

```
base (all medians)          38,318
churner share   10%..90%    31,334  to  45,022   (range 13,688)
save rate       10%..90%     2,455  to  84,972   (range 82,517)
saved value     10%..90%    17,370  to  69,264   (range 51,894)
```

- **Answer** The save rate dominates: moving it across its plausible range swings profit by 82,517 rupees, six times more than the churner share. Check: 84,972 − 2,455 = 82,517.
- **Why it matters** The save rate is an *assumption* (0.30). Run a small A/B test of the offer before the big rollout (Chapter 7) because that is where the money risk is. The model's AUC barely matters compared to it.

**Discrete-event simulation: a queue at a bank counter.** Chapter 11.6 gave the formula `R = S / (1 − ρ)` for the M/M/1 queue. Now simulate it with no formula at all. Each customer waits for the previous one: `wait_i = max(0, wait_{i−1} + service_{i−1} − gap_i)`, a rule called *Lindley's recursion*. Time in the bank is wait plus service. We simulate 200,000 customers, a service rate of 10 customers per hour (mean 0.1 hour, 6 minutes), at three utilisations. All times below are in hours.

```
def mm1(lam, mu, n=200000, seed=0):
    r = np.random.default_rng(seed)
    ia = r.exponential(1/lam, n); sv = r.exponential(1/mu, n)
    w = np.zeros(n)
    for i in range(1, n): w[i] = max(0, w[i-1] + sv[i-1] - ia[i])
    resp = w + sv
    return resp.mean(), np.percentile(resp, 95)
for rho in [0.5, 0.8, 0.9]:
    m, p95 = mm1(rho * 10, 10)
    print(rho, "sim mean", round(m, 3), "theory", round(0.1/(1-rho), 3), "sim p95",
round(p95, 3))
```

```
0.5 sim mean 0.202 theory 0.2 sim p95 0.609 theory p95 0.599
0.8 sim mean 0.505 theory 0.5 sim p95 1.498 theory p95 1.498
0.9 sim mean 1.019 theory 1.0 sim p95 3.005 theory p95 2.996
```

- **Answer** Simulation agrees with theory within about 2%: at 90% utilisation the mean time in the bank is about 1.0 hour and the 95th percentile about 3.0 hours, three times the mean. The theory for the p95 of an M/M/1 queue is `ln(20) × S / (1 − ρ)`, which gives 2.996 × 0.1 / 0.1 = 2.996 at ρ = 0.9.
- **Check** The same method works for a system with no formula (two counters, customers who give up, a priority lane). That is the point of simulation. Libraries such as `simpy` help (not run here).
- **Why it matters** A tail percentile is far worse than the mean: 1 customer in 20 waits about 3 hours while the average is 1. Simulation lets you test staffing and capacity plans before spending money.

**Agent-based simulation** (awareness): simulate many individual actors (customers, drivers) with simple rules and see group behaviour emerge, useful for marketplaces and traffic.

**Connection to the job.** Finance teams ask for P10/P50/P90, not a point forecast. Operations teams use simulation to test peak-season staffing. Data scientists use bootstrap and Monte Carlo to give intervals on any complicated metric.

**Applications**

- Data Science: profit intervals for campaigns, risk quantification for portfolios.
- Operations: queue and capacity what-ifs.
- Supply chain: inventory simulations with random demand and lead times.
- Hospitals and banks: testing staffing plans on simulated arrivals before changing the roster.

| COMMON MISTAKE | Choosing input distributions that look fancy but are not based on data, then trusting the output range. Garbage distributions give confident garbage. State where each distribution came from (past campaigns, an experiment, expert judgment) and show sensitivity. |
|---|---|

**Check yourself**

1. Why can a single plugged-in profit number mislead, even if it is computed carefully? Answer: It hides the spread (here a 10% chance of loss), and it depends on whether you plug in means or medians of skewed inputs.
2. In a tornado analysis, which input should you study first? Answer: The one with the widest swing (here the save rate).
3. What is the benefit of simulating a queue instead of using `S/(1−ρ)`? Answer: It works for systems with no neat formula (customers who give up, several counters, priority lanes).

## 12.4 Decisions Under Uncertainty, Bandits and Feedback Loops

| INTUITION | A restaurant owner tries three new dishes. Serving each equally for a month (an A/B test) teaches her the best dish but wastes sales on the bad ones. A smarter owner shifts more tables to the dish that looks best while still trying the others now and then. That is a *bandit* (named for slot machines, "one-armed bandits"). And once a dish is on the menu, customers' orders change what she sees next, which is a *feedback loop*. |
|---|---|

**A/B test as a decision tool.** A test result is not "significant or not". It is an input to a decision with costs. A Bayesian view makes this direct. Variant A converted 40 of 1,000; variant B 55 of 1,000. Using a flat Beta(1, 1) prior (Chapter 4), the posteriors are Beta(41, 961) and Beta(56, 946). Draw 200,000 samples from each:

```
r = np.random.default_rng(0)
pa = r.beta(1 + 40, 1 + 960, 200000); pb = r.beta(1 + 55, 1 + 945, 200000)
print("P(B>A)", (pb > pa).mean().round(3))
print("expected loss choosing B", np.maximum(pa - pb, 0).mean().round(5),
      "choosing A", np.maximum(pb - pa, 0).mean().round(5))
```

```
P(B>A) 0.941
expected loss choosing B 0.00024
choosing A 0.01519
```

- **Answer** There is a 94.1% chance B is truly better. If we choose B and are wrong, we lose on average only 0.00024 conversion rate points (0.024 percentage points); if we choose A and are wrong we lose 0.01519 (1.5 points). Choosing B has a much smaller expected loss.
- **Check** 0.01519 / 0.00024 is about 63 times. A classical two-sided test gives z = 0.015 / 0.0095 = 1.58, so p is about 0.11 and "not significant". Yet the decision can still be clear when the cost of a wrong choice is this

lopsided.

- **Why it matters** Decision-based thinking uses *expected loss*. Pair it with a guardrail (never ship if a guardrail metric drops, Chapter 7).

**Multi-armed bandits.** With three variants whose true conversion rates are 4.0%, 4.8% and 4.5%, and 20,000 visitors, we compare three policies, averaging 20 runs:

- **Random**: equal split forever.
- **A/B then pick**: split equally for 6,000 visitors, then send all to the best-looking arm.
- **Thompson sampling**: each visitor, draw a plausible rate from each arm's Beta posterior and show the arm with the highest draw. Good arms get more traffic automatically.

```
random  mean conversions 886.0  share of traffic on best arm 0.333
ab      mean conversions 926.7  share on best arm 0.700
ts      mean conversions 929.8  share on best arm 0.675
oracle (always the best arm)    960.0
```

- **Answer** Both smart policies gain about 41 to 44 extra conversions per 20,000 visitors over random (926.7 and 929.8 against 886.0). Thompson sampling is only slightly ahead of A/B-then-pick (3 conversions).
- **Check** Oracle 960 = 0.048 × 20,000. The gap to the oracle is the price of learning: 33 conversions for A/B-then-pick, 30 for Thompson, 74 for random.
- **Why it matters** Be honest: in this setup, where the arms differ by less than one percentage point, the bandit's gain over a plain A/B is small. Bandits shine when you have many arms, a short-lived decision (a headline for a day), or a high cost per wasted visitor. They are weaker when you want a clean, unbiased estimate of effect size for a report, and with delayed rewards.

**Feedback loops and selective labels.** Your model changes the data you will see next.

- **Selective labels:** a lender only sees repayment for approved applicants. We simulated 50,000 applicants where a risk score `x` drives the chance of default, and approved those with `x < 0`.

```
true bad rate, all applicants          0.169
observed bad rate, approved only       0.057
slope of bad-odds on x, all            1.225
slope of bad-odds on x, approved only  1.254
```

- **Read it** The observed bad rate (5.7%) is a third of the real one (16.9%). Anyone who sizes the whole market from approved loans under-estimates risk by a factor of three. But the *slope* (how risk grows with x) is almost the same, 1.254 against 1.225, because we selected on a variable the model sees. Selection on what you *observe* is harmless to a correctly specified model; selection on something you do *not* observe (a

loan officer's gut feeling) is what biases it. And a model has no information at all about the region it never approves.

- **Remedies:** keep a small random "exploration" group (for example approve 5% of the lowest scores under a loss budget) and weight those cases by 1/0.05; use reject-inference methods (Chapter 11.2) with stated assumptions; log every decision and score so future analysis can adjust (not run here).
- **Other loops:** a recommender shows popular items, so they get more clicks (position and popularity bias); a fraud model blocks patterns, so fraudsters change them (drift, Chapter 11.3); a churn campaign removes the churners we would learn from. Designs that help: holdout groups that never get the action, regular exploration, monitoring of input drift (PSI), and retraining on recent data.

**Connection to the job.** Pricing, ads and recommendations are run as bandits at scale. Credit, hiring and healthcare models have selective-label problems that every audit asks about. Interviewers love: "Your model's decisions change future training data. What goes wrong?"

**Applications**

- Growth teams: bandits for headline and creative testing.
- Lenders: exploration with a loss budget and reject inference.
- Recommendation: exploration slots and position-bias correction.
- Clinical: adaptive trials (with strong regulatory oversight).

| COMMON MISTAKE | Using only the observed (approved, shown, treated) data to judge a model that decides who is approved, shown or treated. You are grading the model on the cases it already liked. Keep a random holdout to measure the truth. |
|---|---|

**Check yourself**

1. In the bandit run, why does the oracle (960) beat all policies? Answer: It knows the best arm at the start; others must pay to learn which arm is best.
2. P(B>A) = 0.94, expected loss of choosing B is tiny. Should you ship B? Answer: Likely yes, if guardrail metrics are fine and a wrong choice is cheap; the decision depends on costs, not only on $p < 0.05$.
3. The approved-only bad rate is 5.7% vs a true 16.9%. What does this mean for pricing the whole portfolio? Answer: You under-price risk; the unseen rejected population is much riskier.

# Chapter recap

- A prediction needs a cost matrix to become a decision. Break-even probability = `cost of acting / gain if right` (100 / 450 = 22.2%).

- Threshold choice beat "send to all" by a wide margin (53,250 vs 16,400 rupees) with the same model. The profit curve has a flat top.
- Fixed budget means top K by score. Calibration matters because the EV formula multiplies by a probability.
- LP allocates budget to the best returns up to caps; the shadow price (1.5) tells you the value of one more unit.
- Integer programs choose who to act on. Greedy was 2.9% below the optimal knapsack (3,113 vs 3,205). Assignment solves matching exactly.
- Monte Carlo gives the profit range (mean 44,614, 10.4% chance of loss). Tornado analysis showed the save rate dominates. One plugged-in number hides the range (flaw of averages).
- Simulating an M/M/1 queue reproduces theory (mean about 1.0 hour, p95 about 3.0 hours at 90% utilisation).
- Bayesian decisions use expected loss. Bandits give modest gains in a 3-arm test (about 44 extra conversions per 20,000). Feedback loops and selective labels (5.7% observed vs 16.9% true) need exploration and holdouts.

| Idea | Formula or command | Example |
|---|---|---|
| Break-even probability | `cost / (save_rate × value)` | 100 / 450 = 0.222 |
| Cost matrix profit | `TP × gain_TP + FP × cost_FP` | 179 × 350 − 94 × 100 = 53,250 |
| LP | `linprog(c, A_ub, b_ub, bounds)` | allocation (50, 40, 10) |
| Shadow price | `res.ineqlin.marginals` | 1.5 |
| Knapsack | `milp(−value, LinearConstraint(w, 0, cap), integrality=1)` | 3,205 rupees |
| Assignment | `linear_sum_assignment(C)` | cost 13 |
| Monte Carlo | draw N inputs, compute, `np.percentile` | P(loss) = 0.104 |
| M/M/1 p95 | `ln(20) × S / (1 − ρ)` | 2.996 h at ρ = 0.9 |
| Lindley | `w_i = max(0, w_{i−1} + s_{i−1} − gap_i)` | simulated queue |
| Bayes loss | `E[max(pa − pb, 0)]` | 0.00024 |

# Practice set

**1.** (Easy) Offer cost 50, success rate 25%, value 800. Break-even probability? **Answer:** 50 / (0.25 × 800) = 50 / 200 = 0.25.

**2.** (Easy) A cost matrix has TP = +350, FP = −100, FN = 0, TN = 0. With 179 TP and 94 FP, what is the profit? **Answer:** 179 × 350 − 94 × 100 = 62,650 − 9,400 = 53,250.

**3.** (Easy) In an LP with a budget constraint, adding 1 unit of budget raises profit by 1.5. What is that number called? **Answer:** The shadow price of the budget constraint.

**4.** (Medium) What is the output? `linear_sum_assignment(np.array([[1, 5], [5, 1]]))` gives which pairs and total cost? **Answer:** Pairs (0, 0) and (1, 1), cost 1 + 1 = 2.

**5.** (Medium) A bank counter has a service time of 0.2 hours. Find the mean time in the bank at 75% utilisation and at 95%. **Answer:** 0.2 / 0.25 = 0.8 hours; 0.2 / 0.05 = 4.0 hours. A 20 point utilisation rise multiplies the time by 5.

**6.** (Medium) Find the bug: "We set the threshold to 0.5 because it is the default, and report an F1 of 0.71." **Answer:** The threshold ignores costs. Compute the cost matrix, find the break-even or the profit-maximising threshold, and report profit, not F1.

**7.** (Medium) Items have (value, weight): A (60, 10), B (100, 20), C (120, 30). Capacity 50. Greedy by value/weight takes A, B, then C does not fit (10 + 20 + 30 = 60). What is the optimal set? **Answer:** B + C = 220 with weight 50. Greedy (A + B) gives only 160 (A + B weight 30, then C does not fit). Optimal 220 beats greedy 160, so the heuristic can be badly wrong.

**8.** (Hard) A Monte Carlo of profit gives mean 50,000 but P5 = −20,000. The board wants "at least 90% chance of no loss". Does the project pass? **Answer:** Not yet. P5 below zero means at least a 5% chance of loss, and it may be more than 10%; compute P(profit < 0) directly. If it exceeds 10%, run a pilot to narrow the uncertain input.

**9.** (Hard) A lender sees a 4% default rate on approved loans and plans to approve 3 times more applicants using the same model. What is the risk? **Answer:** The approved-only rate does not describe the new, riskier applicants (selective labels). Start with a controlled expansion plus exploration, and estimate the rate with reject inference with stated assumptions.

**10.** (Interview) "You have a churn model with AUC 0.88. How do you decide who gets the retention offer?" **Answer:** Check calibration; write the cost matrix with the owner (offer cost, success rate, value); compute break-even probability; sweep thresholds for profit; apply budget and capacity limits; test the save rate by A/B; consider uplift modelling; keep a holdout; monitor.

**11.** (Interview) "What is the difference between an A/B test and a bandit? When would you use each?" **Answer:** A/B: fixed split, clean unbiased effect estimate, good for decisions that need evidence and a report. Bandit:

adapts traffic to the better arm while learning, good for many options, short-lived content, or a high cost of showing a bad option. In a 3-arm test with small differences the gain over A/B was only a few conversions in 20,000.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me how to turn a classification model into a business decision: cost matrix, expected value, break-even probability, threshold sweep, top-K under a budget, and why calibration matters. Use a small example with 10 customers and rupee amounts I can compute by hand. After each step, ask me to compute the next number and check it before moving on.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: optimisation for data scientists: linear programming, shadow prices, 0/1 knapsack, assignment problems, and why "predict then optimise" can amplify prediction errors. Give me 8 graded exercises with small numbers, no solutions at first. After I answer, show the full working, explain the corner-point idea in simple words, and show me how to check my answer with scipy.optimize.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: Monte Carlo simulation, sensitivity (tornado) analysis, the flaw of averages and discrete-event simulation of an M/M/1 queue. Walk me through building a profit simulation for a marketing campaign in Python, one small step at a time, explaining each line. Then quiz me with 6 questions about how to choose input distributions and how to report P10, P50 and P90 to a manager.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: decision-making under uncertainty: Bayesian A/B decisions with expected loss, multi-armed bandits (Thompson sampling), selective labels and feedback loops in credit and recommendation systems. Quiz me one question at a time, 10 questions from easy to hard, using small numbers. Wait for my answer, grade it, and explain the intuition in simple English.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles and I am preparing for interviews. Topic: "from prediction to decision" (thresholds from costs, budget allocation, simulation, bandits, feedback loops). Run a mock interview with 8 scenario questions, for example "your model has high AUC but the business sees no profit". After each answer of mine [WAIT FOR MY ANSWER], score it out of 5, point out what a senior data scientist would add, and then ask the next question.
```

CHAPTER 13

# COMMUNICATION, STAKEHOLDERS, ETHICS, REPRODUCIBILITY AND TAKING WORK TO PRODUCTION

*The model is finished on Friday. This chapter is about the Monday after: getting people to understand it, trust it, use it fairly, rerun it, and keep it alive.*

Many good models die in a folder called "final_v3_REAL". The reason is rarely maths. The manager did not understand the slide, the engineer could not rerun the notebook, the risk team found a fairness problem on the day of launch, or nobody noticed the model was broken for six weeks. Senior data scientists are paid largely for avoiding these outcomes. In this chapter you will learn to write an answer-first memo, explain a model with a counterfactual instead of a formula, manage a vague or oversized request, run a basic fairness and proxy check, make a project rerunnable with one command, catch training-serving skew, and write a handover and monitoring plan. We use the shop churn model from Chapters 1 and 12 ( `from ds_common import *` ). Code outputs are real runs (numpy, pandas, scikit-learn, seed 42 data). The snippets assume `d` , `X` , `y` and the cost-based threshold are built as in Chapter 12.1 (features `rec` , `logf` , `pro` , `ads` ) and that `p` holds the model scores. The FastAPI and Streamlit snippets are standard but "not run here".

## 13.1 Communicating Results: Answer First, Uncertainty Honest

**INTUITION**

You are the pilot's voice on the intercom. Passengers do not want the engine temperature log. They want: "We will land 20 minutes late, because of wind, and here is what that means for connections." Start from the decision. Then give the evidence. Then the risk. The same order works for slides, emails and interviews.

**Words you need.**

| Word | Meaning | Example |
|---|---|---|
| Answer first (BLUF) | State the recommendation in the first line | "Send the offer to 273 customers" |
| Counterfactual explanation | "If X were different, the result would change by Y" | "If she had ordered 10 days ago, risk falls from 31% to 18%" |

| Word | Meaning | Example |
|---|---|---|
| Model card | A 1 to 2 page record of what a model is for, how it was tested, and its limits | Section 13.4 template |
| Guardrail | A metric that must not get worse | Unsubscribe rate |
| Handover doc | What the next owner needs to run and fix your work | README plus runbook |

**The one-page memo.** Use this shape for any analysis. Each part has one job.

```
Title: one sentence that is the conclusion, not the topic
1. Recommendation (2 lines)   What should we do, by whom, by when?
2. Why (3 bullets)            The three facts that matter most, each with a number
3. How sure are we?           Range, assumptions, what would change my mind
4. Risks and next steps       What could go wrong, what we will measure
5. Appendix                   Method, data, links to code (readers can skip)
```

**Problem 13.1.1** Turn this raw finding into a memo opening. Raw: "Logistic regression on 1,700 customers, AUC 0.878, threshold 0.22, test profit 53,250 rupees, save rate assumed 30%, 10.4% chance of loss in simulation (Chapter 12)."

| | |
|---|---|
| **Step 1** | Find the decision: who gets the offer? |
| **Step 2** | Put one money number and one risk number in the opening line. |
| **Step 3** | Keep AUC out of the first lines. It goes in the appendix. |
| **Answer** | "Recommendation: send the retention offer to the 273 highest-risk customers (about 32% of the active base). We expect about 53,000 rupees of extra profit on the test sample. The main uncertainty is whether the offer really saves 30% of churners: if the save rate is lower, there is about a 1 in 10 chance we lose money. We propose a two-week A/B test on 20% of the list first." |
| **Check** | Every number in the paragraph appears in Chapters 12.1 and 12.3 (273 of 862 customers is 31.7%, profit 53,250, P(loss) 0.104). A reader can act on it without knowing what AUC means. |
| **Why it matters** | Executives decide in the first 30 seconds of reading. Put the decision there. |

**Explain a model with counterfactuals, not formulas.** We fit logistic regression for churn on four features (days since last order, log of order count, pro plan, ads channel) and take one customer with a risk near 31%.

```
feats = ["rec", "logf", "pro", "ads"]
mu, sd = X.mean(0), X.std(0)
lr = LogisticRegression().fit((X - mu) / sd, y)
p = lr.predict_proba((X - mu) / sd)[:, 1]
i = int(np.argmin(np.abs(p - 0.31)))
print("cust", d.customer_id[i], "p", p[i].round(3), d.loc[i, feats].to_dict())
x = X[i].copy(); x[0] -= 10                      # she ordered 10 days more recently
print("rec - 10 days:", lr.predict_proba(((x - mu) / sd)[None, :])[0, 1].round(3))
x = X[i].copy(); x[1] = np.log1p(d.f[i] + 3)       # she has 3 more orders in history
print("+3 orders   :", lr.predict_proba(((x - mu) / sd)[None, :])[0, 1].round(3))
print("odds ratio per 10 days:", np.exp(lr.coef_[0][0] / sd[0] * 10).round(3))
```

Before you run it: which change helps more, 10 days more recent, or 3 more orders?

```
cust 215 p 0.311 {'rec': 26.0, 'logf': 2.70805020110221, 'pro': 0.0, 'ads': 0.0}
rec - 10 days: 0.179
+3 orders   : 0.306
odds ratio per 10 days: 2.072
```

- **Answer** Customer 215 has been silent for 26 days with a 31.1% churn risk. If she had ordered 10 days ago (16 days of silence) the risk would be 17.9%. Three more past orders would hardly change it (30.6%).
- **Check** The logistic coefficient on days is 1.647 per standard deviation (22.61 days). Per 10 days that is `exp(1.647 / 22.61 × 10) = exp(0.7285) = 2.072`, so every 10 extra silent days doubles the odds of churn.
- **Why it matters** "Recency is what the model watches; history length hardly matters" is a sentence a marketing manager can use. "Coefficient 1.647" is not. Note: this says what *the model* uses, not that ordering earlier would *cause* lower churn (Chapter 4 and 8).

**Uncertainty, in words.** Replace "the lift is 3.2%" with: "We think the lift is between 1% and 5%. The most likely value is 3%. If we repeated this test 100 times, about 95 intervals would contain the true effect." Also say what would change your mind ("a drop in order count among ads customers would make me retest"). For a boss, three ranges are enough: pessimistic, likely, optimistic (Chapter 12.3).

**Charts.** One message per chart. The title is the message ("Customers silent for over 60 days churn far more often than active ones"). Label the axes with units. Avoid dual axes. Show the baseline. If you need a long legend, you need two charts.

**Connection to the job.** In interviews, case-study rounds and take-home tasks are graded mostly on clarity of recommendation and honesty about limits. In the job, the same skill decides if your project is funded next

quarter.

**Applications**

- Data Analytics: weekly insight memos, dashboards with annotated charts.
- Data Science: experiment reports, model cards, executive summaries.
- ML Engineering: design docs and runbooks.
- Consulting-style roles: slide decks (one message per slide).

**COMMON MISTAKE**

Opening with method ("We trained several models...") and putting the recommendation on slide 12. The reader who stops at slide 3 learned nothing. Another version: saying "the model is 88% accurate" with no baseline, which sounds good until someone asks what the simple rule scores.

**Check yourself**

1. Where does AUC belong in a business memo? Answer: In the appendix; the opening should state the decision, the money and the main risk.
2. What does an odds ratio of 2.07 per 10 days mean in plain words? Answer: Ten more silent days roughly double the odds of churn, other things equal.
3. Why add "what would change my mind"? Answer: It shows the limits of the claim and tells people what to monitor.

## 13.2 Stakeholder Management and Working in Small Iterations

**INTUITION**

A restaurant customer who says "something light" is not giving a recipe. A good waiter asks two questions, brings a small sample, and adjusts. Stakeholders are the same: the first request is a starting point, not a specification. Think of a builder: "make the kitchen bigger" becomes "add 2 metres to the east wall, budget 4 lakh rupees, finish before Diwali" only after a conversation.

**Who they are.**

| Stakeholder | Cares about | Typical worry | What helps |
|---|---|---|---|
| Business owner (marketing, risk head) | Revenue, cost, deadlines | "Will this move my KPI?" | Money numbers, a pilot |

| Stakeholder | Cares about | Typical worry | What helps |
|---|---|---|---|
| Product manager | User impact, roadmap | "How will users see it?" | Clear scope, guardrails |
| Engineering | Latency, maintainability | "Who supports this at 2 am?" | Interfaces, tests, runbook |
| Data engineering | Data contracts, pipelines | "Why do you need a new table?" | Early requirements, volumes |
| Legal and compliance | Privacy, fairness, audit | "Can we explain a rejection?" | Model card, explainability |
| Leadership | Strategy, risk | "Do I trust the number?" | Honest ranges, one clear ask |

**Clarify the ask.** Five questions that rescue most projects:

1. What decision will this change, and who makes it?
2. What do you do today, and how good is it?
3. What would a successful result look like in rupees, hours or risk?
4. What data exists, and who owns it?
5. By when is the answer useful, and what is the smallest useful version?

**Problem 13.2.1** A manager asks: "Build a model to predict customer churn, 3 months, full pipeline." You estimate: data and framing 2 weeks, baseline and first model 1 week, validation and thresholding 1 week, deployment 4 weeks, monitoring 1 week. Propose a smaller plan that delivers value in 4 weeks.

| | |
|---|---|
| **Step 1** | Find the smallest piece that changes a decision: a weekly list of high-risk customers in a spreadsheet needs no deployment. |
| **Step 2** | Plan: week 1 framing and data; week 2 baseline rule (silent over 60 days) and first model; week 3 threshold and cost analysis (Chapter 12); week 4 pilot list to marketing, with a holdout of 20%. |
| **Step 3** | Move deployment (4 weeks) and monitoring (1 week) to phase 2, to be funded if the pilot shows lift. |
| **Compute** | Original: 2 + 1 + 1 + 4 + 1 = 9 weeks of work, results at the end. New: 4 weeks to first value. Weeks spent before the first business feedback fall from 9 to 4 (a cut of 55.6%). |
| **Answer** | Offer a 4-week pilot with a manually delivered list and a holdout, and plan production only after the pilot works. |
| **Check** | If the pilot fails (no lift), we have spent 4 weeks instead of 9, and learned that early. |
| **Why it matters** | "Baseline, improve, ship, measure" beats a three-month black box. Stakeholders see progress, and wrong assumptions die early. |

**Saying no without saying no.** Use "not yet, here is what we need".

```
Request: "Predict which employees will resign, by name."
Reply:   "I can build a team-level attrition forecast now. Named predictions need
         HR policy, consent and a plan for how managers act on them. If we agree
         on those, I can scope phase 2."
```

**Managing expectations.** Share the baseline early ("a simple rule gets 0.88 AUC; the model will probably add a little more"). Share the effort and risk in ranges. Never promise an accuracy number before you have seen the data.

**Connection to the job.** The usual failure of junior data scientists is silent work followed by a big reveal. Seniors give a weekly 5-line update: done, next, blocked, risks, decisions needed.

**Applications**

- Product analytics: scoping experiments with PMs.
- ML Engineering: agreeing on interfaces and SLAs with consumers.
- Risk and compliance: early review meetings avoid late vetoes.

| COMMON MISTAKE | Saying "yes" to everything, then delivering late with a weak result. Equally bad: saying "no" without an alternative. Offer a smaller deliverable and a clear condition for the bigger one. |
|---|---|

**Check yourself**

1. Name the first question to ask a vague request. Answer: What decision will it change, and who makes it?
2. Why plan a pilot with a holdout? Answer: It measures the true effect and limits the cost of being wrong.
3. What is wrong with "I will tell you the accuracy in advance"? Answer: Accuracy depends on data you have not examined; promise a process and a baseline instead.

## 13.3 Ethics and Fairness in Practice

| INTUITION | A bridge engineer does not only ask "will it stand?" but "who walks on it, and what if it fails?" A model that decides who gets a loan, a job interview or a medical test is a bridge for people. Ethics here is a checklist you run before launch, not a lecture. |
|---|---|

**Words you need.**

| Word | Meaning | Example |
|---|---|---|
| Protected attribute | A trait where unequal treatment is harmful or illegal (gender, caste, religion, disability, in India also region in some settings) | Caste, gender |
| Proxy variable | An ordinary feature that carries a protected attribute | Pincode standing in for religion or caste |
| Selection rate | Share of a group receiving the favourable decision | 30% of group B approved |
| Disparate impact ratio | Lower group selection rate / higher group selection rate | 0.30 / 0.40 = 0.75 |
| Equal opportunity | Equal true positive rate across groups | TPR 79% vs 76% |
| Appeal / override | A human way to challenge a decision | Manual review of a rejection |

**Four-fifths rule of thumb (80% rule).** In US hiring law a selection-rate ratio below 0.8 is treated as a warning sign. It is a screening heuristic, not a legal test in India, and a pass does not prove fairness. Many fairness definitions exist (equal selection rate, equal opportunity, calibration within groups), and they cannot all hold together when base rates differ. Pick the one that matches the harm, and write down why.

**A fairness check on the churn offer.** In our synthetic shop, we create a group label: group B = customers in Pune and Chennai (510 of 2,562), group A = everyone else. The model never sees the group or city. We compute rates at the cost-based threshold 0.222 (Chapter 12).

```
d["group"] = np.where(d.city.isin(["Pune", "Chennai"]), "B", "A")
s = p >= 0.222
for gr in ["A", "B"]:
    m = (d.group == gr).values
    print(gr, "n", m.sum(), "selection rate", round(s[m].mean(), 3),
          "TPR", round(s[m & (y == 1)].mean(), 3), "FPR", round(s[m & (y == 0)].mean(),
3))
print("churn rate by group", d.groupby("group").churn.mean().round(3).to_dict())
```

```
A n 2052 selection rate 0.305 TPR 0.793 FPR 0.143
B n 510 selection rate 0.324 TPR 0.757 FPR 0.159
churn rate by group {'A': 0.249, 'B': 0.275}
```

**Problem 13.3.1** Compute the disparate impact ratio and the TPR gap from the table, and decide whether to worry.

| | |
|---|---|
| **Formula** | ratio = lower selection rate / higher selection rate |
| **Plug in** | 0.305 / 0.324 |
| **Compute** | 0.305 / 0.324 = 0.941. TPR gap = 0.793 − 0.757 = 0.036 |
| **Answer** | The ratio of 0.94 is above 0.8, and the TPR gap is 3.6 points. For a discount offer (a benefit with small harm) this is acceptable, and we document it. Group B has slightly higher base churn (27.5% vs 24.9%), which explains why they are selected a bit more often. |
| **Check** | If the offer were a loan approval and ratios were 0.30 and 0.40, then 0.30 / 0.40 = 0.75, below 0.8, and we would investigate. |
| **Why it matters** | The same number is a different level of alarm depending on the harm. Rate the harm first (a discount versus a rejected loan). |

**Proxy check.** Even if you drop a sensitive attribute, other features may rebuild it. Test: train a model to predict the group from your features; if it does well, you have proxies.

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import roc_auc_score
gy = (d.group == "B").astype(int).values
pp = cross_val_predict(LogisticRegression(), (X - mu) / sd, gy, cv=5,
method="predict_proba")[:, 1]
print("proxy AUC, model features -> group:", round(roc_auc_score(gy, pp), 3))
C = pd.get_dummies(d.city).values.astype(float)
pp2 = cross_val_predict(LogisticRegression(), np.c_[(X - mu) / sd, C], gy, cv=5,
method="predict_proba")[:, 1]
print("proxy AUC, with city added:", round(roc_auc_score(gy, pp2), 3))
```

```
proxy AUC, model features -> group: 0.485
proxy AUC, with city added: 1.0
```

- **Answer** Our four features carry no information about the group (AUC 0.485 is about a coin flip), but adding `city` gives AUC 1.0 because the group was built from city. In real data the proxy is rarely that perfect, but pincode, college name, device price and first name often give 0.7 or more.
- **Check** AUC 0.5 means no signal. 1.0 means perfect prediction. We designed this case, so 1.0 is expected; the lesson is the test, not the number.
- **Why it matters** "We do not use caste or religion" is not a defence if pincode does the job.

**The ethics checklist before launch.**

1. Who could be harmed, and how badly? (Rejected loan, wrongly flagged fraud, a missed diagnosis.)
2. Which groups are under-represented in the training data? Check the metrics per group, with sample sizes and intervals.
3. What does a mistake cost, in each direction, and for whom?
4. Can the person affected get an explanation, an appeal and a human override?
5. Are we using only the data we need, with consent for this purpose? (Chapter 2.4: DPDP Act and GDPR basics.)
6. Is there a proxy for a sensitive attribute? (Run the test above.)
7. How will we notice harm after launch? (Monitoring by group.)
8. Who signs off, and where is the decision recorded?

**Responsible use.** Scraping, face recognition, emotion detection and surveillance applications need explicit legal and ethical review before any work starts. Do not collect personal data you cannot protect or justify.

**Connection to the job.** Banks, insurers, health and HR teams have review boards that ask these exact questions. In India, RBI expectations on model risk and fair practice, and the DPDP Act on personal data, set the frame; exact rules change, so read the current text for your sector.

**Applications**

- Lending: adverse-action reasons and group metrics.
- Hiring and HR analytics: screening tools need strict review.
- Healthcare: performance by age, sex and region.
- Ads and recommendations: avoid exploiting vulnerable users.

| COMMON MISTAKE | Treating fairness as a number that is computed once. Fairness needs a defined harm, a chosen metric, per-group checks with sample sizes, and ongoing monitoring. A model that passes on average can fail on a small group where the sample is tiny, so show intervals. |
|---|---|

**Check yourself**

1. Selection rates of 0.45 and 0.30: the ratio? Answer: 0.30 / 0.45 = 0.667, below 0.8, so investigate.
2. A model does not use gender, but predicts gender with AUC 0.85 from its inputs. What does that show? Answer: The inputs contain proxies for gender; the model can learn its effects indirectly.
3. Why include an appeal process? Answer: Models make mistakes; people need a way to correct decisions that affect them.

## 13.4 Reproducibility: One Command, Same Answer

| INTUITION | A good recipe card can be cooked by a friend next month in a different kitchen and give the same dish, because the ingredients, the quantities, the oven temperature and the timing are all written down. A notebook that runs only on your laptop, on today's data, with a lucky seed, is the opposite. |
|---|---|

**The five pillars.**

| Pillar | What to do | Tool or command |
|---|---|---|
| Code | Version control; small commits; no notebooks with hidden state | `git` |
| Environment | Pin library versions | `pip freeze > requirements.txt` or `uv` or `conda env export` |
| Seeds | Set random seeds everywhere | `np.random.default_rng(42)`, `random_state=42` |

| Pillar | What to do | Tool or command |
|---|---|---|
| Data | Snapshot it or record its fingerprint and query | Parquet snapshot plus a hash |
| Run | One command from raw data to report | `make report` or `python run.py` |

**Seeds.** Randomness in training, splitting and sampling must be repeatable:

```
def run(seed):
    r = np.random.default_rng(seed)
    return r.normal(size=5).round(3).tolist()
print(run(1)); print(run(1)); print(run(2))
```

```
[0.346, 0.822, 0.33, -1.303, 0.905]
[0.346, 0.822, 0.33, -1.303, 0.905]
[0.189, -0.523, -0.413, -2.441, 1.8]
```

Same seed, same numbers. A different seed gives a different result, which is why you should also report the spread over several seeds for anything important (Chapter 4.4).

**Data fingerprint.** Store a hash of the exact training table next to every model, so you can tell later if the data changed:

```
import hashlib
h = hashlib.sha256(pd.util.hash_pandas_object(d[["customer_id", "churn"]],
index=False).values.tobytes()).hexdigest()[:16]
print("data fingerprint", h)
```

```
data fingerprint caebcf2047b44370
```

(The hash depends on the pandas version's hashing method; record the pandas version too.)

**Problem 13.4.1** Design the folder and the one-command rerun for the churn project.

| | |
|---|---|
| **Step 1** | Structure the folder so every file has one job. |
| **Step 2** | Write a `Makefile` so a newcomer needs only `make report`. |
| **Step 3** | Write a short README on how to set up and run. |

```
churn_project/
  README.md             what, why, how to run, owner, last run date
  requirements.txt      pinned versions (pip freeze)
  Makefile              make data / make train / make report
  config.yaml           dates, thresholds, costs (no magic numbers in code)
  src/
    data.py             builds the point-in-time table (Chapter 2)
    features.py         one definition, used for training AND serving
    train.py            fits, saves model + metrics + data fingerprint
    evaluate.py         cost-based evaluation and fairness check
  tests/                small tests (no leakage, label window, schema)
  reports/              memo, model card
  models/               saved models, with run id
```

```
report: train
        python src/evaluate.py --config config.yaml
train: data
        python src/train.py --config config.yaml --seed 42
data:
        python src/data.py --config config.yaml
```

- **Answer** A colleague runs `pip install -r requirements.txt` and `make report`; they get the same metrics and memo. (Makefile not run here: the commands are standard but the scripts are a template.)
- **Check** Delete `models/` and `reports/`, run `make report`, and compare the printed metrics with the saved ones. If they differ, something is not pinned.
- **Why it matters** The day an auditor, a manager or you in 6 months ask "how did you get this number?", the answer is one command.

**A model card.** One to two pages, kept with the model:

```
Model card: churn_60d v1.2
Purpose        Rank active customers for retention offers. Not for pricing or credit.
Data           Orders Jan 2023 to Dec 2024; label = no order in the next 60 days; run id,
fingerprint
Performance    AUC 0.878 (test); profit 53,250 at threshold 0.222 (cost matrix in config)
Fairness       Selection ratio by city group 0.94; TPR gap 3.6 points
Limits         Not tested on new customers (< 90 days); needs retraining if offers change
Monitoring     Score PSI weekly, churn rate by decile monthly, owner: ds-team
Owner / date   name, 2025-01-15
```

**Notebooks.** Notebooks are excellent for exploring. Before sharing: restart the kernel and run all cells in order; move reusable code to `src/`; remove paths such as `C:/Users/me/`; never store secrets in a notebook (use environment variables).

**Connection to the job.** Reproducibility is what lets another person take over your project, an auditor review it, or MLOps automate it (Chapter 7 of the MLOps material).

**Applications**

- Regulated domains (banking, health): audit trails of data and code versions.
- Research-style roles: papers and experiments others must reproduce.
- Teams: onboarding in an hour instead of a week.

| COMMON MISTAKE | Pinning nothing: "scikit-learn" in requirements with no version, so six months later a new release changes a default and your numbers move. Another: saving a model without the data and code version that made it. |
|---|---|

**Check yourself**

1. Why store a data fingerprint? Answer: To prove later that a model was trained on exactly this data, and to detect silent changes.
2. What is the risk of running a notebook out of order? Answer: Hidden state: results depend on cells run earlier, so a fresh run gives different results.
3. Where do thresholds and costs belong? Answer: In a config file under version control, not hard-coded in several places.

## 13.5 Prototype to Production, Monitoring and Handover

| INTUITION | A new bus route that works on paper is not yet a bus service. Production is real passengers, odd inputs and breakdowns at 3 am. A prototype shows an idea works; production keeps it working. The gap between them is mostly boring things: same features in both places, monitoring, rollback and an owner. |
|---|---|

**Words you need.**

| Word | Meaning |
|---|---|
| Prototype | A demo that shows value (Streamlit app, notebook, spreadsheet list) |
| Batch scoring | Score many rows on a schedule (nightly) |
| Online / real-time scoring | Score one request in milliseconds through an API |

| Word | Meaning |
|---|---|
| Training-serving skew | The feature is computed differently in training and serving |
| Drift | Inputs or outcomes change over time |
| Runbook | Steps to follow when the model misbehaves |

**Pick the simplest serving mode.** If the business decision happens weekly (retention list), a batch job and a CSV is production. Real-time APIs cost far more to build and run. Choose online only when the decision must happen during a user request (fraud at payment, recommendations on a page).

**A prototype in 20 lines (not run here).**

```
# app.py   run with:  streamlit run app.py       (standard usage, not run here)
import streamlit as st, joblib, numpy as np
model = joblib.load("models/churn_v1.joblib")
st.title("Churn risk")
rec = st.slider("Days since last order", 0, 90, 26)
orders = st.number_input("Past orders", 1, 200, 14)
pro = st.checkbox("Pro plan"); ads = st.checkbox("Acquired via ads")
p = model.predict_proba([[rec, np.log1p(orders), int(pro), int(ads)]])[0, 1]
st.metric("Churn probability", f"{p:.1%}")
```

```
# api.py   run with:  uvicorn api:app           (standard usage, not run here)
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, numpy as np
app = FastAPI(); model = joblib.load("models/churn_v1.joblib")
class Customer(BaseModel):
    rec: float; n_orders: int; pro: int; ads: int
@app.post("/score")
def score(c: Customer):
    return {"p_churn": float(model.predict_proba([[c.rec, np.log1p(c.n_orders), c.pro,
c.ads]])[0, 1])}
```

A prototype app helps stakeholders play with the model and find odd behaviour before launch. Handing over to ML engineering means giving them the saved model, the feature code, the model card, test cases, expected latency and the monitoring plan.

**Training-serving skew: a real failure.** The most common silent bug: the feature `rec` (days since last order) is computed in training relative to the prediction date T = 2025-01-01. A serving script computes it relative to "today" (the last data date, 30 April 2025).

```
d["rec_bad"] = (END_DATE - d["last"]).dt.days            # serving bug: wrong reference
date
Xs = d[feats].copy(); Xs["rec"] = d.rec_bad
ps = lr.predict_proba((Xs.values - mu) / sd)[:, 1]
print("mean days since last order:", d.rec.mean().round(1), "vs",
d.rec_bad.mean().round(1))
print("mean risk:", p.mean().round(3), "vs", ps.mean().round(3))
print("share flagged:", (p >= .222).mean().round(3), "vs", (ps >= .222).mean().round(3))
```

```
mean days since last order: 19.4 vs 138.4
mean risk: 0.254 vs 0.998
share flagged: 0.309 vs 1.0
```

- **Answer** With the wrong reference date the model sees every customer as silent for 138 days, assigns a 99.8% churn risk to everyone, and flags 100% of customers. No error message appears. The code ran perfectly.
- **Check** 138.4 − 19.4 = 119.0 days, exactly the gap between 1 January and 30 April 2025 (31 + 28 + 31 + 29 = 119). The bug adds the same constant to every customer.
- **Why it matters** This is why a feature must have one definition, in one place ( `features.py` ), used by both training and serving, and why monitoring must watch input distributions, not only errors.

**What to monitor.** Three layers, from fast to slow:

| Layer | Examples | Alert when |
|---|---|---|
| Service health | Response time of the scoring service (95th percentile), error rate, requests per minute, null inputs | An agreed service target is breached |
| Data and score drift | Input means, missing share, PSI of the score (Chapter 11.2) | PSI above 0.25, or feature missing share jumps |
| Outcome quality | Realised churn by score decile, profit vs holdout, per-group metrics | Quality drops once labels arrive (60 days later here) |

Also keep a **holdout** group that never receives the action, so you can measure true impact, not just model accuracy. Have a **rollback**: the previous model, or a simple rule, one command away. Define **retraining triggers** (PSI, calendar, drop in performance) and an **owner** who reads the alerts.

**Problem 13.5.1** Weekly score PSI is 0.04, 0.06, 0.08, 0.31. Which week needs action, and what is your first move?

| | |
|---|---|
| **Rule** | Below 0.1 stable; 0.1 to 0.25 watch; above 0.25 significant. |
| **Compute** | 0.04, 0.06, 0.08 are stable. 0.31 exceeds 0.25. |
| **Answer** | Week 4 needs action. First, check data pipelines and feature definitions (is it skew or a missing column?), then look at which features moved (PSI per feature), then decide on a recalibration or a retrain. Do not retrain blindly: a data bug retrained into the model makes the bug permanent. |
| **Check** | The trend 0.04 to 0.08 rising slowly might deserve a note even before the jump. |
| **Why it matters** | Alerts without a defined response become noise. |

**Handover checklist.** README with purpose and owner; how to run; model card; data dictionary; list of features with definitions; tests; monitoring dashboard link; runbook ("if PSI is high, do this"); known limits; contact.

**Connection to the job.** In many Indian GCCs and product companies, "data scientist" means you own the model after launch, at least in a review role. ML engineers and MLOps (Chapter 7 in the MLOps material) own the platform. Showing you understand their concerns is a hiring signal.

### Applications

- Retail and e-commerce: nightly batch scores to CRM.
- Payments: real-time scoring under a latency budget.
- Healthcare: decision support with human sign-off.
- Any team: shadow mode first (score but do not act), then pilot, then full rollout.

| | |
|---|---|
| **COMMON MISTAKE** | Launching with no way to tell if the model is working. Labels arrive late, so people assume "no news is good news". Monitor inputs and scores immediately; monitor outcomes as soon as labels arrive. |

### Check yourself

1. What is training-serving skew? Answer: A feature is computed differently when training than when serving, so the model sees data that does not match what it learned from.
2. Why start in shadow mode? Answer: You compare model decisions with real outcomes and with the current process without any risk.

3. When is a batch job better than an API? Answer: When decisions are made on a schedule (weekly list), because it is simpler, cheaper and easier to test.

## Chapter recap

- Answer first: recommendation, money number, main risk, then evidence. AUC goes in the appendix.
- Explain a model with counterfactuals: 10 fewer silent days moved one customer from 31.1% to 17.9% risk; odds double per 10 silent days (2.07).
- Clarify the ask with five questions; deliver small pilots with a holdout (9 weeks to first feedback became 4).
- Ethics: define the harm, check per-group selection rate and TPR (ratio 0.94 here), test for proxies (AUC 0.485 without city, 1.0 with city), keep an appeal path.
- Reproducibility: git, pinned environment, seeds, data fingerprint, one command (`make report`), model card.
- Choose the simplest serving mode; use one feature definition for training and serving.
- A single wrong reference date made every customer look silent: mean risk 0.254 became 0.998 with no error message.
- Monitor service health, input and score drift (PSI), and outcomes; keep a holdout, a rollback, retraining triggers and an owner.

| Pattern | Rule or command | Example |
|---|---|---|
| Memo order | Recommendation, why, how sure, risks, appendix | One page |
| Odds ratio per step | `exp(coef / sd × step)` | `exp(1.647 / 22.61 × 10) = 2.07` |
| Disparate impact | `lower rate / higher rate`, warn below 0.8 | 0.305 / 0.324 = 0.94 |
| Proxy test | Predict the group from the features, AUC near 0.5 is good | 0.485 |
| Seeds | `np.random.default_rng(42)`, `random_state=42` | Same output twice |
| Environment | `pip freeze > requirements.txt` | Pinned versions |
| Data fingerprint | `hashlib.sha256(...)` | `caebcf2047b44370` |
| PSI action | below 0.1 ok, 0.1 to 0.25 watch, above 0.25 act | 0.31 triggers review |

| Pattern | Rule or command | Example |
|---|---|---|
| Serve app | `streamlit run app.py`, `uvicorn api:app` | Not run here |

## Practice set

**1.** (Easy) Rewrite this as an answer-first sentence: "We analysed 12 months of data and built three models; XGBoost had AUC 0.91; the best threshold is 0.30." Add what is missing. **Answer:** Start with the decision and money: "Send the offer to customers with risk above 30%; we expect X rupees profit, assuming Y save rate." AUC moves to the appendix. What is missing is the money, the assumption and the risk.

**2.** (Easy) Selection rates are 0.50 for group A and 0.35 for group B. Disparate impact ratio and verdict? **Answer:** 0.35 / 0.50 = 0.70, below 0.8; investigate.

**3.** (Easy) What is the odds ratio for 20 silent days if it is 2.07 per 10 days? **Answer:** $2.07^2 = 4.28$.

**4.** (Medium) What is wrong with this requirements file? `pandas` / `scikit-learn` / `numpy` **Answer:** No versions. A future install can get different versions and change results. Pin with `pip freeze`.

**5.** (Medium) Find the bug: training computes `days_since_signup = (T - signup).days` with T = 2025-01-01. The API computes `(datetime.now() - signup).days`. **Answer:** Training-serving skew. The API uses "now", so the feature keeps growing after training. Fix: pass the reference date explicitly, or share one function, and add a test on the distribution of the feature in production.

**6.** (Medium) A model is said to be "fair because it does not use religion". Give two ways to test the claim. **Answer:** (1) Train a model to predict religion from the inputs and check the AUC; (2) compare selection rate, TPR and FPR across groups using a held-out set that has the attribute.

**7.** (Medium) Project plan: 12 weeks, first feedback at week 12. Rewrite it with the first business feedback at week 4 and say what you cut. **Answer:** Weeks 1 to 4: framing, baseline rule, first model, list to the business with a holdout. Cut or defer: deployment, API, dashboards and fine-tuning. Phase 2 only if the pilot shows lift.

**8.** (Medium) Weekly PSI of a score is 0.02, 0.03, 0.12. What do you do? **Answer:** Watch zone (0.1 to 0.25): look at PSI per feature, check data changes (launches, pipeline edits), and watch next week; no retrain yet.

**9.** (Hard) The model passes fairness overall, but group C has only 40 customers. What do you report? **Answer:** Show the group's metrics with a confidence interval (with n = 40 the interval is wide) and say that the evidence is weak. Collect more data or monitor this group closely after launch instead of declaring it fair.

**10.** (Interview) "You built a great model and the business does not use it. What do you do?" **Answer:** Find the reason: unclear decision, no trust, wrong format, no capacity to act. Talk to users, show a counterfactual

explanation and a pilot result, fit the output into their workflow (a weekly list, not an API), define the metric of success together, and rerun a small pilot.

**11.** (Interview) "How do you make sure your analysis can be reproduced by a teammate?" **Answer:** Version control, a pinned environment file, fixed seeds, a recorded data snapshot or query and fingerprint, config for parameters, one command to rerun, a README, tests, and a model card.

**12.** (Interview) "Walk me through taking a churn model to production." **Answer:** Decide batch or online from the decision frequency; one feature definition shared by training and serving; test cases; shadow mode, then a pilot with a holdout; monitoring of service health, drift (PSI) and outcomes; rollback and a retraining trigger; handover docs (README, model card, runbook) and a named owner.

## Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Teach me how to communicate data science results to non-technical managers: the answer-first memo, explaining a model with counterfactuals, communicating uncertainty, and chart titles that state the message. Use this finding as the running example: "churn model AUC 0.88, 273 customers flagged, profit about 53,000 rupees, 10% chance of loss". Rewrite it with me step by step, then ask me to rewrite a second example myself and correct it.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: stakeholder management for data scientists: clarifying a vague request, scoping a small pilot, saying "not yet, here is what we need", and giving weekly updates. Role-play a manager who asks vague things (for example "build an AI to reduce churn in 3 months"). I will answer as the data scientist. After each reply, give feedback and score me out of 5. Run 6 rounds, getting harder each round.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: ethics and fairness in applied machine learning: protected attributes, proxy variables, selection rate, disparate impact ratio (the 80% rule), equal opportunity, appeals and overrides, and India's DPDP Act at a basic level. Give me 8 graded exercises with small numeric tables and no solutions. After I answer, grade me, show every calculation, and tell me which real-world harm each case represents.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: reproducibility for data science projects: git, pinned environments, random seeds, data snapshots and hashes, config files, Makefile or one-command rerun, notebooks to scripts, and model cards. Review my project structure and README below and tell me what a teammate could not reproduce, in order of importance. [PASTE HERE MY FOLDER LISTING, README AND requirements.txt]. Then show me a better version.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, and I want to understand how real systems behave after launch. Topic: taking a model from prototype to production: batch vs online scoring, training-serving skew, FastAPI and Streamlit prototypes, shadow mode, monitoring (service health, PSI drift, outcome quality), holdouts, rollback, retraining triggers and handover docs. Run a mock interview of 8 questions, one at a time, for a Data Scientist role at a product company. Score each answer out of 5 and show a model answer.
```

CHAPTER 14

# CAPSTONE PROJECTS AND THE DATA SCIENCE INTERVIEW BANK

*Eight portfolio projects with measurable acceptance criteria, then the questions interviewers actually ask, with worked answers.*

Reading about data science does not make you hireable. A finished project does: one that a stranger can open, rerun, understand in five minutes, and trust. This chapter gives you eight capstone briefs. Each brief says what business question to answer, what data to use, what to deliver, and, most important, the **acceptance criteria**: checks that are either passed or failed, so you know when you are done. After the projects comes an interview bank. For each common question you get the key points of a strong answer and, where numbers are involved, a worked and verified calculation. By the end you will have a plan for a portfolio of three to four finished projects and a set of answers you can say aloud.

## 14.1 How to Run a Capstone: Definition of Done and the Baseline Gate

**INTUITION**

A building inspection is "done" when the report answers: is it safe, what must be repaired, what does it cost? Not when the inspector has walked through every room. A data science project is the same. It is done when the decision is answered, the result beats the simple alternative, and someone else can rerun it.

**Words you need.**

| Word | Meaning | Example |
|---|---|---|
| Brief | One page that states the problem, data, metric, baseline, risks (Chapter 1) | Capstone 3 brief |
| Acceptance criterion | A pass/fail check written before you start | "Model AUC beats recency rule with CI above 0" |
| Baseline gate | The rule that your model must beat the simple baseline, or you ship the baseline | Recency rule AUC |
| Deliverable | A file someone else will read or run | Memo, notebook, model card |

| Word | Meaning | Example |
|---|---|---|
| Stretch goal | Extra credit after all acceptance criteria pass | A Streamlit demo |

**The standard folder and the five deliverables.** Every capstone uses the folder from Chapter 13.4 (`README`, `requirements.txt`, `Makefile`, `config`, `src/`, `tests/`, `reports/`). Five deliverables are common to all eight projects:

1. A one-page **brief** written first (problem, decision, metric, baseline, risks, timeline).
2. A **memo** with the answer first (Chapter 13.1).
3. A **rerunnable pipeline** (one command; seeds, pinned environment).
4. A **model card or analysis card** with limits and monitoring plan.
5. A **5-minute talk** without formulas (record yourself).

**The baseline gate, run for real.** Before you call any model a success, ask whether it beats the simple rule. We test this on the shop churn data (Chapter 1; `from ds_common import *`). The baseline is "rank by days since last order". The model is logistic regression on four features. Same split of 1,700 train and 862 test customers, AUC on the test part, and a bootstrap of the difference (1,000 resamples).

```
from ds_common import *
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
customers, orders = make_data()
d = make_label_table(customers, orders)
past = orders[orders.order_ts < PRED_DATE]
g = past.groupby("customer_id").agg(last=("order_ts", "max"), f=("order_id", "count"))
d = d.merge(g, left_on="customer_id", right_index=True)
d["rec"] = (PRED_DATE - d["last"]).dt.days; d["logf"] = np.log1p(d.f)
d["pro"] = (d.plan == "pro").astype(int); d["ads"] = (d.channel == "ads").astype(int)
idx = np.random.default_rng(0).permutation(len(d)); tr, te = idx[:1700], idx[1700:]
y = d.churn.values; X = d[["rec", "logf", "pro", "ads"]].values
mu, sd = X[tr].mean(0), X[tr].std(0)
lr = LogisticRegression().fit((X[tr] - mu) / sd, y[tr])
pt = lr.predict_proba((X[te] - mu) / sd)[:, 1]; rt = d.rec.values[te]; yt = y[te]
base, model = roc_auc_score(yt, rt), roc_auc_score(yt, pt)
print("baseline recency AUC", round(base, 3), "model", round(model, 3), "gain",
round(model - base, 3))
rng = np.random.default_rng(1); gains = []
for _ in range(1000):
    i = rng.integers(0, len(te), len(te))
    gains.append(roc_auc_score(yt[i], pt[i]) - roc_auc_score(yt[i], rt[i]))
print("gain 95% CI", np.percentile(gains, [2.5, 97.5]).round(3))
```

Before you run it: do you expect the four-feature model to beat the one-line rule?

```
baseline recency AUC 0.884 model 0.878 gain −0.006
gain 95% CI [−0.016  0.005]
```

- **Answer** The model does *not* beat the recency rule. The gain is −0.006 and the interval, −0.016 to +0.005, contains zero. The honest conclusion is "no evidence the model improves ranking".
- **Check** Chapter 1 reported the same pattern: a one-line rule almost matches ML on this data. Here we measured it with an interval.
- **Why it matters** A capstone whose acceptance criterion is "beats the baseline" may fail it, and that is a valid, publishable result. Reporting "I tried, here is the interval, ship the rule" shows more judgment than a model that was never compared. (The model can still be useful for other reasons: calibrated probabilities for the cost analysis of Chapter 12.)

**Problem 14.1.1** Write three acceptance criteria for a churn capstone, so that each is a pass/fail check.

| | |
|---|---|
| **Step 1** | Pick a measurable claim about performance: "On the out-of-time test set, AUC is reported with a 95% bootstrap interval, compared with the recency baseline." |
| **Step 2** | Pick a measurable claim about the decision: "The memo gives profit at the cost-based threshold, with a Monte Carlo range and P(loss)." |
| **Step 3** | Pick a measurable claim about delivery: "Deleting `models/` and `reports/` and running `make report` reproduces the numbers within rounding." |
| **Answer** | Criteria: (1) AUC with interval against baseline; (2) profit with range and P(loss); (3) one-command reproduction. |
| **Check** | Each can be answered yes or no by a reviewer in two minutes. "The model is good" cannot. |
| **Why it matters** | Writing criteria before coding stops you from moving the goalposts after you see the results. |

**Time budget.** Each capstone is about 15 to 25 study hours, roughly 3 to 4 weeks at one hour per day. Do three to four properly instead of eight badly. A good order for most learners: 3, 2, 7, then the rest, choosing the domain you want to work in.

**Connection to the job.** Hiring managers open the README, the memo and the first notebook, in that order. Many reject a repository within a minute if there is no README, no results table, or no clear question.

**Applications**

- Job search: portfolio pieces for data science, analytics and ML roles.

- Interview prep: each project becomes a 3-minute story (problem, approach, result, limit).
- Team work: the same folder structure is used in real projects.

| COMMON MISTAKE | Starting with a Kaggle dataset and the sentence "predict the target". Without a decision, a cost and a baseline, the project cannot be judged. Start from the brief. |
|---|---|

**Check yourself**

1. What do you do if your model does not beat the baseline? Answer: Report it honestly with the interval, and recommend the simpler baseline or find a new signal.
2. Why write acceptance criteria before the work? Answer: So the result cannot quietly redefine success.
3. How many capstones should you finish? Answer: Three or four, done properly, beat eight half-done.

## 14.2 Capstones 1 to 4: Analysis, Experiment, Churn Decision, Forecast

| INTUITION | These four are the "core" projects: understand data and write a memo, design and judge an experiment, turn a prediction into a decision, and forecast with honest backtesting. If you finish only these four, you cover most of what a data science interview tests. |
|---|---|

Each brief below has the same shape: *Question, Data, Deliverables, Acceptance criteria, Stretch, Pitfalls, Chapters used.*

### Capstone 1: EDA and business recommendations on a messy dataset

- **Question** "What should the business change, based on this data?" Choose a messy real dataset (public e-commerce, transactions, ride or listing data) or the shop dataset with injected problems from Chapter 2.
- **Data** At least 50,000 rows, at least 8 columns, with missing values, duplicates and odd codes.
- **Deliverables** An EDA notebook that someone else can rerun, plus a 2-page memo with 3 findings in the form *hypothesis, evidence, so what* (Chapter 3.4).
- **Acceptance criteria**
  1. A data-quality table: nulls by column and segment, duplicates, impossible values, with the action taken for each.
  2. Row counts reconcile with a known total or a stated source.
  3. At least three findings, each with a number, a chart, a stated uncertainty and a recommended action.
  4. A Simpson's-paradox check on one key trend, and a leakage check if a target exists.

5. The notebook runs top to bottom after a kernel restart.

- **Stretch** One A/B-test suggestion based on a finding.
- **Pitfalls** Charts without a message; findings with no "so what"; deleting outliers without checking if they are real.
- **Chapters used** 2, 3, 4, 13.

## Capstone 2: A/B test analysis with a ship or no-ship decision

- **Question** "Should we ship the new checkout (or similar) change?" Use a public experiment dataset or simulate one with fixed seed, including a planted sample-ratio mismatch (SRM) or a novelty effect for extra difficulty.
- **Deliverables** A pre-registered test plan (hypothesis, primary metric, guardrails, unit, MDE, sample size, duration), an analysis notebook, and a decision memo.
- **Acceptance criteria**
  1. Sample size computed and shown before analysis.
  2. SRM check performed and reported.
  3. Lift reported as absolute and relative, with a 95% interval.
  4. One variance-reduction method (CUPED or stratification) shown with its effect on the interval width.
  5. Guardrails reported; a clear ship, no-ship or iterate decision with the reason.
- **Stretch** A sequential or Bayesian analysis and a comparison of decisions.
- **Pitfalls** Peeking at the p-value each day; analysing many segments until one is "significant"; ignoring guardrails.
- **Chapters used** 4, 7, 12.4, 13.

**Problem 14.2.1** (Capstone 2 sizing) Baseline conversion 4%. You want to detect an absolute lift of 0.5 percentage points (to 4.5%) with 5% significance (two-sided) and 80% power. How many users per arm?

| | |
|---|---|
| **Formula** | `n = 2 × (z_α/2 + z_β)² × p(1 − p) / δ²`, the usual approximation with the same variance in both arms |
| **Plug in** | z_α/2 = 1.960, z_β = 0.8416, p = 0.04, δ = 0.005 |
| **Compute** | (1.960 + 0.8416)² = 2.8016² = 7.849. Then 2 × 7.849 × 0.04 × 0.96 = 0.6028. Divide by 0.005² = 0.000025: 0.6028 / 0.000025 = 24,112 |
| **Answer** | About 24,112 users per arm, so about 48,200 in total. The more exact formula (separate variances for 4.0% and 4.5%) gives 25,551 per arm, so plan for about 25,600 per arm. |
| **Check** | Both numbers were produced by code (24,112 and 25,551). Halving the detectable lift to 0.25 points roughly quadruples the sample size, because n depends on 1/δ². |
| **Why it matters** | If the shop gets 3,000 checkout users a day, 51,200 users takes about 17 days, so you must cover at least two full weekly cycles anyway. A sample-size number that is not turned into days is not yet a plan. |

## Capstone 3: Churn model to retention decision

- **Question** "Whom should we send a retention offer, and will it pay?" Use the shop dataset (Chapter 1) or a public telecom churn dataset.
- **Deliverables** Point-in-time label table, baseline rule, model, calibration plot, cost matrix, profit-by-threshold curve, Monte Carlo profit range, memo, model card with a fairness check.
- **Acceptance criteria**
  1. Prediction time and label window written down; no feature from after prediction time (a leakage test in `tests/`).
  2. Baseline gate: model vs recency rule with an interval (Section 14.1).
  3. Calibration table by quintile; recalibrate if the gap is large.
  4. Cost matrix with stated assumptions; the threshold from break-even and a sweep; profit in rupees.
  5. A simulation showing P(loss) and a tornado of the assumptions (Chapter 12.3).
  6. Group-wise selection rate and TPR (Chapter 13.3); a monitoring plan with PSI.
- **Stretch** An uplift model on simulated A/B data (Chapter 8).
- **Pitfalls** Using `cancel_ticket`-like leakage; reporting AUC with no money; assuming the save rate without a test.
- **Chapters used** 1, 2, 3, 8, 12, 13.

## Capstone 4: Demand forecasting with backtesting and an inventory decision

- **Question** "How much should we stock next week, per item?" Use a public retail demand dataset or simulate 20 daily series with weekly seasonality, trend and promotions.
- **Deliverables** Baselines (naive, seasonal naive, moving average), at least one classical and one ML model, rolling-origin backtest, interval forecasts, and a stock rule.
- **Acceptance criteria**
  1. Rolling-origin backtest (never a random split), at least 5 origins, with the metric by horizon.
  2. Beats seasonal naive on WAPE, or an honest statement that it does not.
  3. Prediction intervals with a measured coverage (does the 80% interval cover about 80%?).
  4. Safety stock and reorder point computed from forecast errors (Chapter 11.5), with the service level stated.
  5. A bias check (mean error), not only absolute error.
- **Stretch** Hierarchical reconciliation or intermittent-demand handling.
- **Pitfalls** MAPE near zero demand; leaking future holidays as unknowns; ignoring that forecasts must be made with data available at forecast time.
- **Chapters used** 9, 11.5, 12.

**Problem 14.2.2** (Capstone 4 metric) Actual demand over three days is 100, 120, 80 and the forecast is 110, 100, 90. Compute WAPE and the mean absolute error.

| | |
|---|---|
| **Formula** | WAPE = Σ\|actual − forecast\| / Σ actual |
| **Compute** | Errors are 10, 20, 10, so Σ\|error\| = 40. Σ actual = 300. WAPE = 40 / 300 = 0.1333. MAE = 40 / 3 = 13.33 |
| **Answer** | WAPE is 13.3%: forecasts are off by about 13% of total demand. MAE is 13.3 units per day. |
| **Check** | Here WAPE (as a fraction) and MAE equal each other numerically only because mean demand is 100, so 13.33 / 100 = 0.1333. They differ in general. |
| **Why it matters** | WAPE weights big items more and is stable when one day has zero demand, unlike per-row MAPE. |

**Connection to the job.** These four projects cover the daily work of most analytics and product data scientists: understand data, test changes, decide actions, plan supply.

**Applications**

- Product and growth teams: capstones 1 to 3.
- Retail, e-commerce and supply chain: capstone 4.

- Consulting: capstone 1 as a sample case.

| COMMON MISTAKE | Skipping the memo and the decision step because "the model is the project". The brief says the deliverable is a decision, so a notebook without a memo is half a capstone. |
|---|---|

**Check yourself**

1. In capstone 2, what comes first: analysis or sample size? Answer: Sample size and the plan, before any data is looked at.
2. Why use rolling-origin backtesting in capstone 4? Answer: It respects time order; random splits leak future patterns into training.
3. What turns AUC into something a manager can use in capstone 3? Answer: The cost matrix and profit in rupees at a chosen threshold.

## 14.3 Capstones 5 to 8: Recommender, Causal, Scorecard, Operations Data

| INTUITION | The second four are the "specialist" projects that make a portfolio stand out: a two-stage recommender, a causal analysis with sensitivity checks, a bank-style scorecard with a model card, and an operations-data analysis of queues and capacity. Pick the ones that match the jobs you want. |
|---|---|

### Capstone 5: Recommender system (two-stage) with offline metrics and cold start

- **Question** "What ten items should we show each user?" Use MovieLens-style or a public implicit-feedback dataset (clicks, purchases).
- **Deliverables** Popularity and item-item baselines, matrix factorization (or another collaborative method), a candidate-then-rank design, time-split offline evaluation, a cold-start strategy, and an A/B test plan.
- **Acceptance criteria**
  1. A time-based split (no random split), with leakage of future interactions ruled out.
  2. Baselines: popularity and recently popular; the new model's Recall@K and NDCG@K compared with an interval or several seeds.
  3. Coverage and diversity reported next to accuracy.
  4. A cold-start rule for new users and for new items.
  5. A written online test plan (primary metric, guardrails), because offline gains often do not carry over.
- **Stretch** A learning-to-rank re-ranker with business rules.

- **Pitfalls** Random splits; evaluating only on popular items; ignoring exposure and position bias.
- **Chapters used** 10, 7, 12.4.

**Problem 14.3.1** (Capstone 5 metric) A user has 2 relevant items. Your top-3 list has relevant items at ranks 1 and 3. Compute NDCG@3.

| | |
|---|---|
| **Formula** | DCG = Σ rel_i / $\log_2(i + 1)$; NDCG = DCG / ideal DCG |
| **Compute** | DCG = $1/\log_2(2) + 1/\log_2(4) = 1 + 0.5 = 1.5$. Ideal: relevant items at ranks 1 and 2: $1 + 1/\log_2(3) = 1 + 0.6309 = 1.6309$. NDCG = 1.5 / 1.6309 = 0.920 |
| **Answer** | NDCG@3 is 0.92: nearly ideal, penalised a little for placing one relevant item third. |
| **Check** | If the relevant items were at ranks 1 and 2, NDCG would be exactly 1.0. |
| **Why it matters** | Ranking metrics reward getting good items to the top, which accuracy cannot see. |

## Capstone 6: Causal analysis of observational data

- **Question** "Did programme X cause outcome Y?" Use a public dataset with a treatment (a policy, a feature roll-out) or a simulated one with known true effect and planted confounding.
- **Deliverables** A DAG, a naive estimate, adjusted estimates (regression, propensity weighting), balance table, and a sensitivity analysis. A difference-in-differences version if you have before/after data.
- **Acceptance criteria**
  1. A drawn DAG with confounders, mediators and colliders labelled, and the reason for each control choice.
  2. Overlap check and covariate balance (standardised mean difference) before and after weighting.
  3. At least two estimators that agree (or an explanation if they do not).
  4. If simulated, the estimate is compared with the known true effect.
  5. A sensitivity analysis for unobserved confounding and a plain-language statement of assumptions.
- **Stretch** A synthetic-control or regression-discontinuity analysis.
- **Pitfalls** Controlling for a collider or mediator; claiming causation without stating assumptions.
- **Chapters used** 4, 8.

**Problem 14.3.2** (Capstone 6 check) Treated stores average 10 units before and 16 after. Control stores average 9 before and 12 after. What is the difference-in-differences estimate?

| | |
|---|---|
| **Formula** | DiD = (treated after − treated before) − (control after − control before) |
| **Compute** | (16 − 10) − (12 − 9) = 6 − 3 = 3 |
| **Answer** | The estimated effect is 3 units. Control stores grew by 3 anyway, so only 3 of the treated stores' 6-unit growth is credited to the programme. |
| **Check** | The naive before/after change was 6, double the DiD estimate. The assumption is *parallel trends*: without the programme, treated stores would have grown by 3 too. |
| **Why it matters** | Always state the assumption; a visual check of pre-period trends is part of the acceptance criteria. |

## Capstone 7: Credit-risk scorecard with a model card

- **Question** "Which applicants should we approve, and how is the score monitored?" Use a public credit dataset (for example the German credit or similar), or the synthetic generator of Chapter 11.2.
- **Deliverables** WoE binning with IV table, logistic scorecard, points table (base score and PDO), KS, Gini, PSI over time, a model card, and a reject-inference discussion.
- **Acceptance criteria**
  1. Out-of-time split; bins and coefficients fitted on train only.
  2. WoE monotonic for the main variables, or a justification for each exception.
  3. KS, Gini and AUC on train and test, with a statement on overfitting.
  4. Score points table that a person can recompute by hand for one applicant (show the example).
  5. PSI of the score and of each variable between train and a later period.
  6. Model card with limits, fairness check, reason-code logic and a section on selective labels.
- **Stretch** Compare with gradient boosting and discuss explainability trade-offs.
- **Pitfalls** Any variable with IV above 0.5 (check leakage); using test data to choose bins; no reason codes.
- **Chapters used** 11.2, 12.4, 13.3, 13.4.

**Problem 14.3.3** (Capstone 7 check) Test AUC is 0.74. Report Gini. Then, at base score 600 with odds 50:1 and PDO 20, how many points does a +1 change in log-odds of good represent?

| | |
|---|---|
| **Compute** | Gini = 2 × 0.74 − 1 = 0.48. Factor = PDO / ln 2 = 20 / 0.6931 = 28.85 |
| **Answer** | Gini is 0.48, and one unit of log-odds is worth 28.85 points (doubling the odds is 20 points). |
| **Check** | 28.85 × 0.6931 = 20.0, which is the PDO. |
| **Why it matters** | This is the sentence that shows you really understand the points scale. |

## Capstone 8: Operations-data analytics (queues, delivery times, capacity)

- **Question** "When will this operation run out of capacity, and when does it misbehave?" Use a public operations dataset (taxi trips, hospital appointments, delivery logs, bike-share) with no personal data. If you have none, generate synthetic data as in Chapter 11.6.
- **Deliverables** Percentiles of waiting or delivery time by hour, a plot of waiting time against utilisation with the point where it explodes, a regression of waiting time on load, anomaly detection, a capacity-date forecast with a bootstrap interval, and a one-page memo for an operations manager.
- **Acceptance criteria**
  1. p50, p90, p99 by hour or segment; never averages alone.
  2. Little's law check: items in the system ≈ arrival rate × time in the system, within a stated tolerance.
  3. The utilisation level where waiting explodes, compared with the M/M/1 shape (Chapter 11.6).
  4. Anomaly rule evaluated on injected or known incidents, with detection rate and false alarms reported, and compared with a simple global threshold.
  5. Capacity date with a 90% interval and the assumptions (linear growth, no campaigns).
  6. Memo with a recommendation: add capacity by date X, or fix step Y.
- **Stretch** A discrete-event simulation of a proposed staffing plan (Chapter 12.3).
- **Pitfalls** Aggregating percentiles by averaging them; personal data in a public repo; ignoring the start-up period when the system is still empty.
- **Chapters used** 11.6, 12.3, 13.

**Problem 14.3.4** (Capstone 8 check) A hospital emergency room records 80 arrivals per hour and an average stay of 0.5 hours. It has 30 beds and no waiting room is counted. How many patients are in the ER on average, and is the record consistent?

| | |
|---|---|
| **Formula** | $L = \lambda \times W$ |
| **Compute** | $L = 80 \times 0.5 = 40$ |
| **Answer** | About 40 patients must be present on average, but only 30 beds exist, so the numbers cannot all be right: the stay may exclude time in the corridor, the arrival rate may cover a different time window, or patients are counted twice. Investigate before trusting any conclusion. |
| **Check** | With 30 beds and a 0.5 hour stay, the maximum rate the ER can handle is 30 / 0.5 = 60 patients per hour, below 80. |
| **Why it matters** | Little's law is a cheap consistency check on operations data; failing it means the data or the setup is wrong. |

**Connection to the job.** Capstones 5 to 8 map to recommendation teams, causal/experimentation teams, BFSI risk teams and operations or supply-chain analytics teams.

### Applications

- E-commerce and media: capstone 5.
- Product and marketing science: capstone 6.
- Banks, NBFCs and fintech: capstone 7.
- Logistics, healthcare and operations teams: capstone 8.

| | |
|---|---|
| **COMMON MISTAKE** | Publishing a public repo with real company data, credentials or customer information from your work. Use public or synthetic data, remove secrets, and check your employer's rules before sharing anything derived from work. |

### Check yourself

1. In capstone 6, why compare the estimate with the known truth in a simulation? Answer: It shows whether your method actually recovers the effect, which real data cannot show.
2. Why is IV above 0.5 a warning in a scorecard? Answer: It is suspiciously strong; check for leakage.
3. What does Little's law check in operations data? Answer: Whether the number in the system, the arrival rate and the time spent in the system are consistent with each other.

# 14.4 Mastery Tests and Scoring Your Own Work

| INTUITION | A driving test has a checklist, a time limit, and an examiner who does not know you. Mastery tests are the same: timed tasks with a rubric, so you can see honestly whether a topic is obvious yet (the rule from the start of this document). |
|---|---|

**The six mastery tests.** Do each under time, then score yourself with 0 (not done), 1 (partly) or 2 (clear and correct) on every rubric line. Convert to a percentage: points / (2 × number of lines).

| Test | Time | Rubric lines (score 0 to 2 each) |
|---|---|---|
| 1. One-page brief from a vague ask | 30 min | Decision named; framing; prediction time and label window; metric at 3 levels; baseline; risks |
| 2. Find leakage in an unfamiliar notebook | 45 min | Finds the leaking feature; explains why; fixes the split; re-measures; writes what changed |
| 3. Design an A/B test end to end | 45 min | Hypothesis; primary and guardrail metrics; unit; sample size; analysis plan; pitfalls |
| 4. Explain Simpson's paradox, confounding, selection bias | 20 min | Numeric example each; which view is right; how to check |
| 5. Backtest a forecast against naive | 60 min | Rolling origin; baselines; metric by horizon; honest result; interval coverage |
| 6. Five-minute stakeholder talk with no formulas | 30 min | Answer first; one number per slide; uncertainty; risks; clear ask |

A reasonable target before interviews is 75% or more on every test.

**Problem 14.4.1** You score tests 1 to 6 as 83%, 67%, 92%, 75%, 58%, 50%. Which should you redo first, and what is your average?

| | |
|---|---|
| **Compute** | Sum = 83 + 67 + 92 + 75 + 58 + 50 = 425. Average = 425 / 6 = 70.8% |
| **Answer** | Your average is 70.8%. Redo test 6 (50%, the lowest: the stakeholder talk) first, then test 5 (58%) and test 2 (67%). Test 4 (75%) just meets the target. |
| **Check** | Three tests are below the 75% target, so the redo list has three items. |
| **Why it matters** | A written score removes the feeling of "I think I know it". Repeat a test after a week; the second score is the real one. |

**Problem 14.4.2** Using the leakage test as the example, what would you check in a notebook where a "days to cancel" column is a feature in a churn model?

| | |
|---|---|
| **Step 1** | Ask when each feature is created: "days to cancel" exists only after the customer has cancelled. |
| **Step 2** | Measure its single-feature AUC: a near-perfect value (like the 94% precision of `cancel_ticket` in Chapter 3.4) is the alarm. |
| **Step 3** | Drop it, rebuild the validation as time-ordered, and report the AUC before and after. |
| **Answer** | The feature is leakage, because it is created by the outcome. Remove it and re-measure. |
| **Check** | After removal the best single feature should be a plain behaviour one such as recency. |
| **Why it matters** | Leakage is the most common reason for "too good to be true" results. |

**Connection to the job.** These tests mirror live case rounds. Doing them alone, under time, with a rubric is the cheapest interview practice you can get.

**Applications**

- Self-assessment before applying for roles.
- Study groups: score each other's briefs.
- Mock interviews with an AI partner (Prompt pack).

| | |
|---|---|
| **COMMON MISTAKE** | Scoring yourself generously because you "could do it with more time". The test includes the time limit. Interviews do too. |

**Check yourself**

1. What do you do when one test scores low? Answer: Redo the matching chapter's worked problems, then repeat the test after a week.
2. Why time the tests? Answer: Interviews are timed; speed reveals if the idea is truly obvious to you.
3. Who should read your brief? Answer: Someone who does not know the project; if they cannot state the decision after one read, rewrite it.

## 14.5 The Data Science Interview Bank

| INTUITION | Interview questions are not random. They test whether you can frame a problem, spot traps, use statistics correctly, and communicate. A strong answer has a structure you can reuse: clarify, frame, method, risks, decision. Practise saying them aloud, in short sentences. |
|---|---|

**A reusable answer shape.** (1) Clarify the goal and the decision. (2) Define the metric and the baseline. (3) State the method and why. (4) Name the main risks. (5) Say what you would do next and how you would know it worked.

### Product and metrics questions

**Question 1: "DAU dropped 10%. What do you do?"**

Key points: first check the data (pipeline delay, logging change, bots, timezones), then the shape (sudden or gradual, all segments or one, new vs returning, platform, country, acquisition channel), then external causes (holiday, outage, release, competitor), then act.

**Problem 14.5.1** Daily active users fell from 100,000 to 90,000. New users fell from 20,000 to 19,000; returning users fell from 80,000 to 71,000. Where is the drop?

| | |
|---|---|
| **Step 1** | Total drop = 100,000 − 90,000 = 10,000. |
| **Step 2** | New users explain 20,000 − 19,000 = 1,000, which is 1,000 / 10,000 = 10% of the drop. |
| **Step 3** | Returning users explain 80,000 − 71,000 = 9,000, which is 90%. Returning fell 9,000 / 80,000 = 11.25%. |
| **Answer** | 90% of the drop is returning users, so look at retention, notifications, product bugs, or a segment (a platform or country) rather than at acquisition. |
| **Check** | 1,000 + 9,000 = 10,000. Correct. |
| **Why it matters** | Decomposing a total into parts is the first move; it narrows the search before you form a hypothesis. |

**Question 2: "How would you measure the success of feature X?"**

Key points: state the goal of the feature; one primary metric plus guardrails; method (A/B test if possible, else a quasi-experiment); sample size and duration (Problem 14.2.1); watch for novelty and cannibalisation; decide in advance what result means ship.

**Question 3: "Design an experiment for a new recommendation widget."**

Key points: randomise by user; metric (click-through is a proxy; prefer conversion or retention), guardrails (page latency, returns), SRM check, run full weeks, avoid peeking, segment analysis only as a hypothesis, pre-define the ship rule (Chapter 7).

**Question 4: "Offline AUC is great, but online results are flat. Why?"**

Key points: offline metric is not the business metric; selection bias in training data; position and exposure bias; feedback loops (Chapter 12.4); training-serving skew (Chapter 13.5); novelty; the action does not change when the score changes; wrong threshold; low traffic and no power.

## Statistics and data questions

**Question 5: "Explain a p-value to a manager."**

Key points: "If nothing really changed, how often would we see a difference this big just by luck? A p-value of 0.03 means about 3 times in 100." It is not the chance the idea is true, and it does not say how big or how useful the effect is. Follow with the interval and the effect size.

**Question 6: "A test is 95% sensitive and 95% specific. The condition affects 1% of people. A person tests positive. What is the chance they have it?"**

**Problem 14.5.2** Compute the positive predictive value.

| | |
|---|---|
| **Formula** | PPV = sens × prev / (sens × prev + (1 − spec) × (1 − prev)) |
| **Plug in** | sensitivity 0.95, specificity 0.95, prevalence 0.01. True positives: 0.95 × 0.01 = 0.0095. False positives: 0.05 × 0.99 = 0.0495 |
| **Compute** | PPV = 0.0095 / (0.0095 + 0.0495) = 0.0095 / 0.0590 = 0.161 |
| **Answer** | About 16%. Out of every 100 positives, only about 16 are real, because the condition is rare. (With sensitivity 0.90 instead, the answer is 0.154, which we also confirmed by code.) |
| **Check** | In 10,000 people: 100 sick, 95 detected; 9,900 healthy, 495 false alarms. 95 / (95 + 495) = 0.161. Same. |
| **Why it matters** | Base-rate neglect (Chapter 4.2) explains why fraud and screening alerts are mostly false alarms. |

**Question 7: "How do you handle missing data, imbalanced classes and outliers?"**

Key points. Missing: first ask *why* it is missing (not applicable, not recorded, error); use indicators, impute inside the pipeline only on train; do not drop silently. Imbalance: use PR-AUC and cost-based thresholds, stratified or time splits, class weights; resampling is a last resort and must be applied only to train. Outliers: check if error or real; transform (log) or cap, but keep the evidence (Chapter 3).

**Question 8: "What is data leakage? Give an example."**

Key points: using information at training time that will not exist at prediction time. Example: a `cancel_ticket` feature created after the customer decided to leave scored 94% precision; once removed, the best feature was recency with AUC 0.88 (Chapter 3.4). Detect with single-feature AUC, time-ordered validation and asking "when is this created?".

**Question 9: "How do you decide between two models?"**

Key points: same split, the metric that matches the decision (profit, recall at capacity), intervals on the difference (Section 14.1), calibration, latency and cost, simplicity, explainability, stability over time; and the baseline gate.

**Question 10: "Correlation versus causation: how do you answer a causal question without an experiment?"**

Key points: DAG, adjust for confounders, propensity or DiD, check overlap and balance, state assumptions, run sensitivity analyses, and say what an experiment would add (Chapter 8).

## Modelling-in-business questions

**Question 11: "How would you predict churn?"** Key points: define churn and the prediction time and label window; build a point-in-time table; baseline rule; features such as recency, frequency, trends; model; calibration; cost matrix and threshold; fairness; monitoring (Chapters 1 to 3, 11, 12, 13).

**Question 12: "How would you forecast demand?"** Key points: baselines (seasonal naive), rolling-origin backtest, features such as calendar, price and promotions, intervals, forecast to decision (safety stock), bias check (Chapters 9, 11.5).

**Question 13: "How would you detect fraud?"** Key points: extreme imbalance, precision at review capacity, cost threshold `c_review / (c_review + c_miss)`, time-based validation, graph and velocity features, latency, drift (Chapter 11.3).

**Problem 14.5.3** A bank loses 3,000 per missed fraud and spends 60 per review. A model has probability 0.03 for a transaction. Review it?

| | |
|---|---|
| **Formula** | Break-even p* = c_review / (c_review + c_miss) |
| **Compute** | p* = 60 / (60 + 3,000) = 60 / 3,060 = 0.0196 |
| **Answer** | Yes: 0.03 is above 0.0196. Expected loss if ignored is 0.03 × 3,000 = 90, against a review cost of about 0.97 × 60 = 58.2. |
| **Check** | At p = 0.0196 the two costs are equal (about 58.8 each), so any larger p favours reviewing. |
| **Why it matters** | The threshold comes from costs, not from 0.5. |

## Behavioural and career questions

**Question 14: "Tell me about a project you are proud of."** Use the shape: situation, task, action (with numbers), result (with numbers), what you learned, what you would change. Use a capstone from this chapter. Include a limit or a failure: honest answers are trusted.

**Question 15: "Why are you moving into data science?"** A strong answer has four parts: what you did before and a number that shows it ("I cut the monthly reporting time from three days to four hours"), what you enjoyed (asking why, predicting what happens next), the skills you added (statistics, machine learning, decision analysis), and the project that proves it (one capstone, with its result and its limit). Keep it to 45 seconds and honest.

**Question 16: "A stakeholder disagrees with your result. What do you do?"** Listen; check if they know something about the data you do not; reproduce your result; show uncertainty and what would change your mind; propose a small test to settle it (Chapter 13.2).

**Your 60-second story.** Prepare one story from your own past work: the problem you solved, the numbers (before and after), the method, and the decision it enabled. Then link it to the role: a reporting automation links to analytics engineering; a forecast links to demand planning; a quality check links to data quality and monitoring.

**Connection to the job.** The same structured thinking appears in design reviews and stakeholder meetings. Interview practice is job practice.

### Applications

- Data analyst and product analyst interviews: questions 1 to 5, 7.
- Data scientist interviews: all of the above.
- Applied scientist and ML engineer interviews: add the ML and system design material from the other skill maps.

| COMMON MISTAKE | Memorising answers. Interviewers change the numbers, add a constraint, and ask "why". Practise the structure and the calculations, not the script. |
|---|---|

### Check yourself

1. What is the first thing to check when a metric suddenly drops? Answer: Data and logging problems, before any business explanation.
2. In the screening problem, why is PPV only 16% with 95% accuracy on both sides? Answer: The condition is rare, so false positives from the many healthy people outnumber true positives.
3. What makes a "project I am proud of" answer credible? Answer: Specific numbers, your own actions, and an honest limit.

## Chapter recap

- A capstone is done when the decision is answered, the baseline gate is passed (or its failure is reported honestly), and someone else can rerun it with one command.
- On shop churn the four-feature model did not beat the recency rule: AUC 0.878 vs 0.884, gain interval −0.016 to +0.005.
- Write acceptance criteria before coding: pass/fail checks, not "the model is good".
- Capstones 1 to 4: EDA memo, A/B test (24,112 per arm for a 0.5 point lift on 4%), churn decision, forecast with backtest and stock rule.
- Capstones 5 to 8: recommender (NDCG 0.92 example), causal analysis (DiD estimate 3), scorecard (Gini 0.48 from AUC 0.74) and operations analytics (Little's law check).
- Mastery tests are timed and rubric-scored; redo the lowest first (70.8% average in the example, target 75% on each).

- Interview answers share one shape: clarify, metric and baseline, method, risks, next step; practise the calculations (DAU decomposition, PPV 16%, fraud threshold 1.96%).
- Prepare your own story: your past domain work plus data skills is a distinctive profile.

| Item | Formula or rule | Example |
|---|---|---|
| Sample size per arm | `2 (z_α/2 + z_β)² p(1−p) / δ²` | 24,112 for p = 0.04, δ = 0.005 |
| WAPE | `Σ | a − f |
| NDCG | `DCG / ideal DCG`, `rel / log₂(i + 1)` | 1.5 / 1.631 = 0.92 |
| DiD | `(T₁ − T₀) − (C₁ − C₀)` | (16 − 10) − (12 − 9) = 3 |
| Gini | `2 × AUC − 1` | 0.48 |
| PPV | `sens × prev / (sens × prev + (1 − spec)(1 − prev))` | 0.161 |
| Little's law | `L = λ × W` | 80 × 0.5 = 40 |
| Fraud break-even | `c_review / (c_review + c_miss)` | 60 / 3,060 = 0.0196 |

## Practice set

**1.** (Easy) Baseline AUC 0.80, model AUC 0.83. Gini of each? **Answer:** Baseline 2 × 0.80 − 1 = 0.60; model 2 × 0.83 − 1 = 0.66.

**2.** (Easy) Forecast 90, actual 100 on one day. Absolute error and percentage error? **Answer:** 10 units; 10 / 100 = 10%.

**3.** (Easy) Control grows 5 to 8 and treated grows 6 to 13. DiD? **Answer:** (13 − 6) − (8 − 5) = 7 − 3 = 4.

**4.** (Medium) What is the output? `2 * 0.62 - 1` **Answer:** 0.24, the Gini for AUC 0.62.

**5.** (Medium) Find the bug: "Our model beat the baseline: 0.881 vs 0.879 AUC on one test split. Ship it." **Answer:** The gain of 0.002 is well inside noise (our own gain interval was about ±0.01). Bootstrap the difference; if the interval contains zero, report no evidence of improvement.

**6.** (Medium) A test must detect 2 points on a 10% baseline, alpha 0.05, power 0.8. Approximate n per arm with the simple formula. **Answer:** 2 × 7.849 × 0.10 × 0.90 / 0.02² = 1.4128 / 0.0004 = 3,532 per arm.

**7.** (Medium) A call centre gets 200 calls per hour and each call lasts 0.3 hours including waiting. It has 50 phone lines. Consistent? **Answer:** L = 200 × 0.3 = 60 calls in the system, more than 50 lines. Not consistent: check the measurement window, what the duration includes, or the line count.

**8.** (Medium) Users: 50,000 yesterday, 45,000 today; mobile 30,000 to 24,000 and web 20,000 to 21,000. Where is the problem? **Answer:** Total drop 5,000. Mobile fell 6,000 (20%), web rose 1,000. Mobile explains more than the whole drop; investigate a mobile release, outage or tracking change.

**9.** (Hard) A screening test has sensitivity 0.99 and specificity 0.99 and prevalence 0.1%. PPV? **Answer:** TP = 0.99 × 0.001 = 0.00099; FP = 0.01 × 0.999 = 0.00999. PPV = 0.00099 / 0.01098 = 0.090, about 9%.

**10.** (Interview) "Walk me through one of your projects and what you would do differently." **Answer:** Use situation, action, result with numbers, then the limit (for example the baseline gate showing the model did not beat the simple rule) and what you would change (collect new features, run an A/B test of the offer).

**11.** (Interview) "Your A/B test is stat-sig on the primary metric but a guardrail dropped. What do you do?" **Answer:** Do not ship yet. Check the size and the interval of the guardrail drop, the cost of that drop in money, segments where it concentrates, and whether it is a novelty effect; decide using expected value, discuss with the owner, and consider iterating on the design.

**12.** (Interview) "How would you explain overfitting to a manager?" **Answer:** "The model memorised last year's customers instead of learning general patterns, like a student who memorised old exam papers. We test it on new customers, ones it has never seen, to check."

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles, and English is my second language. Help me plan a data science portfolio of 3 capstone projects from these options: EDA memo, A/B test analysis, churn to retention decision, demand forecast with inventory, recommender, causal analysis, credit scorecard, operations-data analytics. Ask me about my target roles, then choose the 3 projects, and for each write a one-page brief with a question, data, deliverables and 5 pass/fail acceptance criteria. Then review my brief when I paste it.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Topic: the baseline gate for capstone projects: comparing a model with a simple rule, using the same split, bootstrap intervals on the difference in AUC, and reporting honestly when the model does not win. Explain it step by step with a small numeric
```

example, then give me 6 exercises where I decide, from a gain and its interval, whether to ship the model or the baseline. Grade my answers and explain the reasoning.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles. Topic: capstone 8, operations-data analytics: percentiles of waiting time by hour, Little's law consistency check, the M/M/1 utilisation curve, anomaly detection on delivery times and a capacity-date forecast with a bootstrap interval. Review my project plan below and list what is missing against these acceptance criteria. [PASTE HERE MY PLAN OR README]. Then suggest the three most impressive additions that fit in one week.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles and preparing for interviews. Topic: data science product and statistics interview questions (metric drops, success metrics for a feature, experiment design, p-value explanation, base rates, leakage, imbalanced data, choosing between models). Run a mock interview of 10 questions, one at a time, with follow-up questions that change the numbers. After each answer, score me out of 5, show a model answer with calculations, and tell me what to practise.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles. Topic: behavioural interview answers for a career switcher: "tell me about a project", "why are you moving into data science", and "a stakeholder disagrees with your result". Help me write a 60-second story that links my past work to data roles, using the situation, action, result shape with numbers. Ask me first what my past work was. Then role-play the interviewer for three rounds and give me feedback on clarity, honesty and structure.

CHAPTER 15

# PROJECT LAB 1: CHURN, A/B TOOLKIT AND DEMAND FORECASTING

*A practical guide to planning, building, testing and showing portfolio projects in data science, with three complete builds you can run today.*

Reading about data science does not get you hired. A small number of finished, honest, well-explained projects does. A hiring manager opens your repository for about two minutes, so those two minutes must show a clear question, clean code, a fair baseline, a number you can defend and a clear sentence about what to do next. This chapter first gives you a method for running any project in this area. Then it walks through three projects: a churn model that ends in a spending decision (#3), an A/B test toolkit (#25) and a demand forecast with a proper backtest (#26). Every script below was run in this session with `numpy`, `pandas`, `scipy` and `scikit-learn`, and every output shown is the real output. All data is synthetic with fixed seeds, so you get the same numbers.

## 15.1 How to run a project in this area

**INTUITION**

A project is like cooking a meal for a guest. You decide the dish first (the question), buy fresh ingredients (data), cook in a clean kitchen (repo), taste as you go (tests and baselines), and serve it on a plate the guest understands (README and post). Nobody judges the meal by how many pots you used.

### Plan the question and success criteria

Write one sentence before any code: "For [decision maker], I will [predict or estimate X] so that they can [take action Y], and I will call it a success if [measurable result]." A good example: "For the retention team, I will rank customers by churn risk so they can send a 5-rupee offer only where it pays back, and I will call it a success if expected profit beats 'send to everyone' on held-out data." Also write the **baseline** (the dumb method you must beat) and the **metric** you will use. Do it on day one and do not change it after you see results.

| Item | Example | Where it appears in the job |
|---|---|---|
| Decision | Who gets the retention offer | The sentence your manager repeats in meetings |
| Metric | Expected profit, NDCG@10, MAE | The one number on the slide |

| Item | Example | Where it appears in the job |
|---|---|---|
| Baseline | Send to all, popularity, last value | What a sceptic compares you to |
| Stop rule | Done when criteria in 15.1 "Definition of done" pass | Stops endless tweaking |

## Repo layout

```
project-name/
  README.md              the front door (template below)
  requirements.txt       pinned library versions
  Makefile               make data, make test, make run
  data/
    README.md            where data comes from; no big files in git
  src/
    make_data.py         generator with a fixed seed
    features.py          pure functions, no hidden state
    model.py             training and prediction
    evaluate.py          metrics and plots
  tests/
    test_features.py     small checks on tiny inputs
    test_metrics.py
  notebooks/
    01_explore.ipynb     exploration only; final logic lives in src/
  reports/
    figures/             PNG files used in README and post
    results.md           final numbers, copied from a script run
```

## Environment and dependencies

Use a virtual environment: `python3 -m venv .venv`, activate it, then list only what you import in `requirements.txt`. Pin versions with `pip freeze` once the project works, and write the Python version in the README. Set every random seed in one place ( `SEED = 7` ). If a stranger cannot run `make data && make test` in five minutes, the project is not finished.

## Data sourcing or generation

Prefer a public dataset with a clear licence, and write the licence and download date in `data/README.md`. When none fits, **generate data with a documented rule**. A generator is a strength, not a cheat: you know the true answer, so you can check that your method recovers it. The three projects here use generators. State plainly in the README that the data is synthetic and that real data would be messier.

## Commit rhythm and good commit messages

Commit when one small thing works, usually every 30 to 60 minutes. A good message says what and why in one line, in the imperative: `Add SRM check with chi-square test`, `Fix leakage: compute tenure at snapshot date`. Bad messages: `update`, `final`, `fix stuff`. A short history that reads like a story is a portfolio asset.

```
a1f3c2e  Add synthetic customer generator with fixed seed
b7d9011  Add logistic regression baseline and AUC check
c40e8aa  Add calibration table; gradient boosting is not better
d55f2b9  Add cost-based targeting and profit comparison
e91a7c4  Write README with results and limits
```

## Definition of done

- The question, baseline and metric from your plan are answered in `reports/results.md`.
- `make test` passes; at least one test checks a number you computed by hand.
- The README has a result table, one chart, the limits and how to reproduce.
- A stranger can run it from a clean clone.
- You can explain every line to an interviewer, including why you rejected one idea.

## README template

```
# Project title: one line that states the decision

## Question
For <who>, I <predict/estimate> <what> so that <action>.

## Result (headline)
| Method | Metric | Value |
|---|---|---|
| Baseline | ... | ... |
| Mine | ... | ... |

One chart here.

## Data
Synthetic (generator in src/make_data.py, seed 7) or source + licence + date.

## Method
Three to five bullets: features, model, validation, why.

## What I would do next / limits
Honest list: what the data cannot tell you.

## Reproduce
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
make data && make test && make run
```

## Demo and deploy options

Free and simple options: a Streamlit app on a free community hosting tier, a static HTML report from a notebook, or a short screen recording of the notebook running. The core ideas for hosting do not change when the platform does, so check current documentation before you choose. Deployment is optional for analysis projects; a clean README with figures is usually enough. Not run here: Streamlit and hosting.

## Write the LinkedIn or blog post

Use five short parts: (1) the question in plain words, (2) one surprising finding with a number, (3) what you did in three bullets, (4) one mistake you caught and how, (5) link and one honest limit. Example opening: "A retention model with AUC 0.75 made less money than a simpler rule. Here is why." Avoid jargon walls. Show one chart.

### Testing and debugging checklist for this area

1. **Leakage:** does any feature use information from after the prediction date? Rebuild features with a hard cut-off date.
2. **Split:** random split for independent customers, time split for forecasts, group split when one user has many rows.
3. **Baseline first:** if your model cannot beat a simple rule, say so.
4. **Sanity numbers:** class balance, row counts before and after each join, no duplicate IDs.
5. **Known-answer test:** generate data with a known effect and check that the code finds it.
6. **Metric vs decision:** does the metric you report match the business action?
7. **Seeds:** rerun twice; numbers should match exactly or within a stated tolerance.
8. **Uncertainty:** report an interval or a range, never only a point.

| COMMON MISTAKE | Tuning on the test set "just once more". Each peek makes the test score a little less honest. Fix the test set on day one and look at it at the end. |
|---|---|

**Check yourself**

1. Why write the success criteria before coding? Answer: Otherwise you can bend the goal to fit whatever result appears.
2. Why is a data generator useful? Answer: You know the true answer, so you can test whether your method finds it.
3. Name two things that must be in a README. Answer: A result table with a baseline, and the limits, plus how to reproduce.

## 15.2 Project: Churn analysis and retention decision (#3)

**Goal and why employers care.** Predict which shop customers will stop ordering, then decide whom to contact. Most candidates stop at "AUC = 0.75". Employers want the next step: turning a probability into a rupee decision. This is the exact thread of a real retention team.

**Skills used.** Chapter 3, *Exploratory Data Analysis for Modelling*; Chapter 6, *Feature Engineering II* (leakage-safe features); Chapter 11, *Applied Domains* (churn); Chapter 12, *From Prediction to Decision* (cost matrices and thresholds).

**Data.** The generator in the script below makes 20,000 customers with tenure, recent orders, days since last order, support tickets and basket size. Churn is drawn from a known formula, so higher recency and more tickets raise risk. Real alternative: any public telecom or subscription churn file; check its licence.

**Architecture.**

```
make_customers() -> split 70/30 (stratified) -> baselines
        |                                          |
        v                                          v
 logistic regression + gradient boosting      majority class, send-to-all
        |
        v
 AUC + Brier + calibration table -> pick model
        |
        v
 expected gain = P(churn) x P(save) x value - offer cost
        -> send only where gain > 0 -> compare profit
```

**Step-by-step build.**

1. Generate data, check the churn rate (15.4%) and compute the majority-class accuracy (84.6%). A model must beat the *ranking* quality, because 84.6% accuracy is free.
2. Fit logistic regression (simple, explainable) and gradient boosting (more flexible). Score both with AUC (can the model rank churners above non-churners) and Brier score (average squared error of the probability; lower is better).
3. Make a calibration table: sort customers by predicted risk, cut into 5 equal bins, compare average predicted risk to the actual churn rate. If a bin says 12% but 10.7% churn, the probabilities are roughly honest.
4. Convert probability to money. Assumptions (state them in the README): an offer costs 5, it saves 30% of would-be churners, and a saved customer is worth `value`. Send if `P × 0.30 × value − 5 > 0`.

**Before you run it, predict:** will gradient boosting win? Many people expect so. Check the output.

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score, brier_score_loss
from sklearn.calibration import calibration_curve

def make_customers(n=20000, seed=7):
    r = np.random.default_rng(seed)
    df = pd.DataFrame({
        "tenure_months": r.integers(1, 61, n),
        "orders_90d": r.poisson(3, n),
        "days_since_last": r.integers(0, 120, n),
        "support_tickets": r.poisson(0.8, n),
        "avg_basket": r.gamma(4, 15, n).round(2)})
    z = (-2.0 - 0.03*df.tenure_months - 0.25*df.orders_90d + 0.02*df.days_since_last
         + 0.30*df.support_tickets + 0.004*df.avg_basket)
    p = 1/(1+np.exp(-z))
    df["churned"] = (r.random(n) < p).astype(int)
    df["value"] = (df.avg_basket*df.orders_90d/3*4).round(2)   # rough yearly margin proxy
    return df
# Step 1: data and baseline
df = make_customers()
print(df.shape, "churn rate", round(df.churned.mean(),3))
X = df.drop(columns=["churned","value"]); y = df.churned
Xtr, Xte, ytr, yte, vtr, vte = train_test_split(X, y, df.value, test_size=0.3,
random_state=0, stratify=y)
base = max(yte.mean(), 1-yte.mean())
print("majority-class accuracy", round(base,3))
# Step 2: two models, judged by ranking (AUC) and probability quality (Brier)
lr = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
gb = HistGradientBoostingClassifier(random_state=0).fit(Xtr, ytr)
for name, m in [("logreg", lr), ("gboost", gb)]:
    p = m.predict_proba(Xte)[:,1]
    print(name, "AUC", round(roc_auc_score(yte,p),3), "Brier",
round(brier_score_loss(yte,p),4))
# Step 3: calibration table, 5 equal-size bins
p = gb.predict_proba(Xte)[:,1]
fr, mp = calibration_curve(yte, p, n_bins=5, strategy="quantile")
print("gb calibration (pred -> actual):", [(round(float(a),3),round(float(b),3)) for a,b
in zip(mp,fr)])
# Step 4: cost-based targeting: offer cost 5, saves customer with prob 30%, saved value =
vte
cost, save = 5.0, 0.30
pl = lr.predict_proba(Xte)[:,1]
gain = pl*save*vte.values - cost          # expected profit per customer if we send
print("send if gain>0:", (gain>0).sum(), "of", len(gain), "profit",
```

```
round(gain[gain>0].sum(),0))
print("send to all   :", len(gain), "profit", round(gain.sum(),0))
top = np.argsort(-pl)[:int(0.2*len(pl))]
print("top-20% by prob profit", round(gain[top].sum(),0))
```

Real output:

```
(20000, 7) churn rate 0.154
majority-class accuracy 0.846
logreg AUC 0.752 Brier 0.1154
gboost AUC 0.737 Brier 0.1175
gb calibration (pred -> actual): [(0.043, 0.037), (0.079, 0.075), (0.12, 0.107), (0.186,
0.209), (0.355, 0.34)]
send if gain>0: 3424 of 6000 profit 35390.0
send to all   : 6000 profit 28825.0
top-20% by prob profit 17123.0
```

**Reading the output.** Logistic regression (AUC 0.752) beats gradient boosting (0.737) here. This is not a bug. The generator is a logistic formula, so the simple model is the right shape, and boosting adds only variance. The calibration pairs sit close to the diagonal. The profit line is the real finding: sending only where the expected gain is positive earns 35,390 on 3,424 customers, against 28,825 when sending to all 6,000, and a "top 20% by probability" rule earns only 17,123, because the riskiest customers are not always the most valuable or the most savable. These profit numbers depend entirely on the assumptions (5 cost, 30% save rate), so show a small sensitivity table in your README.

**Evaluation and acceptance criteria.**

- Test AUC at least 0.70 and above the baseline you wrote down.
- Brier score reported; calibration table within about 3 percentage points per bin.
- Profit under the decision rule beats send-to-all on the held-out 30%, by a stated margin.
- A sensitivity table: profit when save rate is 10%, 20%, 30% and offer cost is 3, 5, 8.

**Common pitfalls and how to detect them.**

1. *Leakage*: a feature like "cancelled_flag_date". Detect: one feature has implausibly high importance, or AUC above 0.95.
2. *Accuracy trap*: reporting 84.6% accuracy. Detect: compare with the majority-class number first.
3. *Probability as truth*: using uncalibrated scores in a cost formula. Detect: the calibration table.
4. *Treating "will churn" as "will respond"*: the offer works only on persuadable customers. Detect: ask whether you have any experiment data (Chapter 7).
5. *Random split on time-ordered data*: detect by training on early months and testing on later ones.

**Three extensions.**

1. Add isotonic or Platt calibration ( `CalibratedClassifierCV` , not run here) and compare Brier scores.
2. Replace the fixed 30% save rate with an uplift model trained on A/B test data (Chapter 8).
3. Add a SHAP or permutation-importance chart and explain the top 3 drivers in plain words.

**Interview talking points.**

- *Q: Why did logistic regression win?* A: The true pattern was close to linear, so extra flexibility only added noise. I would check this on real data too, not assume.
- *Q: How do you pick the contact threshold?* A: I do not use 0.5. I compute expected gain per customer from probability, save rate, value and cost, and contact only when it is positive.
- *Q: How do you know the probabilities are trustworthy?* A: A calibration table. In each risk bin, predicted and actual churn should match.

**README outline.** Question; baseline (send to all); results table (AUC, Brier, profit); calibration plot; assumptions and sensitivity; limits (synthetic data, no causal effect of the offer); how to reproduce.

## 15.3 Project: A/B test toolkit (#25)

**Goal and why employers care.** Build a small, tested library that does the four jobs every experimentation team needs: size a test, detect a broken split, reduce noise with CUPED, and stop safely. It shows statistical judgement, which is rarer than modelling skill.

**Skills used.** Chapter 4, *Statistical Thinking: Ways Data Fools You*; Chapter 7, *Experimentation: A/B Testing Done Properly* (sample size, SRM, CUPED, peeking).

**Data.** Simulated in the script: a binary conversion metric for sizing, two arm counts for SRM, a continuous spend metric with a pre-period spend for CUPED, and A/A simulations (both arms identical) for the peeking study.

**Architecture.**

```
abtoolkit/
  sizing.py     n_per_arm(p1, p2, alpha, power)
  srm.py        srm_p(n_a, n_b, expected)
  cuped.py      theta, adjust(), diff_se()
  sequential.py aa_false_positive(looks, alpha)
  tests/        known-answer tests for each module
notebooks/demo.ipynb   runs all four on one example
```

**Step-by-step build.**

1. **Sample size.** Implement the formula from Chapter 7. Test it: 10% to 11% must give 14,751 per arm.

2. **SRM.** The chi-square test compares the counts you got with the split you intended. A tiny p-value means the assignment or logging is broken, so do not read the results.
3. **CUPED.** Fit `θ = cov(post, pre) / var(pre)`, subtract `θ × (pre − mean pre)` from the outcome, then compare arms. The adjusted metric has the same expected lift and less noise.
4. **Peeking.** Run many A/A tests (no true effect) and stop at the first `p < 0.05` over 5 looks. Count false alarms.
5. **Fix.** Use a stricter alpha per look; here 0.016 is found by simulation, in the spirit of Pocock boundaries. For a real programme use a vetted group-sequential method.

```
import numpy as np
from scipy import stats

# Step 1: sample size
def n_per_arm(p1, p2, alpha=0.05, power=0.8):
    za, zb = stats.norm.ppf(1-alpha/2), stats.norm.ppf(power)
    pb = (p1+p2)/2
    return (za*np.sqrt(2*pb*(1-pb)) + zb*np.sqrt(p1*(1-p1)+p2*(1-p2)))**2/(p2-p1)**2
print("n per arm:", round(n_per_arm(.10,.11)))

# Step 2: sample-ratio-mismatch check
def srm_p(n_a, n_b, expected=0.5):
    n = n_a+n_b
    return stats.chisquare([n_a,n_b],[n*expected, n*(1-expected)]).pvalue
print("SRM ok  :", round(srm_p(5030,4970),3))
print("SRM bad :", srm_p(5300,4700))

# Step 3: CUPED on a continuous metric (spend), pre-period spend as covariate
r = np.random.default_rng(1)
n = 4000
pre = r.gamma(4, 10, 2*n)
post = 0.8*pre + r.normal(0, 8, 2*n)
t = np.r_[np.zeros(n), np.ones(n)]
post = post + 1.0*t                                  # true lift = +1.0
def diff_se(y, t):
    a, b = y[t==0], y[t==1]
    return b.mean()-a.mean(), np.sqrt(a.var(ddof=1)/len(a)+b.var(ddof=1)/len(b))
theta = np.cov(post, pre)[0,1]/pre.var(ddof=1)
adj = post - theta*(pre-pre.mean())
d0,s0 = diff_se(post,t); d1,s1 = diff_se(adj,t)
print(f"theta={theta:.3f}")
print(f"raw   lift={d0:.3f} se={s0:.3f}")
print(f"CUPED lift={d1:.3f} se={s1:.3f}  var cut={1-(s1/s0)**2:.1%}  corr^2=
{np.corrcoef(post,pre)[0,1]**2:.3f}")

# Step 4: peeking inflates false alarms: A/A tests, 5 looks, stop at first p<0.05
def aa_false_positive(looks, per_look=400, sims=2000, alpha=0.05, seed=2):
    g = np.random.default_rng(seed); hits=0
    for _ in range(sims):
        a = g.normal(size=per_look*looks); b = g.normal(size=per_look*looks)
        for k in range(1, looks+1):
            m = k*per_look
            p = stats.ttest_ind(a[:m], b[:m]).pvalue
            if p < alpha: hits += 1; break
    return hits/sims
print("one look  FPR:", aa_false_positive(1))
print("five looks FPR:", aa_false_positive(5))
```

```
# Step 5: fix: stricter alpha per look: same alpha per look, calibrated by simulation
print("five looks, alpha=0.016 each FPR:", aa_false_positive(5, alpha=0.016))
```

Real output:

```
n per arm: 14751
SRM ok  : 0.549
SRM bad : 1.973175290075393e-09
theta=0.802
raw   lift=1.758 se=0.401
CUPED lift=1.086 se=0.176  var cut=80.7%  corr^2=0.806
one look  FPR: 0.0465
five looks FPR: 0.1495
five looks, alpha=0.016 each FPR: 0.05
```

**Reading the output.** The size function returns 14,751. A 5,030 to 4,970 split passes SRM (p = 0.549), while 5,300 to 4,700 fails with p about 2e-9. In the CUPED step the pre-period explains 80.6% of the variance of the outcome, so the standard error falls from 0.401 to 0.176, a variance cut of 80.7%. Notice that the raw lift (1.758) is far from the true lift of 1.0, and CUPED (1.086) is much closer: less noise means less luck. Peeking five times inflated false alarms from 4.7% to 15.0%. Using 0.016 per look brings it back to 5.0%.

**Evaluation and acceptance criteria.**

- Sizing agrees with a simulation: at the computed n, power is 0.80 plus or minus 0.02 (use 2,000 runs).
- SRM test rejects 5,300/4,700 and accepts 5,030/4,970.
- CUPED variance reduction is close to the squared correlation $\rho^2$ of pre and post metrics.
- A/A false positive rate: about 5% for one look, and about 5% for five looks with the corrected alpha.

**Common pitfalls and how to detect them.**

1. *Using post-treatment data as the covariate*: CUPED needs a covariate measured before assignment. Detect: the covariate differs between arms.
2. *Skipping SRM*: detect by always running it first and refusing to report if it fails.
3. *Peeking without correction*: detect with the A/A simulation above.
4. *Wrong unit*: randomise by customer but analyse by order. Detect: compare the number of rows with the number of customers.
5. *Underpowered "no effect"*: a non-significant result at low power says little. Report the confidence interval.

**Three extensions.**

1. Add a two-sided confidence interval and a Bayesian credible interval to the output.
2. Add a multiple-metric correction (Bonferroni and Benjamini-Hochberg).

3. Add a sequential method with a proper alpha-spending function and compare with the simulation.

**Interview talking points.**

- *Q: What is SRM and why check it first?* A: Sample ratio mismatch means the arm sizes differ from the plan more than chance allows. It means the experiment is broken, so any lift number is untrustworthy.
- *Q: How does CUPED help?* A: It uses each customer's past behaviour to remove noise that has nothing to do with the treatment, so the same traffic gives a tighter answer or the test ends earlier.
- *Q: Why is peeking bad?* A: Each look is another chance to see a fluke. With five looks, false alarms went from 5% to about 15% in my simulation.

**README outline.** Purpose; four tools with one-line examples; the table of simulation checks; how to read an SRM failure; limits (not a replacement for a full platform); how to test.

## 15.4 Project: Demand forecasting with backtesting and intervals (#26)

**Goal and why employers care.** Forecast daily shop demand and prove the model is better than naive guesses, with an honest range around each number. Stock, staffing and delivery plans depend on forecasts, and a forecast without a baseline is just a story.

**Skills used.** Chapter 9, *Forecasting and Time Series for Decisions* (rolling-origin backtests, MASE, intervals); Chapter 5, *Feature Engineering I* (dates, lags); Chapter 12, *From Prediction to Decision*.

**Data.** The generator makes 730 days of orders: a slow upward trend, a yearly wave and a weekend bump, with Poisson noise (counts of orders). Real alternative: any public daily retail sales file; check the licence.

**Architecture.**

```
series y (730 days)
   |
   +-- fold k: train on days [0, cut), forecast the next 14 days
   |     models: naive | seasonal naive | ridge on lag1, lag7, weekday
   |     cut moves forward 14 days each fold (26 folds)
   v
errors per fold -> MAE, MAPE, MASE -> interval from past errors
```

**Step-by-step build.**

1. Generate the series. Keep the last value, last week and the model all as callable functions with the same signature `f(h, hist)`.

2. Use a **rolling-origin backtest**: at each cut, use only the past, predict 14 days, record the error, move the cut forward. This copies real use. A random split would leak the future.
3. Always include two baselines: *naive* (repeat the last value) and *seasonal naive* (repeat the same weekday last week).
4. Report MAE (average absolute miss), MAPE (average percent miss) and MASE (error divided by the error of the in-sample seasonal naive; below 1 is better).
5. Build an 80% prediction interval from past backtest errors. Check its coverage: about 80% of actuals should fall inside.

```
import numpy as np, pandas as pd
from sklearn.linear_model import Ridge

# Step 1: data
def make_sales(n_days=730, seed=3):
    r = np.random.default_rng(seed)
    d = pd.date_range("2023-01-01", periods=n_days, freq="D")
    t = np.arange(n_days)
    mu = 100 + 0.05*t + 15*np.sin(2*np.pi*t/365.25) + np.where(d.dayofweek>=5, 20, 0)
    return pd.Series(r.poisson(mu).astype(float), index=d)
y = make_sales()

# Step 2: models
def naive(h, hist): return np.repeat(hist.iloc[-1], h)
def seasonal_naive(h, hist): return np.array([hist.iloc[-7 + (i % 7)] for i in range(h)])
def lag_model(h, hist):
    # recursive model: lag1, lag7, dow
    idx = hist.index; v = hist.values
    rows = [(v[i-1], v[i-7], *np.eye(7)[idx[i].dayofweek]) for i in range(7, len(v))]
    m = Ridge(alpha=1.0).fit(np.array(rows), v[7:])
    out=[]; buf=list(v); day=idx[-1]
    for _ in range(h):
        day += pd.Timedelta(days=1)
        out.append(m.predict([[buf[-1], buf[-7], *np.eye(7)[day.dayofweek]]])[0]);
buf.append(out[-1])
    return np.array(out)

# Step 3: rolling-origin backtest, 14-day horizon, origin moves 14 days each fold
H, step, start = 14, 14, 365
res = {k: [] for k in ["naive","seasonal_naive","lag_ridge"]}; actuals=[]; resid=[]
for cut in range(start, len(y)-H+1, step):
    hist, fut = y.iloc[:cut], y.iloc[cut:cut+H].values
    actuals.append(fut)
    for k, f in [("naive",naive),("seasonal_naive",seasonal_naive),
("lag_ridge",lag_model)]:
        res[k].append(f(H, hist))
A = np.array(actuals)
print("days", len(y), "mean daily sales", round(y.mean(),1))
print("folds", len(A))
for k in res:
    P = np.array(res[k]); mae = np.abs(A-P).mean()
    print(f"{k:15s} MAE={mae:6.2f}  MAPE={np.mean(np.abs(A-P)/A)*100:5.2f}%")
# Step 4: MASE vs naive-7 in-sample scale
scale = np.abs(y.values[7:]-y.values[:-7]).mean()
P = np.array(res["lag_ridge"]); print("MASE lag_ridge:", round(np.abs(A-
P).mean()/scale,3))
# Step 5: empirical 80% prediction interval prediction interval from backtest residuals
```

```
(first half folds calibrate, second half test)
E = A-P; half = len(E)//2
lo, hi = np.quantile(E[:half], [0.1, 0.9])
cov = np.mean((E[half:]>=lo)&(E[half:]<=hi))
print(f"80% interval offsets [{lo:.1f}, {hi:.1f}] coverage on later folds: {cov:.3f}")
# Step 6: rolling calibration: use residuals from the 8 most recent finished folds
hits=[]
for k in range(8, len(E)):
    lo, hi = np.quantile(E[k-8:k], [0.1, 0.9])
    hits.append(np.mean((E[k]>=lo)&(E[k]<=hi)))
print(f"rolling-8-fold 80% interval coverage: {np.mean(hits):.3f} over {len(hits)} folds")
```

Real output:

```
days 730 mean daily sales 124.5
folds 26
naive           MAE= 21.07  MAPE=16.61%
seasonal_naive  MAE= 13.12  MAPE= 9.93%
lag_ridge       MAE= 12.61  MAPE= 9.14%
MASE lag_ridge: 1.005
80% interval offsets [-6.6, 28.0] coverage on later folds: 0.692
rolling-8-fold 80% interval coverage: 0.750 over 18 folds
```

**Reading the output.** Naive misses by 21.07 orders a day, seasonal naive by 13.12, the ridge model by 12.61. So the model beats the strong baseline by only about 4%. That is an honest and useful message: most of the gain comes from knowing the weekly pattern, and a fancy model adds little. MASE of 1.005 says the model is about as good as the in-sample seasonal naive. Interval coverage was the surprise: an interval learned from the first half of the folds covered only 69.2% of later actuals, because the early errors were biased (the model underestimates a growing trend). Calibrating on only the 8 most recent folds gave 75.0%, closer to the 80% target. Say this in your README: intervals must be re-checked, and they drift when the world drifts.

**Evaluation and acceptance criteria.**

- The model beats naive and seasonal naive in MAE on at least 60% of folds (count it; not printed above).
- MASE reported; result below 1.1 and the README says whether it beats seasonal naive.
- 80% interval coverage between 70% and 90% on held-out folds.
- Backtest has at least 20 folds, and a time plot of forecast vs actual for 3 folds.

**Common pitfalls and how to detect them.**

1. *Shuffled split*: future leaks into the past. Detect: the score is better than the rolling-origin score.
2. *Beating only naive*: always compare with seasonal naive. Detect: print all baselines side by side.
3. *MAPE with small or zero values*: huge or undefined errors. Detect: min of actuals; use MASE or MAE.

4. *Recursive forecast error compounding*: errors at day 14 are bigger than day 1. Detect: error by horizon.
5. *Intervals from in-sample residuals*: too narrow. Use backtest errors as shown.

**Three extensions.**

1. Add holiday and promotion flags and measure the gain in MAE.
2. Replace Ridge with gradient boosting on lag features and compare to the baselines.
3. Add a quantile-regression interval and compare its coverage with the empirical one.

**Interview talking points.**

- *Q: Why a rolling-origin backtest?* A: It repeats how the forecast is used, always predicting forward from the past. Shuffled splits let future patterns leak and flatter the model.
- *Q: Your model is only 4% better than seasonal naive. Ship it?* A: Maybe not. I would weigh the 4% against the extra maintenance, and check whether the gain holds on busy weeks.
- *Q: How do you know the interval is honest?* A: I count how often the actual falls inside across many backtest folds. Mine was 75% for an 80% target, so I would widen it slightly.

**README outline.** Question; baselines table (naive, seasonal naive, model); backtest design diagram; interval coverage; error by horizon; limits; how to rerun.

## 15.5 Worked Debugging Problems and a Self-Marking Rubric

The three projects above tell you what to build. This section shows what usually goes wrong and how to find it. Each problem is a bug that real teams ship. Try to spot the bug before you read the steps. Then use the rubric at the end to mark your own project.

| INTUITION | A result that looks too good is a bug until proven otherwise. A held-out AUC of 1.0, a forecast that beats a seasonal naive by half, a lift that appears only after you drop some rows: each is a signal that something about time, selection or counting is wrong. |
| --- | --- |

**Problem 15.5.1** (churn project, leaky feature) You build "days since last order" for 6,000 customers. The label is "no order in the 30 days after day 100". Your model scores AUC 1.0 on a held-out set. What is wrong?

| | |
|---|---|
| **Suspect** | Which date was the feature measured at? If the pull date is day 130, "days since last order" at day 130 already knows whether an order happened in days 101 to 130. |
| **Test** | Rebuild the same feature at the cutoff (day 100) using only orders on or before that day. Compare AUC. |
| **Rule** | Every feature is computed from rows with timestamp at or before the cutoff. Every label from rows after it. |

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
rng = np.random.default_rng(10)
n, cutoff, window = 6000, 100, 30
rate = rng.gamma(2.0, 0.02, n) + 0.005                      # orders per day
last_before = np.zeros(n); cnt90 = np.zeros(n); last_after = np.zeros(n); churn =
np.zeros(n)
for i in range(n):
    times = np.cumsum(rng.exponential(1/rate[i], 60))
    times = times[times < cutoff + window]
    before = times[times <= cutoff]; after = times[times > cutoff]
    lb = before.max() if len(before) else 0.0
    last_before[i] = cutoff - lb
    cnt90[i] = ((before > cutoff - 90)).sum()
    churn[i] = float(len(after) == 0)                       # label: no order in next 30
days
    allt = np.r_[before, after]
    last_after[i] = (cutoff + window) - (allt.max() if len(allt) else 0.0)   # recency
measured at pull date (leak)
tr, te = slice(0, 4500), slice(4500, 6000)
def auc(X):
    m = LogisticRegression(max_iter=1000).fit(X[tr], churn[tr]); return
roc_auc_score(churn[te], m.predict_proba(X[te])[:, 1])
honest = np.c_[last_before, cnt90]
leaky = np.c_[last_after, cnt90]
print("churn rate:", round(churn.mean(), 3))
print("honest features (recency and count at cutoff) AUC:", round(auc(honest), 3))
print("leaky feature (recency at the pull date) AUC:     ", round(auc(leaky), 3))
print("correlation of leaky recency with label:", round(np.corrcoef(last_after, churn)[0,
1], 3))
```

```
churn rate: 0.336
honest features (recency and count at cutoff) AUC: 0.685
leaky feature (recency at the pull date) AUC:      1.0
correlation of leaky recency with label: 0.796
```

- **Step 1** The leaky feature gives AUC 1.0 and correlates 0.796 with the label. A single feature that perfect is the giveaway.
- **Step 2** The honest features give 0.685, a believable churn model.
- **Step 3** Add a unit test: for a customer with a known order on day 110, assert that the feature at cutoff 100 does not change when that order is added.

| COMMON MISTAKE | Fixing the leak in the training data but not in the scoring path. In production the feature is computed "as of now", which is right. In training, "as of the pull date" is wrong. Build both from one function that takes the as-of date as an argument. |
|---|---|

**Problem 15.5.2** (A/B toolkit, a bug that fakes a win) 20,000 users are assigned 50/50. There is no real effect. The analyst drops users with no logged events and finds a lift of about 1.6 in mean spend with p below 0.001. Find the bug.

| | |
|---|---|
| **Step 1** | Run the SRM check on the assigned counts, then on the analysed counts. |
| **Step 2** | If the second check fails, filtering is related to treatment. Ask what the filter removes. |
| **Step 3** | Re-estimate using everyone assigned (intention to treat). |

```
import numpy as np
from scipy import stats
rng = np.random.default_rng(11)
n = 20000
arm = rng.integers(0, 2, n)                                    # fair coin
spend = rng.gamma(2, 10, n)                                    # NO true effect of the
treatment
logged = ~((arm == 1) & (spend < 5))                           # bug: treated users with
tiny spend lose their record
nA, nB = (arm == 0).sum(), (arm == 1).sum()
print("assigned:", nA, nB, " SRM p =", round(stats.chisquare([nA, nB]).pvalue, 3))
kA, kB = ((arm == 0) & logged).sum(), ((arm == 1) & logged).sum()
print("analysed:", kA, kB, " SRM p =", format(stats.chisquare([kA, kB]).pvalue, ".2e"))
a, b = spend[(arm == 0) & logged], spend[(arm == 1) & logged]
t = stats.ttest_ind(b, a)
print("fake lift in mean spend:", round(b.mean() - a.mean(), 2), " p =", format(t.pvalue,
".3f"))
a0, b0 = spend[arm == 0], spend[arm == 1]
print("lift using everyone assigned:", round(b0.mean() - a0.mean(), 2))
```

```
assigned: 9990 10010  SRM p = 0.888
analysed: 9990 9122  SRM p = 3.42e-10
fake lift in mean spend: 1.57  p = 0.000
lift using everyone assigned: -0.04
```

- **Step 1** The assignment is fine (p = 0.888). The analysed counts are not: 9,990 against 9,122, with p near 3e-10.
- **Step 2** In this simulation a bug dropped the treated users with the lowest spend. Removing the lowest values of one arm raises its mean, so a fake lift appears.
- **Step 3** Using everyone assigned, the lift is −0.04, effectively zero.

| KEY IDEA | Analyse everyone you randomised. Run SRM at each stage (assigned, exposed, analysed). If a stage fails, fix the logging bug and do not trust the effect. |
| --- | --- |

**Problem 15.5.3** (forecasting project, a backtest that cheats) You smooth the series with a centred 3-day average, then forecast tomorrow as yesterday's smoothed value. Compared with the naive "yesterday" forecast, your MAE looks 35% better. Is it real?

| | |
|---|---|
| **Suspect** | A centred window at day t−1 uses days t−2, t−1 and t. The forecast has seen the day it predicts. |
| **Test** | Replace the centred window with a trailing one (only days up to the forecast origin). |
| **Expectation** | The honest smoother may be no better than naive. That is a normal outcome and a valid finding. |

```
import numpy as np, pandas as pd
rng = np.random.default_rng(14)
n = 300
d = pd.Series(100 + 0.1*np.arange(n) + 10*np.sin(2*np.pi*np.arange(n)/7) + rng.normal(0,
6, n))
train, test = d[:240], d[240:]
# bug: smooth the WHOLE series with a centred window, then forecast from the smoothed
history
bad_smooth = d.rolling(3, center=True, min_periods=1).mean()
# one-step-ahead forecast = yesterday's smoothed value; compare MAE
f_bad = bad_smooth.shift(1)[240:]
f_ok = d.rolling(3, min_periods=1).mean().shift(1)[240:]          # trailing window only
f_naive = d.shift(1)[240:]
mae = lambda f: float(np.abs(f - test).mean())
print("naive yesterday    MAE:", round(mae(f_naive), 2))
print("trailing mean      MAE:", round(mae(f_ok), 2))
print("centred mean (leak) MAE:", round(mae(f_bad), 2))
```

```
naive yesterday    MAE: 8.0
trailing mean      MAE: 8.26
centred mean (leak) MAE: 5.2
```

- **Step 1** The centred version reports MAE 5.20 against 8.00 for naive, which looks like a 35% gain.
- **Step 2** The trailing version reports 8.26, slightly worse than naive. The gain was entirely the leak.

### Reference ranges to check yourself against

| Project | Healthy signal | Red flag |
|---|---|---|
| Churn analysis (15.2) | AUC 0.65 to 0.80 on a time-based split; top decile captures 2x to 3x the base churn rate | AUC above 0.95; one feature carrying almost all importance |
| A/B toolkit (15.3) | Sizing test returns 14,751 for 10% to 11%; A/A false positive near 5% with one look | A/A false positives far from 5%; SRM p below 0.001 ignored |
| Demand forecast (15.4) | Beats seasonal naive by a few percent on rolling origins; interval coverage within a few points of target | Beats naive by 30% or more with no new information; coverage far below target |

### Rubric: mark your own project (0 to 2 each)

| Criterion | 0 | 1 | 2 |
|---|---|---|---|
| Question framed | No decision named | Decision named, no cost | Decision, metric and cost of errors stated |
| Baseline | None | Naive baseline | Naive and strong simple baseline, same split |
| Validation honesty | Random split on time data | Time split | Time split plus a leak test in code |
| Uncertainty | Point numbers only | Intervals once | Intervals checked for coverage |
| Tests | None | Some smoke tests | Known-answer tests for every module |
| Communication | Code dump | Readable README | One-page decision memo with caveats |

Score 10 to 12: portfolio-ready. 6 to 9: fix the lowest rows first. Below 6: start again from the baseline.

## Chapter recap

- A project starts with one sentence: who decides, what you predict, what action follows, and what number means success.
- The repo should let a stranger run it in five minutes; commits should read like a story.

- Generated data is fine if you say so, and it gives you a known answer to test against.
- Churn: judge ranking (AUC) and honesty (calibration), then convert probability to money with a cost rule.
- A simpler model can win when the true pattern is simple. Report that, not hide it.
- A/B toolkit: size first, check SRM first, use CUPED to cut noise, never peek without a correction.
- Forecasting: rolling-origin backtest, two baselines, MASE, and check interval coverage.
- Every project needs limits written honestly.

| Pattern | Formula or command |
|---|---|
| Sample size per arm (rule) | $n \approx 16 \cdot p(1-p)/\delta^2$ |
| SRM | chi-square on arm counts vs plan |
| CUPED | $y_{adj} = y - \theta(x_{pre} - \text{mean } x_{pre})$, $\theta = \text{cov}/\text{var}$ |
| Expected gain of a contact | $P(\text{churn}) \times P(\text{save}) \times \text{value} - \text{cost}$ |
| MASE | $\text{MAE} / \text{MAE of in-sample seasonal naive}$ |

## Practice set

**1.** (easy) Using the rule of thumb, how many customers per arm to detect a lift of 2 points on a 10% baseline? **Answer:** $16 \times 0.1 \times 0.9 / 0.02^2 = 3{,}600$ per arm.

**2.** (easy) Arms of 5,300 and 4,700 were planned as 50/50. Compute the chi-square statistic. **Answer:** expected 5,000 each; $(300^2/5000) + (300^2/5000) = 18 + 18 = 36$. With 1 degree of freedom that is far beyond 3.84, so SRM fails.

**3.** (easy) A customer has churn probability 0.20, a save rate of 30%, a value of 100 and an offer cost of 5. Send or not? **Answer:** $0.2 \times 0.3 \times 100 - 5 = +1$, so send. With probability 0.04 it is $-3.8$, so do not send.

**4.** (medium) Pre and post metrics have correlation 0.7. How much variance does CUPED remove? **Answer:** about $\rho^2 = 0.49$, so 49% is removed and 51% remains.

**5.** (medium) What is the maximum false alarm rate if you check 5 times at alpha 0.05 and the looks were independent? Why did our simulation show 15%, not this? **Answer:** $1 - 0.95^5 = 22.6\%$. Looks share the same accumulating data, so they are positively correlated, and the real rate is lower (15% in our run).

**6.** (find the bug) A forecaster scores a model with `train_test_split(shuffle=True)` on daily sales and reports MAPE 3%. What is wrong and what do you do? **Answer:** shuffling leaks neighbouring days into training. Use a rolling-origin backtest; expect the error to rise.

**7.** (scenario) The churn model has AUC 0.82, but the marketing team says the offer made no difference. Give two reasons. **Answer:** (a) The model finds customers likely to leave, not customers who are persuadable; the save rate may be near zero. (b) No control group existed, so the effect cannot be measured. Fix: run a randomised holdout.

**8.** (interview) "Your A/B test shows p = 0.04 after the fifth daily look. Ship?" **Answer:** Not yet. Ask whether the stopping rule was fixed in advance. With five unplanned looks, the false alarm rate is far above 5%. Use the planned sample size or a sequential boundary, and check SRM first.

**9. (medium, find the bug)** A churn model scores AUC 0.99 after you add "number of support tickets in the last 30 days". The tickets are counted up to the pull date, which is 30 days after the cutoff. What is wrong and what is the fix? **Answer:** The count includes tickets raised after the cutoff, and customers who are about to leave raise many tickets. It is label leakage through time. Count tickets only up to the cutoff, check by comparing AUC.

**10. (hard)** Your SRM check on assigned users passes, but it fails on users who reached the checkout page. Is the experiment broken? **Answer:** Not necessarily. Reaching checkout can be an effect of the treatment (a better product page sends more people there). Conditioning on a post-treatment variable breaks balance by itself. Analyse everyone assigned and treat checkout as a funnel metric.

## Prompt pack

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. I want to plan a portfolio project on customer churn with a decision step (contact or not). Ask me five questions one at a time about my data, the action the business can take and the cost of that action. Then help me write a one-sentence project question, a baseline, a metric and three acceptance criteria. Keep your English simple and define any jargon.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Review my churn project for leakage, split mistakes and calibration problems. Go through these checks one by one: feature timing, train/test split, class balance, baseline, calibration table and cost assumptions. For each check say pass, fail or unknown and what evidence you need. My code and results: [PASTE HERE]. Use simple English.
```

✨ **AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Teach me to debug an A/B test toolkit step by step: sample size, sample ratio
```

mismatch, CUPED and peeking. Give me a known-answer test for each tool that I can write in Python with a fixed seed, and tell me what number I should see. Then quiz me on one failure at a time. My code: [PASTE HERE]. Use simple English.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles. I built a daily demand forecast. Review my backtest design: rolling origin, folds, baselines (naive and seasonal naive), MASE and prediction interval coverage. Tell me what is missing, what could leak the future, and what extra chart would convince a sceptical manager. My setup and numbers: [PASTE HERE]. Use simple English.

**✨ AI prompt: copy and paste into any AI**

You are my patient, practical professor. I am a working professional moving into data roles. Run a mock interview about my three portfolio projects (churn decision, A/B test toolkit, demand forecasting). Ask one question at a time, wait for my answer, then give feedback on correctness, clarity and one thing I should add. Start with the churn project. Keep your questions practical and use simple English.

CHAPTER 16

# PROJECT LAB 2: RECOMMENDER, CREDIT SCORECARD AND CAUSAL ANALYSIS

*Three more portfolio builds, each with runnable code, real output, acceptance criteria and interview answers.*

The first lab chapter gave you the method for running a project. This one uses it on three projects that employers like because each hides a trap. A recommender can look good on a number while ignoring new items. A credit scorecard can have a high score and still fail once the applicants change. A causal analysis can give a tidy answer that is wrong because something was never measured. In each project you will build the working version, then deliberately measure how it can mislead. Use the method, repo layout, README template and checklists from Chapter 15, section 15.1. All scripts were run in this session with `numpy`, `pandas`, `scipy` and `scikit-learn` and all data is synthetic with fixed seeds.

## 16.1 Project: Two-stage recommender with NDCG@10 and cold start (#27)

**Goal and why employers care.** Real recommenders cannot score every item for every user in real time. They use two stages: a fast *candidate generator* picks about 100 items, then a slower *ranker* orders them. Building this, measuring it with a ranking metric and handling brand-new items shows you understand how large systems work.

**Skills used.** Chapter 10, *Recommendation, Ranking and Search* (retrieval, ranking, NDCG, cold start); Chapter 5, *Feature Engineering I* (item features); Chapter 4, *Statistical Thinking* (baselines and fair comparison).

**Data.** A MovieLens-style table made by the generator: 600 users, 300 items, about 40 ratings per user, ratings 1 to 5 from hidden taste factors plus item popularity. The last 30 items are *cold*: they have no training interactions. For each user, the 5 highest-rated liked items (rating 4 or 5) are held out as the test. The real MovieLens data (free, with a research licence; read it before use) can replace the generator. Not run here: real MovieLens.

**Architecture.**

```
ratings -> train / test (per-user holdout; cold items only in test)
             |
   stage 1:  item-item similarity (70 candidates) + popularity (30)  -> 100 candidates
             |
   stage 2:  ranker = global mean + item bias + user bias + ALS factors
             |                          cold items: item features -> predicted scores
             v
   top 10 -> NDCG@10     (compare: popularity baseline, random cold ranking)
```

**Step-by-step build.**

1. Generate the data. Make interactions depend on taste ( `exp(1.2 × score)` ), otherwise popularity wins and no personal model can show value.
2. Define **NDCG@10**: reward each relevant item in the top 10 by `1/log2(rank+1)`, divide by the best possible. Build the popularity baseline first.
3. Stage 1: cosine similarity between items using the "liked" matrix; for each user sum the similarity to items they liked; add the most popular unseen items so a user with a thin history still gets candidates.
4. Stage 2: a small matrix-factorisation model by *alternating least squares* (ALS: fix user factors, solve items by least squares, then swap) on ratings after removing the global mean and biases.
5. Cold start: fit a Ridge regression from item features to the predicted-score column on warm items, then predict score columns for cold items.
6. Judge on three views: all test items, warm-only items, and a cold-only ranking.

**Before you run it, predict:** will the two-stage system beat popularity on all items? On warm items only?

```
import numpy as np
# Step 1: synthetic MovieLens-style data (users, items, ratings, cold items)
r = np.random.default_rng(11)
U, I, K = 600, 300, 6
uf = r.normal(size=(U,K)); itf = r.normal(size=(I,K))
pop = r.normal(size=I)*0.8
score = uf@itf.T/np.sqrt(K) + pop
# user rates 40 items drawn with popularity bias; rating = 1..5 from score
cold = set(range(I-30, I))                          # last 30 items are cold: no training
interactions
rows=[]
for u in range(U):
    pu = np.exp(1.2*score[u]); pu/=pu.sum()
    items = r.choice(I, 40, replace=False, p=pu)
    for i in items:
        rating = int(np.clip(np.round(3+score[u,i]+r.normal(0,.5)),1,5))
        rows.append((u,i,rating))
rows = np.array(rows)
# hold out each user's 5 highest-rated items (relevant = rating>=4) as test; cold items
only in test
test=[]; train=[]
for u in range(U):
    ru = rows[rows[:,0]==u]
    rel = ru[(ru[:,2]>=4)]
    t_items = set(rel[:5,1]) if len(rel) else set()
    for (_,i,rt) in ru:
        (test if (i in t_items or i in cold) else train).append((u,i,rt))
test=np.array([x for x in test if x[2]>=4]); train=np.array(train)
print("train", len(train), "test positives", len(test), "cold-item test positives",
int(np.isin(test[:,1], list(cold)).sum()))
seen = {u:set(train[train[:,0]==u][:,1]) for u in range(U)}
rel_by_user = {u:set(test[test[:,0]==u][:,1]) for u in range(U)}

# Step 2: metric and popularity baseline
def ndcg_at_k(ranked, relevant, k=10):
    dcg = sum(1/np.log2(i+2) for i,x in enumerate(ranked[:k]) if x in relevant)
    ideal = sum(1/np.log2(i+2) for i in range(min(k,len(relevant))))
    return dcg/ideal if ideal else np.nan
def evaluate(rank_fn, k=10):
    v=[ndcg_at_k(rank_fn(u), rel_by_user[u], k) for u in range(U) if rel_by_user[u]]
    return float(np.nanmean(v))

# baseline: popularity
cnt = np.bincount(train[:,1], minlength=I)
poprank = np.argsort(-cnt)
pop_fn = lambda u: [i for i in poprank if i not in seen[u]]
print("popularity NDCG@10:", round(evaluate(pop_fn),3))
```

```
# stage 1: candidate generation = item-item cosine on 'liked' (>=4) matrix, 100 candidates
M = np.zeros((U,I)); M[train[train[:,2]>=4][:,0], train[train[:,2]>=4][:,1]] = 1
norm = np.linalg.norm(M,axis=0)+1e-9
S = (M.T@M)/np.outer(norm,norm)
# Step 3: stage 1, candidate generation (item-item + popularity)
feat = itf + r.normal(0,.5,itf.shape)                    # item content features (noisy)
def candidates(u, n_cf=70, n_pop=30):
    liked = list(np.where(M[u]>0)[0])
    s_ = S[:, liked].sum(axis=1) if liked else np.zeros(I)
    s_ = s_.copy(); s_[list(seen[u])] = -1
    cf = list(np.argsort(-s_)[:n_cf])
    pp = [i for i in poprank if i not in seen[u] and i not in cf][:n_pop]
    return cf + pp
cg_fn = lambda u: candidates(u)
print("candidate gen only NDCG@10:", round(evaluate(cg_fn),3))
recall = np.mean([len(set(candidates(u)) & rel_by_user[u])/len(rel_by_user[u]) for u in
range(U) if rel_by_user[u]])
print("candidate recall@100 (item-item + popularity):", round(float(recall),3))

# Step 4: stage 2, ranker (biases + ALS factors): ranker = matrix factorisation by ALS-
lite (SVD of mean-centred ratings) blended with popularity
R = np.zeros((U,I)); Mk = np.zeros((U,I))
R[train[:,0],train[:,1]] = train[:,2]; Mk[train[:,0],train[:,1]] = 1
mu = R.sum()/Mk.sum()
bi = ((R-mu)*Mk).sum(0)/(Mk.sum(0)+5)                    # item bias, shrunk
bu = (((R-mu)-bi)*Mk).sum(1)/(Mk.sum(1)+5)               # user bias
def als(Rc, Mk, k=6, lam=8.0, iters=12, seed=0):
    g = np.random.default_rng(seed); P = g.normal(0,.1,(Rc.shape[0],k)); Q =
g.normal(0,.1,(Rc.shape[1],k))
    for _ in range(iters):
        for u in range(P.shape[0]):
            m = Mk[u]>0; A = Q[m]; P[u] = np.linalg.solve(A.T@A+lam*np.eye(k),
A.T@Rc[u,m])
        for i in range(Q.shape[0]):
            m = Mk[:,i]>0
            if m.any(): A = P[m]; Q[i] = np.linalg.solve(A.T@A+lam*np.eye(k), A.T@Rc[m,i])
    return P, Q
Pf, Qf = als(R-mu-bi-bu[:,None], Mk)
pred = mu + bi[None,:] + bu[:,None] + Pf@Qf.T
# Step 5: cold items: feature -> predicted-score columns learn feature -> predicted-score
columns on warm items, apply to cold items
from sklearn.linear_model import Ridge
warm = [i for i in range(I) if i not in cold]; cl = sorted(cold)
W = Ridge(alpha=5).fit(feat[warm], pred[:, warm].T)
pred_full = pred.copy(); pred_full[:, cl] = W.predict(feat[cl]).T
# add a small cold-item content source to candidates
def cands2(u):
```

```
    c = candidates(u, 60, 20)
    liked = [i for i in seen[u] if R[u,i]>=4]
    w = feat[liked].mean(axis=0) if liked else np.zeros(K)
    extra = [i for i in np.argsort(-(feat[cl]@w)) ]
    return c + [cl[j] for j in extra[:20]]
def rank_fn(u):
    return sorted(cands2(u), key=lambda i: -pred_full[u,i])
print("two-stage NDCG@10:", round(evaluate(rank_fn),3))
cold_rel = {u: rel_by_user[u] & cold for u in range(U)}
def cold_eval(fn):
    v=[ndcg_at_k(fn(u), cold_rel[u]) for u in range(U) if cold_rel[u]]; return
float(np.mean(v)), len(v)
# Step 6: fair view: judge only on warm positives (cold items cannot be found from
history)
warm_rel = {u: rel_by_user[u]-cold for u in range(U)}
def warm_eval(fn):
    return float(np.mean([ndcg_at_k(fn(u), warm_rel[u]) for u in range(U) if
warm_rel[u]]))
print("warm-only NDCG@10 popularity:", round(warm_eval(pop_fn),3))
print("warm-only NDCG@10 two-stage :", round(warm_eval(rank_fn),3))
# Step 7: cold-start check: rank only the 30 new items for each user who liked some of
them
g = np.random.default_rng(0)
rand_fn = lambda u: list(g.permutation(cl))
feat_fn = lambda u: sorted(cl, key=lambda i: -pred_full[u,i])
print("cold-catalogue NDCG@10 random      :", tuple(round(x,3) for x in
cold_eval(rand_fn)))
print("cold-catalogue NDCG@10 feature->emb:", tuple(round(x,3) for x in
cold_eval(feat_fn)))
```

Real output:

```
train 18905 test positives 4623 cold-item test positives 1944
popularity NDCG@10: 0.198
candidate gen only NDCG@10: 0.293
candidate recall@100 (item-item + popularity): 0.536
two-stage NDCG@10: 0.297
warm-only NDCG@10 popularity: 0.273
warm-only NDCG@10 two-stage : 0.409
cold-catalogue NDCG@10 random      : (0.227, 585)
cold-catalogue NDCG@10 feature->emb: (0.388, 585)
```

**Reading the output.** Popularity scores NDCG@10 of 0.198. The candidate stage alone gives 0.293 and the two-stage system 0.297. The ranker adds little overall because 1,944 of the 4,623 test positives are cold items, which stage 1 cannot find from history. On warm items only, the gap is clear: 0.273 for popularity against 0.409 for two-stage. For cold items, ranking the 30 new items with the feature-based predictor gives 0.388, against

0.227 for random order. Candidate recall at 100 is only 0.536, which caps what any ranker can do, so improving stage 1 is the next win. Always report recall of stage 1 and NDCG of stage 2 separately.

**Evaluation and acceptance criteria.**

- NDCG@10 of two-stage greater than the popularity baseline on the warm-only view, with the numbers in a table.
- Candidate recall@100 reported; target above 0.5 here.
- Cold-catalogue NDCG@10 above the random-order value (0.227 here) by a clear margin.
- Items the user already saw are never recommended (add a test).

**Common pitfalls and how to detect them.**

1. *Weak baseline*: forgetting popularity. Detect: print it first.
2. *Test items in training*: leakage. Detect: assert that no (user, item) pair is in both sets.
3. *Recommending seen items*: detect with an assertion.
4. *Evaluating the ranker on a bad candidate list*: low recall hides ranker quality. Detect: report recall.
5. *Judging cold start with all-item NDCG*: cold items never reach the top 10, so the score is zero. Detect: use a cold-only ranking, as above.

**Three extensions.**

1. Replace ALS with a two-tower neural model and compare NDCG (not run here).
2. Add diversity: limit to 3 items from one category in the top 10 and measure the NDCG cost.
3. Simulate an online test of the recommender with a click model and size it with your Chapter 15 A/B toolkit.

**Interview talking points.**

- *Q: Why two stages?* A: Scoring every item for every user is too slow. A cheap first stage picks a few hundred plausible items and a richer second stage orders them.
- *Q: How do you handle a new item with no ratings?* A: Use its features (genre, text, price) to predict how users would score it, mix in some exploration, and switch to behaviour data as it arrives.
- *Q: Why can NDCG go up offline and fail online?* A: Offline data only shows what users saw before. The model may favour popular items, and real users react to the new list differently, so you confirm with an A/B test.

**README outline.** Question; data and split diagram; baselines; two-stage design; three-view results table; cold-start method; limits (synthetic ratings, offline only); how to run.

# 16.2 Project: Credit-risk scorecard with WoE, KS, Gini and PSI (#28)

**Goal and why employers care.** Banks still rely on scorecards: a points table that anyone can audit. Building one proves you can bin variables safely, fit a logistic model, measure separation (KS, Gini), convert to points and monitor drift (PSI). It is a classic of regulated data science.

**Skills used.** Chapter 5, *Feature Engineering I* (binning and encoding); Chapter 11, *Applied Domains* (credit risk); Chapter 12, *From Prediction to Decision* (cut-offs); Chapter 13, *Communication, Stakeholders, Ethics, Reproducibility and Taking Work to Production* (monitoring and explainability).

**Data.** The generator makes 30,000 loan applicants with age, income, credit utilisation and number of past late payments, and a default flag ( `bad` , 12.5%) from a known formula. A free real alternative is a public credit-default file; check its licence. Note that fairness rules apply to real credit data; do not use protected attributes.

**Architecture.**

```
applicants -> first 20,000 train / last 10,000 test
   |
   v
bin each variable (quantile bins, rule for late payments)
   |
   v
WoE per bin = ln(share of goods / share of bads);  IV = sum (diff of shares x WoE)
   |
   v
logistic regression on WoE columns -> probability of bad
   |
   v
points = offset + factor x ln(odds good)   ->  KS, Gini, PSI
```

**Step-by-step build.**

1. Split in a fixed order (train earlier, test later), like a real lending timeline.
2. For each variable, make about 5 bins from the *training* data only, count goods and bads per bin (add 0.5 to avoid log of zero), and compute WoE and IV. A common rule: IV below 0.02 is useless, 0.1 to 0.3 medium, above 0.3 strong. Here the IVs are modest.
3. Replace each value with its bin's WoE and fit logistic regression. Because WoE already points the right way, the coefficients should be near -1 (the sign comes from modelling bad as 1 while WoE is defined with goods on top).
4. Report AUC, Gini = `2 × AUC − 1` and KS (the biggest gap between the score distributions of bad and good customers).

5. Scale to points: 600 points at odds 50 to 1, and 20 points to double the odds ( `PDO = 20` ).
6. Compute PSI (population stability index): `Σ (actual% − expected%) × ln(actual% / expected%)` across 10 score bins, for the test set and for a drifted applicant group.

```
import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from scipy.stats import ks_2samp

def make_loans(n=30000, seed=5, shift=0.0):
    r = np.random.default_rng(seed)
    age = np.clip(r.normal(40+shift*8,11,n),21,70).round()
    income = r.lognormal(10.6-0.15*shift,0.5,n).round(-2)
    util = np.clip(r.beta(2,3,n)+0.1*shift,0,1.2).round(2)
    late = r.poisson(0.5+0.4*shift,n)
    z = -3.2 - 0.025*(age-40) - 0.6*np.log(income/40000) + 2.0*util + 0.55*late
    bad = (r.random(n) < 1/(1+np.exp(-z))).astype(int)
    return pd.DataFrame(dict(age=age,income=income,util=util,late=late,bad=bad))
# Step 1: data and split
df = make_loans(); print("bad rate", round(df.bad.mean(),3))
train = df.iloc[:20000].copy(); test = df.iloc[20000:].copy()

# Step 2: WoE / IV
def woe_table(x, y, edges):
    b = pd.cut(x, edges, include_lowest=True)
    g = pd.crosstab(b, y).rename(columns={0:"good",1:"bad"}) + 0.5     # smoothing avoids
log(0)
    g.columns.name=None
    g["dist_good"]=g.good/g.good.sum(); g["dist_bad"]=g.bad/g.bad.sum()
    g["woe"]=np.log(g.dist_good/g.dist_bad)
    g["iv"]=(g.dist_good-g.dist_bad)*g.woe
    return g
def quant_edges(x, k=5): return sorted(set(np.quantile(x, np.linspace(0,1,k+1))))
edges = {c: quant_edges(train[c]) for c in ["age","income","util"]}
edges["late"] = [-0.5,0.5,1.5,2.5,99]
ivs = {}
for c,e in edges.items():
    t = woe_table(train[c], train.bad, e); ivs[c] = t.iv.sum()
print("IV:", {k: round(float(v),3) for k,v in ivs.items()})
print(woe_table(train["util"], train.bad, edges["util"]).round(3)
[["good","bad","woe","iv"]])

# Step 3: model on WoE features
def to_woe(d):
    out = pd.DataFrame(index=d.index)
    for c,e in edges.items():
        t = woe_table(train[c], train.bad, e)["woe"]
        b = pd.cut(d[c].clip(e[0], e[-1]), e, include_lowest=True)
        out[c] = b.map(t).astype(float)
    return out
Xtr, Xte = to_woe(train), to_woe(test)
```

```
m = LogisticRegression(C=1.0, max_iter=1000).fit(Xtr, train.bad)
print("coefs:", dict(zip(Xtr.columns, [round(float(c),2) for c in m.coef_[0]])))
ptr, pte = m.predict_proba(Xtr)[:,1], m.predict_proba(Xte)[:,1]
def ks(y,p): return ks_2samp(p[y==1], p[y==0]).statistic
for nm,y,p in [("train",train.bad,ptr),("test",test.bad,pte)]:
    auc=roc_auc_score(y,p); print(f"{nm}: AUC={auc:.3f} Gini={2*auc-1:.3f} KS=
{ks(y.values,p):.3f}")

# Step 4: points; Step 5: PSI: score = 600 + 20/ln2*ln(odds_good) , PDO=20, base odds 50:1
at 600
pdo, base, odds0 = 20, 600, 50
factor = pdo/np.log(2); offset = base - factor*np.log(odds0)
def score(p_bad): return offset + factor*np.log((1-p_bad)/p_bad)
s_tr, s_te = score(ptr), score(pte)
print("score range test:", int(s_te.min()), int(s_te.max()), "mean", round(s_te.mean(),1))

def psi(expected, actual, edges):
    e = np.histogram(expected, edges)[0]/len(expected); a = np.histogram(actual, edges)
[0]/len(actual)
    e, a = np.clip(e,1e-4,None), np.clip(a,1e-4,None)
    return float(np.sum((a-e)*np.log(a/e)))
sedges = np.quantile(s_tr, np.linspace(0,1,11)); sedges[0]-=1; sedges[-1]+=1
print("PSI train vs test:", round(psi(s_tr, s_te, sedges),4))
drift = make_loans(10000, seed=9, shift=1.0)
s_dr = score(m.predict_proba(to_woe(drift))[:,1])
print("PSI train vs drifted applicants:", round(psi(s_tr, s_dr, sedges),3))
```

Real output:

```
bad rate 0.125
IV: {'age': 0.064, 'income': 0.071, 'util': 0.142, 'late': 0.197}
                 good    bad    woe     iv
util
(-0.001, 0.21]  3856.5  309.5  0.580  0.056
(0.21, 0.33]    3700.5  411.5  0.254  0.012
(0.33, 0.44]    3316.5  447.5  0.061  0.001
(0.44, 0.58]    3427.5  554.5 -0.121  0.003
(0.58, 0.97]    3195.5  785.5 -0.539  0.070
coefs: {'age': -1.01, 'income': -1.06, 'util': -1.02, 'late': -1.02}
train: AUC=0.691 Gini=0.383 KS=0.283
test: AUC=0.671 Gini=0.342 KS=0.247
score range test: 463 596 mean 547.9
PSI train vs test: 0.0015
PSI train vs drifted applicants: 0.167
```

**Reading the output.** Late payments carry the most information (IV 0.197) and utilisation next (0.142), matching the generator. In the utilisation table WoE falls steadily from +0.580 for the lowest-use bin to -0.539 for the highest: a clean, monotonic pattern that an auditor can read. All four coefficients are near -1, as expected. Train Gini is 0.383 and test Gini is 0.342, KS 0.283 versus 0.247. A small drop is normal; a large drop means overfitting. PSI between train and test scores is 0.0015 (stable). For the drifted group it is 0.167, which a common rule of thumb places between 0.1 and 0.25: investigate. Cut-offs for PSI are conventions, not laws.

**Evaluation and acceptance criteria.**

- Test Gini within 0.05 of train Gini.
- WoE monotonic for the main continuous variables, or a written reason when it is not.
- KS and Gini reported on train and test.
- PSI computed monthly in a `monitor.py` function; alert level written in the README.
- Points table exported as a CSV a non-technical reader can follow.

**Common pitfalls and how to detect them.**

1. *Bins fitted on all data*: leaks the test set. Detect: code review; fit bins on train only, as here.
2. *Zero count in a bin*: infinite WoE. Detect: min count per bin; add smoothing.
3. *Non-monotonic WoE ignored*: confusing for a reviewer. Detect: print WoE per bin.
4. *Good Gini but bad calibration*: separation is not the same as correct probabilities. Detect: compare predicted and actual bad rate by score band.
5. *Reject inference ignored*: you only see outcomes for approved loans. Detect: ask how the data was selected; say it in the limits.

**Three extensions.**

1. Compare with gradient boosting and report the Gini gap; discuss the value of explainability.
2. Add an approval cut-off from a cost table: loss from a bad loan versus profit from a good one (Chapter 12).
3. Add a characteristic stability index (PSI per variable) to find which input drifted.

**Interview talking points.**

- *Q: What is WoE and why use it?* A: Weight of evidence turns each bin into a number that says how much it favours good over bad customers. It handles missing values, outliers and non-linear effects, and keeps the model easy to audit.
- *Q: KS versus Gini?* A: Both measure how well the score separates bad from good. Gini is a rescaled AUC over all thresholds. KS is the single best gap at one threshold. A model can have both a good Gini and a poor business cut-off.
- *Q: What does PSI of 0.17 mean?* A: The score distribution of today's applicants differs noticeably from the build sample. I would check which inputs moved, and re-validate the model before trusting it.

**README outline.** Question; data rules; binning and WoE tables; model and points table; train/test metrics; PSI monitoring; fairness and limits; how to run.

## 16.3 Project: Causal analysis with propensity scores and difference-in-differences (#30)

**Goal and why employers care.** Many business questions ("did the loyalty club raise spending?") cannot be tested with a coin flip. Employers want to see that you know the difference between correlation and effect, that you can use two standard tools, and, most of all, that you test your own answer for weak spots.

**Skills used.** Chapter 8, *Causal Inference When You Cannot Randomise* (confounding, propensity scores, difference-in-differences); Chapter 4, *Statistical Thinking: Ways Data Fools You*; Chapter 7, *Experimentation* (what the ideal test would be).

**Data.** Two generators in one script. Part A: 20,000 shoppers, where "engagement" drives both joining a loyalty club and spending, and the true effect of joining is +10. We observe only a noisy proxy (prior spend) and age. Part B: 20 cities over 12 months; 10 get free delivery from month 7 with a true effect of +8 orders per 1,000 customers. Real data alternatives: public policy-evaluation datasets; check licences.

**Architecture.**

```
Part A (observational):  naive difference -> propensity model -> balance check (SMD)
                         -> IPW estimate + bootstrap CI -> sensitivity: hide confounder
Part B (panel):          2x2 means -> DiD -> city-level bootstrap CI
                         -> placebo test + pre-trend check
```

**Step-by-step build.**

1. Compute the naive difference in means. It is wrong when joiners differ from non-joiners.
2. Fit logistic regression for the chance of joining from the observed covariates (the *propensity score*). Check overlap (both groups have similar score ranges) and covariate balance with the standardised mean difference (SMD = gap in means divided by pooled spread; below 0.1 is the common target).
3. Estimate the effect with inverse-probability weighting (IPW): weight treated people by `1/ps` and others by `1/(1-ps)`. Trim extreme scores. Bootstrap a confidence interval.
4. Sensitivity: remove the best covariate and see how far the answer moves. This shows what hidden confounding would do.
5. DiD: compute the change over time in treated cities minus the change in control cities.
6. Uncertainty: resample *cities*, not rows, because rows in one city are not independent.

7. Placebo and pre-trend: pretend the policy began earlier, using pre-period data only, and compare pre-period slopes.

```
import numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression, LinearRegression

# Step 1: data: propensity scores. Treatment = joined loyalty club; outcome = spend next 3
months. True effect = +10
r = np.random.default_rng(21); n = 20000
engage = r.normal(size=n); age = r.normal(40,10,n); prior = 50 + 15*engage +
r.normal(0,10,n)
p_join = 1/(1+np.exp(-(-1.0 + 1.2*engage + 0.02*(age-40))))
T = (r.random(n) < p_join).astype(int)
Y = 60 + 12*engage + 0.3*(age-40) + 10*T + r.normal(0,15,n)
df = pd.DataFrame(dict(engage=engage, age=age, prior=prior, T=T, Y=Y))
print("naive diff:", round(df.Y[df['T']==1].mean()-df.Y[df['T']==0].mean(),2), "(true
effect 10)")

# Step 2: propensity score and balance observe: prior spend (proxy of engagement) and age
Xc = df[["prior","age"]]
Xs = (Xc-Xc.mean())/Xc.std()
ps = LogisticRegression().fit(Xs, df['T']).predict_proba(Xs)[:,1]
df["ps"]=ps
print("propensity range treated:",
df.ps[df['T']==1].quantile([.01,.99]).round(2).tolist(), "control:",
df.ps[df['T']==0].quantile([.01,.99]).round(2).tolist())
def smd(x, t):
    a,b = x[t==1], x[t==0]; return (a.mean()-b.mean())/np.sqrt((a.var()+b.var())/2)
print("SMD before:", {c: round(float(smd(df[c], df['T'])),2) for c in ["prior","age"]})
# Step 3: IPW with trimming
d = df[(df.ps>0.02)&(df.ps<0.98)]
w = np.where(d['T']==1, 1/d.ps, 1/(1-d.ps))
ate = np.average(d.Y[d['T']==1], weights=w[d['T']==1]) - np.average(d.Y[d['T']==0],
weights=w[d['T']==0])
print("IPW ATE:", round(ate,2))
def wsmd(x,t,w):
    m1=np.average(x[t==1],weights=w[t==1]); m0=np.average(x[t==0],weights=w[t==0])
    return (m1-m0)/np.sqrt((x[t==1].var()+x[t==0].var())/2)
print("SMD after weighting:", {c: round(float(wsmd(d[c].values, d['T'].values, w)),2) for
c in ["prior","age"]})
# bootstrap CI
bs=[]
g=np.random.default_rng(0)
for _ in range(200):
    s=d.sample(len(d), replace=True, random_state=int(g.integers(1e9)))
    ww=np.where(s['T']==1,1/s.ps,1/(1-s.ps))
    bs.append(np.average(s.Y[s['T']==1],weights=ww[s['T']==1])-
np.average(s.Y[s['T']==0],weights=ww[s['T']==0]))
print("bootstrap 95% CI:", np.round(np.percentile(bs,[2.5,97.5]),2).tolist())
# Step 4: sensitivity the proxy (hidden confounder) -> estimate with age only
```

```
Xa = ((df[["age"]]-df.age.mean())/df.age.std())
pa = LogisticRegression().fit(Xa, df['T']).predict_proba(Xa)[:,1]
wa = np.where(df['T']==1,1/pa,1/(1-pa))
ate_h = np.average(df.Y[df['T']==1],weights=wa[df['T']==1]) -
np.average(df.Y[df['T']==0],weights=wa[df['T']==0])
print("IPW with engagement hidden (age only):", round(ate_h,2))

# Step 5: DiD data: difference-in-differences. 20 cities, 10 get free delivery from month
7. true effect = +8 orders/1000 customers
r = np.random.default_rng(4)
rows=[]
for c in range(20):
    treated = int(c<10); base = 100 + 5*r.normal() + (6 if treated else 0)   # treated
cities start higher (levels differ: fine)
    for m in range(12):
        post = int(m>=6)
        y = base + 1.5*m + 8*treated*post + r.normal(0,2)
        rows.append((c,m,treated,post,y))
p = pd.DataFrame(rows, columns=["city","month","treated","post","y"])
means = p.groupby(["treated","post"]).y.mean().unstack()
print(means.round(2))
did = (means.loc[1,1]-means.loc[1,0]) - (means.loc[0,1]-means.loc[0,0])
print("DiD estimate:", round(did,2))
# Step 6: uncertainty with city-clustered bootstrap
def did_est(q):
    mm = q.groupby(["treated","post"]).y.mean().unstack(); return (mm.loc[1,1]-
mm.loc[1,0])-(mm.loc[0,1]-mm.loc[0,0])
g=np.random.default_rng(1); est=[]
for _ in range(500):
    cs_t = g.choice(range(10),10); cs_c = g.choice(range(10,20),10)
    q = pd.concat([p[p.city==c] for c in list(cs_t)+list(cs_c)])
    est.append(did_est(q))
print("cluster bootstrap 95% CI:", np.round(np.percentile(est,[2.5,97.5]),2).tolist())
# Step 7: placebo and pre-trend pretend treatment began at month 3, use only pre months 0-
5
pre = p[p.month<6].copy(); pre["post"]=(pre.month>=3).astype(int)
print("placebo DiD (fake start month 3, pre-data only):", round(did_est(pre),2))
# parallel-trend check: slope of pre-period by group
for tr in [1,0]:
    q = p[(p.treated==tr)&(p.post==0)].groupby("month").y.mean()
    print("pre-trend slope treated=%d:"%tr, round(np.polyfit(q.index,q.values,1)[0],2))
```

Real output:

```
naive diff: 21.74 (true effect 10)
propensity range treated: [0.08, 0.84] control: [0.03, 0.71]
SMD before: {'prior': 0.87, 'age': 0.17}
IPW ATE: 14.02
SMD after weighting: {'prior': 0.01, 'age': 0.0}
bootstrap 95% CI: [13.36, 14.74]
IPW with engagement hidden (age only): 21.26
post          0       1
treated
0        104.74  113.92
1        107.49  124.92
DiD estimate: 8.25
cluster bootstrap 95% CI: [7.54, 8.88]
placebo DiD (fake start month 3, pre-data only): 0.13
pre-trend slope treated=1: 1.6
pre-trend slope treated=0: 1.53
```

**Reading the output.** The naive difference is 21.74, more than double the truth (10). Before weighting, prior spend differs between groups by 0.87 SMD; after weighting both SMDs are about 0. Even so, the IPW estimate is 14.02 with a CI of 13.36 to 14.74, which **excludes the truth of 10**. Balance on what we measured did not remove bias, because prior spend is only a noisy proxy for engagement. This is the key lesson: good balance does not prove there is no hidden confounder. In the sensitivity run, using age alone gives 21.26, nearly the naive number, which shows how much the proxy mattered. DiD gives 8.25 (true 8), with a city-bootstrap CI of 7.54 to 8.88. The placebo effect is 0.13, close to 0, and pre-trend slopes are 1.60 and 1.53: parallel. The treated cities start higher, and DiD does not mind, because it uses changes.

**Evaluation and acceptance criteria.**

- Report naive, adjusted and true (for synthetic data) side by side.
- SMD after weighting below 0.1 for all observed covariates; overlap plot shown.
- DiD placebo effect within its CI of zero; pre-trend slopes within 0.2 of each other.
- A written paragraph naming at least two unmeasured confounders and which direction each would bias the result.
- Cluster-level resampling used for panel data.

**Common pitfalls and how to detect them.**

1. *Conditioning on a post-treatment variable*: detect by drawing the timeline; only use pre-treatment covariates.
2. *No overlap*: some people have almost no chance of treatment. Detect: histogram of scores by group; trim.
3. *Treating balance as proof*: detect with sensitivity analysis, as in step 4.
4. *Parallel trends assumed, not checked*: detect with pre-period slopes and a placebo.
5. *Row-level standard errors in a panel*: intervals too narrow. Detect: compare with a city-level bootstrap.

**Three extensions.**

1. Add propensity-score matching and doubly robust estimation and compare all estimates.
2. Add an event-study plot with an effect for each month before and after.
3. Add an E-value or similar sensitivity measure (not run here) and explain it in one paragraph.

**Interview talking points.**

- *Q: Why not just compare joiners with non-joiners?* A: Joiners are different: more engaged people join and also spend more. The gap mixes the effect of the club with the effect of who joins.
- *Q: What does difference-in-differences assume?* A: That without the policy, treated and control groups would have moved in parallel. I check pre-period trends and a placebo date, but I can never prove it.
- *Q: Your adjusted estimate is still off. How would you tell?* A: In real data I cannot see the truth. I would remove covariates one at a time, compare methods and ask how strong a hidden confounder must be to erase the result. Then I would say what experiment would settle it.

**README outline.** Question; why randomising was impossible; table of naive, adjusted and DiD estimates; balance and overlap figures; placebo and pre-trend; the list of unmeasured confounders; limits; how to run.

## 16.4 Worked Debugging Problems and Marking Guide

Each project in this chapter has a classic failure that produces a flattering number. Here are three, with a test that catches each. Spot the bug first, then read the steps.

| INTUITION | Ask of every number: "which rows and which dates did this calculation see?" Metrics break when the evaluation sees information the real system will not have, or when the formula quietly changes the target. |
|---|---|

**Problem 16.4.1** (recommender, wrong NDCG) A user has 3 relevant items in the catalogue. Your ranked top 5 has relevance [0, 1, 0, 1, 0]. The code reports NDCG@5 of 0.651. Is that right?

| | |
|---|---|
| **Formula** | DCG = Σ rel / log2(rank + 1). Ideal DCG is the DCG of the best possible top 5 for that user, using ALL their relevant items. |
| **DCG** | Rank 2: 1/log2(3) = 0.6309. Rank 4: 1/log2(5) = 0.4307. Sum 1.0616. |
| **Wrong ideal** | Sorting the shown list gives [1, 1, 0, 0, 0]: 1 + 0.6309 = 1.6309. NDCG 1.0616 / 1.6309 = 0.651. |
| **Right ideal** | Three relevant items fit in the top 5: 1 + 0.6309 + 0.5 = 2.1309. NDCG 1.0616 / 2.1309 = 0.498. |

```
import numpy as np
def dcg(rels): return sum(r / np.log2(i + 2) for i, r in enumerate(rels))
shown = [0, 1, 0, 1, 0]            # relevance of the 5 items we ranked
total_relevant = 3                 # the user actually has 3 relevant items in the
catalogue
wrong_ideal = dcg(sorted(shown, reverse=True))
right_ideal = dcg([1] * min(total_relevant, 5))
print("DCG@5:", round(dcg(shown), 4))
print("wrong ideal (sorts the shown list):", round(wrong_ideal, 4), " NDCG",
round(dcg(shown) / wrong_ideal, 3))
print("right ideal (all relevant items):  ", round(right_ideal, 4), " NDCG",
round(dcg(shown) / right_ideal, 3))
```

```
DCG@5: 1.0616
wrong ideal (sorts the shown list): 1.6309  NDCG 0.651
right ideal (all relevant items):   2.1309  NDCG 0.498
```

- **Step 1** The bug inflates NDCG from 0.498 to 0.651 because the ideal list was built only from items the system chose to show. A system that retrieves poorly looks good.
- **Step 2** The test: a ranking that shows none of the user's relevant items must score 0, and a perfect one must score 1 regardless of what is retrieved.

| | |
|---|---|
| **COMMON MISTAKE** | Computing the ideal from the retrieved list. Candidate generation errors then disappear from your metric. Ideal DCG must come from the ground truth, truncated to k. |

**Problem 16.4.2** (scorecard, leakage through binning) A noise feature with no relationship to default is cut into 60 quantile bins, WoE is computed, and the score is tested on a held-out set. Gini is 0.435. Real or leak?

| | |
|---|---|
| **Suspect** | Were the bins and WoE values learned on all 1,000 rows before the split? Then each held-out row's WoE was computed partly from its own label. |
| **Test** | Learn bins and WoE on training rows only, then apply them to the held-out rows. |
| **Rule** | Anything that learns from labels (WoE, target encoding, binning for IV) belongs inside the training fold. |

```
import numpy as np
from sklearn.metrics import roc_auc_score
rng = np.random.default_rng(15)
n = 1000
x = rng.normal(size=n)                                  # a pure-noise feature
y = (rng.random(n) < 0.2).astype(int)                   # labels unrelated to x
idx = rng.permutation(n); tr, te = idx[:600], idx[600:]
def woe_map(x_fit, y_fit, bins=60):
    edges = np.quantile(x_fit, np.linspace(0, 1, bins + 1)); edges[0], edges[-1] = -
np.inf, np.inf
    b = np.digitize(x_fit, edges[1:-1])
    g = np.bincount(b, weights=1 - y_fit, minlength=bins) + 0.5
    bd = np.bincount(b, weights=y_fit, minlength=bins) + 0.5
    return edges, np.log((g / g.sum()) / (bd / bd.sum()))
def apply(x_new, edges, woe): return woe[np.digitize(x_new, edges[1:-1])]
# bug: WOE learned from ALL rows, then evaluated on the test rows
e_all, w_all = woe_map(x, y)
leaky = roc_auc_score(y[te], -apply(x[te], e_all, w_all))
e_tr, w_tr = woe_map(x[tr], y[tr])
honest = roc_auc_score(y[te], -apply(x[te], e_tr, w_tr))
print("test AUC with WOE fitted on all rows (leak):", round(leaky, 3), " Gini",
round(2*leaky - 1, 3))
print("test AUC with WOE fitted on train only:     ", round(honest, 3), " Gini",
round(2*honest - 1, 3))
```

```
test AUC with WOE fitted on all rows (leak): 0.718  Gini 0.435
test AUC with WOE fitted on train only:      0.542  Gini 0.083
```

- **Step 1** The leaky Gini is 0.435 on a feature that is pure noise. The honest Gini is 0.083, which is just small-sample luck.
- **Step 2** More bins and smaller samples make this leak bigger. With 60 bins and about 17 rows per bin, each bin's WoE mostly memorises its own labels.

**Problem 16.4.3** (causal project, parallel trends fail) A pilot region has been growing faster than the control region even before launch. The true effect of the launch is +4. Show that the plain DiD overstates it and what to do.

| | |
|---|---|
| **Step 1** | Plot or fit the gap between treated and control in the pre-period. If it trends, parallel trends fail. |
| **Step 2** | The plain DiD attributes the pre-existing extra growth to the launch. |
| **Step 3** | Option: extrapolate the pre-period gap trend and subtract it. This relies on the trend continuing, which is its own assumption. |

```
import numpy as np
rng = np.random.default_rng(13)
T, T0 = 16, 10
t = np.arange(T)
ctrl = 100 + 1.0*t + rng.normal(0, 0.5, T)
treat = 110 + 2.0*t + rng.normal(0, 0.5, T)                # already growing faster
BEFORE launch
treat[T0:] += 4.0                                          # true effect 4
pre_gap = (treat - ctrl)[:T0]
slope = np.polyfit(t[:T0], pre_gap, 1)[0]
did = (treat[T0:].mean() - treat[:T0].mean()) - (ctrl[T0:].mean() - ctrl[:T0].mean())
print("pre-period gap slope per period:", round(slope, 3), "(should be about 0)")
print("naive DiD estimate:", round(did, 2), " true effect: 4.0")
# fix: allow a group-specific linear trend estimated on the pre-period
cf = np.polyval(np.polyfit(t[:T0], treat[:T0] - ctrl[:T0], 1), t[T0:])
est = ((treat - ctrl)[T0:] - cf).mean()
print("trend-adjusted estimate:", round(est, 2))
```

```
pre-period gap slope per period: 1.091 (should be about 0)
naive DiD estimate: 11.83  true effect: 4.0
trend-adjusted estimate: 3.1
```

- **Step 1** The pre-launch gap grows by 1.09 per period, so the "should be about 0" check fails.
- **Step 2** The plain DiD returns 11.83, nearly three times the truth of 4.
- **Step 3** The trend-adjusted estimate is 3.10. It is much closer, but extrapolating a noisy slope adds error. Report both, with the pre-trend check, and prefer a better control group or a synthetic control when you can build one.

| KEY IDEA | A causal estimate is only as good as its pre-launch diagnostics. Publish the pre-trend plot and the placebo test next to the headline number. |
|---|---|

### Marking guide for the three projects

| Project | Must have | Healthy ranges | Red flags |
|---|---|---|---|
| Recommender (16.1) | Popularity baseline, recall for stage 1, NDCG for stage 2, cold-start plan | Beats popularity on NDCG; recall at 100 candidates well above the ranker's top-10 hit rate | NDCG above 0.9 on real-looking data; ideal DCG from the shown list |
| Scorecard (16.2) | Time-based split, WoE fitted on train only, KS, Gini, PSI | Gini 0.3 to 0.6 on retail credit data; PSI under 0.1 stable, 0.1 to 0.25 watch, above 0.25 act | Gini above 0.8; train Gini far above test Gini |
| Causal (16.3) | DAG, overlap plot, balance table (SMD under 0.1), pre-trend and placebo checks | Estimates stable across adjustment, IPW and DiD | Single estimate, no diagnostics, SMD above 0.1 left unaddressed |

## Chapter recap

- Two-stage recommenders pair a fast candidate generator with a better ranker; report recall for stage 1 and NDCG for stage 2.
- Compare with popularity; judge warm and cold items separately; test cold start with a cold-only ranking.
- A scorecard is bins, WoE, logistic regression and points; every step is auditable.
- Gini and KS measure separation; PSI measures drift; none of them measures calibration.
- Fit bins on training data only; watch zero counts and non-monotonic WoE.
- Naive group comparisons overstate effects when the groups differ.
- Propensity weighting fixes imbalance on measured variables only; test with sensitivity runs.
- DiD needs parallel trends; check pre-trends and a placebo, and resample units, not rows.

| Pattern | Formula |
|---|---|
| NDCG@k | `DCG/ideal DCG`, `DCG = Σ rel_i / log2(i+1)` |
| WoE and IV | `WoE = ln(%good/%bad)`, `IV = Σ (%good − %bad) × WoE` |
| Gini | `2 × AUC − 1` |
| Score points | `offset + (PDO/ln 2) × ln(odds good)` |

| Pattern | Formula |
| --- | --- |
| PSI | `Σ (a − e) × ln(a/e)` |
| IPW weights | treated `1/ps`, control `1/(1−ps)` |
| DiD | `(treated post − treated pre) − (control post − control pre)` |

## Practice set

**1.** (easy) A relevant item appears at ranks 1 and 3 of the top 10; the user has 2 relevant items in total. Compute NDCG@10. **Answer:** `DCG = 1/log2(2) + 1/log2(4) = 1 + 0.5 = 1.5`. Ideal `= 1 + 1/log2(3) = 1.631`. NDCG `= 1.5/1.631 = 0.920`.

**2.** (easy) A bin holds 60% of goods and 30% of bads. Compute its WoE and its IV contribution. **Answer:** `WoE = ln(0.6/0.3) = ln 2 = 0.693`. IV `= (0.6 − 0.3) × 0.693 = 0.208`.

**3.** (easy) Treated sales went from 100 to 110; control from 90 to 95. DiD estimate? **Answer:** `(110 − 100) − (95 − 90) = 5`.

**4.** (medium) Score is 600 at odds 50 to 1 with PDO of 20. What score has odds 100 to 1? **Answer:** doubling the odds adds 20 points, so 620.

**5.** (medium) Expected score share per bin is [0.5, 0.5]; today it is [0.6, 0.4]. Compute PSI. **Answer:** `0.1 × ln(1.2) + (−0.1) × ln(0.8) = 0.0182 + 0.0223 = 0.0405`. Below 0.1, stable.

**6.** (find the bug) A scorecard builder computes quantile bins and WoE on the full dataset, then splits into train and test. Test Gini is 0.40, but production Gini is 0.31. Why? **Answer:** the bins and WoE values saw the test labels, which leaked information. Fit bins and WoE on training data only.

**7.** (scenario) After weighting, SMD for all measured covariates is below 0.05, and a manager says the estimate is causal. Reply. **Answer:** balance holds only on measured variables. Ask about unmeasured drivers, show what happens when a covariate is dropped, report a sensitivity analysis and say what test would confirm it.

**8.** (interview) "Your recommender has NDCG@10 of 0.30 offline. Why might a live test show no gain?" **Answer:** offline data reflects past exposure, so the metric rewards items users already saw. Candidate recall may be limited, popularity may already capture most gain and users react to new lists. Run an A/B test with a pre-set sample size and guardrails.

**9.** (medium) A ranker retrieves only 1 of a user's 4 relevant items, at rank 1. Compute NDCG@5 correctly. **Answer:** DCG = 1. Ideal DCG at k = 5 with 4 relevant items = 1 + 0.6309 + 0.5 + 0.4307 = 2.5616. NDCG = 1 / 2.5616 = 0.390. A buggy ideal built from the shown list would give 1.0 and hide the missed items.

**10.** (hard) Pre-launch trends differ and the trend-adjusted DiD gives 3.1 while the plain DiD gives 11.8. Which do you report to the board? **Answer:** Report the adjusted estimate as the main figure, show the plain DiD and the pre-trend plot, and state that the adjustment assumes the pre-period trend gap would have continued. Offer a synthetic control or a holdout in the next rollout to remove that assumption.

## Prompt pack

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Help me plan a two-stage recommender project on a MovieLens-style dataset: candidate generation, ranking, NDCG@10 and cold start. Ask me three questions about my data first, one at a time. Then propose the folder layout, the baselines I must beat and three acceptance criteria with numbers. Keep your English simple and explain every technical word.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Review my recommender evaluation for leakage and weak baselines. Check: train/test split by user, whether seen items are excluded, popularity baseline, candidate recall, NDCG@10 definition and how cold items are judged. Say pass, fail or unknown for each and what to fix. My code and numbers: [PASTE HERE]. Use simple English.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Teach me to debug a credit scorecard: WoE and IV, monotonic bins, logistic regression on WoE, KS, Gini and PSI. Give me eight graded exercises with a tiny table of goods and bads, without solutions, then check my answers one at a time. If I make a mistake, ask a question that helps me find it. Use simple English.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Review my causal analysis (propensity scores and difference-in-differences). Check overlap, covariate balance, parallel trends, placebo tests, clustering of standard errors and unmeasured confounders. Then write the three hardest questions a sceptical reviewer would ask. My analysis and output: [PASTE HERE]. Use simple English and define any jargon.
```

**✨ AI prompt: copy and paste into any AI**

```
You are my patient, practical professor. I am a working professional moving into data roles. Run a mock interview about my recommender, credit scorecard and causal analysis projects. Ask one question at a time, wait for my answer, then score it for correctness and clarity and show a better model answer in simple English. Start with the credit scorecard.
```

APPENDIX

# CHEAT SHEET

*One page per idea: the formulas, patterns and rules from every chapter, with the number from the chapter next to each so you can check your memory.*

Use this appendix after you have read the chapters. If a line feels unfamiliar, go back to the chapter named in the heading and redo its worked problem. A formula you cannot explain in plain words is not yet yours. Symbols: `T` is the prediction time, `p` a rate or probability, `n` a count, `δ` a difference.

## Chapter 1: The Job, the Lifecycle, Framing and Baselines

| Idea | Formula or pattern | Example |
|---|---|---|
| Improvement % | `(old − new) / old × 100` | 480 to 360 is 25% |
| Base rate | `positives / rows` | 651 / 2,562 = 0.254 |
| Lift | `rate in group / base rate` | 0.778 / 0.254 = 3.07 |
| Precision, recall | `true flagged / flagged` , `true flagged / all true` | 0.759 and 0.662 for the 30-day rule |
| Expected value per target | `rate × save rate × value − cost` | 0.778 × 0.30 × 600 − 100 = 40 |
| Timeline rule | features before `T` , label after `T` | T = 2025-01-01, 60-day window |
| Brief (one page) | problem, users, data, metric, baseline, risks, plan | Chapter 1.2 |

## Chapter 2: Trustworthy Data

| Idea | Pattern | Example |
|---|---|---|
| Point-in-time filter | `WHERE ts < :T` | `order_ts < '2025-01-01'` |
| Label window | `ts >= :T AND ts < :T + horizon` | next 60 days |

| Idea | Pattern | Example |
|---|---|---|
| Row-count guard | compare `len` before and after a join | 2 customers became 3 rows |
| Duplicate check | `df.key.duplicated().sum()` | 300 |
| Reconcile | `(yours − finance) / finance` | 0.17% |
| Cohen's kappa | `(po − pe) / (1 − pe)` | (0.85 − 0.5) / 0.5 = 0.7 |
| k-anonymity | `groupby(quasi_ids).size().min()` | 15 |

# Chapter 3: EDA for Modelling

| Task | Pattern | Meaning |
|---|---|---|
| Profile | `df.shape`, `df.describe()`, `df.nunique()` | size, summary, cardinality |
| Missing by target | `df[c].isna().groupby(df.y).mean()` | informative blanks |
| Mode spike | `(df[c] == −1).mean()` | default codes |
| Rate by bin | `pd.cut` then `groupby().y.agg(["mean","size"])` | feature against target |
| Single-feature AUC | `max(a, 1 − a)` | leak or signal |
| Simpson check | `groupby([seg, treat]).y.mean().unstack()` | does the sign flip? |
| Finding format | hypothesis, evidence, so what | every finding |

# Chapter 4: Statistical Thinking

| Idea | Formula | Example |
|---|---|---|
| Rate standard error | $\sqrt{p(1-p)/n}$ | 0.009 for p = 0.25, n = 2,300 |
| Two-rate CI | $\text{diff} \pm 1.96 \times \sqrt{p_0 q_0/n_0 + p_1 q_1/n_1}$ | 0.143 ± 0.033 |
| Regression to the mean | $\text{mean} + r \times (x_1 - \text{mean})$ | 100 + 0.5 × (73.7 − 100) = 86.85 |

| Idea | Formula | Example |
|---|---|---|
| PPV | `TP / (TP + FP)` | 99 / 1,098 = 0.090 |
| Family error | `1 − (1 − α)^m` | 0.642 for 20 tests |
| Bonferroni | `α / m` | 0.05 / 20 = 0.0025 |
| Beta update | `Beta(a + k, b + n − k)` | Beta(1,1) + 3 of 10 gives Beta(4, 8) |

## Chapters 5 and 6: Feature Engineering

| Task | Pattern | Example |
|---|---|---|
| Log of skewed value | `np.log1p(x)` | skew 2.17 to −0.43 |
| Clip outliers | `x.clip(q01, q99)` | max 41,357 to 24,601 |
| One-hot | `pd.get_dummies(df, drop_first=True)` | `plan_pro` |
| Smoothed target mean | `(sum + m·gm) / (count + m)` | (2 + 20 × 0.254) / 23 = 0.308 |
| Out-of-fold encoding | encode fold k with the other folds | `KFold(5)` |
| Cyclical time | `sin(2πh/24)` , `cos(2πh/24)` | hours 23 and 0 are close |
| Haversine | `2R·arcsin(√a)` , R = 6371 | Mumbai to Pune 120.2 km |
| Window mask | `past.age_d <= w` , `age_d = (T − ts).dt.days` | 7, 30, 90 days |
| Trend ratio | `(a_now + 1) / (a_before + 1)` | 3.33 |
| RFM score | `pd.qcut(x, 5, labels=...)` | R, F, M in 1 to 5 |
| PSI | `Σ (a − e) × ln(a / e)` | 0.0629 by hand; below 0.1 stable |

| Task | Pattern | Example |
|---|---|---|
| Permutation importance | `permutation_importance(m, X, y, scoring="roc_auc")` | recency drop 0.198 |
| Leakage-safe pipeline | `Pipeline([("pre", ColumnTransformer(...)), ("clf", model)])` | AUC 0.893 |

# Chapter 7: Experimentation

| Item | Formula or pattern |
|---|---|
| Sample size (rule of thumb) | $n \approx 16\, p(1-p) / \delta^2$ per arm |
| Sample size (exact) | $(z_{\alpha/2} \sqrt{(2\bar{p}(1-\bar{p}))} + z_{\beta} \sqrt{(p_1 q_1 + p_2 q_2)})^2 / \delta^2$ |
| z-test | $z = (pB - pA) / \sqrt{(p(1-p)(1/nA + 1/nB))}$ |
| CUPED | $Y_{adj} = Y - \theta(X - \text{mean } X)$, $\theta = \text{Cov}(Y,X)/\text{Var}(X)$; variance × $(1 - \rho^2)$ |
| SRM check | $\chi^2 = \Sigma\, (O - E)^2 / E$; alarm if $p < 0.001$ |
| Peeking fix | one look, Pocock-style boundaries, or a sequential method |
| Ratio metric | delta method or user-level bootstrap |
| Bandit (Thompson) | draw from Beta(s, f), play the max, update |

# Chapter 8: Causal Inference

| Method | Use when | Key assumption | Quick check |
|---|---|---|---|
| Adjustment, IPW, AIPW | all confounders measured | no hidden confounding, overlap | SMD below 0.1, propensity range |
| Difference-in-differences | treated and control, before and after | parallel trends | pre-trends, placebo |

| Method | Use when | Key assumption | Quick check |
|---|---|---|---|
| Synthetic control | one or few treated units | good pre-fit | placebo ranking |
| Regression discontinuity | rule with a cut-off | no manipulation | window sweep |
| Instrumental variable | hidden confounding plus a random lever | relevance, independence, exclusion | first-stage F |
| Uplift | who to target | random or adjusted assignment | decile uplift, Qini |

Tiny DiD: `(16 − 10) − (12 − 9) = 3`.

# Chapter 9: Forecasting

| Item | Formula or pattern |
|---|---|
| Seasonal naive | `f(t+h) = y(t+h−m)` |
| Simple exponential smoothing | `level = α·y + (1−α)·level` |
| WAPE | `Σ |
| MASE | `MAE / MAE(naive on train)` |
| Differencing | `y(t) − y(t−1)`, seasonal `y(t) − y(t−m)` |
| Lag features | `shift(k)`, `shift(1).rolling(w).mean()` |
| Backtest | train to an origin, forecast h steps, roll the origin forward |
| Empirical interval | percentiles of backtest relative errors; check coverage |
| Safety stock | `z × σ_daily × √lead time` = 1.645 × 30 × 2 = 98.7 |
| Newsvendor | order the quantile at `c_under / (c_under + c_over)` = 0.80, Q = 584 |

# Chapter 10: Recommendation and Search

| Item | Formula or pattern |
|---|---|
| Cosine similarity | `a·b / (‖a‖ ‖b‖)` |
| Item-item score | `X · Sim` |
| Precision@K, Recall@K | hits / K, hits / all relevant |
| NDCG | `Σ rel / log₂(pos + 1)` divided by the ideal; relevant at ranks 1 and 3 gives 0.92 |
| MRR | `1 / rank of first hit` |
| Matrix factorization score | `P[u] · Q[i]` |
| ALS confidence | `c = 1 + α·x` |
| IPS | weight `1 / P(examined)` |
| BM25 | `idf × tf(k1+1) / (tf + k1(1 − b + b·len/avglen))` |
| Evaluation rule | time split, never random; A/B test decides |

# Chapter 11: Applied Domains

| Idea | Formula or pattern | Example |
|---|---|---|
| Kaplan-Meier | `S × (1 − d/n)` at each churn time | 0.8, 0.6, 0.3 |
| Cohort CLV | 12-month revenue per customer × margin | 7,610 × 0.20 = 1,522 |
| WoE | `ln(share of goods / share of bads)` | ln(0.5) = −0.693 |
| IV | `Σ (pg − pb) × WoE` ; under 0.02 useless, 0.1 to 0.3 medium, over 0.5 suspicious | 0.168 |
| Gini | `2 × AUC − 1` | 0.3706 |
| KS | max gap between cumulative bad and good curves | 0.295 |

| Idea | Formula or pattern | Example |
|---|---|---|
| Score scale | `factor = PDO / ln 2`, `offset = base − factor × ln(odds)` | 28.854, 487.123 |
| PSI | `Σ (a − e) ln(a / e)` | 0.0405 by hand; 0.456 after a shift |
| Expected loss | `PD × LGD × EAD` | 0.03 × 0.45 × 200,000 = 2,700 |
| Fraud threshold | `c_review / (c_review + c_miss)` | 100 / 5,100 = 0.0196 |
| Adstock, saturation | `a × prev + x`, `x / (x + k)` | 100, 50, 25; 0.25, 0.5, 0.75 |
| Elasticity | slope of `ln(units)` on `ln(price)`; `p∗ = c·e/(1+e)` | −1.778; do not extrapolate |
| Net benefit | `TP/N − FP/N × p_t/(1 − p_t)` | 0.0354 at 0.10 |
| Little's law | `L = λ × W` | 50 × 0.4 = 20 |
| M/M/1 time in system | `R = S / (1 − ρ)` | 0.1 h gives 2.0 h at 95% |
| Robust z-score | `0.6745 × (x − rolling median) / rolling MAD` | flag above 6 |
| Capacity date | `(limit − intercept) / slope`, bootstrap for an interval | day 122.7, 90% range 119 to 126 |

# Chapter 12: From Prediction to Decision

| Idea | Formula or command | Example |
|---|---|---|
| Break-even probability | `cost / (save rate × value)` | 100 / 450 = 0.222 |
| Cost-matrix profit | `TP × gain + FP × cost` | 179 × 350 − 94 × 100 = 53,250 |
| Linear program | `linprog(c, A_ub, b_ub, bounds)`; negate to maximise | allocation (50, 40, 10), profit 253 |

| Idea | Formula or command | Example |
|---|---|---|
| Shadow price | `res.ineqlin.marginals` | 1.5 per extra unit of budget |
| Knapsack | `milp(−value, LinearConstraint(w, 0, cap), integrality=1)` | 3,205 against greedy 3,113 |
| Assignment | `linear_sum_assignment(C)` | cost 13 |
| Monte Carlo | draw inputs, compute, `np.percentile` | mean 44,614, P(loss) 0.104 |
| Tornado | move one input from P10 to P90 | save rate range 82,517 |
| Queue simulation | `w_i = max(0, w_{i−1} + s_{i−1} − gap_i)` | mean about 1.0 h at 90% |
| M/M/1 95th percentile | `ln(20) × S / (1 − ρ)` | 2.996 h at ρ = 0.9 |
| Bayes decision | expected loss `E[max(pa − pb, 0)]` | 0.00024 for B, 0.01519 for A |
| Selective labels | keep a random holdout and an exploration group | observed 5.7% against true 16.9% |

# Chapter 13: Communication, Ethics, Reproducibility, Production

| Pattern | Rule or command | Example |
|---|---|---|
| Memo order | recommendation, why, how sure, risks, appendix | one page, answer first |
| Odds ratio per step | `exp(coef / sd × step)` | `exp(1.647 / 22.61 × 10) = 2.07` |
| Disparate impact | lower selection rate / higher; warn below 0.8 | 0.305 / 0.324 = |

| Pattern | Rule or command | Example |
|---|---|---|
| | | 0.94 |
| Proxy test | predict the group from the features; AUC near 0.5 is good | 0.485, then 1.0 with `city` |
| Seeds | `np.random.default_rng(42)`, `random_state=42` | same output twice |
| Environment | `pip freeze > requirements.txt` | pinned versions |
| One-command rerun | `make report` | README, config, tests |
| Data fingerprint | `hashlib.sha256(pd.util.hash_pandas_object(...).values.tobytes())` | record with the model |
| Serving prototypes | `streamlit run app.py`, `uvicorn api:app` | not run here |
| Skew check | one feature function for training and serving | mean risk 0.254 became 0.998 with the wrong date |
| Monitoring | service health, score and input drift (PSI), outcomes, holdout, rollback | PSI above 0.25 acts |

# Chapter 14: Capstones and Interviews

| Item | Formula or rule | Example |
|---|---|---|
| Baseline gate | model must beat the simple rule, with an interval on the gain | gain −0.006, interval −0.016 to +0.005 |

| Item | Formula or rule | Example |
|---|---|---|
| Sample size per arm | $2\,(z_{\alpha/2} + z_{\beta})^2\, p(1-p) / \delta^2$ | 24,112 for p = 0.04, δ = 0.005 |
| PPV | $\text{sens} \times \text{prev} / (\text{sens} \times \text{prev} + (1 - \text{spec})(1 - \text{prev}))$ | 0.161 |
| Decomposing a drop | total change = change in each part | 9,000 of 10,000 from returning users |
| Mastery test score | $\text{points} / (2 \times \text{lines})$ | target 75% each |
| Answer shape | clarify, metric and baseline, method, risks, next step | every interview answer |

## Common Code Patterns

```
# point-in-time label table (Chapters 1, 2)
past = orders[orders.order_ts < T]
future = orders[(orders.order_ts >= T) & (orders.order_ts < T + pd.Timedelta(days=60))]

# honest comparison with a baseline (Chapter 14.1)
gains = [roc_auc_score(y[i], model_p[i]) - roc_auc_score(y[i], base_p[i])
         for i in (rng.integers(0, n, n) for _ in range(1000))]
np.percentile(gains, [2.5, 97.5])

# threshold from costs (Chapter 12.1)
p_star = cost_act / (success_rate * value_if_saved)
send = p >= p_star
```

## An Honest Closing Note

If you worked through the chapters, you can now do something many people with "data science" on their résumé cannot. You can turn a vague request into a one-page brief with a decision, a metric and a baseline. You can build a training table that does not leak the future. You can check data quality, explore a dataset for hidden traps, and explain why a number could fool you. You can build features, test an idea with an experiment or a causal method, forecast with a fair backtest, and rank items for a user. You can work in a domain such as churn, credit, fraud or operations. Most important, you can take a probability and turn it into a decision with costs, a budget and a range of outcomes, then explain it in plain language, keep it fair, rerun it with one command, and watch it after launch.

This document does **not** make you an expert. It is a map with worked examples, built on small synthetic data where the true answer is known. Real data is messier, bosses are less patient, and many problems have no clean answer. Three honest limits to remember. First, the numbers in this document come from simulated data; real results will differ, so always test your own data. Second, we did not cover deep learning, language models and computer vision, large-scale data engineering, SQL tuning, cloud platforms or full MLOps pipelines; those have their own study documents. Third, domain depth takes years: one chapter on credit risk or healthcare gives you the vocabulary, not the judgment of a person who has seen a hundred failures.

What to practise next, in order:

1. Finish one capstone from Chapter 14 properly. A single complete project with a README, a memo and a baseline comparison beats ten notebooks.
2. Redo the mastery tests with a timer and score yourself honestly. Repeat the lowest one after a week.
3. Pick one domain from Chapter 11 and spend a month on it: read its terms, build one project, talk to someone who works there.
4. Practise explaining each chapter aloud in five minutes, in simple words. If you cannot, the topic is not yet obvious, and the rule of this document applies: do not move on.
5. Use the Prompt packs, but do the work first. Ask an AI to quiz you, not to do your thinking.

You do not need to know everything. You need to be honest about what you know, careful with data, and clear with people. Keep building, keep checking, and be patient with yourself.